CHAPTER 7

Relational-Database Design

This chapter continues our discussion of design issues in relational databases. In general, the goal of a relational-database design is to generate a set of relation schemas that allows us to store information without unnecessary redundancy, yet also allows us to retrieve information easily. One approach is to design schemas that are in an appropriate *normal form*. To determine whether a relation schema is in one of the desirable normal forms, we need additional information about the real-world enterprise that we are modeling with the database. In this chapter, we introduce the notion of functional dependencies. We then define normal forms in terms of functional dependencies and other types of data dependencies.

7.1 First Normal Form

The first of the normal forms that we study, **first normal form**, imposes a very basic requirement on relations; unlike the other normal forms, it does not require additional information such as functional dependencies.

A domain is **atomic** if elements of the domain are considered to be indivisible units. We say that a relation schema R is in **first normal form** (1NF) if the domains of all attributes of R are atomic.

A set of names is an example of a nonatomic value. For example, if the schema of a relation *employee* included an attribute *children* whose domain elements are sets of names, the schema would not be in first normal form.

Composite attributes, such as an attribute *address* with component attributes *street* and *city*, also have nonatomic domains.

Integers are assumed to be atomic, so the set of integers is an atomic domain; the set of all sets of integers is a nonatomic domain. The distinction is that we do not normally consider integers to have subparts, but we consider sets of integers to have subparts—namely, the integers making up the set. But the important issue is not what the domain itself is, but rather how we use domain elements in our database.

The domain of all integers would be nonatomic if we considered each integer to be an ordered list of digits.

As a practical illustration of the above point, consider an organization that assigns employees identification numbers of the following form: The first two letters specify the department and the remaining four digits are a unique number within the department for the employee. Examples of such numbers would be $CS0012$ and $EE1127$. Such identification numbers can be divided into smaller units, and are therefore nonatomic. If a relation schema had an attribute whose domain consists of identification numbers encoded as above, the schema would not be in first normal form.

When such identification numbers are used, the department of an employee can be found by writing code that breaks up the structure of an identification number. Doing so requires extra programming, and information gets encoded in the application program rather than in the database. Further problems arise if such identification numbers are used as primary keys: When an employee changes department, the employee's identification number must be changed everywhere it occurs, which can be a difficult task, or the code that interprets the number would give a wrong result.

The use of set valued attributes can lead to designs with redundant storage of data, which in turn can result in inconsistencies. For instance, instead of the relationship between accounts and customers being represented as a separate relation *depositor*, a database designer may be tempted to store a set of *owners* with each account, and a set of *accounts* with each customer. Whenever an account is created, or the set of owners of an account is updated, the update has to be performed at two places; failure to perform both updates can leave the database in an inconsistent state. Keeping only one of these sets would avoid repeated information, but would complicate some queries. Set valued attributes are also more complicated to write queries with, and more complicated to reason about.

In this chapter we consider only atomic domains, and assume that relations are in first normal form. Although we have not mentioned first normal form earlier, when we introduced the relational model in Chapter 3 we stated that attribute values must be atomic.

Some types of nonatomic values can be useful, although they should be used with care. For example, composite valued attributes are often useful, and set valued attributes are also useful in many cases, which is why both are supported in the E-R model. In many domains where entities have a complex structure, forcing a first normal form representation represents an unnecessary burden on the application programmer, who has to write code to convert data into atomic form. There is also a runtime overhead of converting data back and forth from the atomic form. Support for nonatomic values can thus be very useful in such domains. In fact, modern database systems do support many types of nonatomic values, as we will see in Chapters 8 and 9. However, in this chapter we restrict ourselves to relations in first normal form.

7.2 Pitfalls in Relational-Database Design

Before we continue our discussion of normal forms, let us look at what can go wrong in a bad database design. Among the undesirable properties that a bad design may have are:

- Repetition of information
- Inability to represent certain information

We shall discuss these problems with the help of a modified database design for our banking example: In contrast to the relation schema used in Chapters 3 to 6, suppose the information concerning loans is kept in one single relation, *lending*, which is defined over the relation schema

$$\textit{Lending-schema} = (\textit{branch-name}, \textit{branch-city}, \textit{assets}, \textit{customer-name}, \textit{loan-number}, \textit{amount})$$

Figure 7.1 shows an instance of the relation *lending* (*Lending-schema*). A tuple t in the *lending* relation has the following intuitive meaning:

- t[*assets*] is the asset figure for the branch named t[*branch-name*].
- t[*branch-city*] is the city in which the branch named t[*branch-name*] is located.
- t[*loan-number*] is the number assigned to a loan made by the branch named t[*branch-name*] to the customer named t[*customer-name*].
- t[*amount*] is the amount of the loan whose number is t[*loan-number*].

Suppose that we wish to add a new loan to our database. Say that the loan is made by the Perryridge branch to Adams in the amount of $1500. Let the *loan-number* be L-31. In our design, we need a tuple with values on all the attributes of *Lending-schema*. Thus, we must repeat the asset and city data for the Perryridge branch, and must add the tuple

(Perryridge, Horseneck, 1700000, Adams, L-31, 1500)

branch-name	*branch-city*	*assets*	*customer-name*	*loan-number*	*amount*
Downtown	Brooklyn	9000000	Jones	L-17	1000
Redwood	Palo Alto	2100000	Smith	L-23	2000
Perryridge	Horseneck	1700000	Hayes	L-15	1500
Downtown	Brooklyn	9000000	Jackson	L-14	1500
Mianus	Horseneck	400000	Jones	L-93	500
Round Hill	Horseneck	8000000	Turner	L-11	900
Pownal	Bennington	300000	Williams	L-29	1200
North Town	Rye	3700000	Hayes	L-16	1300
Downtown	Brooklyn	9000000	Johnson	L-18	2000
Perryridge	Horseneck	1700000	Glenn	L-25	2500
Brighton	Brooklyn	7100000	Brooks	L-10	2200

Figure 7.1 Sample *lending* relation.

to the *lending* relation. In general, the asset and city data for a branch must appear once for each loan made by that branch.

The repetition of information in our alternative design is undesirable. Repeating information wastes space. Furthermore, it complicates updating the database. Suppose, for example, that the assets of the Perryridge branch change from 1700000 to 1900000. Under our original design, one tuple of the *branch* relation needs to be changed. Under our alternative design, many tuples of the *lending* relation need to be changed. Thus, updates are more costly under the alternative design than under the original design. When we perform the update in the alternative database, we must ensure that *every* tuple pertaining to the Perryridge branch is updated, or else our database will show two different asset values for the Perryridge branch.

That observation is central to understanding why the alternative design is bad. We know that a bank branch has a unique value of assets, so given a branch name we can uniquely identify the assets value. On the other hand, we know that a branch may make many loans, so given a branch name, we cannot uniquely determine a loan number. In other words, we say that the *functional dependency*

$$branch\text{-}name \rightarrow assets$$

holds on *Lending-schema*, but we do not expect the functional dependency *branch-name* $\rightarrow$ *loan-number* to hold. The fact that a branch has a particular value of assets, and the fact that a branch makes a loan are independent, and, as we have seen, these facts are best represented in separate relations. We shall see that we can use functional dependencies to specify formally when a database design is good.

Another problem with the *Lending-schema* design is that we cannot represent directly the information concerning a branch (*branch-name*, *branch-city*, *assets*) unless there exists at least one loan at the branch. This is because tuples in the *lending* relation require values for *loan-number*, *amount*, and *customer-name*.

One solution to this problem is to introduce *null values*, as we did to handle updates through views. Recall, however, that null values are difficult to handle, as we saw in Section 3.3.4. If we are not willing to deal with null values, then we can create the branch information only when the first loan application at that branch is made. Worse, we would have to delete this information when all the loans have been paid. Clearly, this situation is undesirable, since, under our original database design, the branch information would be available regardless of whether or not loans are currently maintained in the branch, and without resorting to null values.

7.3 Functional Dependencies

Functional dependencies play a key role in differentiating good database designs from bad database designs. A **functional dependency** is a type of constraint that is a generalization of the notion of *key*, as discussed in Chapters 2 and 3.

7.3.1 Basic Concepts

Functional dependencies are constraints on the set of legal relations. They allow us to express facts about the enterprise that we are modeling with our database.

In Chapter 2, we defined the notion of a *superkey* as follows. Let R be a relation schema. A subset K of R is a **superkey** of R if, in any legal relation $r(R)$, for all pairs t_1 and t_2 of tuples in r such that $t_1 \neq t_2$, then $t_1[K] \neq t_2[K]$. That is, no two tuples in any legal relation $r(R)$ may have the same value on attribute set K.

The notion of functional dependency generalizes the notion of superkey. Consider a relation schema R, and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **functional dependency**

$$\alpha \rightarrow \beta$$

holds on schema R if, in any legal relation $r(R)$, for all pairs of tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, it is also the case that $t_1[\beta] = t_2[\beta]$.

Using the functional-dependency notation, we say that K is a superkey of R if $K \rightarrow R$. That is, K is a superkey if, whenever $t_1[K] = t_2[K]$, it is also the case that $t_1[R] = t_2[R]$ (that is, $t_1 = t_2$).

Functional dependencies allow us to express constraints that we cannot express with superkeys. Consider the schema

$$\textit{Loan-info-schema} = (\textit{loan-number}, \textit{branch-name}, \textit{customer-name}, \textit{amount})$$

which is simplification of the *Lending-schema* that we saw earlier. The set of functional dependencies that we expect to hold on this relation schema is

$$\textit{loan-number} \rightarrow \textit{amount}$$
$$\textit{loan-number} \rightarrow \textit{branch-name}$$

We would not, however, expect the functional dependency

$$\textit{loan-number} \rightarrow \textit{customer-name}$$

to hold, since, in general, a given loan can be made to more than one customer (for example, to both members of a husband–wife pair).

We shall use functional dependencies in two ways:

1. To test relations to see whether they are legal under a given set of functional dependencies. If a relation r is legal under a set F of functional dependencies, we say that r **satisfies** F.
2. To specify constraints on the set of legal relations. We shall thus concern ourselves with *only* those relations that satisfy a given set of functional dependencies. If we wish to constrain ourselves to relations on schema R that satisfy a set F of functional dependencies, we say that F **holds** on R.

Let us consider the relation r of Figure 7.2, to see which functional dependencies are satisfied. Observe that $A \rightarrow C$ is satisfied. There are two tuples that have an A value of a_1. These tuples have the same C value—namely, c_1. Similarly, the two tuples with an A value of a_2 have the same C value, c_2. There are no other pairs of distinct tuples that have the same A value. The functional dependency $C \rightarrow A$ is not satisfied, however. To see that it is not, consider the tuples $t_1 = (a_2, b_3, c_2, d_3)$ and

A	B	C	D
a_1	b_1	c_1	d_1
a_1	b_2	c_1	d_2
a_2	b_2	c_2	d_2
a_2	b_2	c_2	d_3
a_3	b_3	c_2	d_4

Figure 7.2 Sample relation r.

$t_2 = (a_3, b_3, c_2, d_4)$. These two tuples have the same C values, c_2, but they have different A values, a_2 and a_3, respectively. Thus, we have found a pair of tuples t_1 and t_2 such that $t_1[C] = t_2[C]$, but $t_1[A] \neq t_2[A]$.

Many other functional dependencies are satisfied by r, including, for example, the functional dependency $AB \rightarrow D$. Note that we use AB as a shorthand for $\{A,B\}$, to conform with standard practice. Observe that there is no pair of distinct tuples t_1 and t_2 such that $t_1[AB] = t_2[AB]$. Therefore, if $t_1[AB] = t_2[AB]$, it must be that $t_1 = t_2$ and, thus, $t_1[D] = t_2[D]$. So, r satisfies $AB \rightarrow D$.

Some functional dependencies are said to be **trivial** because they are satisfied by all relations. For example, $A \rightarrow A$ is satisfied by all relations involving attribute A. Reading the definition of functional dependency literally, we see that, for all tuples t_1 and t_2 such that $t_1[A] = t_2[A]$, it is the case that $t_1[A] = t_2[A]$. Similarly, $AB \rightarrow A$ is satisfied by all relations involving attribute A. In general, a functional dependency of the form $\alpha \rightarrow \beta$ is **trivial** if $\beta \subseteq \alpha$.

To distinguish between the concepts of a relation satisfying a dependency and a dependency holding on a schema, we return to the banking example. If we consider the *customer* relation (on *Customer-schema*) in Figure 7.3, we see that *customer-street* $\rightarrow$ *customer-city* is satisfied. However, we believe that, in the real world, two cities

customer-name	*customer-street*	*customer-city*
Jones	Main	Harrison
Smith	North	Rye
Hayes	Main	Harrison
Curry	North	Rye
Lindsay	Park	Pittsfield
Turner	Putnam	Stamford
Williams	Nassau	Princeton
Adams	Spring	Pittsfield
Johnson	Alma	Palo Alto
Glenn	Sand Hill	Woodside
Brooks	Senator	Brooklyn
Green	Walnut	Stamford

Figure 7.3 The *customer* relation.

loan-number	*branch-name*	*amount*
L-17	Downtown	1000
L-23	Redwood	2000
L-15	Perryridge	1500
L-14	Downtown	1500
L-93	Mianus	500
L-11	Round Hill	900
L-29	Pownal	1200
L-16	North Town	1300
L-18	Downtown	2000
L-25	Perryridge	2500
L-10	Brighton	2200

Figure 7.4 The *loan* relation.

can have streets with the same name. Thus, it is possible, at some time, to have an instance of the *customer* relation in which *customer-street* → *customer-city* is not satisfied. So, we would not include *customer-street* → *customer-city* in the set of functional dependencies that hold on *Customer-schema*.

In the *loan* relation (on *Loan-schema*) of Figure 7.4, we see that the dependency *loan-number* → *amount* is satisfied. In contrast to the case of *customer-city* and *customer-street* in *Customer-schema*, we do believe that the real-world enterprise that we are modeling requires each loan to have only one amount. Therefore, we want to require that *loan-number* → *amount* be satisfied by the *loan* relation at all times. In other words, we require that the constraint *loan-number* → *amount* hold on *Loan-schema*.

In the *branch* relation of Figure 7.5, we see that *branch-name* → *assets* is satisfied, as is *assets* → *branch-name*. We want to require that *branch-name* → *assets* hold on *Branch-schema*. However, we do not wish to require that *assets* → *branch-name* hold, since it is possible to have several branches that have the same asset value.

In what follows, we assume that, when we design a relational database, we first list those functional dependencies that must always hold. In the banking example, our list of dependencies includes the following:

branch-name	*branch-city*	*assets*
Downtown	Brooklyn	9000000
Redwood	Palo Alto	2100000
Perryridge	Horseneck	1700000
Mianus	Horseneck	400000
Round Hill	Horseneck	8000000
Pownal	Bennington	300000
North Town	Rye	3700000
Brighton	Brooklyn	7100000

Figure 7.5 The *branch* relation.

- On *Branch-schema*:

  branch-name → *branch-city*
  branch-name → *assets*

- On *Customer-schema*:

  customer-name → *customer-city*
  customer-name → *customer-street*

- On *Loan-schema*:

  loan-number → *amount*
  loan-number → *branch-name*

- On *Borrower-schema*:

  No functional dependencies

- On *Account-schema*:

  account-number → *branch-name*
  account-number → *balance*

- On *Depositor-schema*:

  No functional dependencies

7.3.2 Closure of a Set of Functional Dependencies

It is not sufficient to consider the given set of functional dependencies. Rather, we need to consider *all* functional dependencies that hold. We shall see that, given a set F of functional dependencies, we can prove that certain other functional dependencies hold. We say that such functional dependencies are "logically implied" by F.

More formally, given a relational schema R, a functional dependency f on R is **logically implied** by a set of functional dependencies F on R if every relation instance $r(R)$ that satisfies F also satisfies f.

Suppose we are given a relation schema $R = (A, B, C, G, H, I)$ and the set of functional dependencies

$$A \rightarrow B$$
$$A \rightarrow C$$
$$CG \rightarrow H$$
$$CG \rightarrow I$$
$$B \rightarrow H$$

The functional dependency

$$A \rightarrow H$$

is logically implied. That is, we can show that, whenever our given set of functional dependencies holds on a relation, $A \rightarrow H$ must also hold on the relation. Suppose that t_1 and t_2 are tuples such that

$$t_1[A] = t_2[A]$$

Since we are given that $A \rightarrow B$, it follows from the definition of functional dependency that

$$t_1[B] = t_2[B]$$

Then, since we are given that $B \rightarrow H$, it follows from the definition of functional dependency that

$$t_1[H] = t_2[H]$$

Therefore, we have shown that, whenever t_1 and t_2 are tuples such that $t_1[A] = t_2[A]$, it must be that $t_1[H] = t_2[H]$. But that is exactly the definition of $A \rightarrow H$.

Let F be a set of functional dependencies. The **closure** of F, denoted by F^+, is the set of all functional dependencies logically implied by F. Given F, we can compute F^+ directly from the formal definition of functional dependency. If F were large, this process would be lengthy and difficult. Such a computation of F^+ requires arguments of the type just used to show that $A \rightarrow H$ is in the closure of our example set of dependencies.

Axioms, or rules of inference, provide a simpler technique for reasoning about functional dependencies. In the rules that follow, we use Greek letters (α, β, γ, ...) for sets of attributes, and uppercase Roman letters from the beginning of the alphabet for individual attributes. We use $\alpha\beta$ to denote $\alpha \cup \beta$.

We can use the following three rules to find logically implied functional dependencies. By applying these rules *repeatedly*, we can find all of F^+, given F. This collection of rules is called **Armstrong's axioms** in honor of the person who first proposed it.

- **Reflexivity rule**. If α is a set of attributes and $\beta \subseteq \alpha$, then $\alpha \rightarrow \beta$ holds.
- **Augmentation rule**. If $\alpha \rightarrow \beta$ holds and γ is a set of attributes, then $\gamma\alpha \rightarrow \gamma\beta$ holds.
- **Transitivity rule**. If $\alpha \rightarrow \beta$ holds and $\beta \rightarrow \gamma$ holds, then $\alpha \rightarrow \gamma$ holds.

Armstrong's axioms are **sound**, because they do not generate any incorrect functional dependencies. They are **complete**, because, for a given set F of functional dependencies, they allow us to generate all F^+. The bibliographical notes provide references for proofs of soundness and completeness.

Although Armstrong's axioms are complete, it is tiresome to use them directly for the computation of F^+. To simplify matters further, we list additional rules. It is possible to use Armstrong's axioms to prove that these rules are correct (see Exercises 7.8, 7.9, and 7.10).

- **Union rule**. If $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds, then $\alpha \rightarrow \beta\gamma$ holds.
- **Decomposition rule**. If $\alpha \rightarrow \beta\gamma$ holds, then $\alpha \rightarrow \beta$ holds and $\alpha \rightarrow \gamma$ holds.
- **Pseudotransitivity rule**. If $\alpha \rightarrow \beta$ holds and $\gamma\beta \rightarrow \delta$ holds, then $\alpha\gamma \rightarrow \delta$ holds.

Let us apply our rules to the example of schema $R = (A, B, C, G, H, I)$ and the set F of functional dependencies $\{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$. We list several members of F^+ here:

- $A \rightarrow H$. Since $A \rightarrow B$ and $B \rightarrow H$ hold, we apply the transitivity rule. Observe that it was much easier to use Armstrong's axioms to show that $A \rightarrow H$ holds than it was to argue directly from the definitions, as we did earlier in this section.
- $CG \rightarrow HI$. Since $CG \rightarrow H$ and $CG \rightarrow I$, the union rule implies that $CG \rightarrow HI$.
- $AG \rightarrow I$. Since $A \rightarrow C$ and $CG \rightarrow I$, the pseudotransitivity rule implies that $AG \rightarrow I$ holds.

  Another way of finding that $AG \rightarrow I$ holds is as follows. We use the augmentation rule on $A \rightarrow C$ to infer $AG \rightarrow CG$. Applying the transitivity rule to this dependency and $CG \rightarrow I$, we infer $AG \rightarrow I$.

Figure 7.6 shows a procedure that demonstrates formally how to use Armstrong's axioms to compute F^+. In this procedure, when a functional dependency is added to F^+, it may be already present, and in that case there is no change to F^+. We will also see an alternative way of computing F^+ in Section 7.3.3.

The left-hand and right-hand sides of a functional dependency are both subsets of R. Since a set of size n has 2^n subsets, there are a total of $2 \times 2^n = 2^{n+1}$ possible functional dependencies, where n is the number of attributes in R. Each iteration of the repeat loop of the procedure, except the last iteration, adds at least one functional dependency to F^+. Thus, the procedure is guaranteed to terminate.

7.3.3 Closure of Attribute Sets

To test whether a set α is a superkey, we must devise an algorithm for computing the set of attributes functionally determined by α. One way of doing this is to compute F^+, take all functional dependencies with α as the left-hand side, and take the union of the right-hand sides of all such dependencies. However, doing so can be expensive, since F^+ can be large.

```
F+ = F
repeat
    for each functional dependency f in F+
        apply reflexivity and augmentation rules on f
        add the resulting functional dependencies to F+
    for each pair of functional dependencies f1 and f2 in F+
        if f1 and f2 can be combined using transitivity
            add the resulting functional dependency to F+
until F+ does not change any further
```

Figure 7.6 A procedure to compute F^+.

An efficient algorithm for computing the set of attributes functionally determined by α is useful not only for testing whether α is a superkey, but also for several other tasks, as we will see later in this section.

Let α be a set of attributes. We call the set of all attributes functionally determined by α under a set F of functional dependencies the **closure** of α under F; we denote it by α^+. Figure 7.7 shows an algorithm, written in pseudocode, to compute α^+. The input is a set F of functional dependencies and the set α of attributes. The output is stored in the variable *result*.

To illustrate how the algorithm works, we shall use it to compute $(AG)^+$ with the functional dependencies defined in Section 7.3.2. We start with $result = AG$. The first time that we execute the **while** loop to test each functional dependency, we find that

- $A \rightarrow B$ causes us to include B in *result*. To see this fact, we observe that $A \rightarrow B$ is in F, $A \subseteq result$ (which is AG), so $result := result \cup B$.
- $A \rightarrow C$ causes *result* to become *ABCG*.
- $CG \rightarrow H$ causes *result* to become *ABCGH*.
- $CG \rightarrow I$ causes *result* to become *ABCGHI*.

The second time that we execute the **while** loop, no new attributes are added to *result*, and the algorithm terminates.

Let us see why the algorithm of Figure 7.7 is correct. The first step is correct, since $\alpha \rightarrow \alpha$ always holds (by the reflexivity rule). We claim that, for any subset β of *result*, $\alpha \rightarrow \beta$. Since we start the **while** loop with $\alpha \rightarrow result$ being true, we can add γ to *result* only if $\beta \subseteq result$ and $\beta \rightarrow \gamma$. But then $result \rightarrow \beta$ by the reflexivity rule, so $\alpha \rightarrow \beta$ by transitivity. Another application of transitivity shows that $\alpha \rightarrow \gamma$ (using $\alpha \rightarrow \beta$ and $\beta \rightarrow \gamma$). The union rule implies that $\alpha \rightarrow result \cup \gamma$, so α functionally determines any new result generated in the **while** loop. Thus, any attribute returned by the algorithm is in α^+.

It is easy to see that the algorithm finds all α^+. If there is an attribute in α^+ that is not yet in *result*, then there must be a functional dependency $\beta \rightarrow \gamma$ for which $\beta \subseteq result$, and at least one attribute in γ is not in *result*.

It turns out that, in the worst case, this algorithm may take an amount of time quadratic in the size of F. There is a faster (although slightly more complex) algorithm that runs in time linear in the size of F; that algorithm is presented as part of Exercise 7.14.

```
result := α;
while (changes to result) do
        for each functional dependency β → γ in F do
            begin
                if β ⊆ result then result := result ∪ γ;
            end
```

Figure 7.7 An algorithm to compute α^+, the closure of α under F.

There are several uses of the attribute closure algorithm:

- To test if α is a superkey, we compute α^+, and check if α^+ contains all attributes of R.
- We can check if a functional dependency $\alpha \rightarrow \beta$ holds (or, in other words, is in F^+), by checking if $\beta \subseteq \alpha^+$. That is, we compute α^+ by using attribute closure, and then check if it contains β. This test is particularly useful, as we will see later in this chapter.
- It gives us an alternative way to compute F^+: For each $\gamma \subseteq R$, we find the closure γ^+, and for each $S \subseteq \gamma^+$, we output a functional dependency $\gamma \rightarrow S$.

7.3.4 Canonical Cover

Suppose that we have a set of functional dependencies F on a relation schema. Whenever a user performs an update on the relation, the database system must ensure that the update does not violate any functional dependencies, that is, all the functional dependencies in F are satisfied in the new database state.

The system must roll back the update if it violates any functional dependencies in the set F.

We can reduce the effort spent in checking for violations by testing a simplified set of functional dependencies that has the same closure as the given set. Any database that satisfies the simplified set of functional dependencies will also satisfy the original set, and vice versa, since the two sets have the same closure. However, the simplified set is easier to test. We shall see how the simplified set can be constructed in a moment. First, we need some definitions.

An attribute of a functional dependency is said to be **extraneous** if we can remove it without changing the closure of the set of functional dependencies. The formal definition of **extraneous attributes** is as follows. Consider a set F of functional dependencies and the functional dependency $\alpha \rightarrow \beta$ in F.

- Attribute A is extraneous in α if $A \in \alpha$, and F logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$.
- Attribute A is extraneous in β if $A \in \beta$, and the set of functional dependencies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$ logically implies F.

For example, suppose we have the functional dependencies $AB \rightarrow C$ and $A \rightarrow C$ in F. Then, B is extraneous in $AB \rightarrow C$. As another example, suppose we have the functional dependencies $AB \rightarrow CD$ and $A \rightarrow C$ in F. Then C would be extraneous in the right-hand side of $AB \rightarrow CD$.

Beware of the direction of the implications when using the definition of extraneous attributes: If you exchange the left-hand side with right-hand side, the implication will *always* hold. That is, $(F - \{\alpha \rightarrow \beta\}) \cup \{(\alpha - A) \rightarrow \beta\}$ always logically implies F, and also F always logically implies $(F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$

Here is how we can test efficiently if an attribute is extraneous. Let R be the relation schema, and let F be the given set of functional dependencies that hold on R. Consider an attribute A in a dependency $\alpha \rightarrow \beta$.

- If $A \in \beta$, to check if A is extraneous consider the set
  $$F' = (F - \{\alpha \rightarrow \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$$
  and check if $\alpha \rightarrow A$ can be inferred from F'. To do so, compute α^+ (the closure of α) under F'; if α^+ includes A, then A is extraneous in β.
- If $A \in \alpha$, to check if A is extraneous, let $\gamma = \alpha - \{A\}$, and check if $\gamma \rightarrow \beta$ can be inferred from F. To do so, compute γ^+ (the closure of γ) under F; if γ^+ includes all attributes in β, then A is extraneous in α.

For example, suppose F contains $AB \rightarrow CD$, $A \rightarrow E$, and $E \rightarrow C$. To check if C is extraneous in $AB \rightarrow CD$, we compute the attribute closure of AB under $F' = \{AB \rightarrow D, A \rightarrow E, \text{and } E \rightarrow C\}$. The closure is $ABCDE$, which includes CD, so we infer that C is extraneous.

A **canonical cover** F_c for F is a set of dependencies such that F logically implies all dependencies in F_c, and F_c logically implies all dependencies in F. Furthermore, F_c must have the following properties:

- No functional dependency in F_c contains an extraneous attribute.
- Each left side of a functional dependency in F_c is unique. That is, there are no two dependencies $\alpha_1 \rightarrow \beta_1$ and $\alpha_2 \rightarrow \beta_2$ in F_c such that $\alpha_1 = \alpha_2$.

A canonical cover for a set of functional dependencies F can be computed as depicted in Figure 7.8. It is important to note that when checking if an attribute is extraneous, the check uses the dependencies in the current value of F_c, and **not** the dependencies in F. If a functional dependency contains only one attribute in its right-hand side, for example $A \rightarrow C$, and that attribute is found to be extraneous, we would get a functional dependency with an empty right-hand side. Such functional dependencies should be deleted.

The canonical cover of F, F_c, can be shown to have the same closure as F; hence, testing whether F_c is satisfied is equivalent to testing whether F is satisfied. However, F_c is minimal in a certain sense—it does not contain extraneous attributes, and it

$F_c = F$
repeat
  Use the union rule to replace any dependencies in F_c of the form
    $\alpha_1 \rightarrow \beta_1$ and $\alpha_1 \rightarrow \beta_2$ with $\alpha_1 \rightarrow \beta_1 \beta_2$.
  Find a functional dependency $\alpha \rightarrow \beta$ in F_c with an extraneous
    attribute either in α or in β.
    /* Note: the test for extraneous attributes is done using F_c, not F */
  If an extraneous attribute is found, delete it from $\alpha \rightarrow \beta$.
until F_c does not change.

Figure 7.8 Computing canonical cover

combines functional dependencies with the same left side. It is cheaper to test F_c than it is to test F itself.

Consider the following set F of functional dependencies on schema (A, B, C):

$$A \rightarrow BC$$
$$B \rightarrow C$$
$$A \rightarrow B$$
$$AB \rightarrow C$$

Let us compute the canonical cover for F.

- There are two functional dependencies with the same set of attributes on the left side of the arrow:

  $$A \rightarrow BC$$
  $$A \rightarrow B$$

  We combine these functional dependencies into $A \rightarrow BC$.

- A is extraneous in $AB \rightarrow C$ because F logically implies $(F - \{AB \rightarrow C\}) \cup \{B \rightarrow C\}$. This assertion is true because $B \rightarrow C$ is already in our set of functional dependencies.

- C is extraneous in $A \rightarrow BC$, since $A \rightarrow BC$ is logically implied by $A \rightarrow B$ and $B \rightarrow C$.

Thus, our canonical cover is

$$A \rightarrow B$$
$$B \rightarrow C$$

Given a set F of functional dependencies, it may be that an entire functional dependency in the set is extraneous, in the sense that dropping it does not change the closure of F. We can show that a canonical cover F_c of F contains no such extraneous functional dependency. Suppose that, to the contrary, there were such an extraneous functional dependency in F_c. The right-side attributes of the dependency would then be extraneous, which is not possible by the definition of canonical covers.

A canonical cover might not be unique. For instance, consider the set of functional dependencies $F = \{A \rightarrow BC, B \rightarrow AC$, and $C \rightarrow AB\}$. If we apply the extraneity test to $A \rightarrow BC$, we find that both B and C are extraneous under F. However, it is incorrect to delete both! The algorithm for finding the canonical cover picks one of the two, and deletes it. Then,

1. If C is deleted, we get the set $F' = \{A \rightarrow B, B \rightarrow AC$, and $C \rightarrow AB\}$. Now, B is not extraneous in the righthand side of $A \rightarrow B$ under F'. Continuing the algorithm, we find A and B are extraneous in the right-hand side of $C \rightarrow AB$, leading to two canonical covers

   $$F_c = \{A \rightarrow B, B \rightarrow C, \text{and } C \rightarrow A\}, \text{ and}$$
   $$F_c = \{A \rightarrow B, B \rightarrow AC, \text{and } C \rightarrow B\}.$$

2. If B is deleted, we get the set $\{A \rightarrow C, B \rightarrow AC$, and $C \rightarrow AB\}$. This case is symmetrical to the previous case, leading to the canonical covers

$$F_c = \{A \rightarrow C, C \rightarrow B, \text{ and } B \rightarrow A\}, \text{ and}$$
$$F_c = \{A \rightarrow C, B \rightarrow C, \text{ and } C \rightarrow AB\}.$$

As an exercise, can you find one more canonical cover for F?

7.4 Decomposition

The bad design of Section 7.2 suggests that we should *decompose* a relation schema that has many attributes into several schemas with fewer attributes. Careless decomposition, however, may lead to another form of bad design.

Consider an alternative design in which we decompose *Lending-schema* into the following two schemas:

Branch-customer-schema = (*branch-name, branch-city, assets, customer-name*)
Customer-loan-schema = (*customer-name, loan-number, amount*)

Using the *lending* relation of Figure 7.1, we construct our new relations *branch-customer* (*Branch-customer*) and *customer-loan* (*Customer-loan-schema*):

$$branch\text{-}customer = \Pi_{branch\text{-}name,\ branch\text{-}city,\ assets,\ customer\text{-}name}\ (lending)$$
$$customer\text{-}loan = \Pi_{customer\text{-}name,\ loan\text{-}number,\ amount}\ (lending)$$

Figures 7.9 and 7.10, respectively, show the resulting *branch-customer* and *customer-name* relations.

Of course, there are cases in which we need to reconstruct the *loan* relation. For example, suppose that we wish to find all branches that have loans with amounts less than $1000. No relation in our alternative database contains these data. We need to reconstruct the *lending* relation. It appears that we can do so by writing

$$branch\text{-}customer \bowtie customer\text{-}loan$$

branch-name	*branch-city*	*assets*	*customer-name*
Downtown	Brooklyn	9000000	Jones
Redwood	Palo Alto	2100000	Smith
Perryridge	Horseneck	1700000	Hayes
Downtown	Brooklyn	9000000	Jackson
Mianus	Horseneck	400000	Jones
Round Hill	Horseneck	8000000	Turner
Pownal	Bennington	300000	Williams
North Town	Rye	3700000	Hayes
Downtown	Brooklyn	9000000	Johnson
Perryridge	Horseneck	1700000	Glenn
Brighton	Brooklyn	7100000	Brooks

Figure 7.9 The relation *branch-customer*.

customer-name	*loan-number*	*amount*
Jones	L-17	1000
Smith	L-23	2000
Hayes	L-15	1500
Jackson	L-14	1500
Jones	L-93	500
Turner	L-11	900
Williams	L-29	1200
Hayes	L-16	1300
Johnson	L-18	2000
Glenn	L-25	2500
Brooks	L-10	2200

Figure 7.10 The relation *customer-loan*.

Figure 7.11 shows the result of computing $branch\text{-}customer \bowtie customer\text{-}loan$. When we compare this relation and the *lending* relation with which we started (Figure 7.1), we notice a difference: Although every tuple that appears in the *lending* relation appears in $branch\text{-}customer \bowtie customer\text{-}loan$, there are tuples in $branch\text{-}customer \bowtie customer\text{-}loan$ that are not in *lending*. In our example, *branch-customer* ⋈ *customer-loan* has the following additional tuples:

(Downtown, Brooklyn, 9000000, Jones, L-93, 500)
(Perryridge, Horseneck, 1700000, Hayes, L-16, 1300)
(Mianus, Horseneck, 400000, Jones, L-17, 1000)
(North Town, Rye, 3700000, Hayes, L-15, 1500)

Consider the query, "Find all bank branches that have made a loan in an amount less than $1000." If we look back at Figure 7.1, we see that the only branches with loan amounts less than $1000 are Mianus and Round Hill. However, when we apply the expression

$$\Pi_{branch\text{-}name} (\sigma_{amount < 1000} (branch\text{-}customer \bowtie customer\text{-}loan))$$

we obtain *three* branch names: Mianus, Round Hill, and Downtown.

A closer examination of this example shows why. If a customer happens to have several loans from different branches, we cannot tell which loan belongs to which branch. Thus, when we join *branch-customer* and *customer-loan*, we obtain not only the tuples we had originally in *lending*, but also several additional tuples. Although we have *more* tuples in $branch\text{-}customer \bowtie customer\text{-}loan$, we actually have *less* information. We are no longer able, in general, to represent in the database information about which customers are borrowers from which branch. Because of this loss of information, we call the decomposition of *Lending-schema* into *Branch-customer-schema* and *customer-loan-schema* a **lossy decomposition**, or a **lossy-join decomposition**. A decomposition that is not a lossy-join decomposition is a **lossless-join decomposi-**

branch-name	*branch-city*	*assets*	*customer-name*	*loan-number*	*amount*
Downtown	Brooklyn	9000000	Jones	L-17	1000
Downtown	Brooklyn	9000000	Jones	L-93	500
Redwood	Palo Alto	2100000	Smith	L-23	2000
Perryridge	Horseneck	1700000	Hayes	L-15	1500
Perryridge	Horseneck	1700000	Hayes	L-16	1300
Downtown	Brooklyn	9000000	Jackson	L-14	1500
Mianus	Horseneck	400000	Jones	L-17	1000
Mianus	Horseneck	400000	Jones	L-93	500
Round Hill	Horseneck	8000000	Turner	L-11	900
Pownal	Bennington	300000	Williams	L-29	1200
North Town	Rye	3700000	Hayes	L-15	1500
North Town	Rye	3700000	Hayes	L-16	1300
Downtown	Brooklyn	9000000	Johnson	L-18	2000
Perryridge	Horseneck	1700000	Glenn	L-25	2500
Brighton	Brooklyn	7100000	Brooks	L-10	2200

Figure 7.11 The relation $branch\text{-}customer \bowtie customer\text{-}loan$.

tion. It should be clear from our example that a lossy-join decomposition is, in general, a bad database design.

Why is the decomposition lossy? There is one attribute in common between *Branch-customer-schema* and *Customer-loan-schema*:

$$Branch\text{-}customer\text{-}schema \cap Customer\text{-}loan\text{-}schema = \{customer\text{-}name\}$$

The only way that we can represent a relationship between, for example, *loan-number* and *branch-name* is through *customer-name*. This representation is not adequate because a customer may have several loans, yet these loans are not necessarily obtained from the same branch.

Let us consider another alternative design, in which we decompose *Lending-schema* into the following two schemas:

$$Branch\text{-}schema = (branch\text{-}name, branch\text{-}city, assets)$$
$$Loan\text{-}info\text{-}schema = (branch\text{-}name, customer\text{-}name, loan\text{-}number, amount)$$

There is one attribute in common between these two schemas:

$$Branch\text{-}loan\text{-}schema \cap Customer\text{-}loan\text{-}schema = \{branch\text{-}name\}$$

Thus, the only way that we can represent a relationship between, for example, *customer-name* and *assets* is through *branch-name*. The difference between this example and the preceding one is that the assets of a branch are the same, regardless of the customer to which we are referring, whereas the lending branch associated with a certain loan amount *does* depend on the customer to which we are referring. For a given *branch-name*, there is exactly one *assets* value and exactly one *branch-city*;

whereas a similar statement cannot be made for *customer-name*. That is, the functional dependency

$$\textit{branch-name} \rightarrow \textit{assets branch-city}$$

holds, but *customer-name* does not functionally determine *loan-number*.

The notion of lossless joins is central to much of relational-database design. Therefore, we restate the preceding examples more concisely and more formally. Let R be a relation schema. A set of relation schemas $\{R_1, R_2, \ldots, R_n\}$ is a **decomposition** of R if

$$R = R_1 \cup R_2 \cup \cdots \cup R_n$$

That is, $\{R_1, R_2, \ldots, R_n\}$ is a decomposition of R if, for $i = 1, 2, \ldots, n$, each R_i is a subset of R, and every attribute in R appears in at least one R_i.

Let r be a relation on schema R, and let $r_i = \Pi_{R_i}(r)$ for $i = 1, 2, \ldots, n$. That is, $\{r_1, r_2, \ldots, r_n\}$ is the database that results from decomposing R into $\{R_1, R_2, \ldots, R_n\}$. It is always the case that

$$r \subseteq r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$$

To see that this assertion is true, consider a tuple t in relation r. When we compute the relations $r_1, r_2, \ldots, r_n$, the tuple t gives rise to one tuple t_i in each r_i, $i = 1, 2, \ldots, n$. These n tuples combine to regenerate t when we compute $r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$. The details are left for you to complete as an exercise. Therefore, every tuple in r appears in $r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$.

In general, $r \neq r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$. As an illustration, consider our earlier example, in which

- $n = 2$.
- $R =$ *Lending-schema*.
- $R_1 =$ *Branch-customer-schema*.
- $R_2 =$ *Customer-loan-schema*.
- $r =$ the relation shown in Figure 7.1.
- $r_1 =$ the relation shown in Figure 7.9.
- $r_2 =$ the relation shown in Figure 7.10.
- $r_1 \bowtie r_2 =$ the relation shown in Figure 7.11.

Note that the relations in Figures 7.1 and 7.11 are not the same.

To have a lossless-join decomposition, we need to impose constraints on the set of possible relations. We found that the decomposition of *Lending-schema* into *Branch-schema* and *Loan-info-schema* is lossless because the functional dependency

$$\textit{branch-name} \rightarrow \textit{branch-city assets}$$

holds on *Branch-schema*.

Later in this chapter, we shall introduce constraints other than functional dependencies. We say that a relation is **legal** if it satisfies all rules, or constraints, that we impose on our database.

Let *C* represent a set of constraints on the database, and let R be a relation schema. A decomposition $\{R_1, R_2, \ldots, R_n\}$ of R is a **lossless-join decomposition** if, for all relations *r* on schema *R* that are legal under *C*,

$$r = \Pi_{R_1}(r) \bowtie \Pi_{R_2}(r) \bowtie \cdots \bowtie \Pi_{R_n}(r)$$

We shall show how to test whether a decomposition is a lossless-join decomposition in the next few sections. A major part of this chapter deals with the questions of how to specify constraints on the database, and how to obtain lossless-join decompositions that avoid the pitfalls represented by the examples of bad database designs that we have seen in this section.

7.5 Desirable Properties of Decomposition

We can use a given set of functional dependencies in designing a relational database in which most of the undesirable properties discussed in Section 7.2 do not occur. When we design such systems, it may become necessary to decompose a relation into several smaller relations.

In this section, we outline the desirable properties of a decomposition of a relational schema. In later sections, we outline specific ways of decomposing a relational schema to get the properties we desire. We illustrate our concepts with the *Lending-schema* schema of Section 7.2:

Lending-schema = (*branch-name, branch-city, assets, customer-name, loan-number, amount*)

The set *F* of functional dependencies that we require to hold on *Lending-schema* are

branch-name → *branch-city assets*
loan-number → *amount branch-name*

As we discussed in Section 7.2, *Lending-schema* is an example of a bad database design. Assume that we decompose it to the following three relations:

Branch-schema = (*branch-name, branch-city, assets*)
Loan-schema = (*loan-number, branch-name, amount*)
Borrower-schema = (*customer-name, loan-number*)

We claim that this decomposition has several desirable properties, which we discuss next. Note that these three relation schemas are precisely the ones that we used previously, in Chapters 3 through 5.

7.5.1 Lossless-Join Decomposition

In Section 7.2, we argued that, when we decompose a relation into a number of smaller relations, it is crucial that the decomposition be lossless. We claim that the

decomposition in Section 7.5 is indeed lossless. To demonstrate our claim, we must first present a criterion for determining whether a decomposition is lossy.

Let R be a relation schema, and let F be a set of functional dependencies on R. Let R_1 and R_2 form a decomposition of R. This decomposition is a lossless-join decomposition of R if at least one of the following functional dependencies is in F^+:

- $R_1 \cap R_2 \rightarrow R_1$
- $R_1 \cap R_2 \rightarrow R_2$

In other words, if $R_1 \cap R_2$ forms a superkey of either R_1 or R_2, the decomposition of R is a lossless-join decomposition. We can use attribute closure to efficiently test for superkeys, as we have seen earlier.

We now demonstrate that our decomposition of *Lending-schema* is a lossless-join decomposition by showing a sequence of steps that generate the decomposition. We begin by decomposing *Lending-schema* into two schemas:

Branch-schema = *(branch-name, branch-city, assets)*
Loan-info-schema = *(branch-name, customer-name, loan-number, amount)*

Since *branch-name* $\rightarrow$ *branch-city assets*, the augmentation rule for functional dependencies (Section 7.3.2) implies that

branch-name $\rightarrow$ *branch-name branch-city assets*

Since *Branch-schema* $\cap$ *Loan-info-schema* $= \{$*branch-name*$\}$, it follows that our initial decomposition is a lossless-join decomposition.

Next, we decompose *Loan-info-schema* into

Loan-schema = *(loan-number, branch-name, amount)*
Borrower-schema = *(customer-name, loan-number)*

This step results in a lossless-join decomposition, since *loan-number* is a common attribute and *loan-number* $\rightarrow$ *amount branch-name*.

For the general case of decomposition of a relation into multiple parts at once, the test for lossless join decomposition is more complicated. See the bibliographical notes for references on the topic.

While the test for binary decomposition is clearly a sufficient condition for lossless join, it is a necessary condition only if all constraints are functional dependencies. We shall see other types of constraints later (in particular, a type of constraint called multivalued dependencies), that can ensure that a decomposition is lossless join even if no functional dependencies are present.

7.5.2 Dependency Preservation

There is another goal in relational-database design: *dependency preservation*. When an update is made to the database, the system should be able to check that the update will not create an illegal relation—that is, one that does not satisfy all the given

functional dependencies. If we are to check updates efficiently, we should design relational-database schemas that allow update validation without the computation of joins.

To decide whether joins must be computed to check an update, we need to determine what functional dependencies can be tested by checking each relation individually. Let F be a set of functional dependencies on a schema R, and let $R_1, R_2, \ldots, R_n$ be a decomposition of R. The **restriction** of F to R_i is the set F_i of all functional dependencies in F^+ that include *only* attributes of R_i. Since all functional dependencies in a restriction involve attributes of only one relation schema, it is possible to test such a dependency for satisfaction by checking only one relation.

Note that the definition of restriction uses all dependencies in F^+, not just those in F. For instance, suppose $F = \{A \rightarrow B, B \rightarrow C\}$, and we have a decomposition into AC and AB. The restriction of F to AC is then $A \rightarrow C$, since $A \rightarrow C$ is in F^+, even though it is not in F.

The set of restrictions $F_1, F_2, \ldots, F_n$ is the set of dependencies that can be checked efficiently. We now must ask whether testing only the restrictions is sufficient. Let $F' = F_1 \cup F_2 \cup \cdots \cup F_n$. F' is a set of functional dependencies on schema R, but, in general, $F' \neq F$. However, even if $F' \neq F$, it may be that $F'^+ = F^+$. If the latter is true, then every dependency in F is logically implied by F', and, if we verify that F' is satisfied, we have verified that F is satisfied. We say that a decomposition having the property $F'^+ = F^+$ is a **dependency-preserving decomposition**.

Figure 7.12 shows an algorithm for testing dependency preservation. The input is a set $D = \{R_1, R_2, \ldots, R_n\}$ of decomposed relation schemas, and a set F of functional dependencies. This algorithm is expensive since it requires computation of F^+; we will describe another algorithm that is more efficient after giving an example of testing for dependency preservation.

We can now show that our decomposition of *Lending-schema* is dependency preserving. Instead of applying the algorithm of Figure 7.12, we consider an easier alternative: We consider each member of the set F of functional dependencies that we

```
compute F+;
for each schema Ri in D do
    begin
        Fi := the restriction of F+ to Ri;
    end
F' := ∅
for each restriction Fi do
    begin
        F' = F' ∪ Fi
    end
compute F'+;
if (F'+ = F+) then return (true)
                  else return (false);
```

Figure 7.12 Testing for dependency preservation.

require to hold on *Lending-schema*, and show that each one can be tested in at least one relation in the decomposition.

- We can test the functional dependency: *branch-name* → *branch-city assets* using *Branch-schema* = (*branch-name*, *branch-city*, *assets*).
- We can test the functional dependency: *loan-number* → *amount branch-name* using *Loan-schema* = (*branch-name*, *loan-number*, *amount*).

If each member of F can be tested on one of the relations of the decomposition, then the decomposition is dependency preserving. However, there are cases where, even though the decomposition is dependency preserving, there is a dependency in F that cannot be tested in any one relation in the decomposition. The alternative test can therefore be used as a sufficient condition that is easy to check; if it fails we cannot conclude that the decomposition is not dependency preserving, instead we will have to apply the general test.

We now give a more efficient test for dependency preservation, which avoids computing F^+. The idea is to test each functional dependency $\alpha \rightarrow \beta$ in F by using a modified form of attribute closure to see if it is preserved by the decomposition. We apply the following procedure to each $\alpha \rightarrow \beta$ in F.

result = α
while (changes to *result*) **do**
  for each R_i in the decomposition
    t = (*result* $\cap R_i$)$^+$ $\cap$ R_i
    result = *result* $\cup$ t

The attribute closure is with respect to the functional dependencies in F. If *result* contains all attributes in β, then the functional dependency $\alpha \rightarrow \beta$ is preserved. The decomposition is dependency preserving if and only if all the dependencies in F are preserved.

Note that instead of precomputing the restriction of F on R_i and using it for computing the attribute closure of *result*, we use attribute closure on (*result* $\cap R_i$) with respect to F, and then intersect it with R_i, to get an equivalent result. This procedure takes polynomial time, instead of the exponential time required to compute F^+.

7.5.3 Repetition of Information

The decomposition of *Lending-schema* does not suffer from the problem of repetition of information that we discussed in Section 7.2. In *Lending-schema*, it was necessary to repeat the city and assets of a branch for each loan. The decomposition separates branch and loan data into distinct relations, thereby eliminating this redundancy. Similarly, observe that, if a single loan is made to several customers, we must repeat the amount of the loan once for each customer (as well as the city and assets of the branch) in *lending-schema*. In the decomposition, the relation on schema *Borrower-schema* contains the *loan-number*, *customer-name* relationship, and no other schema does. Therefore, we have one tuple for each customer for a loan in only the relation

on *Borrower-schema*. In the other relations involving *loan-number* (those on schemas *Loan-schema* and *Borrower-schema*), only one tuple per loan needs to appear.

Clearly, the lack of redundancy in our decomposition is desirable. The degree to which we can achieve this lack of redundancy is represented by several *normal forms*, which we shall discuss in the remainder of this chapter.

7.6 Boyce—Codd Normal Form

Using functional dependencies, we can define several *normal forms* that represent "good" database designs. In this section we cover BCNF (defined below), and later, in Section 7.7, we cover 3NF.

7.6.1 Definition

One of the more desirable normal forms that we can obtain is **Boyce–Codd normal form** (**BCNF**). A relation schema R is in BCNF with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency (that is, $\beta \subseteq \alpha$).
- α is a superkey for schema R.

A database design is in BCNF if each member of the set of relation schemas that constitutes the design is in BCNF.

As an illustration, consider the following relation schemas and their respective functional dependencies:

- *Customer-schema* = (*customer-name*, *customer-street*, *customer-city*)
  customer-name → *customer-street customer-city*
- *Branch-schema* = (*branch-name*, *assets*, *branch-city*)
  branch-name → *assets branch-city*
- *Loan-info-schema* = (*branch-name*, *customer-name*, *loan-number*, *amount*)
  loan-number → *amount branch-name*

We claim that *Customer-schema* is in BCNF. We note that a candidate key for the schema is *customer-name*. The only nontrivial functional dependencies that hold on *Customer-schema* have *customer-name* on the left side of the arrow. Since *customer-name* is a candidate key, functional dependencies with *customer-name* on the left side do not violate the definition of BCNF. Similarly, it can be shown easily that the relation schema *Branch-schema* is in BCNF.

The schema *Loan-info-schema*, however, is *not* in BCNF. First, note that *loan-number* is not a superkey for *Loan-info-schema*, since we *could* have a pair of tuples representing a single loan made to two people—for example,

(Downtown, John Bell, L-44, 1000)
(Downtown, Jane Bell, L-44, 1000)

Because we did not list functional dependencies that rule out the preceding case, *loan-number* is not a candidate key. However, the functional dependency *loan-number* $\rightarrow$ *amount* is nontrivial. Therefore, *Loan-info-schema* does not satisfy the definition of BCNF.

We claim that *Loan-info-schema* is not in a desirable form, since it suffers from the problem of *repetition of information* that we described in Section 7.2. We observe that, if there are several customer names associated with a loan, in a relation on *Loan-info-schema*, then we are forced to repeat the branch name and the amount once for each customer. We can eliminate this redundancy by redesigning our database such that all schemas are in BCNF. One approach to this problem is to take the existing non-BCNF design as a starting point, and to decompose those schemas that are not in BCNF. Consider the decomposition of *Loan-info-schema* into two schemas:

Loan-schema = (*loan-number, branch-name, amount*)
Borrower-schema = (*customer-name, loan-number*)

This decomposition is a lossless-join decomposition.

To determine whether these schemas are in BCNF, we need to determine what functional dependencies apply to them. In this example, it is easy to see that

loan-number $\rightarrow$ *amount branch-name*

applies to the *Loan-schema*, and that only trivial functional dependencies apply to *Borrower-schema*. Although *loan-number* is not a superkey for *Loan-info-schema*, it is a candidate key for *Loan-schema*. Thus, both schemas of our decomposition are in BCNF.

It is now possible to avoid redundancy in the case where there are several customers associated with a loan. There is exactly one tuple for each loan in the relation on *Loan-schema*, and one tuple for each customer of each loan in the relation on *Borrower-schema*. Thus, we do not have to repeat the branch name and the amount once for each customer associated with a loan.

Often testing of a relation to see if it satisfies BCNF can be simplified:

- To check if a nontrivial dependency $\alpha \rightarrow \beta$ causes a violation of BCNF, compute α^+ (the attribute closure of α), and verify that it includes all attributes of R; that is, it is a superkey of R.
- To check if a relation schema R is in BCNF, it suffices to check only the dependencies in the given set F for violation of BCNF, rather than check all dependencies in F^+.

  We can show that if none of the dependencies in F causes a violation of BCNF, then none of the dependencies in F^+ will cause a violation of BCNF either.

Unfortunately, the latter procedure does not work when a relation is decomposed. That is, it *does not* suffice to use F when we test a relation R_i, in a decomposition of R, for violation of BCNF. For example, consider relation schema $R\ (A, B, C, D, E)$, with functional dependencies F containing $A \rightarrow B$ and $BC \rightarrow D$. Suppose this were

decomposed into $R1(A, B)$ and $R2(A, C, D, E)$. Now, neither of the dependencies in F contains only attributes from (A, C, D, E) so we might be misled into thinking $R2$ satisfies BCNF. In fact, there is a dependency $AC \rightarrow D$ in F^+ (which can be inferred using the pseudotransitivity rule from the two dependencies in F), which shows that $R2$ is not in BCNF. Thus, we may need a dependency that is in F^+, but is not in F, to show that a decomposed relation is not in BCNF.

An alternative BCNF test is sometimes easier than computing every dependency in F^+. To check if a relation R_i in a decomposition of R is in BCNF, we apply this test:

- For every subset α of attributes in R_i, check that α^+ (the attribute closure of α under F) either includes no attribute of $R_i - \alpha$, or includes all attributes of R_i.

If the condition is violated by some set of attributes α in R_i, consider the following functional dependency, which can be shown to be present in F^+:

$$\alpha \rightarrow (\alpha^+ - \alpha) \cap R_i.$$

The above dependency shows that R_i violates BCNF, and is a "witness" for the violation. The BCNF decomposition algorithm, which we shall see in Section 7.6.2, makes use of the witness.

7.6.2 Decomposition Algorithm

We are now able to state a general method to decompose a relation schema so as to satisfy BCNF. Figure 7.13 shows an algorithm for this task. If R is not in BCNF, we can decompose R into a collection of BCNF schemas $R_1, R_2, \ldots, R_n$ by the algorithm. The algorithm uses dependencies ("witnesses") that demonstrate violation of BCNF to perform the decomposition.

The decomposition that the algorithm generates is not only in BCNF, but is also a lossless-join decomposition. To see why our algorithm generates only lossless-join decompositions, we note that, when we replace a schema R_i with $(R_i - \beta)$ and (α, β), the dependency $\alpha \rightarrow \beta$ holds, and $(R_i - \beta) \cap (\alpha, \beta) = \alpha$.

```
result := {R};
done := false;
compute F+;
while (not done) do
    if (there is a schema Ri in result that is not in BCNF)
      then begin
              let α → β be a nontrivial functional dependency that holds
              on Ri such that α → Ri is not in F+, and α ∩ β = ∅;
              result := (result − Ri) ∪ (Ri − β) ∪ ( α, β);
           end
      else done := true;
```

Figure 7.13 BCNF decomposition algorithm.

We apply the BCNF decomposition algorithm to the *Lending-schema* schema that we used in Section 7.2 as an example of a poor database design:

$$\textit{Lending-schema} = (\textit{branch-name}, \textit{branch-city}, \textit{assets}, \textit{customer-name}, \textit{loan-number}, \textit{amount})$$

The set of functional dependencies that we require to hold on *Lending-schema* are

$$\textit{branch-name} \rightarrow \textit{assets branch-city}$$
$$\textit{loan-number} \rightarrow \textit{amount branch-name}$$

A candidate key for this schema is $\{\textit{loan-number}, \textit{customer-name}\}$.

We can apply the algorithm of Figure 7.13 to the *Lending-schema* example as follows:

- The functional dependency

  $$\textit{branch-name} \rightarrow \textit{assets branch-city}$$

  holds on *Lending-schema*, but *branch-name* is not a superkey. Thus, *Lending-schema* is not in BCNF. We replace *Lending-schema* by

  $$\textit{Branch-schema} = (\textit{branch-name}, \textit{branch-city}, \textit{assets})$$
  $$\textit{Loan-info-schema} = (\textit{branch-name}, \textit{customer-name}, \textit{loan-number}, \textit{amount})$$

- The only nontrivial functional dependencies that hold on *Branch-schema* include *branch-name* on the left side of the arrow. Since *branch-name* is a key for *Branch-schema*, the relation *Branch-schema* is in BCNF.

- The functional dependency

  $$\textit{loan-number} \rightarrow \textit{amount branch-name}$$

  holds on *Loan-info-schema*, but *loan-number* is not a key for *Loan-info-schema*. We replace *Loan-info-schema* by

  $$\textit{Loan-schema} = (\textit{loan-number}, \textit{branch-name}, \textit{amount})$$
  $$\textit{Borrower-schema} = (\textit{customer-name}, \textit{loan-number})$$

- *Loan-schema* and *Borrower-schema* are in BCNF.

Thus, the decomposition of *Lending-schema* results in the three relation schemas *Branch-schema*, *Loan-schema*, and *Borrower-schema*, each of which is in BCNF. These relation schemas are the same as those in Section 7.5, where we demonstrated that the resulting decomposition is both a lossless-join decomposition and a dependency-preserving decomposition.

The BCNF decomposition algorithm takes time exponential in the size of the initial schema, since the algorithm for checking if a relation in the decomposition satisfies BCNF can take exponential time. The bibliographical notes provide references to an

algorithm that can compute a BCNF decomposition in polynomial time. However, the algorithm may "overnormalize," that is, decompose a relation unnecessarily.

7.6.3 Dependency Preservation

Not every BCNF decomposition is dependency preserving. As an illustration, consider the relation schema

$$\textit{Banker-schema} = (\textit{branch-name, customer-name, banker-name})$$

which indicates that a customer has a "personal banker" in a particular branch. The set F of functional dependencies that we require to hold on the *Banker-schema* is

$$\textit{banker-name} \rightarrow \textit{branch-name}$$
$$\textit{branch-name customer-name} \rightarrow \textit{banker-name}$$

Clearly, *Banker-schema* is not in BCNF since *banker-name* is not a superkey.

If we apply the algorithm of Figure 7.13, we obtain the following BCNF decomposition:

$$\textit{Banker-branch-schema} = (\textit{banker-name}, \textit{branch-name})$$
$$\textit{Customer-banker-schema} = (\textit{customer-name}, \textit{banker-name})$$

The decomposed schemas preserve only $\textit{banker-name} \rightarrow \textit{branch-name}$ (and trivial dependencies), but the closure of $\{\textit{banker-name} \rightarrow \textit{branch-name}\}$ does not include $\textit{customer-name branch-name} \rightarrow \textit{banker-name}$. The violation of this dependency cannot be detected unless a join is computed.

To see why the decomposition of *Banker-schema* into the schemas *Banker-branch-schema* and *Customer-banker-schema* is not dependency preserving, we apply the algorithm of Figure 7.12. We find that the restrictions F_1 and F_2 of F to each schema are:

$F_1 = \{\textit{banker-name} \rightarrow \textit{branch-name}\}$
$F_2 = \emptyset$ (only trivial dependencies hold on *Customer-banker-schema*)

(For brevity, we do not show trivial functional dependencies.) It is easy to see that the dependency *customer-name branch-name* $\rightarrow$ *banker-name* is not in $(F_1 \cup F_2)^+$ even though it *is* in F^+. Therefore, $(F_1 \cup F_2)^+ \neq F^+$, and the decomposition is not dependency preserving.

This example demonstrates that not every BCNF decomposition is dependency preserving. Moreover, it is easy to see that *any* BCNF decomposition of *Banker-schema* must fail to preserve *customer-name branch-name* $\rightarrow$ *banker-name*. Thus, the example shows that we cannot always satisfy all three design goals:

1. Lossless join
2. BCNF
3. Dependency preservation

Recall that lossless join is an essential condition for a decomposition, to avoid loss of information. We are therefore forced to give up either BCNF or dependency preservation. In Section 7.7 we present an alternative normal form, called **third normal form**, which is a small relaxation of BCNF; the motivation for using third normal form is that there is always a dependency preserving decomposition into third normal form.

There are situations where there is more than one way to decompose a schema into BCNF. Some of these decompositions may be dependency preserving, while others may not. For instance, suppose we have a relation schema $R(A, B, C)$ with the functional dependencies $A \rightarrow B$ and $B \rightarrow C$. From this set we can derive the further dependency $A \rightarrow C$. If we used the dependency $A \rightarrow B$ (or equivalently, $A \rightarrow C$) to decompose R, we would end up with two relations $R1(A, B)$ and $R2(A, C)$; the dependency $B \rightarrow C$ would not be preserved.

If instead we used the dependency $B \rightarrow C$ to decompose R, we would end up with two relations $R1(A, B)$ and $R2(B, C)$, which are in BCNF, and the decomposition is also dependency preserving. Clearly the decomposition into $R1(A, B)$ and $R2(B, C)$ is preferable. In general, the database designer should therefore look at alternative decompositions, and pick a dependency preserving decomposition where possible.

7.7 Third Normal Form

As we saw earlier, there are relational schemas where a BCNF decomposition cannot be dependency preserving. For such schemas, we have two alternatives if we wish to check if an update violates any functional dependencies:

- Pay the extra cost of computing joins to test for violations.
- Use an alternative decomposition, third normal form (3NF), which we present below, which makes testing of updates cheaper. Unlike BCNF, 3NF decompositions may contain some redundancy in the decomposed schema.

We shall see that it is always possible to find a lossless-join, dependency-preserving decomposition that is in 3NF. Which of the two alternatives to choose is a design decision to be made by the database designer on the basis of the application requirements.

7.7.1 Definition

BCNF requires that all nontrivial dependencies be of the form $\alpha \rightarrow \beta$, where α is a superkey. 3NF relaxes this constraint slightly by allowing nontrivial functional dependencies whose left side is not a superkey.

A relation schema R is in **third normal form (3NF)** with respect to a set F of functional dependencies if, for all functional dependencies in F^+ of the form $\alpha \rightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds:

- $\alpha \rightarrow \beta$ is a trivial functional dependency.
- α is a superkey for R.

- Each attribute A in $\beta - \alpha$ is contained in a candidate key for R.

Note that the third condition above does not say that a single candidate key should contain all the attributes in $\beta - \alpha$; each attribute A in $\beta - \alpha$ may be contained in a *different* candidate key.

The first two alternatives are the same as the two alternatives in the definition of BCNF. The third alternative of the 3NF definition seems rather unintuitive, and it is not obvious why it is useful. It represents, in some sense, a minimal relaxation of the BCNF conditions that helps ensure that every schema has a dependency-preserving decomposition into 3NF. Its purpose will become more clear later, when we study decomposition into 3NF.

Observe that any schema that satisfies BCNF also satisfies 3NF, since each of its functional dependencies would satisfy one of the first two alternatives. BCNF is therefore a more restrictive constraint than is 3NF.

The definition of 3NF allows certain functional dependencies that are not allowed in BCNF. A dependency $\alpha \rightarrow \beta$ that satisfies only the third alternative of the 3NF definition is not allowed in BCNF, but is allowed in 3NF.[1]

Let us return to our *Banker-schema* example (Section 7.6). We have shown that this relation schema does not have a dependency-preserving, lossless-join decomposition into BCNF. This schema, however, turns out to be in 3NF. To see that it is, we note that {*customer-name, branch-name*} is a candidate key for *Banker-schema*, so the only attribute not contained in a candidate key for *Banker-schema* is *banker-name*. The only nontrivial functional dependencies of the form

$$\alpha \rightarrow \textit{banker-name}$$

include {*customer-name, branch-name*} as part of α. Since {*customer-name, branch-name*} is a candidate key, these dependencies do not violate the definition of 3NF.

As an optimization when testing for 3NF, we can consider only functional dependencies in the given set F, rather than in F^+. Also, we can decompose the dependencies in F so that their right-hand side consists of only single attributes, and use the resultant set in place of F.

Given a dependency $\alpha \rightarrow \beta$, we can use the same attribute-closure–based technique that we used for BCNF to check if α is a superkey. If α is not a superkey, we have to verify whether each attribute in β is contained in a candidate key of R; this test is rather more expensive, since it involves finding candidate keys. In fact, testing for 3NF has been shown to be NP-hard; thus, it is very unlikely that there is a polynomial time complexity algorithm for the task.

7.7.2 Decomposition Algorithm

Figure 7.14 shows an algorithm for finding a dependency-preserving, lossless-join decomposition into 3NF. The set of dependencies F_c used in the algorithm is a canoni-

1. These dependencies are examples of **transitive dependencies** (see Exercise 7.25). The original definition of 3NF was in terms of transitive dependencies. The definition we use is equivalent but easier to understand.

```
let F_c be a canonical cover for F;
i := 0;
for each functional dependency α → β in F_c do
   if none of the schemas R_j, j = 1, 2, ..., i contains α β
      then begin
               i := i + 1;
               R_i := α β;
           end
if none of the schemas R_j, j = 1, 2, ..., i contains a candidate key for R
   then begin
           i := i + 1;
           R_i := any candidate key for R;
      end
return (R_1, R_2, ..., R_i)
```

Figure 7.14 Dependency-preserving, lossless-join decomposition into 3NF.

cal cover for *F*. Note that the algorithm considers the set of schemas $R_j, j = 1, 2, \ldots, i$; initially $i = 0$, and in this case the set is empty.

To illustrate the algorithm of Figure 7.14, consider the following extension to the *Banker-schema* in Section 7.6:

Banker-info-schema = (*branch-name*, *customer-name*, *banker-name*, *office-number*)

The main difference here is that we include the banker's office number as part of the information. The functional dependencies for this relation schema are

banker-name → *branch-name office-number*
customer-name branch-name → *banker-name*

The **for** loop in the algorithm causes us to include the following schemas in our decomposition:

Banker-office-schema = (*banker-name*, *branch-name*, *office-number*)
Banker-schema = (*customer-name*, *branch-name*, *banker-name*)

Since *Banker-schema* contains a candidate key for *Banker-info-schema*, we are finished with the decomposition process.

The algorithm ensures the preservation of dependencies by explicitly building a schema for each dependency in a canonical cover. It ensures that the decomposition is a lossless-join decomposition by guaranteeing that at least one schema contains a candidate key for the schema being decomposed. Exercise 7.19 provides some insight into the proof that this suffices to guarantee a lossless join.

This algorithm is also called the **3NF synthesis algorithm**, since it takes a set of dependencies and adds one schema at a time, instead of decomposing the initial schema repeatedly. The result is not uniquely defined, since a set of functional dependencies

can have more than one canonical cover, and, further, in some cases the result of the algorithm depends on the order in which it considers the dependencies in F_c.

If a relation R_i is in the decomposition generated by the synthesis algorithm, then R_i is in 3NF. Recall that when we test for 3NF, it suffices to consider functional dependencies whose right-hand side is a single attribute. Therefore, to see that R_i is in 3NF, you must convince yourself that any functional dependency $\gamma \rightarrow B$ that holds on R_i satisfies the definition of 3NF. Assume that the dependency that generated R_i in the synthesis algorithm is $\alpha \rightarrow \beta$. Now, B must be in α or β, since B is in R_i and $\alpha \rightarrow \beta$ generated R_i. Let us consider the three possible cases:

- B is in both α and β. In this case, the dependency $\alpha \rightarrow \beta$ would not have been in F_c since B would be extraneous in β. Thus, this case cannot hold.
- B is in β but not α. Consider two cases:
  - γ is a superkey. The second condition of 3NF is satisfied.
  - γ is not a superkey. Then α must contain some attribute not in γ. Now, since $\gamma \rightarrow B$ is in F^+, it must be derivable from F_c by using the attribute closure algorithm on γ. The derivation could not have used $\alpha \rightarrow \beta$— if it had been used, α must be contained in the attribute closure of γ, which is not possible, since we assumed γ is not a superkey. Now, using $\alpha \rightarrow (\beta - \{B\})$ and $\gamma \rightarrow B$, we can derive $\alpha \rightarrow B$ (since $\gamma \subseteq \alpha\beta$, and γ cannot contain B because $\gamma \rightarrow B$ is nontrivial). This would imply that B is extraneous in the right-hand side of $\alpha \rightarrow \beta$, which is not possible since $\alpha \rightarrow \beta$ is in the canonical cover F_c. Thus, if B is in β, then γ must be a superkey, and the second condition of 3NF must be satisfied.
- B is in α but not β.
  Since α is a candidate key, the third alternative in the definition of 3NF is satisfied.

Interestingly, the algorithm we described for decomposition into 3NF can be implemented in polynomial time, even though testing a given relation to see if it satisfies 3NF is NP-hard.

7.7.3 Comparison of BCNF and 3NF

Of the two normal forms for relational-database schemas, 3NF and BCNF, there are advantages to 3NF in that we know that it is always possible to obtain a 3NF design without sacrificing a lossless join or dependency preservation. Nevertheless, there are disadvantages to 3NF: If we do not eliminate all transitive relations schema dependencies, we may have to use null values to represent some of the possible meaningful relationships among data items, and there is the problem of repetition of information.

As an illustration of the null value problem, consider again the *Banker-schema* and its associated functional dependencies. Since *banker-name* $\rightarrow$ *branch-name*, we may want to represent relationships between values for *banker-name* and values for *branch-name* in our database. If we are to do so, however, either there must be a corresponding value for *customer-name*, or we must use a null value for the attribute *customer-name*.

customer-name	*banker-name*	*branch-name*
Jones	Johnson	Perryridge
Smith	Johnson	Perryridge
Hayes	Johnson	Perryridge
Jackson	Johnson	Perryridge
Curry	Johnson	Perryridge
Turner	Johnson	Perryridge

Figure 7.15 An instance of *Banker-schema*.

As an illustration of the repetition of information problem, consider the instance of *Banker-schema* in Figure 7.15. Notice that the information indicating that Johnson is working at the Perryridge branch is repeated.

Recall that our goals of database design with functional dependencies are:

1. BCNF
2. Lossless join
3. Dependency preservation

Since it is not always possible to satisfy all three, we may be forced to choose between BCNF and dependency preservation with 3NF.

It is worth noting that SQL does not provide a way of specifying functional dependencies, except for the special case of declaring superkeys by using the **primary key** or **unique** constraints. It is possible, although a little complicated, to write assertions that enforce a functional dependency (see Exercise 7.15); unfortunately, testing the assertions would be very expensive in most database systems. Thus even if we had a dependency-preserving decomposition, if we use standard SQL we would not be able to efficiently test a functional dependency whose left-hand side is not a key.

Although testing functional dependencies may involve a join if the decomposition is not dependency preserving, we can reduce the cost by using materialized views, which many database systems support. Given a BCNF decomposition that is not dependency preserving, we consider each dependency in a minimum cover F_c that is not preserved in the decomposition. For each such dependency $\alpha \rightarrow \beta$, we define a materialized view that computes a join of all relations in the decomposition, and projects the result on $\alpha\beta$. The functional dependency can be easily tested on the materialized view, by means of a constraint **unique** (α). On the negative side, there is a space and time overhead due to the materialized view, but on the positive side, the application programmer need not worry about writing code to keep redundant data consistent on updates; it is the job of the database system to maintain the materialized view, that is, keep up up to date when the database is updated. (Later in the book, in Section 14.5, we outline how a database system can perform materialized view maintenance efficiently.)

Thus, in case we are not able to get a dependency-preserving BCNF decomposition, it is generally preferable to opt for BCNF, and use techniques such as materialized views to reduce the cost of checking functional dependencies.

7.8 Fourth Normal Form

Some relation schemas, even though they are in BCNF, do not seem to be sufficiently normalized, in the sense that they still suffer from the problem of repetition of information. Consider again our banking example. Assume that, in an alternative design for the bank database schema, we have the schema

$$\textit{BC-schema} = (\textit{loan-number}, \textit{customer-name}, \textit{customer-street}, \textit{customer-city})$$

The astute reader will recognize this schema as a non-BCNF schema because of the functional dependency

$$\textit{customer-name} \rightarrow \textit{customer-street customer-city}$$

that we asserted earlier, and because *customer-name* is not a key for *BC-schema*. However, assume that our bank is attracting wealthy customers who have several addresses (say, a winter home and a summer home). Then, we no longer wish to enforce the functional dependency *customer-name* $\rightarrow$ *customer-street customer-city*. If we remove this functional dependency, we find *BC-schema* to be in BCNF with respect to our modified set of functional dependencies. Yet, even though *BC-schema* is now in BCNF, we still have the problem of repetition of information that we had earlier.

To deal with this problem, we must define a new form of constraint, called a *multivalued dependency*. As we did for functional dependencies, we shall use multivalued dependencies to define a normal form for relation schemas. This normal form, called **fourth normal form** (4NF), is more restrictive than BCNF. We shall see that every 4NF schema is also in BCNF, but there are BCNF schemas that are not in 4NF.

7.8.1 Multivalued Dependencies

Functional dependencies rule out certain tuples from being in a relation. If $A \rightarrow B$, then we cannot have two tuples with the same A value but different B values. Multivalued dependencies, on the other hand, do not rule out the existence of certain tuples. Instead, they *require* that other tuples of a certain form be present in the relation. For this reason, functional dependencies sometimes are referred to as **equality-generating dependencies**, and multivalued dependencies are referred to as **tuple-generating dependencies**.

Let R be a relation schema and let $\alpha \subseteq R$ and $\beta \subseteq R$. The **multivalued dependency**

$$\alpha \twoheadrightarrow \beta$$

holds on R if, in any legal relation $r(R)$, for all pairs of tuples t_1 and t_2 in r such that $t_1[\alpha] = t_2[\alpha]$, there exist tuples t_3 and t_4 in r such that

$$\begin{aligned}
&t_1[\alpha] = t_2[\alpha] = t_3[\alpha] = t_4[\alpha] \\
&t_3[\beta] = t_1[\beta] \\
&t_3[R - \beta] = t_2[R - \beta] \\
&t_4[\beta] = t_2[\beta] \\
&t_4[R - \beta] = t_1[R - \beta]
\end{aligned}$$

	α	β	$R - \alpha - \beta$
t_1	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$a_{j+1} \dots a_n$
t_2	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$b_{j+1} \dots b_n$
t_3	$a_1 \dots a_i$	$a_{i+1} \dots a_j$	$b_{j+1} \dots b_n$
t_4	$a_1 \dots a_i$	$b_{i+1} \dots b_j$	$a_{j+1} \dots a_n$

Figure 7.16 Tabular representation of $\alpha \twoheadrightarrow \beta$.

This definition is less complicated than it appears to be. Figure 7.16 gives a tabular picture of t_1, t_2, t_3, and t_4. Intuitively, the multivalued dependency $\alpha \twoheadrightarrow \beta$ says that the relationship between α and β is independent of the relationship between α and $R - \beta$. If the multivalued dependency $\alpha \twoheadrightarrow \beta$ is satisfied by all relations on schema R, then $\alpha \twoheadrightarrow \beta$ is a *trivial* multivalued dependency on schema R. Thus, $\alpha \twoheadrightarrow \beta$ is trivial if $\beta \subseteq \alpha$ or $\beta \cup \alpha = R$.

To illustrate the difference between functional and multivalued dependencies, we consider the *BC-schema* again, and the relation *bc* (*BC-schema*) of Figure 7.17. We must repeat the loan number once for each address a customer has, and we must repeat the address for each loan a customer has. This repetition is unnecessary, since the relationship between a customer and his address is independent of the relationship between that customer and a loan. If a customer (say, Smith) has a loan (say, loan number L-23), we want that loan to be associated with all Smith's addresses. Thus, the relation of Figure 7.18 is illegal. To make this relation legal, we need to add the tuples (L-23, Smith, Main, Manchester) and (L-27, Smith, North, Rye) to the *bc* relation of Figure 7.18.

Comparing the preceding example with our definition of multivalued dependency, we see that we want the multivalued dependency

$$\textit{customer-name} \twoheadrightarrow \textit{customer-street customer-city}$$

to hold. (The multivalued dependency *customer-name* $\twoheadrightarrow$ *loan-number* will do as well. We shall soon see that they are equivalent.)

As with functional dependencies, we shall use multivalued dependencies in two ways:

1. To test relations to determine whether they are legal under a given set of functional and multivalued dependencies
2. To specify constraints on the set of legal relations; we shall thus concern ourselves with *only* those relations that satisfy a given set of functional and multivalued dependencies

loan-number	*customer-name*	*customer-street*	*customer-city*
L-23	Smith	North	Rye
L-23	Smith	Main	Manchester
L-93	Curry	Lake	Horseneck

Figure 7.17 Relation *bc*: An example of redundancy in a BCNF relation.

loan-number	*customer-name*	*customer-street*	*customer-city*
L-23	Smith	North	Rye
L-27	Smith	Main	Manchester

Figure 7.18 An illegal *bc* relation.

Note that, if a relation r fails to satisfy a given multivalued dependency, we can construct a relation r' that *does* satisfy the multivalued dependency by adding tuples to r.

Let D denote a set of functional and multivalued dependencies. The **closure** D^+ of D is the set of all functional and multivalued dependencies logically implied by D. As we did for functional dependencies, we can compute D^+ from D, using the formal definitions of functional dependencies and multivalued dependencies. We can manage with such reasoning for very simple multivalued dependencies. Luckily, multivalued dependencies that occur in practice appear to be quite simple. For complex dependencies, it is better to reason about sets of dependencies by using a system of inference rules. (Section C.1.1 of the appendix outlines a system of inference rules for multivalued dependencies.)

From the definition of multivalued dependency, we can derive the following rule:

- If $\alpha \rightarrow \beta$, then $\alpha \twoheadrightarrow \beta$.

In other words, every functional dependency is also a multivalued dependency.

7.8.2 Definition of Fourth Normal Form

Consider again our *BC-schema* example in which the multivalued dependency *customer-name* $\twoheadrightarrow$ *customer-street customer-city* holds, but no nontrivial functional dependencies hold. We saw in the opening paragraphs of Section 7.8 that, although *BC-schema* is in BCNF, the design is not ideal, since we must repeat a customer's address information for each loan. We shall see that we can use the given multivalued dependency to improve the database design, by decomposing *BC-schema* into a **fourth normal form** decomposition.

A relation schema R is in **fourth normal form** (4NF) with respect to a set D of functional and multivalued dependencies if, for all multivalued dependencies in D^+ of the form $\alpha \twoheadrightarrow \beta$, where $\alpha \subseteq R$ and $\beta \subseteq R$, at least one of the following holds

- $\alpha \twoheadrightarrow \beta$ is a trivial multivalued dependency.
- α is a superkey for schema R.

A database design is in 4NF if each member of the set of relation schemas that constitutes the design is in 4NF.

Note that the definition of 4NF differs from the definition of BCNF in only the use of multivalued dependencies instead of functional dependencies. Every 4NF schema is in BCNF. To see this fact, we note that, if a schema R is not in BCNF, then there is

```
result := {R};
done := false;
compute D+; Given schema Ri, let Di denote the restriction of D+ to Ri
while (not done) do
    if (there is a schema Ri in result that is not in 4NF w.r.t. Di)
      then begin
              let α →→ β be a nontrivial multivalued dependency that holds
              on Ri such that α → Ri is not in Di, and α ∩ β = ∅;
              result := (result − Ri) ∪ (Ri − β) ∪ (α, β);
           end
      else done := true;
```

Figure 7.19 4NF decomposition algorithm.

a nontrivial functional dependency $\alpha \rightarrow \beta$ holding on R, where α is not a superkey. Since $\alpha \rightarrow \beta$ implies $\alpha \twoheadrightarrow \beta$, R cannot be in 4NF.

Let R be a relation schema, and let $R_1, R_2, \ldots, R_n$ be a decomposition of R. To check if each relation schema R_i in the decomposition is in 4NF, we need to find what multivalued dependencies hold on each R_i. Recall that, for a set F of functional dependencies, the restriction F_i of F to R_i is all functional dependencies in F^+ that include *only* attributes of R_i. Now consider a set D of both functional and multivalued dependencies. The **restriction** of D to R_i is the set D_i consisting of

1. All functional dependencies in D^+ that include only attributes of R_i
2. All multivalued dependencies of the form

$$\alpha \twoheadrightarrow \beta \cap R_i$$

where $\alpha \subseteq R_i$ and $\alpha \twoheadrightarrow \beta$ is in D^+.

7.8.3 Decomposition Algorithm

The analogy between 4NF and BCNF applies to the algorithm for decomposing a schema into 4NF. Figure 7.19 shows the 4NF decomposition algorithm. It is identical to the BCNF decomposition algorithm of Figure 7.13, except that it uses multivalued, instead of functional, dependencies and uses the restriction of D^+ to R_i.

If we apply the algorithm of Figure 7.19 to *BC-schema*, we find that *customer-name* $\twoheadrightarrow$ *loan-number* is a nontrivial multivalued dependency, and *customer-name* is not a superkey for *BC-schema*. Following the algorithm, we replace *BC-schema* by two schemas:

Borrower-schema = (*customer-name*, *loan-number*)
Customer-schema = (*customer-name*, *customer-street*, *customer-city*).

This pair of schemas, which is in 4NF, eliminates the problem we encountered earlier with the redundancy of *BC-schema*.

As was the case when we were dealing solely with functional dependencies, we are interested in decompositions that are lossless-join decompositions and that preserve dependencies. The following fact about multivalued dependencies and lossless joins shows that the algorithm of Figure 7.19 generates only lossless-join decompositions:

- Let R be a relation schema, and let D be a set of functional and multivalued dependencies on R. Let R_1 and R_2 form a decomposition of R. This decomposition is a lossless-join decomposition of R if and only if at least one of the following multivalued dependencies is in D^+:

$$R_1 \cap R_2 \twoheadrightarrow R_1$$
$$R_1 \cap R_2 \twoheadrightarrow R_2$$

Recall that we stated in Section 7.5.1 that, if $R_1 \cap R_2 \rightarrow R_1$ or $R_1 \cap R_2 \rightarrow R_2$, then R_1 and R_2 are a lossless-join decomposition of R. The preceding fact about multivalued dependencies is a more general statement about lossless joins. It says that, for *every* lossless-join decomposition of R into two schemas R_1 and R_2, one of the two dependencies $R_1 \cap R_2 \twoheadrightarrow R_1$ or $R_1 \cap R_2 \twoheadrightarrow R_2$ must hold.

The issue of dependency preservation when we decompose a relation becomes more complicated in the presence of multivalued dependencies. Section C.1.2 of the appendix pursues this topic.

7.9 More Normal Forms

The fourth normal form is by no means the "ultimate" normal form. As we saw earlier, multivalued dependencies help us understand and tackle some forms of repetition of information that cannot be understood in terms of functional dependencies. There are types of constraints called **join dependencies** that generalize multivalued dependencies, and lead to another normal form called **project-join normal form (PJNF)** (PJNF is called **fifth normal form** in some books). There is a class of even more general constraints, which leads to a normal form called **domain-key normal form**.

A practical problem with the use of these generalized constraints is that they are not only hard to reason with, but there is also no set of sound and complete inference rules for reasoning about the constraints. Hence PJNF and domain-key normal form are used quite rarely. Appendix C provides more details about these normal forms.

Conspicuous by its absence from our discussion of normal forms is **second normal form** (2NF). We have not discussed it, because it is of historical interest only. We simply define it, and let you experiment with it in Exercise 7.26.

7.10 Overall Database Design Process

So far we have looked at detailed issues about normal forms and normalization. In this section we study how normalization fits into the overall database design process.

Earlier in the chapter, starting in Section 7.4, we assumed that a relation schema R is given, and proceeded to normalize it. There are several ways in which we could have come up with the schema R:

1. R could have been generated when converting a E-R diagram to a set of tables.
2. R could have been a single relation containing *all* attributes that are of interest. The normalization process then breaks up R into smaller relations.
3. R could have been the result of some ad hoc design of relations, which we then test to verify that it satisfies a desired normal form.

In the rest of this section we examine the implications of these approaches. We also examine some practical issues in database design, including denormalization for performance and examples of bad design that are not detected by normalization.

7.10.1 E-R Model and Normalization

When we carefully define an E-R diagram, identifying all entities correctly, the tables generated from the E-R diagram should not need further normalization. However, there can be functional dependencies between attributes of an entity. For instance, suppose an *employee* entity had attributes *department-number* and *department-address*, and there is a functional dependency *department-number* $\rightarrow$ *department-address*. We would then need to normalize the relation generated from *employee*.

Most examples of such dependencies arise out of poor E-R diagram design. In the above example, if we did the E-R diagram correctly, we would have created a *department* entity with attribute *department-address* and a relationship between *employee* and *department*. Similarly, a relationship involving more than two entities may not be in a desirable normal form. Since most relationships are binary, such cases are relatively rare. (In fact, some E-R diagram variants actually make it difficult or impossible to specify nonbinary relations.)

Functional dependencies can help us detect poor E-R design. If the generated relations are not in desired normal form, the problem can be fixed in the E-R diagram. That is, normalization can be done formally as part of data modeling. Alternatively, normalization can be left to the designer's intuition during E-R modeling, and can be done formally on the relations generated from the E-R model.

7.10.2 The Universal Relation Approach

The second approach to database design is to start with a single relation schema containing all attributes of interest, and decompose it. One of our goals in choosing a decomposition was that it be a lossless-join decomposition. To consider losslessness, we assumed that it is valid to talk about the join of all the relations of the decomposed database.

Consider the database of Figure 7.20, showing a decomposition of the *loan-info* relation. The figure depicts a situation in which we have not yet determined the amount of loan L-58, but wish to record the remainder of the data on the loan. If we compute the natural join of these relations, we discover that all tuples referring to loan L-58 disappear. In other words, there is no *loan-info* relation corresponding to the relations of Figure 7.20. Tuples that disappear when we compute the join are *dangling tuples* (see Section 6.2.1). Formally, let $r_1(R_1),\ r_2(R_2), \ldots, r_n(R_n)$ be a set of relations. A

branch-name	*loan-number*
Round Hill	L-58

loan-number	*amount*

loan-number	*customer-name*
L-58	Johnson

Figure 7.20 Decomposition of *loan-info*.

tuple t of relation r_i is a dangling tuple if t is not in the relation

$$\Pi_{R_i} \ (r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n)$$

Dangling tuples may occur in practical database applications. They represent incomplete information, as they do in our example, where we wish to store data about a loan that is still in the process of being negotiated. The relation $r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$ is called a **universal relation**, since it involves all the attributes in the universe defined by $R_1 \cup R_2 \cup \cdots \cup R_n$.

The only way that we can write a universal relation for the example of Figure 7.20 is to include *null values* in the universal relation. We saw in Chapter 3 that null values present several difficulties. Because of them, it may be better to view the relations of the decomposed design as representing the database, rather than as the universal relation whose schema we decomposed during the normalization process. (The bibliographical notes discuss research on null values and universal relations.)

Note that we cannot enter all incomplete information into the database of Figure 7.20 without resorting to null values. For example, we cannot enter a loan number unless we know at least one of the following:

- The customer name
- The branch name
- The amount of the loan

Thus, a particular decomposition defines a restricted form of incomplete information that is acceptable in our database.

The normal forms that we have defined generate good database designs from the point of view of representation of incomplete information. Returning again to the example of Figure 7.20, we would not want to allow storage of the following fact: "There is a loan (whose number is unknown) to Jones in the amount of $100." This is because

$$\textit{loan-number} \rightarrow \textit{customer-name amount}$$

and therefore the only way that we can relate *customer-name* and *amount* is through *loan-number*. If we do not know the loan number, we cannot distinguish this loan from other loans with unknown numbers.

In other words, we do not want to store data for which the key attributes are unknown. Observe that the normal forms that we have defined do not allow us to store that type of information unless we use null values. Thus, our normal forms allow representation of acceptable incomplete information via dangling tuples, while prohibiting the storage of undesirable incomplete information.

Another consequence of the universal relation approach to database design is that attribute names must be unique in the universal relation. We cannot use *name* to refer to both *customer-name* and to *branch-name*. It is generally preferable to use unique names, as we have done. Nevertheless, if we defined our relation schemas directly, rather than in terms of a universal relation, we could obtain relations on schemas such as the following for our banking example:

branch-loan (*name, number*)
loan-customer (*number, name*)
amt (*number, amount*)

Observe that, with the preceding relations, expressions such as *branch-loan* $\bowtie$ *loan-customer* are meaningless. Indeed, the expression *branch-loan* $\bowtie$ *loan-customer* finds loans made by branches to customers who have the same name as the name of the branch.

In a language such as SQL, however, a query involving *branch-loan* and *loan-customer* must remove ambiguity in references to *name* by prefixing the relation name. In such environments, the multiple roles for *name* (as branch name and as customer name) are less troublesome and may be simpler to use.

We believe that using the **unique-role assumption**—that each attribute name has a unique meaning in the database—is generally preferable to reusing of the same name in multiple roles. When the unique-role assumption is not made, the database designer must be especially careful when constructing a normalized relational-database design.

7.10.3 Denormalization for Performance

Occasionally database designers choose a schema that has redundant information; that is, it is not normalized. They use the redundancy to improve performance for specific applications. The penalty paid for not using a normalized schema is the extra work (in terms of coding time and execution time) to keep redundant data consistent.

For instance, suppose that the name of an account holder has to be displayed along with the account number and balance, every time the account is accessed. In our normalized schema, this requires a join of *account* with *depositor*.

One alternative to computing the join on the fly is to store a relation containing all the attributes of *account* and *depositor*. This makes displaying the account information faster. However, the balance information for an account is repeated for every person who owns the account, and all copies must be updated by the application, whenever the account balance is updated. The process of taking a normalized schema and making it non-normalized is called **denormalization**, and designers use it to tune performance of systems to support time-critical operations.

A better alternative, supported by many database systems today, is to use the normalized schema, and additionally store the join or *account* and *depositor* as a materialized view. (Recall that a materialized view is a view whose result is stored in the database, and brought up to date when the relations used in the view are updated.) Like denormalization, using materialized view does have space and time overheads; however, it has the advantage that keeping the view up to date is the job of the database system, not the application programmer.

7.10.4 Other Design Issues

There are some aspects of database design that are not addressed by normalization, and can thus lead to bad database design. We give examples here; obviously, such designs should be avoided.

Consider a company database, where we want to store earnings of companies in different years. A relation *earnings*(*company-id*, *year*, *amount*) could be used to store the earnings information. The only functional dependency on this relation is *company-id*, *year* $\rightarrow$ *amount*, and the relation is in BCNF.

An alternative design is to use multiple relations, each storing the earnings for a different year. Let us say the years of interest are 2000, 2001, and 2002; we would then have relations of the form *earnings-2000*, *earnings-2001*, *earnings-2002*, all of which are on the schema (*company-id*, *earnings*). The only functional dependency here on each relation would be *company-id* $\rightarrow$ *earnings*, so these relations are also in BCNF.

However, this alternative design is clearly a bad idea—we would have to create a new relation every year, and would also have to write new queries every year, to take each new relation into account. Queries would also be more complicated since they may have to refer to many relations.

Yet another way of representing the same data is to have a single relation *company-year*(*company-id*, *earnings-2000*, *earnings-2001*, *earnings-2002*). Here the only functional dependencies are from *company-id* to the other attributes, and again the relation is in BCNF. This design is also a bad idea since it has problems similar to the previous design—namely we would have to modify the relation schema and write new queries, every year. Queries would also be more complicated, since they may have to refer to many attributes.

Representations such as those in the *company-year* relation, with one column for each value of an attribute, are called **crosstabs**; they are widely used in spreadsheets and reports and in data analysis tools. While such representations are useful for display to users, for the reasons just given, they are not desirable in a database design. SQL extensions have been proposed to convert data from a normal relational representation to a crosstab, for display.

7.11 Summary

- We showed pitfalls in database design, and how to systematically design a database schema that avoids the pitfalls. The pitfalls included repeated information and inability to represent some information.

- We introduced the concept of functional dependencies, and showed how to reason with functional dependencies. We laid special emphasis on what dependencies are logically implied by a set of dependencies. We also defined the notion of a canonical cover, which is a minimal set of functional dependencies equivalent to a given set of functional dependencies.
- We introduced the concept of decomposition, and showed that decompositions must be lossless-join decompositions, and should preferably be dependency preserving.
- If the decomposition is dependency preserving, given a database update, all functional dependencies can be verifiable from individual relations, without computing a join of relations in the decomposition.
- We then presented Boyce–Codd Normal Form (BCNF); relations in BCNF are free from the pitfalls outlined earlier. We outlined an algorithm for decomposing relations into BCNF. There are relations for which there is no dependency-preserving BCNF decomposition.
- We used the canonical covers to decompose a relation into 3NF, which is a small relaxation of the BCNF condition. Relations in 3NF may have some redundancy, but there is always a dependency-preserving decomposition into 3NF.
- We presented the notion of multivalued dependencies, which specify constraints that cannot be specified with functional dependencies alone. We defined fourth normal form (4NF) with multivalued dependencies. Section C.1.1 of the appendix gives details on reasoning about multivalued dependencies.
- Other normal forms, such as PJNF and DKNF, eliminate more subtle forms of redundancy. However, these are hard to work with and are rarely used. Appendix C gives details on these normal forms.
- In reviewing the issues in this chapter, note that the reason we could define rigorous approaches to relational-database design is that the relational data model rests on a firm mathematical foundation. That is one of the primary advantages of the relational model compared with the other data models that we have studied.

Review Terms

- Atomic domains
- First normal form
- Pitfalls in relational-database design
- Functional dependencies
- Superkey
- F holds on R
- R satisfies F
- Trivial functional dependencies
- Closure of a set of functional dependencies
- Armstrong's axioms

- Closure of attribute sets
- Decomposition
- Lossless-join decomposition
- Legal relations
- Dependency preservation
- Restriction of F to R_i
- Boyce–Codd normal form (BCNF)
- BCNF decomposition algorithm
- Canonical cover
- Extraneous attributes
- Third normal form
- 3NF decomposition algorithm
- Multivalued dependencies
- Fourth normal form
- restriction of a multivalued dependency
- Project-join normal form (PJNF)
- Domain-key normal form
- E-R model and normalization
- Universal relation
- Unique-role assumption
- Denormalization

Exercises

7.1 Explain what is meant by *repetition of information* and *inability to represent information*. Explain why each of these properties may indicate a bad relational-database design.

7.2 Suppose that we decompose the schema $R = (A, B, C, D, E)$ into

$$(A, B, C)$$
$$(A, D, E)$$

Show that this decomposition is a lossless-join decomposition if the following set F of functional dependencies holds:

$$A \rightarrow BC$$
$$CD \rightarrow E$$
$$B \rightarrow D$$
$$E \rightarrow A$$

7.3 Why are certain functional dependencies called *trivial* functional dependencies?

7.4 List all functional dependencies satisfied by the relation of Figure 7.21.

A	B	C
a_1	b_1	c_1
a_1	b_1	c_2
a_2	b_1	c_1
a_2	b_1	c_3

Figure 7.21 Relation of Exercise 7.4.

7.5 Use the definition of functional dependency to argue that each of Armstrong's axioms (reflexivity, augmentation, and transitivity) is sound.

7.6 Explain how functional dependencies can be used to indicate the following:

- A one-to-one relationship set exists between entity sets *account* and *customer*.
- A many-to-one relationship set exists between entity sets *account* and *customer*.

7.7 Consider the following proposed rule for functional dependencies: If $\alpha \rightarrow \beta$ and $\gamma \rightarrow \beta$, then $\alpha \rightarrow \gamma$. Prove that this rule is *not* sound by showing a relation *r* that satisfies $\alpha \rightarrow \beta$ and $\gamma \rightarrow \beta$, but does not satisfy $\alpha \rightarrow \gamma$.

7.8 Use Armstrong's axioms to prove the soundness of the union rule. (*Hint*: Use the augmentation rule to show that, if $\alpha \rightarrow \beta$, then $\alpha \rightarrow \alpha\beta$. Apply the augmentation rule again, using $\alpha \rightarrow \gamma$, and then apply the transitivity rule.)

7.9 Use Armstrong's axioms to prove the soundness of the decomposition rule.

7.10 Use Armstrong's axioms to prove the soundness of the pseudotransitivity rule.

7.11 Compute the closure of the following set *F* of functional dependencies for relation schema $R = (A, B, C, D, E)$.

$$A \rightarrow BC$$
$$CD \rightarrow E$$
$$B \rightarrow D$$
$$E \rightarrow A$$

List the candidate keys for *R*.

7.12 Using the functional dependencies of Exercise 7.11, compute B^+.

7.13 Using the functional dependencies of Exercise 7.11, compute the canonical cover F_c.

7.14 Consider the algorithm in Figure 7.22 to compute α^+. Show that this algorithm is more efficient than the one presented in Figure 7.7 (Section 7.3.3) and that it computes α^+ correctly.

7.15 Given the database schema $R(a, b, c)$, and a relation r on the schema R, write an SQL query to test whether the functional dependency $b \rightarrow c$ holds on relation r. Also write an SQL assertion that enforces the functional dependency. Assume that no null values are present.

7.16 Show that the following decomposition of the schema *R* of Exercise 7.2 is not a lossless-join decomposition:

$$(A, B, C)$$
$$(C, D, E)$$

Hint: Give an example of a relation *r* on schema *R* such that

$$\Pi_{A,B,C}(r) \bowtie \Pi_{C,D,E}(r) \neq r$$

```
result := ∅;
/* fdcount is an array whose ith element contains the number
   of attributes on the left side of the ith FD that are
   not yet known to be in α+ */
for i := 1 to |F| do
   begin
      let β → γ denote the ith FD;
      fdcount [i] := |β|;
   end
/* appears is an array with one entry for each attribute. The
   entry for attribute A is a list of integers. Each integer
   i on the list indicates that A appears on the left side
   of the ith FD */
for each attribute A do
   begin
      appears [A] := NIL;
      for i := 1 to |F| do
         begin
            let β → γ denote the ith FD;
            if A ∈ β then add i to appears [A];
         end
   end
addin (α);
return (result);

procedure addin (α);
for each attribute A in α do
   begin
      if A ∉ result then
         begin
            result := result ∪ {A};
            for each element i of appears[A] do
               begin
                  fdcount [i] := fdcount [i] − 1;
                  if fdcount [i] := 0 then
                     begin
                        let β → γ denote the ith FD;
                        addin (γ);
                     end
               end
         end
   end
```

Figure 7.22 An algorithm to compute α^+.

7.17 Let $R_1, R_2, \ldots, R_n$ be a decomposition of schema U. Let $u(U)$ be a relation, and let $r_i = \Pi_{R_I}(u)$. Show that

$$u \subseteq r_1 \bowtie r_2 \bowtie \cdots \bowtie r_n$$

7.18 Show that the decomposition in Exercise 7.2 is not a dependency-preserving decomposition.

7.19 Show that it is possible to ensure that a dependency-preserving decomposition into 3NF is a lossless-join decomposition by guaranteeing that at least one schema contains a candidate key for the schema being decomposed. (*Hint*: Show that the join of all the projections onto the schemas of the decomposition cannot have more tuples than the original relation.)

7.20 List the three design goals for relational databases, and explain why each is desirable.

7.21 Give a lossless-join decomposition into BCNF of schema R of Exercise 7.2.

7.22 Give an example of a relation schema R' and set F' of functional dependencies such that there are at least three distinct lossless-join decompositions of R' into BCNF.

7.23 In designing a relational database, why might we choose a non-BCNF design?

7.24 Give a lossless-join, dependency-preserving decomposition into 3NF of schema R of Exercise 7.2.

7.25 Let a *prime* attribute be one that appears in at least one candidate key. Let α and β be sets of attributes such that $\alpha \rightarrow \beta$ holds, but $\beta \rightarrow \alpha$ does not hold. Let A be an attribute that is not in α, is not in β, and for which $\beta \rightarrow A$ holds. We say that A is *transitively dependent* on α. We can restate our definition of 3NF as follows: A relation schema R is in 3NF with respect to a set F of functional dependencies if there are no nonprime attributes A in R for which A is transitively dependent on a key for R.

Show that this new definition is equivalent to the original one.

7.26 A functional dependency $\alpha \rightarrow \beta$ is called a **partial dependency** if there is a proper subset γ of α such that $\gamma \rightarrow \beta$. We say that β is *partially dependent* on α. A relation schema R is in **second normal form** (2NF) if each attribute A in R meets one of the following criteria:

- It appears in a candidate key.
- It is not partially dependent on a candidate key.

Show that every 3NF schema is in 2NF. (*Hint*: Show that every partial dependency is a transitive dependency.)

7.27 Given the three goals of relational-database design, is there any reason to design a database schema that is in 2NF, but is in no higher-order normal form? (See Exercise 7.26 for the definition of 2NF.)

7.28 Give an example of a relation schema R and a set of dependencies such that R is in BCNF, but is not in 4NF.

7.29 Explain why 4NF is a normal form more desirable than BCNF.

7.30 Explain how dangling tuples may arise. Explain problems that they may cause.

Bibliographical Notes

The first discussion of relational-database design theory appeared in an early paper by Codd [1970]. In that paper, Codd also introduced functional dependencies, and first, second, and third normal forms.

Armstrong's axioms were introduced in Armstrong [1974]. Ullman [1988] is an easily accessible source of proofs of soundness and completeness of Armstrong's axioms. Ullman [1988] also provides an algorithm for testing for lossless-join decomposition for general (nonbinary) decompositions, and many other algorithms, theorems, and proofs concerning dependency theory. Maier [1983] discusses the theory of functional dependencies.Graham et al. [1986] discusses formal aspects of the concept of a legal relation.

BCNF was introduced in Codd [1972]. The desirability of BCNF is discussed in Bernstein et al. [1980a]. A polynomial-time algorithm for BCNF decomposition appears in Tsou and Fischer [1982], and can also be found in Ullman [1988]. Biskup et al. [1979] gives the algorithm we used to find a lossless-join dependency-preserving decomposition into 3NF. Fundamental results on the lossless-join property appear in Aho et al. [1979a].

Multivalued dependencies are discussed in Zaniolo [1976]. Beeri et al. [1977] gives a set of axioms for multivalued dependencies, and proves that the authors axioms are sound and complete. Our axiomatization is based on theirs. The notions of 4NF, PJNF, and DKNF are from Fagin [1977], Fagin [1979], and Fagin [1981], respectively.

Maier [1983] presents the design theory of relational databases in detail. Ullman [1988] and Abiteboul et al. [1995] present a more theoretic coverage of many of the dependencies and normal forms presented here. See the bibliographical notes of Appendix C for further references to literature on normalization.

PART 3

Object-based Databases and XML

Several application areas for database systems are limited by the restrictions of the relational data model. As a result, researchers have developed several data models to deal with these application domains. In this part, we study the object-oriented data model and the object-relational data model. In addition, we study XML, a language that can represent data that is less structured than that of the other data models.

The object-oriented data model, described in Chapter 8, is based on the object-oriented-programming language paradigm, which is now in wide use. Inheritance, object-identity, and encapsulation (information hiding), with methods to provide an interface to objects, are among the key concepts of object-oriented programming that have found applications in data modeling. The object-oriented data model also supports a rich type system, including structured and collection types. While inheritance and, to some extent, complex types are also present in the E-R model, encapsulation and object-identity distinguish the object-oriented data model from the E-R model.

The object-relational model, described in Chapter 9, combines features of the relational and object-oriented models. This model provides the rich type system of object-oriented databases, combined with relations as the basis for storage of data. It applies inheritance to relations, not just to types. The object-relational data model provides a smooth migration path from relational databases, which is attractive to relational database vendors. As a result, the SQL:1999 standard includes a number of object-oriented features in its type system, while continuing to use the relational model as the underlying model.

The XML language was initially designed as a way of adding markup information to text documents, but has become important because of its applications in data exchange. XML provides a way to represent data that have nested structure, and furthermore allows a great deal of flexibility in structuring of data, which is important for certain kinds of nontraditional data. Chapter 10 describes the XML language, and then presents different ways of expressing queries on data represented in XML, and transforming XML data from one form to another.

© The McGraw–Hill Companies, 2001

CHAPTER 11

Storage and File Structure

In preceding chapters, we have emphasized the higher-level models of a database. For example, at the *conceptual* or *logical* level, we viewed the database, in the relational model, as a collection of tables. Indeed, the logical model of the database is the correct level for database *users* to focus on. This is because the goal of a database system is to simplify and facilitate access to data; users of the system should not be burdened unnecessarily with the physical details of the implementation of the system.

In this chapter, however, as well as in Chapters 12, 13, and 14, we probe below the higher levels as we describe various methods for implementing the data models and languages presented in preceding chapters. We start with characteristics of the underlying storage media, such as disk and tape systems. We then define various data structures that will allow fast access to data. We consider several alternative structures, each best suited to a different kind of access to data. The final choice of data structure needs to be made on the basis of the expected use of the system and of the physical characteristics of the specific machine.

11.1 Overview of Physical Storage Media

Several types of data storage exist in most computer systems. These storage media are classified by the speed with which data can be accessed, by the cost per unit of data to buy the medium, and by the medium's reliability. Among the media typically available are these:

- **Cache**. The cache is the fastest and most costly form of storage. Cache memory is small; its use is managed by the computer system hardware. We shall not be concerned about managing cache storage in the database system.
- **Main memory**. The storage medium used for data that are available to be operated on is main memory. The general-purpose machine instructions operate on main memory. Although main memory may contain many megabytes of

data, or even gigabytes of data in large server systems, it is generally too small (or too expensive) for storing the entire database. The contents of main memory are usually lost if a power failure or system crash occurs.

- **Flash memory**. Also known as *electrically erasable programmable read-only memory* (*EEPROM*), flash memory differs from main memory in that data survive power failure. Reading data from flash memory takes less than 100 nanoseconds (a nanosecond is 1/1000 of a microsecond), which is roughly as fast as reading data from main memory. However, writing data to flash memory is more complicated—data can be written once, which takes about 4 to 10 microseconds, but cannot be overwritten directly. To overwrite memory that has been written already, we have to erase an entire bank of memory at once; it is then ready to be written again. A drawback of flash memory is that it can support only a limited number of erase cycles, ranging from 10,000 to 1 million. Flash memory has found popularity as a replacement for magnetic disks for storing small volumes of data (5 to 10 megabytes) in low-cost computer systems, such as computer systems that are embedded in other devices, in hand-held computers, and in other digital electronic devices such as digital cameras.

- **Magnetic-disk storage**. The primary medium for the long-term on-line storage of data is the magnetic disk. Usually, the entire database is stored on magnetic disk. The system must move the data from disk to main memory so that they can be accessed. After the system has performed the designated operations, the data that have been modified must be written to disk.

  The size of magnetic disks currently ranges from a few gigabytes to 80 gigabytes. Both the lower and upper end of this range have been growing at about 50 percent per year, and we can expect much larger capacity disks every year. Disk storage survives power failures and system crashes. Disk-storage devices themselves may sometimes fail and thus destroy data, but such failures usually occur much less frequently than do system crashes.

- **Optical storage**. The most popular forms of optical storage are the *compact disk* (CD), which can hold about 640 megabytes of data, and the *digital video disk* (DVD) which can hold 4.7 or 8.5 gigabytes of data per side of the disk (or up to 17 gigabytes on a two-sided disk). Data are stored optically on a disk, and are read by a laser. The optical disks used in read-only compact disks (CD-ROM) or read-only digital video disk (DVD-ROM) cannot be written, but are supplied with data prerecorded.

  There are "record-once" versions of compact disk (called CD-R) and digital video disk (called DVD-R), which can be written only once; such disks are also called **write-once, read-many** (WORM) disks. There are also "multiple-write" versions of compact disk (called CD-RW) and digital video disk (DVD-RW and DVD-RAM), which can be written multiple times. Recordable compact disks are magnetic–optical storage devices that use optical means to read magnetically encoded data. Such disks are useful for archival storage of data as well as distribution of data.

Jukebox systems contain a few drives and numerous disks that can be loaded into one of the drives automatically (by a robot arm) on demand.

- **Tape storage**. Tape storage is used primarily for backup and archival data. Although magnetic tape is much cheaper than disks, access to data is much slower, because the tape must be accessed sequentially from the beginning. For this reason, tape storage is referred to as **sequential-access** storage. In contrast, disk storage is referred to as **direct-access** storage because it is possible to read data from any location on disk.

  Tapes have a high capacity (40 gigabyte to 300 gigabytes tapes are currently available), and can be removed from the tape drive, so they are well suited to cheap archival storage. Tape jukeboxes are used to hold exceptionally large collections of data, such as remote-sensing data from satellites, which could include as much as hundreds of terabytes (1 terabyte = 10^{12} bytes), or even a petabyte (1 petabyte = 10^{15} bytes) of data.

The various storage media can be organized in a hierarchy (Figure 11.1) according to their speed and their cost. The higher levels are expensive, but are fast. As we move down the hierarchy, the cost per bit decreases, whereas the access time increases. This trade-off is reasonable; if a given storage system were both faster and less expensive than another—other properties being the same—then there would be no reason to use the slower, more expensive memory. In fact, many early storage devices, including paper tape and core memories, are relegated to museums now that magnetic tape and semiconductor memory have become faster and cheaper. Magnetic tapes themselves were used to store active data back when disks were expensive and had low

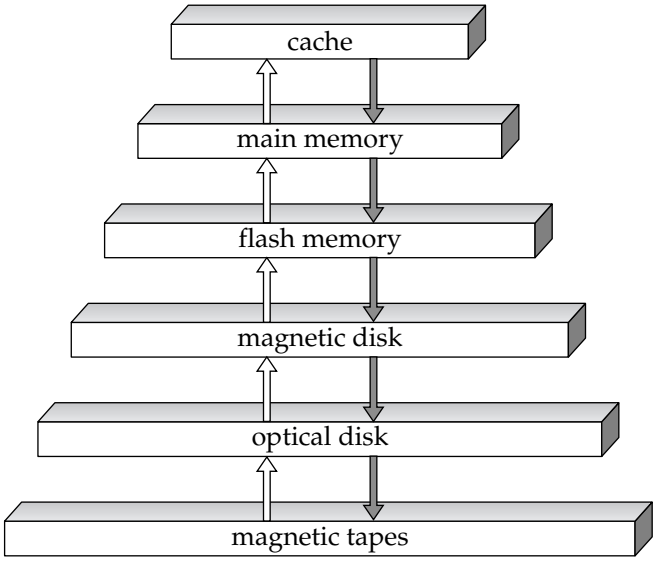


Figure 11.1 Storage-device hierarchy.

storage capacity. Today, almost all active data are stored on disks, except in rare cases where they are stored on tape or in optical jukeboxes.

The fastest storage media—for example, cache and main memory—are referred to as **primary storage**. The media in the next level in the hierarchy—for example, magnetic disks—are referred to as **secondary storage**, or **online storage**. The media in the lowest level in the hierarchy—for example, magnetic tape and optical-disk jukeboxes—are referred to as **tertiary storage**, or **offline storage**.

In addition to the speed and cost of the various storage systems, there is also the issue of storage volatility. **Volatile storage** loses its contents when the power to the device is removed. In the hierarchy shown in Figure 11.1, the storage systems from main memory up are volatile, whereas the storage systems below main memory are nonvolatile. In the absence of expensive battery and generator backup systems, data must be written to **nonvolatile storage** for safekeeping. We shall return to this subject in Chapter 17.

11.2 Magnetic Disks

Magnetic disks provide the bulk of secondary storage for modern computer systems. Disk capacities have been growing at over 50 percent per year, but the storage requirements of large applications have also been growing very fast, in some cases even faster than the growth rate of disk capacities. A large database may require hundreds of disks.

11.2.1 Physical Characteristics of Disks

Physically, disks are relatively simple (Figure 11.2). Each disk **platter** has a flat circular shape. Its two surfaces are covered with a magnetic material, and information is recorded on the surfaces. Platters are made from rigid metal or glass and are covered (usually on both sides) with magnetic recording material. We call such magnetic disks **hard disks**, to distinguish them from **floppy disks**, which are made from flexible material.

When the disk is in use, a drive motor spins it at a constant high speed (usually 60, 90, or 120 revolutions per second, but disks running at 250 revolutions per second are available). There is a read–write head positioned just above the surface of the platter. The disk surface is logically divided into **tracks**, which are subdivided into **sectors**. A **sector** is the smallest unit of information that can be read from or written to the disk. In currently available disks, sector sizes are typically 512 bytes; there are over 16,000 tracks on each platter, and 2 to 4 platters per disk. The inner tracks (closer to the spindle) are of smaller length, and in current-generation disks, the outer tracks contain more sectors than the inner tracks; typical numbers are around 200 sectors per track in the inner tracks, and around 400 sectors per track in the outer tracks. The numbers above vary among different models; higher-capacity models usually have more sectors per track and more tracks on each platter.

The **read–write head** stores information on a sector magnetically as reversals of the direction of magnetization of the magnetic material. There may be hundreds of concentric tracks on a disk surface, containing thousands of sectors.

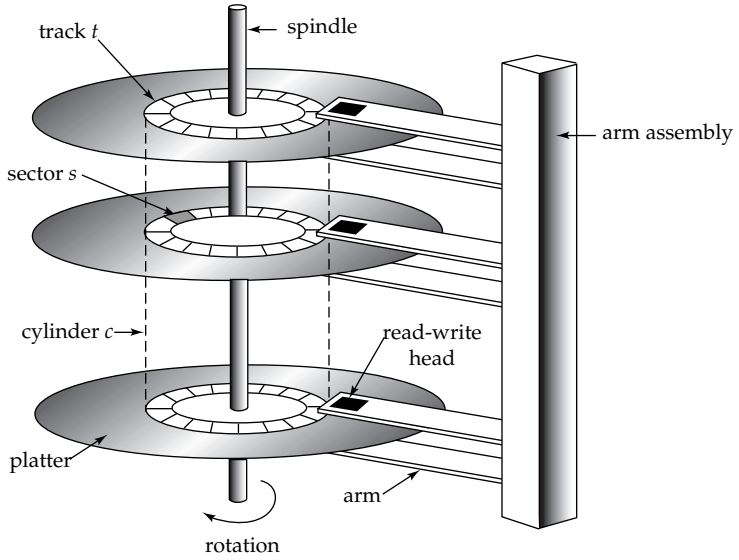


Figure 11.2 Moving-head disk mechanism.

Each side of a platter of a disk has a read–write head, which moves across the platter to access different tracks. A disk typically contains many platters, and the read–write heads of all the tracks are mounted on a single assembly called a **disk arm**, and move together. The disk platters mounted on a spindle and the heads mounted on a disk arm are together known as **head–disk assemblies**. Since the heads on all the platters move together, when the head on one platter is on the ith track, the heads on all other platters are also on the ith track of their respective platters. Hence, the ith tracks of all the platters together are called the ith **cylinder**.

Today, disks with a platter diameter of $3\frac{1}{2}$ inches dominate the market. They have a lower cost and faster seek times (due to smaller seek distances) than do the larger-diameter disks (up to 14 inches) that were common earlier, yet they provide high storage capacity. Smaller-diameter disks are used in portable devices such as laptop computers.

The read–write heads are kept as close as possible to the disk surface to increase the recording density. The head typically floats or flies only microns from the disk surface; the spinning of the disk creates a small breeze, and the head assembly is shaped so that the breeze keeps the head floating just above the disk surface. Because the head floats so close to the surface, platters must be machined carefully to be flat. Head crashes can be a problem. If the head contacts the disk surface, the head can scrape the recording medium off the disk, destroying the data that had been there. Usually, the head touching the surface causes the removed medium to become airborne and to come between the other heads and their platters, causing more crashes. Under normal circumstances, a head crash results in failure of the entire disk, which must then be replaced. Current-generation disk drives use a thin film of magnetic

metal as recording medium. They are much less susceptible to failure by head crashes than the older oxide-coated disks.

A *fixed-head disk* has a separate head for each track. This arrangement allows the computer to switch from track to track quickly, without having to move the head assembly, but because of the large number of heads, the device is extremely expensive. Some disk systems have multiple disk arms, allowing more than one track on the same platter to be accessed at a time. Fixed-head disks and multiple-arm disks were used in high-performance mainframe systems, but are no longer in production.

A **disk controller** interfaces between the computer system and the actual hardware of the disk drive. It accepts high-level commands to read or write a sector, and initiates actions, such as moving the disk arm to the right track and actually reading or writing the data. Disk controllers also attach **checksums** to each sector that is written; the checksum is computed from the data written to the sector. When the sector is read back, the controller computes the checksum again from the retrieved data and compares it with the stored checksum; if the data are corrupted, with a high probability the newly computed checksum will not match the stored checksum. If such an error occurs, the controller will retry the read several times; if the error continues to occur, the controller will signal a read failure.

Another interesting task that disk controllers perform is **remapping of bad sectors**. If the controller detects that a sector is damaged when the disk is initially formatted, or when an attempt is made to write the sector, it can logically map the sector to a different physical location (allocated from a pool of extra sectors set aside for this purpose). The remapping is noted on disk or in nonvolatile memory, and the write is carried out on the new location.

Figure 11.3 shows how disks are connected to a computer system. Like other storage units, disks are connected to a computer system or to a controller through a high-speed interconnection. In modern disk systems, lower-level functions of the disk controller, such as control of the disk arm, computing and verification of checksums, and remapping of bad sectors, are implemented within the disk drive unit.

The **AT attachment** (**ATA**) interface (which is a faster version of the **integrated drive electronics** (IDE) interface used earlier in IBM PCs) and a **small-computer-system interconnect** (SCSI; pronounced "scuzzy") are commonly used to connect

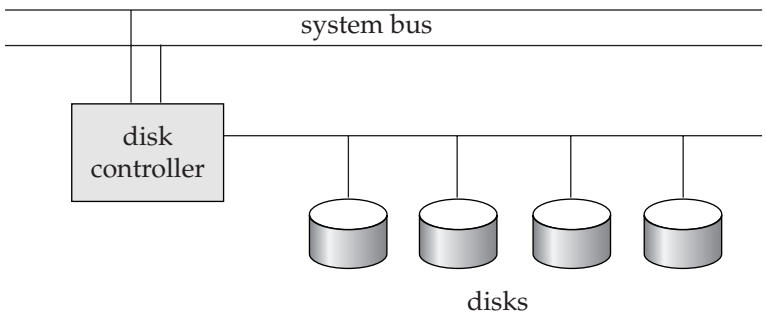


Figure 11.3 Disk subsystem.

disks to personal computers and workstations. Mainframe and server systems usually have a faster and more expensive interface, such as high-capacity versions of the SCSI interface, and the Fibre Channel interface.

While disks are usually connected directly by cables to the disk controller, they can be situated remotely and connected by a high-speed network to the disk controller. In the **storage area network** (**SAN**) architecture, large numbers of disks are connected by a high-speed network to a number of server computers. The disks are usually organized locally using **redundant arrays of independent disks** (**RAID**) storage organizations, but the RAID organization may be hidden from the server computers: the disk subsystems pretend each RAID system is a very large and very reliable disk. The controller and the disk continue to use SCSI or Fibre Channel interfaces to talk with each other, although they may be separated by a network. Remote access to disks across a storage area network means that disks can be shared by multiple computers, which could run different parts of an application in parallel. Remote access also means that disks containing important data can be kept in a central server room where they can be monitored and maintained by system administrators, instead of being scattered in different parts of an organization.

11.2.2 Performance Measures of Disks

The main measures of the qualities of a disk are capacity, access time, data-transfer rate, and reliability.

Access time is the time from when a read or write request is issued to when data transfer begins. To access (that is, to read or write) data on a given sector of a disk, the arm first must move so that it is positioned over the correct track, and then must wait for the sector to appear under it as the disk rotates. The time for repositioning the arm is called the **seek time**, and it increases with the distance that the arm must move. Typical seek times range from 2 to 30 milliseconds, depending on how far the track is from the initial arm position. Smaller disks tend to have lower seek times since the head has to travel a smaller distance.

The **average seek time** is the average of the seek times, measured over a sequence of (uniformly distributed) random requests. If all tracks have the same number of sectors, and we disregard the time required for the head to start moving and to stop moving, we can show that the average seek time is one-third the worst case seek time. Taking these factors into account, the average seek time is around one-half of the maximum seek time. Average seek times currently range between 4 milliseconds and 10 milliseconds, depending on the disk model.

Once the seek has started, the time spent waiting for the sector to be accessed to appear under the head is called the **rotational latency time**. Rotational speeds of disks today range from 5400 rotations per minute (90 rotations per second) up to 15,000 rotations per minute (250 rotations per second), or, equivalently, 4 milliseconds to 11.1 milliseconds per rotation. On an average, one-half of a rotation of the disk is required for the beginning of the desired sector to appear under the head. Thus, the **average latency time** of the disk is one-half the time for a full rotation of the disk.

The access time is then the sum of the seek time and the latency, and ranges from 8 to 20 milliseconds. Once the first sector of the data to be accessed has come under

the head, data transfer begins. The **data-transfer rate** is the rate at which data can be retrieved from or stored to the disk. Current disk systems claim to support maximum transfer rates of about 25 to 40 megabytes per second, although actual transfer rates may be significantly less, at about 4 to 8 megabytes per second.

The final commonly used measure of a disk is the **mean time to failure (MTTF)**, which is a measure of the reliability of the disk. The mean time to failure of a disk (or of any other system) is the amount of time that, on average, we can expect the system to run continuously without any failure. According to vendors' claims, the mean time to failure of disks today ranges from 30,000 to 1,200,000 hours—about 3.4 to 136 years. In practice the claimed mean time to failure is computed on the probability of failure when the disk is new—the figure means that given 1000 relatively new disks, if the MTTF is 1,200,000 hours, on an average one of them will fail in 1200 hours. A mean time to failure of 1,200,000 hours does not imply that the disk can be expected to function for 136 years! Most disks have an expected life span of about 5 years, and have significantly higher rates of failure once they become more than a few years old.

There may be multiple disks sharing a disk interface. The widely used ATA-4 interface standard (also called Ultra-DMA) supports 33 megabytes per second transfer rates, while ATA-5 supports 66 megabytes per second. SCSI-3 (Ultra2 wide SCSI) supports 40 megabytes per second, while the more expensive Fibre Channel interface supports up to 256 megabytes per second. The transfer rate of the interface is shared between all disks attached to the interface.

11.2.3 Optimization of Disk-Block Access

Requests for disk I/O are generated both by the file system and by the virtual memory manager found in most operating systems. Each request specifies the address on the disk to be referenced; that address is in the form of a *block number*. A **block** is a contiguous sequence of sectors from a single track of one platter. Block sizes range from 512 bytes to several kilobytes. Data are transferred between disk and main memory in units of blocks. The lower levels of the file-system manager convert block addresses into the hardware-level cylinder, surface, and sector number.

Since access to data on disk is several orders of magnitude slower than access to data in main memory, equipment designers have focused on techniques for improving the speed of access to blocks on disk. One such technique, buffering of blocks in memory to satisfy future requests, is discussed in Section 11.5. Here, we discuss several other techniques.

- **Scheduling**. If several blocks from a cylinder need to be transferred from disk to main memory, we may be able to save access time by requesting the blocks in the order in which they will pass under the heads. If the desired blocks are on different cylinders, it is advantageous to request the blocks in an order that minimizes disk-arm movement. **Disk-arm–scheduling** algorithms attempt to order accesses to tracks in a fashion that increases the number of accesses that can be processed. A commonly used algorithm is the **elevator algorithm**, which works in the same way many elevators do. Suppose that, initially, the arm is moving from the innermost track toward the outside of the disk. Under the elevator algorithms control, for each track for which there

is an access request, the arm stops at that track, services requests for the track, and then continues moving outward until there are no waiting requests for tracks farther out. At this point, the arm changes direction, and moves toward the inside, again stopping at each track for which there is a request, until it reaches a track where there is no request for tracks farther toward the center. Now, it reverses direction and starts a new cycle. Disk controllers usually perform the task of reordering read requests to improve performance, since they are intimately aware of the organization of blocks on disk, of the rotational position of the disk platters, and of the position of the disk arm.

- **File organization**. To reduce block-access time, we can organize blocks on disk in a way that corresponds closely to the way we expect data to be accessed. For example, if we expect a file to be accessed sequentially, then we should ideally keep all the blocks of the file sequentially on adjacent cylinders. Older operating systems, such as the IBM mainframe operating systems, provided programmers fine control on placement of files, allowing a programmer to reserve a set of cylinders for storing a file. However, this control places a burden on the programmer or system administrator to decide, for example, how many cylinders to allocate for a file, and may require costly reorganization if data are inserted to or deleted from the file.

  Subsequent operating systems, such as Unix and personal-computer operating systems, hide the disk organization from users, and manage the allocation internally. However, over time, a sequential file may become **fragmented**; that is, its blocks become scattered all over the disk. To reduce fragmentation, the system can make a backup copy of the data on disk and restore the entire disk. The restore operation writes back the blocks of each file contiguously (or nearly so). Some systems (such as different versions of the Windows operating system) have utilities that scan the disk and then move blocks to decrease the fragmentation. The performance increases realized from these techniques can be large, but the system is generally unusable while these utilities operate.

- **Nonvolatile write buffers**. Since the contents of main memory are lost in a power failure, information about database updates has to be recorded on disk to survive possible system crashes. For this reason, the performance of update-intensive database applications, such as transaction-processing systems, is heavily dependent on the speed of disk writes.

  We can use **nonvolatile random-access memory** (NV-RAM) to speed up disk writes drastically. The contents of nonvolatile RAM are not lost in power failure. A common way to implement nonvolatile RAM is to use battery–backed-up RAM. The idea is that, when the database system (or the operating system) requests that a block be written to disk, the disk controller writes the block to a nonvolatile RAM buffer, and immediately notifies the operating system that the write completed successfully. The controller writes the data to their destination on disk whenever the disk does not have any other requests, or when the nonvolatile RAM buffer becomes full. When the database system requests a block write, it notices a delay only if the nonvolatile RAM buffer

is full. On recovery from a system crash, any pending buffered writes in the nonvolatile RAM are written back to the disk.

An example illustrates how much nonvolatile RAM improves performance. Assume that write requests are received in a random fashion, with the disk being busy on average 90 percent of the time.[1] If we have a nonvolatile RAM buffer of 50 blocks, then, on average, only once per minute will a write find the buffer to be full (and therefore have to wait for a disk write to finish). Doubling the buffer to 100 blocks results in approximately only one write per hour finding the buffer to be full. Thus, in most cases, disk writes can be executed without the database system waiting for a seek or rotational latency.

- **Log disk**. Another approach to reducing write latencies is to use a log disk—that is, a disk devoted to writing a sequential log—in much the same way as a nonvolatile RAM buffer. All access to the log disk is sequential, essentially eliminating seek time, and several consecutive blocks can be written at once, making writes to the log disk several times faster than random writes. As before, the data have to be written to their actual location on disk as well, but the log disk can do the write later, without the database system having to wait for the write to complete. Furthermore, the log disk can reorder the writes to minimize disk arm movement. If the system crashes before some writes to the actual disk location have completed, when the system comes back up it reads the log disk to find those writes that had not been completed, and carries them out then.

  File systems that support log disks as above are called **journaling file systems**. Journaling file systems can be implemented even without a separate log disk, keeping data and the log on the same disk. Doing so reduces the monetary cost, at the expense of lower performance.

  The **log-based file system** is an extreme version of the log-disk approach. Data are not written back to their original destination on disk; instead, the file system keeps track of where in the log disk the blocks were written most recently, and retrieves them from that location. The log disk itself is compacted periodically, so that old writes that have subsequently been overwritten can be removed. This approach improves write performance, but generates a high degree of fragmentation for files that are updated often. As we noted earlier, such fragmentation increases seek time for sequential reading of files.

11.3 RAID

The data storage requirements of some applications (in particular Web, database, and multimedia data applications) have been growing so fast that a large number of disks are needed to store data for such applications, even though disk drive capacities have been growing very fast.

1. For the statistically inclined reader, we assume Poisson distribution of arrivals. The exact arrival rate and rate of service are not needed since the disk utilization provides enough information for our calculations.

Having a large number of disks in a system presents opportunities for improving the rate at which data can be read or written, if the disks are operated in parallel. Parallelism can also be used to perform several independent reads or writes in parallel. Furthermore, this setup offers the potential for improving the reliability of data storage, because redundant information can be stored on multiple disks. Thus, failure of one disk does not lead to loss of data.

A variety of disk-organization techniques, collectively called **redundant arrays of independent disks (RAID)**, have been proposed to achieve improved performance and reliability.

In the past, system designers viewed storage systems composed of several small cheap disks as a cost-effective alternative to using large, expensive disks; the cost per megabyte of the smaller disks was less than that of larger disks. In fact, the I in RAID, which now stands for *independent*, originally stood for *inexpensive*. Today, however, all disks are physically small, and larger-capacity disks actually have a lower cost per megabyte. RAID systems are used for their higher reliability and higher performance rate, rather than for economic reasons.

11.3.1 Improvement of Reliability via Redundancy

Let us first consider reliability. The chance that some disk out of a set of N disks will fail is much higher than the chance that a specific single disk will fail. Suppose that the mean time to failure of a disk is 100,000 hours, or slightly over 11 years. Then, the mean time to failure of some disk in an array of 100 disks will be 100,000 / 100 = 1000 hours, or around 42 days, which is not long at all! If we store only one copy of the data, then each disk failure will result in loss of a significant amount of data (as discussed in Section 11.2.1). Such a high rate of data loss is unacceptable.

The solution to the problem of reliability is to introduce **redundancy**; that is, we store extra information that is not needed normally, but that can be used in the event of failure of a disk to rebuild the lost information. Thus, even if a disk fails, data are not lost, so the effective mean time to failure is increased, provided that we count only failures that lead to loss of data or to nonavailability of data.

The simplest (but most expensive) approach to introducing redundancy is to duplicate every disk. This technique is called **mirroring** (or, sometimes, *shadowing*). A logical disk then consists of two physical disks, and every write is carried out on both disks. If one of the disks fails, the data can be read from the other. Data will be lost only if the second disk fails before the first failed disk is repaired.

The mean time to failure (where failure is the loss of data) of a mirrored disk depends on the mean time to failure of the individual disks, as well as on the **mean time to repair**, which is the time it takes (on an average) to replace a failed disk and to restore the data on it. Suppose that the failures of the two disks are *independent*; that is, there is no connection between the failure of one disk and the failure of the other. Then, if the mean time to failure of a single disk is 100,000 hours, and the mean time to repair is 10 hours, then the **mean time to data loss** of a mirrored disk system is $100000^2/(2 * 10) = 500 * 10^6$ hours, or 57,000 years! (We do not go into the derivations here; references in the bibliographical notes provide the details.)

You should be aware that the assumption of independence of disk failures is not valid. Power failures, and natural disasters such as earthquakes, fires, and floods, may result in damage to both disks at the same time. As disks age, the probability of failure increases, increasing the chance that a second disk will fail while the first is being repaired. In spite of all these considerations, however, mirrored-disk systems offer much higher reliability than do single-disk systems. Mirrored-disk systems with mean time to data loss of about 500,000 to 1,000,000 hours, or 55 to 110 years, are available today.

Power failures are a particular source of concern, since they occur far more frequently than do natural disasters. Power failures are not a concern if there is no data transfer to disk in progress when they occur. However, even with mirroring of disks, if writes are in progress to the same block in both disks, and power fails before both blocks are fully written, the two blocks can be in an inconsistent state. The solution to this problem is to write one copy first, then the next, so that one of the two copies is always consistent. Some extra actions are required when we restart after a power failure, to recover from incomplete writes. This matter is examined in Exercise 11.4.

11.3.2 Improvement in Performance via Parallelism

Now let us consider the benefit of parallel access to multiple disks. With disk mirroring, the rate at which read requests can be handled is doubled, since read requests can be sent to either disk (as long as both disks in a pair are functional, as is almost always the case). The transfer rate of each read is the same as in a single-disk system, but the number of reads per unit time has doubled.

With multiple disks, we can improve the transfer rate as well (or instead) by **striping data** across multiple disks. In its simplest form, data striping consists of splitting the bits of each byte across multiple disks; such striping is called **bit-level striping**. For example, if we have an array of eight disks, we write bit i of each byte to disk i. The array of eight disks can be treated as a single disk with sectors that are eight times the normal size, and, more important, that has eight times the transfer rate. In such an organization, every disk participates in every access (read or write), so the number of accesses that can be processed per second is about the same as on a single disk, but each access can read eight times as many data in the same time as on a single disk. Bit-level striping can be generalized to a number of disks that either is a multiple of 8 or a factor of 8. For example, if we use an array of four disks, bits i and $4 + i$ of each byte go to disk i.

Block-level striping stripes blocks across multiple disks. It treats the array of disks as a single large disk, and it gives blocks logical numbers; we assume the block numbers start from 0. With an array of n disks, block-level striping assigns logical block i of the disk array to disk $(i \bmod n) + 1$; it uses the $\lfloor i/n \rfloor$th physical block of the disk to store logical block i. For example, with 8 disks, logical block 0 is stored in physical block 0 of disk 1, while logical block 11 is stored in physical block 1 of disk 4. When reading a large file, block-level striping fetches n blocks at a time in parallel from the n disks, giving a high data transfer rate for large reads. When a single block is read, the data transfer rate is the same as on one disk, but the remaining $n - 1$ disks are free to perform other actions.

Block level striping is the most commonly used form of data striping. Other levels of striping, such as bytes of a sector or sectors of a block also are possible.

In summary, there are two main goals of parallelism in a disk system:

1. Load-balance multiple small accesses (block accesses), so that the throughput of such accesses increases.
2. Parallelize large accesses so that the response time of large accesses is reduced.

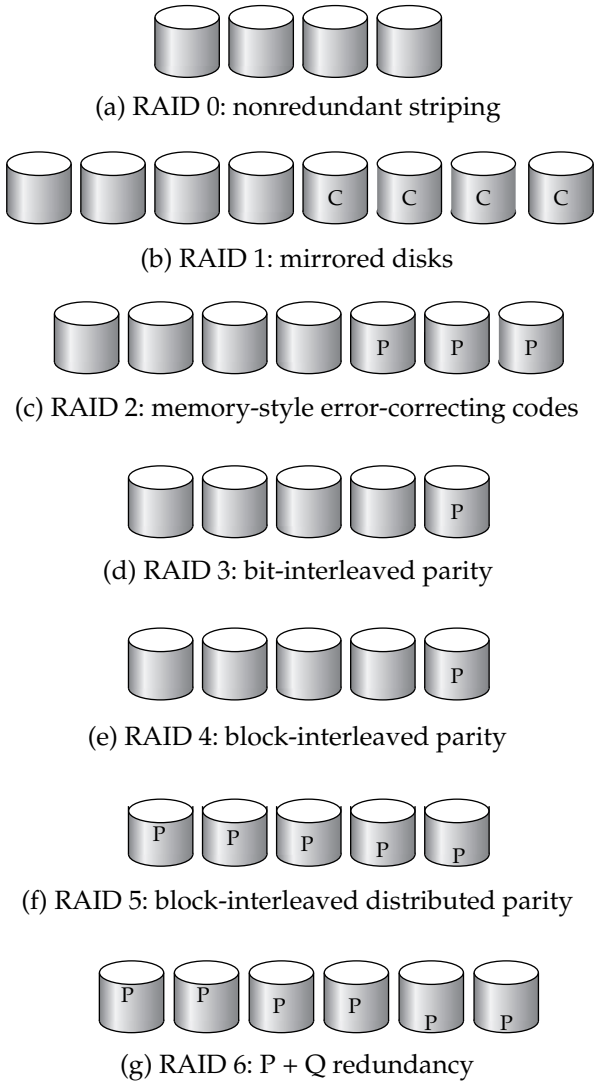
11.3.3 RAID Levels

Mirroring provides high reliability, but it is expensive. Striping provides high data-transfer rates, but does not improve reliability. Various alternative schemes aim to provide redundancy at lower cost by combining disk striping with "parity" bits (which we describe next). These schemes have different cost–performance trade-offs. The schemes are classified into **RAID levels**, as in Figure 11.4. (In the figure, P indicates error-correcting bits, and C indicates a second copy of the data.) For all levels, the figure depicts four disk's worth of data, and the extra disks depicted are used to store redundant information for failure recovery.

- **RAID level 0** refers to disk arrays with striping at the level of blocks, but without any redundancy (such as mirroring or parity bits). Figure 11.4a shows an array of size 4.
- **RAID level 1** refers to disk mirroring with block striping. Figure 11.4b shows a mirrored organization that holds four disks worth of data.
- **RAID level 2**, known as memory-style error-correcting-code (ECC) organization, employs parity bits. Memory systems have long used parity bits for error detection and correction. Each byte in a memory system may have a parity bit associated with it that records whether the numbers of bits in the byte that are set to 1 is even (parity = 0) or odd (parity = 1). If one of the bits in the byte gets damaged (either a 1 becomes a 0, or a 0 becomes a 1), the parity of the byte changes and thus will not match the stored parity. Similarly, if the stored parity bit gets damaged, it will not match the computed parity. Thus, all 1-bit errors will be detected by the memory system. Error-correcting schemes store 2 or more extra bits, and can reconstruct the data if a single bit gets damaged.

  The idea of error-correcting codes can be used directly in disk arrays by striping bytes across disks. For example, the first bit of each byte could be stored in disk 1, the second bit in disk 2, and so on until the eighth bit is stored in disk 8, and the error-correction bits are stored in further disks.

  Figure 11.4c shows the level 2 scheme. The disks labeled P store the error-correction bits. If one of the disks fails, the remaining bits of the byte and the associated error-correction bits can be read from other disks, and can be used to reconstruct the damaged data. Figure 11.4c shows an array of size 4; note RAID level 2 requires only three disks' overhead for four disks of data, unlike RAID level 1, which required four disks' overhead.



(a) RAID 0: nonredundant striping

(b) RAID 1: mirrored disks

(c) RAID 2: memory-style error-correcting codes

(d) RAID 3: bit-interleaved parity

(e) RAID 4: block-interleaved parity

(f) RAID 5: block-interleaved distributed parity

(g) RAID 6: P + Q redundancy

Figure 11.4 RAID levels.

- **RAID level 3**, bit-interleaved parity organization, improves on level 2 by exploiting the fact that disk controllers, unlike memory systems, can detect whether a sector has been read correctly, so a single parity bit can be used for error correction, as well as for detection. The idea is as follows. If one of the sectors gets damaged, the system knows exactly which sector it is, and, for each bit in the sector, the system can figure out whether it is a 1 or a 0 by computing the parity of the corresponding bits from sectors in the other disks. If the parity of the remaining bits is equal to the stored parity, the missing bit is 0; otherwise, it is 1.

RAID level 3 is as good as level 2, but is less expensive in the number of extra disks (it has only a one-disk overhead), so level 2 is not used in practice. Figure 11.4d shows the level 3 scheme.

RAID level 3 has two benefits over level 1. It needs only one parity disk for several regular disks, whereas Level 1 needs one mirror disk for every disk, and thus reduces the storage overhead. Since reads and writes of a byte are spread out over multiple disks, with N-way striping of data, the transfer rate for reading or writing a single block is N times faster than a RAID level 1 organization using N-way striping. On the other hand, RAID level 3 supports a lower number of I/O operations per second, since every disk has to participate in every I/O request.

- **RAID level 4**, block-interleaved parity organization, uses block level striping, like RAID 0, and in addition keeps a parity block on a separate disk for corresponding blocks from N other disks. This scheme is shown pictorially in Figure 11.4e. If one of the disks fails, the parity block can be used with the corresponding blocks from the other disks to restore the blocks of the failed disk.

  A block read accesses only one disk, allowing other requests to be processed by the other disks. Thus, the data-transfer rate for each access is slower, but multiple read accesses can proceed in parallel, leading to a higher overall I/O rate. The transfer rates for large reads is high, since all the disks can be read in parallel; large writes also have high transfer rates, since the data and parity can be written in parallel.

  Small independent writes, on the other hand, cannot be performed in parallel. A write of a block has to access the disk on which the block is stored, as well as the parity disk, since the parity block has to be updated. Moreover, both the old value of the parity block and the old value of the block being written have to be read for the new parity to be computed. Thus, a single write requires four disk accesses: two to read the two old blocks, and two to write the two blocks.

- **RAID level 5**, block-interleaved distributed parity, improves on level 4 by partitioning data and parity among all $N + 1$ disks, instead of storing data in N disks and parity in one disk. In level 5, all disks can participate in satisfying read requests, unlike RAID level 4, where the parity disk cannot participate, so level 5 increases the total number of requests that can be met in a given amount of time. For each set of N logical blocks, one of the disks stores the parity, and the other N disks store the blocks.

  Figure 11.4f shows the setup. The P's are distributed across all the disks. For example, with an array of 5 disks, the parity block, labelled Pk, for logical blocks $4k, 4k+1, 4k+2, 4k+3$ is stored in disk $(k \bmod 5) + 1$; the corresponding blocks of the other four disks store the 4 data blocks $4k$ to $4k + 3$. The following table indicates how the first 20 blocks, numbered 0 to 19, and their parity blocks are laid out. The pattern shown gets repeated on further blocks.

P0	0	1	2	3
4	P1	5	6	7
8	9	P2	10	11
12	13	14	P3	15
16	17	18	19	P4

Note that a parity block cannot store parity for blocks in the same disk, since then a disk failure would result in loss of data as well as of parity, and hence would not be recoverable. Level 5 subsumes level 4, since it offers better read–write performance at the same cost, so level 4 is not used in practice.

- **RAID level 6**, the P + Q redundancy scheme, is much like RAID level 5, but stores extra redundant information to guard against multiple disk failures. Instead of using parity, level 6 uses error-correcting codes such as the Reed–Solomon codes (see the bibliographical notes). In the scheme in Figure 11.4g, 2 bits of redundant data are stored for every 4 bits of data—unlike 1 parity bit in level 5—and the system can tolerate two disk failures.

Finally, we note that several variations have been proposed to the basic RAID schemes described here.

Some vendors use their own terminology to describe their RAID implementations.[2] However, the terminology we have presented is the most widely used.

11.3.4 Choice of RAID Level

The factors to be taken into account when choosing a RAID level are

- Monetary cost of extra disk storage requirements
- Performance requirements in terms of number of I/O operations
- Performance when a disk has failed
- Performance during rebuild (that is, while the data in a failed disk is being rebuilt on a new disk)

The time to rebuild the data of a failed disk can be significant, and varies with the RAID level that is used. Rebuilding is easiest for RAID level 1, since data can be copied from another disk; for the other levels, we need to access all the other disks in the array to rebuild data of a failed disk. The **rebuild performance** of a RAID system may be an important factor if continuous availability of data is required, as it is in high-performance database systems. Furthermore, since rebuild time can form a significant part of the repair time, rebuild performance also influences the mean time to data loss.

2. For example, some products use RAID level 1 to refer to mirroring without striping, and level 1+0 or level 10 to refer to mirroring with striping. Such a distinction is not really necessary since not striping can simply be viewed as a special case of striping, namely striping across 1 disk.

RAID level 0 is used in high-performance applications where data safety is not critical. Since RAID levels 2 and 4 are subsumed by RAID levels 3 and 5, the choice of RAID levels is restricted to the remaining levels. Bit striping (level 3) is rarely used since block striping (level 5) gives as good data transfer rates for large transfers, while using fewer disks for small transfers. For small transfers, the disk access time dominates anyway, so the benefit of parallel reads diminishes. In fact, level 3 may perform worse than level 5 for a small transfer, since the transfer completes only when corresponding sectors on all disks have been fetched; the average latency for the disk array thus becomes very close to the worst-case latency for a single disk, negating the benefits of higher transfer rates. Level 6 is not supported currently by many RAID implementations, but it offers better reliability than level 5 and can be used in applications where data safety is very important.

The choice between RAID level 1 and level 5 is harder to make. RAID level 1 is popular for applications such as storage of log files in a database system, since it offers the best write performance. RAID level 5 has a lower storage overhead than level 1, but has a higher time overhead for writes. For applications where data are read frequently, and written rarely, level 5 is the preferred choice.

Disk storage capacities have been growing at a rate of over 50 percent per year for many years, and the cost per byte has been falling at the same rate. As a result, for many existing database applications with moderate storage requirements, the monetary cost of the extra disk storage needed for mirroring has become relatively small (the extra monetary cost, however, remains a significant issue for storage-intensive applications such as video data storage). Access speeds have improved at a much slower rate (around a factor of 3 over 10 years), while the number of I/O operations required per second has increased tremendously, particularly for Web application servers.

RAID level 5, which increases the number of I/O operations needed to write a single logical block, pays a significant time penalty in terms of write performance. RAID level 1 is therefore the RAID level of choice for many applications with moderate storage requirements, and high I/O requirements.

RAID system designers have to make several other decisions as well. For example, how many disks should there be in an array? How many bits should be protected by each parity bit? If there are more disks in an array, data-transfer rates are higher, but the system would be more expensive. If there are more bits protected by a parity bit, the space overhead due to parity bits is lower, but there is an increased chance that a second disk will fail before the first failed disk is repaired, and that will result in data loss.

11.3.5 Hardware Issues

Another issue in the choice of RAID implementations is at the level of hardware. RAID can be implemented with no change at the hardware level, using only software modification. Such RAID implementations are called **software RAID**. However, there are significant benefits to be had by building special-purpose hardware to support RAID, which we outline below; systems with special hardware support are called **hardware RAID** systems.

Hardware RAID implementations can use nonvolatile RAM to record writes that need to be executed; in case of power failure before a write is completed, when the system comes back up, it retrieves information about incomplete writes from nonvolatile RAM and then completes the writes. Without such hardware support, extra work needs to be done to detect blocks that may have been partially written before power failure (see Exercise 11.4).

Some hardware RAID implementations permit **hot swapping**; that is, faulty disks can be removed and replaced by new ones without turning power off. Hot swapping reduces the mean time to repair, since replacement of a disk does not have to wait until a time when the system can be shut down. In fact many critical systems today run on a 24 × 7 schedule; that is, they run 24 hours a day, 7 days a week, providing no time for shutting down and replacing a failed disk. Further, many RAID implementations assign a spare disk for each array (or for a set of disk arrays). If a disk fails, the spare disk is immediately used as a replacement. As a result, the mean time to repair is reduced greatly, minimizing the chance of any data loss. The failed disk can be replaced at leisure.

The power supply, or the disk controller, or even the system interconnection in a RAID system could become a single point of failure, that could stop functioning of the RAID system. To avoid this possibility, good RAID implementations have multiple redundant power supplies (with battery backups so they continue to function even if power fails). Such RAID systems have multiple disk controllers, and multiple interconnections to connect them to the computer system (or to a network of computer systems). Thus, failure of any single component will not stop the functioning of the RAID system.

11.3.6 Other RAID Applications

The concepts of RAID have been generalized to other storage devices, including arrays of tapes, and even to the broadcast of data over wireless systems. When applied to arrays of tapes, the RAID structures are able to recover data even if one of the tapes in an array of tapes is damaged. When applied to broadcast of data, a block of data is split into short units and is broadcast along with a parity unit; if one of the units is not received for any reason, it can be reconstructed from the other units.

11.4 Tertiary Storage

In a large database system, some of the data may have to reside on tertiary storage. The two most common tertiary storage media are optical disks and magnetic tapes.

11.4.1 Optical Disks

Compact disks are a popular medium for distributing software, multimedia data such as audio and images, and other electronically published information. They have a fairly large capacity (640 megabytes), and they are cheap to mass-produce.

Digital video disks (DVDs) are replacing compact disks in applications that require very large amounts of data. Disks in the DVD-5 format can store 4.7 gigabytes of data

(in one recording layer), while disks in the DVD-9 format can store 8.5 gigabytes of data (in two recording layers). Recording on both sides of a disk yields even larger capacities; DVD-10 and DVD-18 formats, which are the two-sided versions of DVD-5 and DVD-9, can store 9.4 gigabytes and 17 gigabytes respectively.

CD and DVD drives have much longer seek times (100 milliseconds is common) than do magnetic-disk drives, since the head assembly is heavier. Rotational speeds are typically lower than those of magnetic disks, although the faster CD and DVD drives have rotation speeds of about 3000 rotations per minute, which is comparable to speeds of lower-end magnetic-disk drives. Rotational speeds of CD drives originally corresponded to the audio CD standards, and the speeds of DVD drives originally corresponded to the DVD video standards, but current-generation drives rotate at many times the standard rate.

Data transfer rates are somewhat less than for magnetic disks. Current CD drives read at around 3 to 6 megabytes per second, and current DVD drives read at 8 to 15 megabytes per second. Like magnetic disk drives, optical disks store more data in outside tracks and less data in inner tracks. The transfer rate of optical drives is characterized as $n\times$, which means the drive supports transfers at n times the standard rate; rates of around $50\times$ for CD and $12\times$ for DVD are now common.

The record-once versions of optical disks (CD-R, and increasingly, DVD-R) are popular for distribution of data and particularly for archival storage of data because they have a high capacity, have a longer lifetime than magnetic disks, and can be removed and stored at a remote location. Since they cannot be overwritten, they can be used to store information that should not be modified, such as audit trails. The multiple-write versions (CD-RW, DVD-RW, and DVD-RAM) are also used for archival purposes.

Jukeboxes are devices that store a large number of optical disks (up to several hundred) and load them automatically on demand to one of a small number (usually, 1 to 10) of drives. The aggregate storage capacity of such a system can be many terabytes. When a disk is accessed, it is loaded by a mechanical arm from a rack onto a drive (any disk that was already in the drive must first be placed back on the rack). The disk load/unload time is usually of the order of a few seconds—very much slower than disk access times.

11.4.2 Magnetic Tapes

Although magnetic tapes are relatively permanent, and can hold large volumes of data, they are slow in comparison to magnetic and optical disks. Even more important, magnetic tapes are limited to sequential access. Thus, they cannot provide random access for secondary-storage requirements, although historically, prior to the use of magnetic disks, tapes were used as a secondary-storage medium.

Tapes are used mainly for backup, for storage of infrequently used information, and as an offline medium for transferring information from one system to another. Tapes are also used for storing large volumes of data, such as video or image data, that either do not need to be accessible quickly or are so voluminous that magnetic-disk storage would be too expensive.

A tape is kept in a spool, and is wound or rewound past a read–write head. Moving to the correct spot on a tape can take seconds or even minutes, rather than

milliseconds; once positioned, however, tape drives can write data at densities and speeds approaching those of disk drives. Capacities vary, depending on the length and width of the tape and on the density at which the head can read and write. The market is currently fragmented among a wide variety of tape formats. Currently available tape capacities range from a few gigabytes [with the **Digital Audio Tape** (DAT) format], 10 to 40 gigabytes [with the **Digital Linear Tape** (DLT) format], 100 gigabytes and higher (with the **Ultrium** format), to 330 gigabytes (with **Ampex helical scan** tape formats). Data transfer rates are of the order of a few to tens of megabytes per second.

Tape devices are quite reliable, and good tape drive systems perform a read of the just-written data to ensure that it has been recorded correctly. Tapes, however, have limits on the number of times that they can be read or written reliably.

Some tape formats (such as the **Accelis** format) support faster seek times (of the order of tens of seconds), which is important for applications that need quick access to very large amounts of data, larger than what would fit economically on a disk drive. Most other tape formats provide larger capacities, at the cost of slower access; such formats are ideal for data backup, where fast seeks are not important.

Tape jukeboxes, like optical disk jukeboxes, hold large numbers of tapes, with a few drives onto which the tapes can be mounted; they are used for storing large volumes of data, ranging up to many terabytes (10^{12} bytes), with access times on the order of seconds to a few minutes. Applications that need such enormous data storage include imaging systems that gather data by remote-sensing satellites, and large video libraries for television broadcasters.

11.5 Storage Access

A database is mapped into a number of different files, which are maintained by the underlying operating system. These files reside permanently on disks, with backups on tapes. Each file is partitioned into fixed-length storage units called **blocks**, which are the units of both storage allocation and data transfer. We shall discuss in Section 11.6 various ways to organize the data logically in files.

A block may contain several data items. The exact set of data items that a block contains is determined by the form of physical data organization being used (see Section 11.6). We shall assume that no data item spans two or more blocks. This assumption is realistic for most data-processing applications, such as our banking example.

A major goal of the database system is to minimize the number of block transfers between the disk and memory. One way to reduce the number of disk accesses is to keep as many blocks as possible in main memory. The goal is to maximize the chance that, when a block is accessed, it is already in main memory, and, thus, no disk access is required.

Since it is not possible to keep all blocks in main memory, we need to manage the allocation of the space available in main memory for the storage of blocks. The **buffer** is that part of main memory available for storage of copies of disk blocks. There is always a copy kept on disk of every block, but the copy on disk may be a version

of the block older than the version in the buffer. The subsystem responsible for the allocation of buffer space is called the **buffer manager**.

11.5.1 Buffer Manager

Programs in a database system make requests (that is, calls) on the buffer manager when they need a block from disk. If the block is already in the buffer, the buffer manager passes the address of the block in main memory to the requester. If the block is not in the buffer, the buffer manager first allocates space in the buffer for the block, throwing out some other block, if necessary, to make space for the new block. The thrown-out block is written back to disk only if it has been modified since the most recent time that it was written to the disk. Then, the buffer manager reads in the requested block from the disk to the buffer, and passes the address of the block in main memory to the requester. The internal actions of the buffer manager are transparent to the programs that issue disk-block requests.

If you are familiar with operating-system concepts, you will note that the buffer manager appears to be nothing more than a virtual-memory manager, like those found in most operating systems. One difference is that the size of the database may be much more than the hardware address space of a machine, so memory addresses are not sufficient to address all disk blocks. Further, to serve the database system well, the buffer manager must use techniques more sophisticated than typical virtual-memory management schemes:

- **Buffer replacement strategy**. When there is no room left in the buffer, a block must be removed from the buffer before a new one can be read in. Most operating systems use a **least recently used** (**LRU**) scheme, in which the block that was referenced least recently is written back to disk and is removed from the buffer. This simple approach can be improved on for database applications.
- **Pinned blocks**. For the database system to be able to recover from crashes (Chapter 17), it is necessary to restrict those times when a block may be written back to disk. For instance, most recovery systems require that a block should not be written to disk while an update on the block is in progress. A block that is not allowed to be written back to disk is said to be **pinned**. Although many operating systems do not support pinned blocks, such a feature is essential for a database system that is resilient to crashes.
- **Forced output of blocks**. There are situations in which it is necessary to write back the block to disk, even though the buffer space that it occupies is not needed. This write is called the **forced output** of a block. We shall see the reason for forced output in Chapter 17; briefly, main-memory contents and thus buffer contents are lost in a crash, whereas data on disk usually survive a crash.

11.5.2 Buffer-Replacement Policies

The goal of a replacement strategy for blocks in the buffer is to minimize accesses to the disk. For general-purpose programs, it is not possible to predict accurately

```
for each tuple b of borrower do
    for each tuple c of customer do
        if b[customer-name] = c[customer-name]
        then begin
                let x be a tuple defined as follows:
                x[customer-name] := b[customer-name]
                x[loan-number] := b[loan-number]
                x[customer-street] := c[customer-street]
                x[customer-city] := c[customer-city]
                include tuple x as part of result of borrower ⋈ customer
             end
    end
end
```

Figure 11.5 Procedure for computing join.

which blocks will be referenced. Therefore, operating systems use the past pattern of block references as a predictor of future references. The assumption generally made is that blocks that have been referenced recently are likely to be referenced again. Therefore, if a block must be replaced, the least recently referenced block is replaced. This approach is called the **least recently used** (**LRU**) block-replacement scheme.

LRU is an acceptable replacement scheme in operating systems. However, a database system is able to predict the pattern of future references more accurately than an operating system. A user request to the database system involves several steps. The database system is often able to determine in advance which blocks will be needed by looking at each of the steps required to perform the user-requested operation. Thus, unlike operating systems, which must rely on the past to predict the future, database systems may have information regarding at least the short-term future.

To illustrate how information about future block access allows us to improve the LRU strategy, consider the processing of the relational-algebra expression

$$borrower \bowtie customer$$

Assume that the strategy chosen to process this request is given by the pseudocode program shown in Figure 11.5. (We shall study other strategies in Chapter 13.)

Assume that the two relations of this example are stored in separate files. In this example, we can see that, once a tuple of *borrower* has been processed, that tuple is not needed again. Therefore, once processing of an entire block of *borrower* tuples is completed, that block is no longer needed in main memory, even though it has been used recently. The buffer manager should be instructed to free the space occupied by a *borrower* block as soon as the final tuple has been processed. This buffer-management strategy is called the **toss-immediate** strategy.

Now consider blocks containing *customer* tuples. We need to examine every block of *customer* tuples once for each tuple of the *borrower* relation. When processing of a *customer* block is completed, we know that that block will not be accessed again until all other *customer* blocks have been processed. Thus, the most recently used *customer* block will be the final block to be re-referenced, and the least recently used

customer block is the block that will be referenced next. This assumption set is the exact opposite of the one that forms the basis for the LRU strategy. Indeed, the optimal strategy for block replacement is the **most recently used** (**MRU**) strategy. If a *customer* block must be removed from the buffer, the MRU strategy chooses the most recently used block.

For the MRU strategy to work correctly for our example, the system must pin the *customer* block currently being processed. After the final *customer* tuple has been processed, the block is unpinned, and it becomes the most recently used block.

In addition to using knowledge that the system may have about the request being processed, the buffer manager can use statistical information about the probability that a request will reference a particular relation. For example, the data dictionary that (as we will see in detail in Section 11.8) keeps track of the logical schema of the relations as well as their physical storage information is one of the most frequently accessed parts of the database. Thus, the buffer manager should try not to remove data-dictionary blocks from main memory, unless other factors dictate that it do so. In Chapter 12, we discuss indices for files. Since an index for a file may be accessed more frequently than the file itself, the buffer manager should, in general, not remove index blocks from main memory if alternatives are available.

The ideal database block-replacement strategy needs knowledge of the database operations—both those being performed and those that will be performed in the future. No single strategy is known that handles all the possible scenarios well. Indeed, a surprisingly large number of database systems use LRU, despite that strategy's faults. The exercises explore alternative strategies.

The strategy that the buffer manager uses for block replacement is influenced by factors other than the time at which the block will be referenced again. If the system is processing requests by several users concurrently, the concurrency-control subsystem (Chapter 16) may need to delay certain requests, to ensure preservation of database consistency. If the buffer manager is given information from the concurrency-control subsystem indicating which requests are being delayed, it can use this information to alter its block-replacement strategy. Specifically, blocks needed by active (nondelayed) requests can be retained in the buffer at the expense of blocks needed by the delayed requests.

The crash-recovery subsystem (Chapter 17) imposes stringent constraints on block replacement. If a block has been modified, the buffer manager is not allowed to write back the new version of the block in the buffer to disk, since that would destroy the old version. Instead, the block manager must seek permission from the crash-recovery subsystem before writing out a block. The crash-recovery subsystem may demand that certain other blocks be force-output before it grants permission to the buffer manager to output the block requested. In Chapter 17, we define precisely the interaction between the buffer manager and the crash-recovery subsystem.

11.6 File Organization

A **file** is organized logically as a sequence of records. These records are mapped onto disk blocks. Files are provided as a basic construct in operating systems, so we shall

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 2	A-215	Mianus	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700

Figure 11.6 File containing *account* records.

assume the existence of an underlying *file system*. We need to consider ways of representing logical data models in terms of files.

Although blocks are of a fixed size determined by the physical properties of the disk and by the operating system, record sizes vary. In a relational database, tuples of distinct relations are generally of different sizes.

One approach to mapping the database to files is to use several files, and to store records of only one fixed length in any given file. An alternative is to structure our files so that we can accommodate multiple lengths for records; however, files of fixed-length records are easier to implement than are files of variable-length records. Many of the techniques used for the former can be applied to the variable-length case. Thus, we begin by considering a file of fixed-length records.

11.6.1 Fixed-Length Records

As an example, let us consider a file of *account* records for our bank database. Each record of this file is defined as:

```
type deposit = record
                   account-number : char(10);
                   branch-name : char (22);
                   balance : real;
               end
```

If we assume that each character occupies 1 byte and that a real occupies 8 bytes, our *account* record is 40 bytes long. A simple approach is to use the first 40 bytes for the first record, the next 40 bytes for the second record, and so on (Figure 11.6). However, there are two problems with this simple approach:

1. It is difficult to delete a record from this structure. The space occupied by the record to be deleted must be filled with some other record of the file, or we must have a way of marking deleted records so that they can be ignored.

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600
record 8	A-218	Perryridge	700

Figure 11.7 File of Figure 11.6, with record 2 deleted and all records moved.

2. Unless the block size happens to be a multiple of 40 (which is unlikely), some records will cross block boundaries. That is, part of the record will be stored in one block and part in another. It would thus require two block accesses to read or write such a record.

When a record is deleted, we could move the record that came after it into the space formerly occupied by the deleted record, and so on, until every record following the deleted record has been moved ahead (Figure 11.7). Such an approach requires moving a large number of records. It might be easier simply to move the final record of the file into the space occupied by the deleted record (Figure 11.8).

It is undesirable to move records to occupy the space freed by a deleted record, since doing so requires additional block accesses. Since insertions tend to be more frequent than deletions, it is acceptable to leave open the space occupied by the deleted record, and to wait for a subsequent insertion before reusing the space. A simple marker on a deleted record is not sufficient, since it is hard to find this available space when an insertion is being done. Thus, we need to introduce an additional structure.

At the beginning of the file, we allocate a certain number of bytes as a **file header**. The header will contain a variety of information about the file. For now, all we need to store there is the address of the first record whose contents are deleted. We use this

record 0	A-102	Perryridge	400
record 1	A-305	Round Hill	350
record 8	A-218	Perryridge	700
record 3	A-101	Downtown	500
record 4	A-222	Redwood	700
record 5	A-201	Perryridge	900
record 6	A-217	Brighton	750
record 7	A-110	Downtown	600

Figure 11.8 File of Figure 11.6, with record 2 deleted and final record moved.

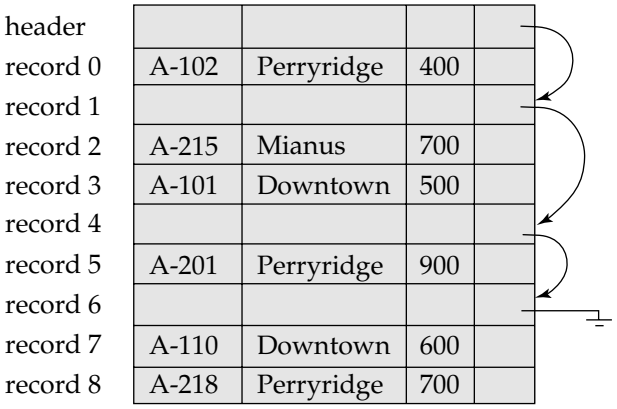


Figure 11.9 File of Figure 11.6, with free list after deletion of records 1, 4, and 6.

first record to store the address of the second available record, and so on. Intuitively, we can think of these stored addresses as *pointers*, since they point to the location of a record. The deleted records thus form a linked list, which is often referred to as a **free list**. Figure 11.9 shows the file of Figure 11.6, with the free list, after records 1, 4, and 6 have been deleted.

On insertion of a new record, we use the record pointed to by the header. We change the header pointer to point to the next available record. If no space is available, we add the new record to the end of the file.

Insertion and deletion for files of fixed-length records are simple to implement, because the space made available by a deleted record is exactly the space needed to insert a record. If we allow records of variable length in a file, this match no longer holds. An inserted record may not fit in the space left free by a deleted record, or it may fill only part of that space.

11.6.2 Variable-Length Records

Variable-length records arise in database systems in several ways:

- Storage of multiple record types in a file
- Record types that allow variable lengths for one or more fields
- Record types that allow repeating fields

Different techniques for implementing variable-length records exist. For purposes of illustration, we shall use one example to demonstrate the various implementation techniques. We shall consider a different representation of the *account* information stored in the file of Figure 11.6, in which we use one variable-length record for each branch name and for all the account information for that branch. The format of the record is

```
type account-list = record
                        branch-name : char (22);
                        account-info : array [1 .. ∞] of
                                           record;
                                               account-number : char(10);
                                               balance : real;
                                           end
                    end
```

We define *account-info* as an array with an arbitrary number of elements. That is, the type definition does not limit the number of elements in the array, although any actual record will have a specific number of elements in its array. There is no limit on how large a record can be (up to, of course, the size of the disk storage!).

11.6.2.1 Byte-String Representation

A simple method for implementing variable-length records is to attach a special *end-of-record* (⊥) symbol to the end of each record. We can then store each record as a string of consecutive bytes. Figure 11.10 shows such an organization to represent the file of fixed-length records of Figure 11.6 as variable-length records. An alternative version of the byte-string representation stores the record length at the beginning of each record, instead of using end-of-record symbols.

The byte-string representation as described in Figure 11.10 has some disadvantages:

- It is not easy to reuse space occupied formerly by a deleted record. Although techniques exist to manage insertion and deletion, they lead to a large number of small fragments of disk storage that are wasted.
- There is no space, in general, for records to grow longer. If a variable-length record becomes longer, it must be moved—movement is costly if pointers to the record are stored elsewhere in the database (e.g., in indices, or in other records), since the pointers must be located and updated.

Thus, the basic byte-string representation described here not usually used for implementing variable-length records. However, a modified form of the byte-string repre-

0	Perryridge	A-102	400	A-201	900	A-218	700	⊥
1	Round Hill	A-305	350	⊥				
2	Mianus	A-215	700	⊥				
3	Downtown	A-101	500	A-110	600	⊥		
4	Redwood	A-222	700	⊥				
5	Brighton	A-217	750	⊥				

Figure 11.10 Byte-string representation of variable-length records.

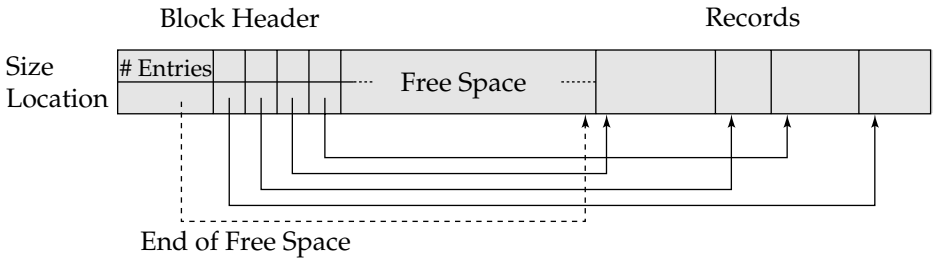


Figure 11.11 Slotted-page structure.

sentation, called the slotted-page structure, is commonly used for organizing records *within* a single block.

The **slotted-page structure** appears in Figure 11.11. There is a header at the beginning of each block, containing the following information:

1. The number of record entries in the header
2. The end of free space in the block
3. An array whose entries contain the location and size of each record

The actual records are allocated *contiguously* in the block, starting from the end of the block. The free space in the block is contiguous, between the final entry in the header array, and the first record. If a record is inserted, space is allocated for it at the end of free space, and an entry containing its size and location is added to the header.

If a record is deleted, the space that it occupies is freed, and its entry is set to deleted (its size is set to -1, for example). Further, the records in the block before the deleted record are moved, so that the free space created by the deletion gets occupied, and all free space is again between the final entry in the header array and the first record. The end-of-free-space pointer in the header is appropriately updated as well. Records can be grown or shrunk by similar techniques, as long as there is space in the block. The cost of moving the records is not too high, since the size of a block is limited: A typical value is 4 kilobytes.

The slotted-page structure requires that there be no pointers that point directly to records. Instead, pointers must point to the entry in the header that contains the actual location of the record. This level of indirection allows records to be moved to prevent fragmentation of space inside a block, while supporting indirect pointers to the record.

11.6.2.2 Fixed-Length Representation

Another way to implement variable-length records efficiently in a file system is to use one or more fixed-length records to represent one variable-length record.

There are two ways of doing this:

1. **Reserved space**. If there is a maximum record length that is never exceeded, we can use fixed-length records of that length. Unused space (for records

0	Perryridge	A-102	400	A-201	900	A-218	700
1	Round Hill	A-305	350	⊥	⊥	⊥	⊥
2	Mianus	A-215	700	⊥	⊥	⊥	⊥
3	Downtown	A-101	500	A-110	600	⊥	⊥
4	Redwood	A-222	700	⊥	⊥	⊥	⊥
5	Brighton	A-217	750	⊥	⊥	⊥	⊥

Figure 11.12 File of Figure 11.10, using the reserved-space method.

shorter than the maximum space) is filled with a special null, or end-of-record, symbol.

2. **List representation**. We can represent variable-length records by lists of fixed-length records, chained together by pointers.

If we choose to apply the reserved-space method to our account example, we need to select a maximum record length. Figure 11.12 shows how the file of Figure 11.10 would be represented if we allowed a maximum of three accounts per branch. A record in this file is of the *account-list* type, but with the array containing exactly three elements. Those branches with fewer than three accounts (for example, Round Hill) have records with null fields. We use the symbol ⊥ to represent this situation in Figure 11.12. In practice, a particular value that can never represent real data is used (for example, an account number that is blank, or a name beginning with "*").

The reserved-space method is useful when most records have a length close to the maximum. Otherwise, a significant amount of space may be wasted. In our bank example, some branches may have many more accounts than others. This situation leads us to consider the linked list method. To represent the file by the linked list method, we add a pointer field as we did in Figure 11.9. The resulting structure appears in Figure 11.13.



Figure 11.13 File of Figure 11.10 using linked lists.



Figure 11.14 Anchor-block and overflow-block structures.

The file structures of Figures 11.9 and 11.13 both use pointers; the difference is that, in Figure 11.9, we use pointers to chain together only deleted records, whereas in Figure 11.13, we chain together all records pertaining to the same branch.

A disadvantage to the structure of Figure 11.13 is that we waste space in all records except the first in a chain. The first record needs to have the *branch-name* value, but subsequent records do not. Nevertheless, we need to include a field for *branch-name* in all records, lest the records not be of fixed length. This wasted space is significant, since we expect, in practice, that each branch has a large number of accounts. To deal with this problem, we allow two kinds of blocks in our file:

1. **Anchor block**, which contains the first record of a chain
2. **Overflow block**, which contains records other than those that are the first record of a chain

Thus, all records *within a block* have the same length, even though not all records in the file have the same length. Figure 11.14 shows this file structure.

11.7 Organization of Records in Files

So far, we have studied how records are represented in a file structure. An instance of a relation is a set of records. Given a set of records, the next question is how to organize them in a file. Several of the possible ways of organizing records in files are:

- **Heap file organization**. Any record can be placed anywhere in the file where there is space for the record. There is no ordering of records. Typically, there is a single file for each relation
- **Sequential file organization**. Records are stored in sequential order, according to the value of a "search key" of each record. Section 11.7.1 describes this organization.

- **Hashing file organization**. A hash function is computed on some attribute of each record. The result of the hash function specifies in which block of the file the record should be placed. Chapter 12 describes this organization; it is closely related to the indexing structures described in that chapter.

Generally, a separate file is used to store the records of each relation. However, in a **clustering file organization**, records of several different relations are stored in the same file; further, related records of the different relations are stored on the same block, so that one I/O operation fetches related records from all the relations. For example, records of the two relations can be considered to be related if they would match in a join of the two relations. Section 11.7.2 describes this organization.

11.7.1 Sequential File Organization

A **sequential file** is designed for efficient processing of records in sorted order based on some search-key. A **search key** is any attribute or set of attributes; it need not be the primary key, or even a superkey. To permit fast retrieval of records in search-key order, we chain together records by pointers. The pointer in each record points to the next record in search-key order. Furthermore, to minimize the number of block accesses in sequential file processing, we store records physically in search-key order, or as close to search-key order as possible.

Figure 11.15 shows a sequential file of *account* records taken from our banking example. In that example, the records are stored in search-key order, using *branch-name* as the search key.

The sequential file organization allows records to be read in sorted order; that can be useful for display purposes, as well as for certain query-processing algorithms that we shall study in Chapter 13.

It is difficult, however, to maintain physical sequential order as records are inserted and deleted, since it is costly to move many records as a result of a single



Figure 11.15 Sequential file for *account* records.

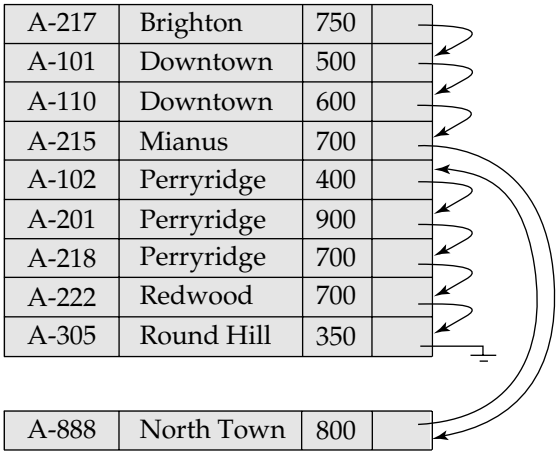


Figure 11.16 Sequential file after an insertion.

insertion or deletion. We can manage deletion by using pointer chains, as we saw previously. For insertion, we apply the following rules:

1. Locate the record in the file that comes before the record to be inserted in search-key order.
2. If there is a free record (that is, space left after a deletion) within the same block as this record, insert the new record there. Otherwise, insert the new record in an *overflow block*. In either case, adjust the pointers so as to chain together the records in search-key order.

Figure 11.16 shows the file of Figure 11.15 after the insertion of the record (North Town, A-888, 800). The structure in Figure 11.16 allows fast insertion of new records, but forces sequential file-processing applications to process records in an order that does not match the physical order of the records.

If relatively few records need to be stored in overflow blocks, this approach works well. Eventually, however, the correspondence between search-key order and physical order may be totally lost, in which case sequential processing will become much less efficient. At this point, the file should be **reorganized** so that it is once again physically in sequential order. Such reorganizations are costly, and must be done during times when the system load is low. The frequency with which reorganizations are needed depends on the frequency of insertion of new records. In the extreme case in which insertions rarely occur, it is possible always to keep the file in physically sorted order. In such a case, the pointer field in Figure 11.15 is not needed.

11.7.2 Clustering File Organization

Many relational-database systems store each relation in a separate file, so that they can take full advantage of the file system that the operating system provides. Usually, tuples of a relation can be represented as fixed-length records. Thus, relations

can be mapped to a simple file structure. This simple implementation of a relational database system is well suited to low-cost database implementations as in, for example, embedded systems or portable devices. In such systems, the size of the database is small, so little is gained from a sophisticated file structure. Furthermore, in such environments, it is essential that the overall size of the object code for the database system be small. A simple file structure reduces the amount of code needed to implement the system.

This simple approach to relational-database implementation becomes less satisfactory as the size of the database increases. We have seen that there are performance advantages to be gained from careful assignment of records to blocks, and from careful organization of the blocks themselves. Clearly, a more complicated file structure may be beneficial, even if we retain the strategy of storing each relation in a separate file.

However, many large-scale database systems do not rely directly on the underlying operating system for file management. Instead, one large operating-system file is allocated to the database system. The database system stores all relations in this one file, and manages the file itself. To see the advantage of storing many relations in one file, consider the following SQL query for the bank database:

select *account-number, customer-name, customer-street, customer-city*
from *depositor, customer*
where *depositor.customer-name* = *customer.customer-name*

This query computes a join of the *depositor* and *customer* relations. Thus, for each tuple of *depositor*, the system must locate the *customer* tuples with the same value for *customer-name*. Ideally, these records will be located with the help of *indices*, which we shall discuss in Chapter 12. Regardless of how these records are located, however, they need to be transferred from disk into main memory. In the worst case, each record will reside on a different block, forcing us to do one block read for each record required by the query.

As a concrete example, consider the *depositor* and *customer* relations of Figures 11.17 and 11.18, respectively. In Figure 11.19, we show a file structure designed for efficient execution of queries involving $depositor \bowtie customer$. The *depositor* tuples for each *customer-name* are stored near the *customer* tuple for the corresponding *customer-name*. This structure mixes together tuples of two relations, but allows for efficient processing of the join. When a tuple of the *customer* relation is read, the entire block containing that tuple is copied from disk into main memory. Since the corresponding

customer-name	*account-number*
Hayes	A-102
Hayes	A-220
Hayes	A-503
Turner	A-305

Figure 11.17 The *depositor* relation.

customer-name	*customer-street*	*customer-city*
Hayes	Main	Brooklyn
Turner	Putnam	Stamford

Figure 11.18 The *customer* relation.

depositor tuples are stored on the disk near the *customer* tuple, the block containing the *customer* tuple contains tuples of the *depositor* relation needed to process the query. If a customer has so many accounts that the *depositor* records do not fit in one block, the remaining records appear on nearby blocks.

A **clustering file organization** is a file organization, such as that illustrated in Figure 11.19 that stores related records of two or more relations in each block. Such a file organization allows us to read records that would satisfy the join condition by using one block read. Thus, we are able to process this particular query more efficiently.

Our use of clustering has enhanced processing of a particular join (*depositor* ⋈ *customer*), but it results in slowing processing of other types of query. For example,

select *
from *customer*

requires more block accesses than it did in the scheme under which we stored each relation in a separate file. Instead of several *customer* records appearing in one block, each record is located in a distinct block. Indeed, simply finding all the *customer* records is not possible without some additional structure. To locate all tuples of the *customer* relation in the structure of Figure 11.19, we need to chain together all the records of that relation using pointers, as in Figure 11.20.

When clustering is to be used depends on the types of query that the database designer believes to be most frequent. Careful use of clustering can produce significant performance gains in query processing.

11.8 Data-Dictionary Storage

So far, we have considered only the representation of the relations themselves. A relational-database system needs to maintain data *about* the relations, such as the

Hayes	Main	Brooklyn
Hayes	A-102	
Hayes	A-220	
Hayes	A-503	
Turner	Putnam	Stamford
Turner	A-305	

Figure 11.19 Clustering file structure.

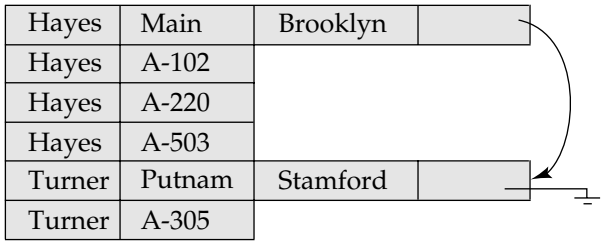


Figure 11.20 Clustering file structure with pointer chains.

schema of the relations. This information is called the **data dictionary**, or **system catalog**. Among the types of information that the system must store are these:

- Names of the relations
- Names of the attributes of each relation
- Domains and lengths of attributes
- Names of views defined on the database, and definitions of those views
- Integrity constraints (for example, key constraints)

In addition, many systems keep the following data on users of the system:

- Names of authorized users
- Accounting information about users
- Passwords or other information used to authenticate users

Further, the database may store statistical and descriptive data about the relations, such as:

- Number of tuples in each relation
- Method of storage for each relation (for example, clustered or nonclustered)

The data dictionary may also note the storage organization (sequential, hash or heap) of relations, and the location where each relation is stored:

- If relations are stored in operating system files, the dictionary would note the names of the file (or files) containing each relation.
- If the database stores all relations in a single file, the dictionary may note the blocks containing records of each relation in a data structure such as a linked list.

In Chapter 12, in which we study indices, we shall see a need to store information about each index on each of the relations:

- Name of the index
- Name of the relation being indexed
- Attributes on which the index is defined
- Type of index formed

All this information constitutes, in effect, a miniature database. Some database systems store this information by using special-purpose data structures and code. It is generally preferable to store the data about the database in the database itself. By using the database to store system data, we simplify the overall structure of the system and harness the full power of the database for fast access to system data.

The exact choice of how to represent system data by relations must be made by the system designers. One possible representation, with primary keys underlined, is

Relation-metadata (*relation-name*, *number-of-attributes*, *storage-organization*, *location*)
Attribute-metadata (*attribute-name*, *relation-name*, *domain-type*, *position*, *length*)
User-metadata (*user-name*, *encrypted-password*, *group*)
Index-metadata (*index-name*, *relation-name*, *index-type*, *index-attributes*)
View-metadata (*view-name*, *definition*)

In this representation, the attribute *index-attributes* of the relation *Index-metadata* is assumed to contain a list of one or more attributes, which can be represented by a character string such as "*branch-name, branch-city*". The *Index-metadata* relation is thus not in first normal form; it can be normalized, but the above representation is likely to be more efficient to access. The data dictionary is often stored in a non-normalized form to achieve fast access.

The storage organization and location of the *Relation-metadata* itself must be recorded elsewhere (for example, in the database code itself), since we need this information to find the contents of *Relation-metadata*.

11.9 Storage for Object-Oriented Databases**

The file-organization techniques described in Section 11.7—the heap, sequential, hashing and clustering organizations—can also be used for storing objects in an object-oriented database. However, some extra features are needed to support object-oriented database features, such as set-valued fields and persistent pointers.

11.9.1 Mapping of Objects to Files

The mapping of objects to files is in many ways like the mapping of tuples to files in a relational system. At the lowest level of data representation, both tuples and the data parts of objects are simply sequences of bytes. We can therefore store object data in the file structures described in this chapter, with some modifications which we note next.

Objects in object-oriented databases may lack the uniformity of tuples in relational databases. For example, fields of records may be sets; in relational databases, in con-

trast, data are typically required to be (at least) in first normal form. Furthermore, objects may be extremely large. Such objects have to be managed differently from records in a relational system.

We can implement set-valued fields that have a small number of elements using data structures such as linked lists. Set-valued fields that have a larger number of elements can be implemented as relations in the database. Set-valued fields of objects can also be eliminated at the *storage level* by normalization: A relation is created containing one tuple for each value of a set-valued field of an object. Each tuple also contains the object identifier of the object. However, this relation is not made visible to the upper levels of the database system. The storage system gives the upper levels of the database system the view of a set-valued field, even though the set-valued field has actually been normalized by creating a new relation.

Some applications include extremely large objects that are not easily decomposed into smaller components. Such large objects may each be stored in a separate file. We discuss this idea further in Section 11.9.6.

11.9.2 Implementation of Object Identifiers

Since objects are identified by object identifiers (OIDs), an object-storage system needs a mechanism to locate an object, given an OID. If the OIDs are **logical OID**s—that is, they do not specify the location of the object—then the storage system must maintain an index that maps OIDs to the actual location of the object. If the OIDs are **physical OID**s—that is, they encode the location of the object—then the object can be found directly. Physical OIDs typically have the following three parts:

1. A volume or file identifier
2. A block identifier within the volume or file
3. An offset within the block

A volume is a logical unit of storage that usually corresponds to a disk.

In addition, physical OIDs may contain a **unique identifier**, which is an integer that distinguishes the OID from the identifiers of other objects that happened to be stored at the same location earlier, and were deleted or moved elsewhere. The unique identifier is also stored with the object, and the identifiers in an OID and the corresponding object should match. If the unique identifier in a physical OID does not match the unique identifier in the object to which that OID points, the system detects that the pointer is a dangling pointer, and signals an error. (A **dangling pointer** is a pointer that does not point to a valid object.) Figure 11.21 illustrates this scheme.

Such pointer errors occur when physical OIDs corresponding to old objects that have been deleted are used accidentally. If the space occupied by the object had been reallocated, there may be a new object in the location, and it may get incorrectly addressed by the identifier of the old object. If a dangling pointer is not detected, it could cause corruption of a new object stored at the same location. The unique identifier helps to detect such errors, since the unique identifiers of the old physical OID and the new object will not match.

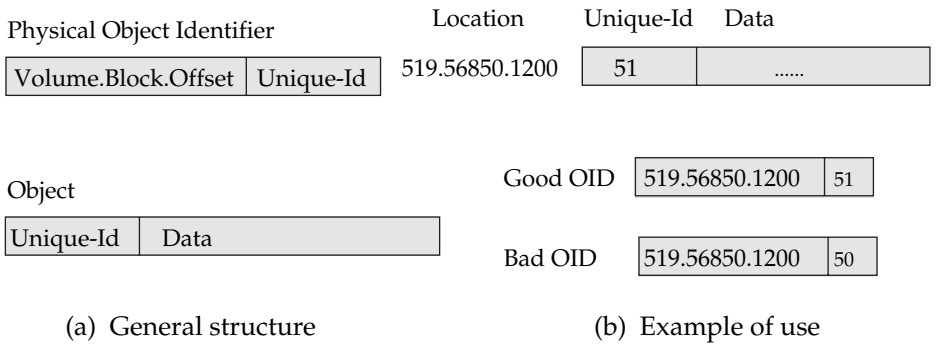


Figure 11.21 Unique identifiers in an OID.

Suppose that an object has to be moved to a new block, perhaps because the size of the object has increased, and the old block has no extra space. Then, the physical OID will point to the old block, which no longer contains the object. Rather than change the OID of the object (which involves changing every object that points to this one), we leave behind a **forwarding address** at the old location. When the database tries to locate the object, it finds the forwarding address instead of the object; it then uses the forwarding address to locate the object.

11.9.3 Management of Persistent Pointers

We implement persistent pointers in a persistent programming language by using OIDs. In some implementations, persistent pointers are physical OIDs; in others, they are logical OIDs. An important difference between persistent pointers and in-memory pointers is the size of the pointer. In-memory pointers need to be only big enough to address all virtual memory. On most current computers, in-memory pointers are usually 4 bytes long, which is sufficient to address 4 gigabytes of memory. The most recent computer architectures have pointers that are 8 bytes long,

Persistent pointers need to address all the data in a database. Since database systems are often bigger than 4 gigabytes, persistent pointers are usually at least 8 bytes long. Many object-oriented databases also provide unique identifiers in persistent pointers, to catch dangling references. This feature further increases the size of persistent pointers. Thus, persistent pointers may be substantially longer than in-memory pointers.

The action of looking up an object, given its identifier, is called **dereferencing**. Given an in-memory pointer (as in C++), looking up the object is merely a memory reference. Given a persistent pointer, dereferencing an object has an extra step—finding the actual location of the object in memory by looking up the persistent pointer in a table. If the object is not already in memory, it has to be loaded from disk. We can implement the table lookup fairly efficiently by using a hash table data structure, but the lookup is still slow compared to a pointer dereference, even if the object is already in memory.

Pointer swizzling is a way to cut down the cost of locating persistent objects that are already present in memory. The idea is that, when a persistent pointer is first dereferenced, the system locates the object and brings it into memory if it is not already there. Now the system carries out an extra step—it stores an in-memory pointer to the object in place of the persistent pointer. The next time that the *same* persistent pointer is dereferenced, the in-memory location can be read out directly, so the costs of locating the object are avoided. (When persistent objects have to be moved from memory back to disk to make space for other persistent objects, the system must carry out an extra step to ensure that the object is still in memory. Correspondingly, when an object is written out, any persistent pointers that it contained and that were swizzled have to be **deswizzled**, that is, converted back to their persistent representation. Pointer swizzling on pointer dereference, as described here, is called **software swizzling**.

Buffer management is more complicated if pointer swizzling is used, since the physical location of an object must not change once that object is brought into the buffer. One way to ensure that it will not change is to pin pages containing swizzled objects in the buffer pool, so that they are never replaced until the program that performed the swizzling has finished execution. See the bibliographical notes for more complex buffer-management schemes, based on virtual-memory mapping techniques, that make it unnecessary to pin the buffer pages.

11.9.4 Hardware Swizzling

Having two types of pointers, persistent and transient (in-memory), is inconvenient. Programmers have to remember the type of the pointers, and may have to write code twice—once for the persistent pointers and once for the in-memory pointers. It would be simpler if both persistent and in-memory pointers were of the same type.

A simple way to merge persistent and in-memory pointer types is just to extend the length of in-memory pointers to the same size as persistent pointers, and to use 1 bit of the identifier to distinguish between persistent and in-memory pointers. However, the storage cost of longer persistent pointers will have to be borne by in-memory pointers as well; understandably, this scheme is unpopular.

We shall describe a technique called **hardware swizzling**, which uses the virtual-memory-management hardware present in most current computer systems to address this problem. When data in a virtual memory page are accessed, and the operating system detects that the page does not have real storage allocated for it, or has been access protected, then a **segmentation violation** is said to occur.[3] Many operating systems provide a mechanism to specify a function to be called when a segmentation violation occurs, and a mechanism to allocate storage for a page in virtual address space, and to set that page's access permissions. In most Unix systems, the `mmap` system call provides this latter functionality. Hardware swizzling makes clever use of the above mechanisms.

3. The term **page fault** is sometimes used instead of *segmentation violation*, although access protection violations are generally not considered to be page faults.

Hardware swizzling has two major advantages over software swizzling:

1. It is able to store persistent pointers in objects in the same amount of space as in-memory pointers require (along with extra storage external to the object).
2. It transparently converts between persistent pointers and in-memory pointers in a clever and efficient way. Software written to deal with in-memory pointers can thereby deal with persistent pointers as well, without any changes.

11.9.4.1 Pointer Representation

Hardware swizzling uses the following representation of persistent pointers contained in objects that are on disk. A persistent pointer is conceptually split into two parts: a page identifier in the database, and an offset within the page.[4] The page identifier in a persistent pointer is actually a small indirect pointer, which we call the short page identifier. Each page (or other unit of storage) has a translation table that provides a mapping from the short page identifiers to full database page identifiers. The system has to look up the short page identifier in a persistent pointer in the translation table to find the full page identifier.

The translation table, in the worst case, will be only as big as the maximum number of pointers that can be contained in objects in a page; with a page size of 4096, and a pointer size of 4 bytes, the maximum number of pointers is 1024. In practice, the translation table is likely to contain much less than the maximum number of elements (1024 in our example) and will not consume excessive space. The short page identifier needs to have only enough bits to identify a row in the table; with a maximum table size of 1024, only 10 bits are required. Hence, a small number of bits is enough to store the short page identifier. Thus, the translation table permits an entire persistent pointer to fit into the same space as an in-memory pointer. Even though only a few bits are needed for the short page identifier, all the bits of an in-memory pointer, other than the page-offset bits, are used as the short page identifier. This architecture facilitates swizzling, as we shall see.

The persistent-pointer representation scheme appears in Figure 11.22, where there are three objects in the page, each containing a persistent pointer. The translation table gives the mapping between short page identifiers and the full database page identifiers for each of the short page identifiers in these persistent pointers. The database page identifiers are shown in the format *volume.page.offset*.

Each page maintains extra information so that all persistent pointers in the page can be found. The system updates the information when an object is created or deleted in the page. The need to locate all the persistent pointers in a page will become clear later.

4. The term **page** is generally used to refer to a real-memory or virtual-memory page, and the term **block** is used to refer to disk blocks in the database. In hardware swizzling, these have to be of the same size, and database blocks are fetched into virtual memory pages. We shall use the terms *page* and *block* interchangeably in this section.

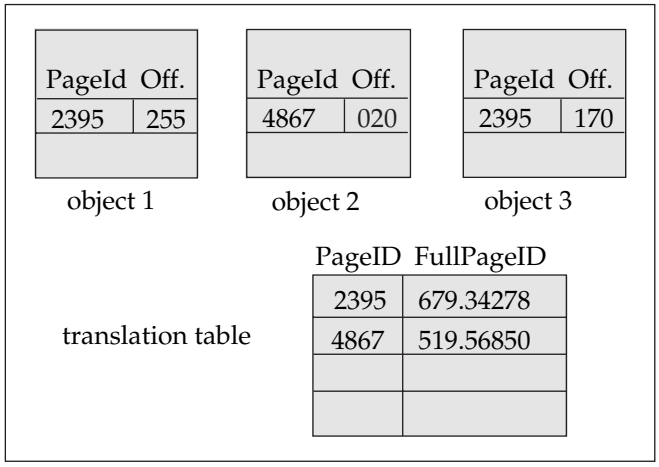


Figure 11.22 Page image before swizzling.

11.9.4.2 Swizzling Pointers on a Page

Initially no page of the database has been allocated a page in virtual memory. Virtual-memory pages may be allocated to database pages even before they are actually loaded, as we will see shortly. Database pages get loaded into virtual-memory when the database system needs to access data on the page. Before a database page is loaded, the system allocates a virtual-memory page to the database page if one has not already been allocated. The system then loads the database page into the virtual-memory page it has allocated to it.

When the system loads a database page P into virtual memory, it does pointer swizzling on the page: It locates all persistent pointers contained in objects in page P, using the extra information stored in the page. It takes the following actions for each persistent pointer in the page. (Let the value of the persistent pointer be $\langle p_i, o_i \rangle$, where p_i is the short page identifier and o_i is the offset within the page. Let P_i be the full page identifier of p_i, found in the translation table in page P.)

1. If page P_i does not already have a virtual-memory page allocated to it, the system now allocates a free page in virtual memory to it. The page P_i will reside at this virtual-memory location if and when it is brought in. At this point, the page in virtual address space does not have any storage allocated for it, either in memory or on disk; it is merely a range of addresses reserved for the database page. The system allocates actual space when it actually loads the database page P_i into virtual memory.

2. Let the virtual-memory page allocated (either earlier or in the preceding step) for P_i be v_i. The system updates the persistent pointer being considered, whose value is $\langle p_i, o_i \rangle$, by replacing p_i with v_i.

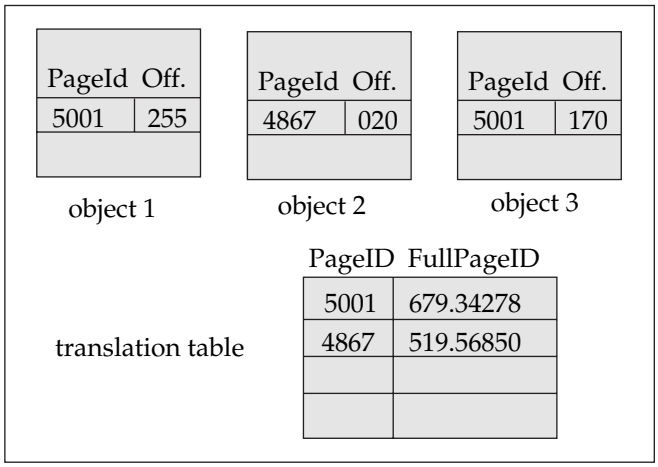


Figure 11.23 Page image after swizzling.

Figure 11.23 shows the state of the page from Figure 11.22 after the system has brought that page into memory and swizzled the pointers in it. Here, we assume that the page whose database page identifier is 679.34278 has been mapped to page 5001 in memory, whereas the page whose identifier is 519.56850 has been mapped to page 4867 (which is the same as the short page identifier). All the pointers in objects have been updated to reflect the new mapping, and can now be used as in-memory pointers.

At the end of the translation phase for a page, the objects in the page satisfy an important property: All persistent pointers contained in objects in the page have been converted to in-memory pointers. Thus, objects in in-memory pages contain only in-memory pointers. Routines that use these objects do not even need to know about the existence of persistent pointers! For example, existing libraries written for in-memory objects can be used unchanged for persistent objects. That is indeed an important advantage!

11.9.4.3 Pointer Dereference

Consider the first time that an in-memory pointer to a virtual-memory page v_i is dereferenced, when storage has not yet been allocated for the page. As we described, a segmentation violation will occur, and will result in a function call on the database system. The database system takes the following actions:

1. It first determines what database page was allocated to virtual-memory page v_i; let the full page identifier of the database page be P_i. (If no database page has been allocated to v_i, the pointer is incorrect, and the system flags an error.)
2. It allocates storage space for page v_i, and loads the database page P_i into virtual-memory page v_i.

3. It carries out pointer swizzling out on page P_i, as described earlier in "Swizzling Pointer on a Page".

4. After swizzling all persistent pointers in P, the system allows the pointer dereference that resulted in the segmentation violation to continue. The pointer dereference will find the object for which it was looking loaded in memory.

If any swizzled pointer that points to an object in page v_i is dereferenced later, the dereference proceeds just like any other virtual-memory access, with no extra overheads. In contrast, if swizzling is not used, there is considerable overhead in locating the buffer page containing the object and then accessing it. This overhead has to be incurred on *every* access to objects in the page, whereas when swizzling is performed, the overhead is incurred only on the *first* access to an object in the page. Later accesses operate at regular virtual-memory access speeds. Hardware swizzling thus gives excellent performance benefits to applications that repeatedly dereference pointers.

11.9.4.4 Optimizations

Software swizzling performs a deswizzling operation when a page in memory has to be written back to the database, to convert in-memory pointers back to persistent pointers. Hardware swizzling can avoid this step—when the system does pointer swizzling for the page, it simply updates the translation table for the page, so that the page-identifier part of the swizzled in-memory pointers can be used to look up the table. For example, as shown in Figure 11.23, database page 679.34278 (with short identifier 2395 in the page shown) is mapped to virtual-memory page 5001. At this point, not only is the pointer in object 1 updated from 2395255 to 5001255, but also the short identifier in the table is updated to 5001. Thus, the short identifier 5001 in object 1 and in the table match each other again. Therefore, the page can be written back to disk without any deswizzling.

Several optimizations can be carried out on the basic scheme described here. When the system swizzles page P, for each page P' referred to by any persistent pointer in P, it attempts to allocate P' to the virtual address location indicated by the short page identifier of P' on page P. If the system can allocate the page in this attempt, pointers to it do not need to be updated. In our swizzling example, page 519.56850 with short page identifier 4867 was mapped to virtual-memory page 4867, which is the same as its short page identifier. We can see that the pointer in object 2 to this page did not need to be changed during swizzling. If every page can be allocated to its appropriate location in virtual address space, none of the pointers need to be translated, and the cost of swizzling is reduced significantly.

Hardware swizzling works even if the database is bigger than virtual memory, but only as long as all the pages that a particular process accesses fit into the virtual memory of the process. If they do not, a page that has been brought into virtual memory will have to be replaced, and that replacement is hard to do, since there may be in-memory pointers to objects in that page.

Hardware swizzling can also be used at the level of sets of pages (often called segments), instead of for a single page. For set-level swizzling, the system uses a single translation table for all pages in the segment. It loads pages in the segment and swizzles them as and when they are required; they need not be loaded all together.

11.9.5 Disk Versus Memory Structure of Objects

The format in which objects are stored in memory may be different from the format in which they are stored on disk in the database. One reason may be the use of software swizzling, where the structures of persistent and in-memory pointers are different. Another reason may be that we want to have the database accessible from different machines, possibly based on different architectures, and from different languages, and from programs compiled under different compilers, all of which result in differences in the in-memory representation.

Consider, for example, a data-structure definition in a programming language such as C++. The physical structure (such as sizes and representation of integers) in the object depends on the machine on which the program is run.[5] Further, the physical structure may also depend on which compiler is used—in a language as complex as C++, different choices for translation from the high-level description to the physical structure are possible, and each compiler can make its own choice.

The solution to this problem is to make the physical representation of objects in the database independent of the machine and of the compiler. The system can convert the object from the disk representation to the form that is required on the specific machine, language, and compiler, when that object is brought into memory. It can do this conversion transparently at the same time that it swizzles pointers in the object, so the programmer does not need to worry about the conversion.

The first step in implementing such a scheme is to define a common language for describing the structure of objects—that is, a data-definition language. One such language is the Object Definition Language (ODL) developed by the Object Database Management Group (ODMG). ODL has mappings defined to the Java, C++, and Smalltalk languages, so potentially we may manipulate objects in an ODMG-compliant database using any of these languages.

The definition of the structure of each class in the database is stored (logically) in the databases. The code to translate an object in the database to the representation that is manipulated with the programming language (and vice versa) depends on the machine as well as on the compiler for the language. We can generate this code automatically, using the stored definition of the class of the object.

An unexpected source of differences between the disk and in-memory representations of data is the hidden-pointers in objects. **Hidden pointers** are transient pointers

5. For instance, the Motorola 680x0 architectures, the IBM 360 architecture, and the Intel 80386/80486/Pentium/Pentium-II/Pentium-III architectures all have 4-byte integers. However, they differ in how the bits of an integer are laid out within a word. In earlier-generation personal computers, integers were 2 bytes long; in newer workstation architectures, such as the Compaq Alpha, Intel Itanium, and Sun UltraSparc architectures, integers can be 8 bytes long.

that compilers generate and store in objects. These pointers point (indirectly) to tables used to implement certain methods of the object. The tables are typically compiled into executable object code, and their exact location depends on the executable object code; hence, they may be different for different processes. Therefore, when a process accesses an object, the hidden pointers must be fixed to point to the correct location. The hidden pointers can be initialized at the same time that data-representation conversions are carried out.

11.9.6 Large Objects

Objects may also be extremely large; for instance, multimedia objects may occupy several megabytes of space. Exceptionally large data items, such as video sequences, may run into gigabytes, although they are usually split into multiple objects, each on the order of a few megabytes or less. Large objects containing binary data are called binary large objects (blobs), while large objects containing character data, are called character large objects (clobs), as we saw in Section 9.2.1.

Most relational databases restrict the size of a record to be no larger than the size of a page, to simplify buffer management and free-space management. Large objects and long fields are often stored in a special file (or collection of files) reserved for long-field storage.

Allocation of buffer pages presents a problem with managing large objects. Large objects may need to be stored in a contiguous sequence of bytes when they are brought into memory; in that case, if an object is bigger than a page, contiguous pages of the buffer pool must be allocated to store it, which makes buffer management more difficult.

We often modify large objects by updating part of the object, or by inserting or deleting parts of the object, rather than by writing the entire object. If inserts and deletes need to be supported, we can handle large objects by using B-tree structures (which we study in Chapter 12). B-tree structures permit us to read the entire object, as well as to insert and delete parts of the object.

For practical reasons, we may manipulate large objects by using application programs, instead of doing so within the database:

- **Text data**. Text is usually treated as a byte string manipulated by editors and formatters.
- **Image/Graphical data**. Graphical data may be represented as a bitmap or as a set of lines, boxes, and other geometric objects. Although some graphical data often are managed within the database system itself, special application software is used for many cases, such as integrated circuit design.
- **Audio and video data**. Audio and video data are typically a digitized, compressed representation created and displayed by separate application software. Data are usually modified with special-purpose editing software, outside the database system.

The most widely used method for updating such data is the **checkout/checkin** method. A user or an application would **check out** a copy of a long-field object, operate on this copy with special-purpose application programs, and then **check in** the modified copy. *Checkout* and a *checkin* correspond roughly to read and write. In some systems, a checkin may create a new version of the object without deleting the old version.

11.10 Summary

- Several types of data storage exist in most computer systems. They are classified by the speed with which they can access data, by their cost per unit of data to buy the memory, and by their reliability. Among the media available are cache, main memory, flash memory, magnetic disks, optical disks, and magnetic tapes.
- Two factors determine the reliability of storage media: whether a power failure or system crash causes data to be lost, and what the likelihood is of physical failure of the storage devise.
- We can reduce the likelihood of physical failure by retaining multiple copies of data. For disks, we can use mirroring. Or we can use more sophisticated methods based on redundant arrays of independent disks (RAIDs). By striping data across disks, these methods offer high throughput rates on large accesses; by introducing redundancy across disks, they improve reliability greatly. Several different RAID organizations are possible, each with different cost, performance and reliability characteristics. RAID level 1 (mirroring) and RAID level 5 are the most commonly used.
- We can organize a file logically as a sequence of records mapped onto disk blocks. One approach to mapping the database to files is to use several files, and to store records of only one fixed length in any given file. An alternative is to structure files so that they can accommodate multiple lengths for records. There are different techniques for implementing variable-length records, including the slotted-page method, the pointer method, and the reserved-space method.
- Since data are transferred between disk storage and main memory in units of a block, it is worthwhile to assign file records to blocks in such a way that a single block contains related records. If we can access several of the records we want with only one block access, we save disk accesses. Since disk accesses are usually the bottleneck in the performance of a database system, careful assignment of records to blocks can pay significant performance dividends.
- One way to reduce the number of disk accesses is to keep as many blocks as possible in main memory. Since it is not possible to keep all blocks in main memory, we need to manage the allocation of the space available in main memory for the storage of blocks. The *buffer* is that part of main memory avail-

able for storage of copies of disk blocks. The subsystem responsible for the allocation of buffer space is called the *buffer manager*.

- Storage systems for object-oriented databases are somewhat different from storage systems for relational databases: They must deal with large objects, for example, and must support persistent pointers. There are schemes to detect dangling persistent pointers.
- Software- and hardware-based swizzling schemes permit efficient dereferencing of persistent pointers. The hardware-based schemes use the virtual-memory-management support implemented in hardware, and made accessible to user programs by many current-generation operating systems.

Review Terms

- Physical storage media
  - Cache
  - Main memory
  - Flash memory
  - Magnetic disk
  - Optical storage
- Magnetic disk
  - Platter
  - Hard disks
  - Floppy disks
  - Tracks
  - Sectors
  - Read–write head
  - Disk arm
  - Cylinder
  - Disk controller
  - Checksums
  - Remapping of bad sectors
- Performance measures of disks
  - Access time
  - Seek time
  - Rotational latency
  - Data-transfer rate
  - Mean time to failure (MTTF)
- Disk block
- Optimization of disk-block access
  - Disk-arm scheduling
  - Elevator algorithm
  - File organization
  - Defragmenting
  - Nonvolatile write buffers
  - Nonvolatile random-access memory (NV-RAM)
  - Log disk
  - Log-based file system
- Redundant arrays of independent disks (RAID)
  - Mirroring
  - Data striping
  - Bit-level striping
  - Block-level striping
- RAID levels
  - Level 0 (block striping, no redundancy)
  - Level 1 (block striping, mirroring)
  - Level 3 (bit striping, parity)
  - Level 5 (block striping, distributed parity)
  - Level 6 (block striping, P + Q redundancy)
- Rebuild performance
- Software RAID
- Hardware RAID
- Hot swapping

- Tertiary storage
  - ☐ Optical disks
  - ☐ Magnetic tapes
  - ☐ Jukeboxes
- Buffer
  - ☐ Buffer manager
  - ☐ Pinned blocks
  - ☐ Forced output of blocks
- Buffer-replacement policies
  - ☐ Least recently used (LRU)
  - ☐ Toss-immediate
  - ☐ Most recently used (MRU)
- File
- File organization
  - ☐ File header
  - ☐ Free list
- Variable-length records
  - ☐ Byte-string representation
  - ☐ Slotted-page structure
  - ☐ Reserved space
  - ☐ List representation
- Heap file organization
- Sequential file organization
- Hashing file organization
- Clustering file organization
- Search key
- Data dictionary
- System catalog
- Storage structures for OODBs
- Object identifier (OID)
  - ☐ Logical OID
  - ☐ Physical OID
  - ☐ Unique identifier
  - ☐ Dangling pointer
  - ☐ Forwarding address
- Pointer swizzling
  - ☐ Dereferencing
  - ☐ Deswizzling
  - ☐ Software swizzling
  - ☐ Hardware swizzling
  - ☐ Segmentation violation
  - ☐ Page fault
- Hidden pointers
- Large objects

Exercises

11.1 List the physical storage media available on the computers you use routinely. Give the speed with which data can be accessed on each medium.

11.2 How does the remapping of bad sectors by disk controllers affect data-retrieval rates?

11.3 Consider the following data and parity-block arrangement on four disks:

Disk 1	Disk 2	Disk 3	Disk 4
B_1	B_2	B_3	B_4
P_1	B_5	B_6	B_7
B_8	P_2	B_9	B_{10}
⋮	⋮	⋮	⋮

The B_i's represent data blocks; the P_i's represent parity blocks. Parity block P_i is the parity block for data blocks B_{4i-3} to B_{4i}. What, if any, problem might this arrangement present?

11.4 A power failure that occurs while a disk block is being written could result in the block being only partially written. Assume that partially written blocks can be detected. An atomic block write is one where either the disk block is fully written or nothing is written (i.e., there are no partial writes). Suggest schemes for getting the effect of atomic block writes with the following RAID schemes. Your schemes should involve work on recovery from failure.

a. RAID level 1 (mirroring)
b. RAID level 5 (block interleaved, distributed parity)

11.5 RAID systems typically allow you to replace failed disks without stopping access to the system. Thus, the data in the failed disk must be rebuilt and written to the replacement disk while the system is in operation. With which of the RAID levels is the amount of interference between the rebuild and ongoing disk accesses least? Explain your answer.

11.6 Give an example of a relational-algebra expression and a query-processing strategy in each of the following situations:

a. MRU is preferable to LRU.
b. LRU is preferable to MRU.

11.7 Consider the deletion of record 5 from the file of Figure 11.8. Compare the relative merits of the following techniques for implementing the deletion:

a. Move record 6 to the space occupied by record 5, and move record 7 to the space occupied by record 6.
b. Move record 7 to the space occupied by record 5.
c. Mark record 5 as deleted, and move no records.

11.8 Show the structure of the file of Figure 11.9 after each of the following steps:

a. Insert (Brighton, A-323, 1600).
b. Delete record 2.
c. Insert (Brighton, A-626, 2000).

11.9 Give an example of a database application in which the reserved-space method of representing variable-length records is preferable to the pointer method. Explain your answer.

11.10 Give an example of a database application in which the pointer method of representing variable-length records is preferable to the reserved-space method. Explain your answer.

11.11 Show the structure of the file of Figure 11.12 after each of the following steps:

a. Insert (Mianus, A-101, 2800).
b. Insert (Brighton, A-323, 1600).
c. Delete (Perryridge, A-102, 400).

11.12 What happens if you attempt to insert the record

(Perryridge, A-929, 3000)

into the file of Figure 11.12?

11.13 Show the structure of the file of Figure 11.13 after each of the following steps:

a. Insert (Mianus, A-101, 2800).

b. Insert (Brighton, A-323, 1600).

c. Delete (Perryridge, A-102, 400).

11.14 Explain why the allocation of records to blocks affects database-system performance significantly.

11.15 If possible, determine the buffer-management strategy used by the operating system running on your local computer system, and what mechanisms it provides to control replacement of pages. Discuss how the control on replacement that it provides would be useful for the implementation of database systems.

11.16 In the sequential file organization, why is an overflow *block* used even if there is, at the moment, only one overflow record?

11.17 List two advantages and two disadvantages of each of the following strategies for storing a relational database:

a. Store each relation in one file.

b. Store multiple relations (perhaps even the entire database) in one file.

11.18 Consider a relational database with two relations:

course (*course-name*, *room*, *instructor*)
enrollment (*course-name*, *student-name*, *grade*)

Define instances of these relations for three courses, each of which enrolls five students. Give a file structure of these relations that uses clustering.

11.19 Consider the following bitmap technique for tracking free space in a file. For each block in the file, two bits are maintained in the bitmap. If the block is between 0 and 30 percent full the bits are 00, between 30 and 60 percent the bits are 01, between 60 and 90 percent the bits are 10, and above 90 percent the bits are 11. Such bitmaps can be kept in memory even for quite large files.

a. Describe how to keep the bitmap up-to-date on record insertions and deletions.

b. Outline the benefit of the bitmap technique over free lists when searching for free space and when updating free space information.

11.20 Give a normalized version of the *Index-metadata* relation, and explain why using the normalized version would result in worse performance.

11.21 Explain why a physical OID must contain more information than a pointer to a physical storage location.

11.22 If physical OIDs are used, an object can be relocated by keeping a forwarding pointer to its new location. In case an object gets forwarded multiple times, what would be the effect on retrieval speed? Suggest a technique to avoid multiple accesses in such a case.

11.23 Define the term *dangling pointer*. Describe how the unique-id scheme helps in detecting dangling pointers in an object-oriented database.

11.24 Consider the example on page 435, which shows that there is no need for deswizzling if hardware swizzling is used. Explain why, in that example, it is safe to change the short identifier of page 679.34278 from 2395 to 5001. Can some other page already have short identifier 5001? If it could, how can you handle that situation?

Bibliographical Notes

Patterson and Hennessy [1995] discusses the hardware aspects of translation look-aside buffers, caches and memory-management units. Rosch and Wethington [1999] presents an excellent overview of computer hardware, including extensive coverage of all types of storage technology such as floppy disks, magnetic disks, optical disks, tapes, and storage interfaces. Ruemmler and Wilkes [1994] presents a survey of magnetic-disk technology. Flash memory is discussed by Dippert and Levy [1993].

The specifications of current-generation disk drives can be obtained from the Web sites of their manufacturers, such as IBM, Seagate, and Maxtor.

Alternative disk organizations that provide a high degree of fault tolerance include those described by Gray et al. [1990] and Bitton and Gray [1988]. Disk striping is described by Salem and Garcia-Molina [1986]. Discussions of redundant arrays of inexpensive disks (RAID) are presented by Patterson et al. [1988] and Chen and Patterson [1990]. Chen et al. [1994] presents an excellent survey of RAID principles and implementation. Reed–Solomon codes are covered in Pless [1989]. The log-based file system, which makes disk access sequential, is described in Rosenblum and Ousterhout [1991].

In systems that support mobile computing, data may be broadcast repeatedly. The broadcast medium can be viewed as a level of the storage hierarchy—as a broadcast disk with high latency. These issues are discussed in Acharya et al. [1995]. Caching and buffer management for mobile computing is discussed in Barbará and Imielinski [1994]. Further discussion of storage issues in mobile computing appears in Douglis et al. [1994].

Basic data structures are discussed in Cormen et al. [1990]. There are several papers describing the storage structure of specific database systems. Astrahan et al. [1976] discusses System R. Chamberlin et al. [1981] reviews System R in retrospect. The *Oracle 8 Concepts Manual* (Oracle [1997]) describes the storage organization of the Oracle 8 database system. The structure of the Wisconsin Storage System (WiSS) is described in Chou et al. [1985]. A software tool for the physical design of relational databases is described by Finkelstein et al. [1988].

Buffer management is discussed in most operating system texts, including in Silberschatz and Galvin [1998]. Stonebraker [1981] discusses the relationship between database-system buffer managers and operating-system buffer managers. Chou and Dewitt [1985] presents algorithms for buffer management in database systems, and describes a performance evaluation. Bridge et al. [1997] describes techniques used in the buffer manager of the Oracle database system.

Descriptions and performance comparisons of different swizzling techniques are given in Wilson [1990], Moss [1990], and White and DeWitt [1992]. White and DeWitt [1994] describes the virtual-memory-mapped buffer-management scheme used in the ObjectStore OODB system and in the QuickStore storage manager. Using this scheme, we can map disk pages to a fixed virtual-memory address, even if they are not pinned in the buffer. The Exodus object storage manager is described in Carey et al. [1986]. Biliris and Orenstein [1994] provides a survey of storage systems for object-oriented databases. Jagadish et al. [1994] describes a storage manager for main-memory databases.

© The McGraw–Hill Companies, 2001

CHAPTER 12

Indexing and Hashing

Many queries reference only a small proportion of the records in a file. For example, a query like "Find all accounts at the Perryridge branch" or "Find the balance of account number A-101" references only a fraction of the account records. It is inefficient for the system to read every record and to check the *branch-name* field for the name "Perryridge," or the *account-number* field for the value A-101. Ideally, the system should be able to locate these records directly. To allow these forms of access, we design additional structures that we associate with files.

12.1 Basic Concepts

An index for a file in a database system works in much the same way as the index in this textbook. If we want to learn about a particular topic (specified by a word or a phrase) in this textbook, we can search for the topic in the index at the back of the book, find the pages where it occurs, and then read the pages to find the information we are looking for. The words in the index are in sorted order, making it easy to find the word we are looking for. Moreover, the index is much smaller than the book, further reducing the effort needed to find the words we are looking for.

Card catalogs in libraries worked in a similar manner (although they are rarely used any longer). To find a book by a particular author, we would search in the author catalog, and a card in the catalog tells us where to find the book. To assist us in searching the catalog, the library would keep the cards in alphabetic order by authors, with one card for each author of each book.

Database system indices play the same role as book indices or card catalogs in libraries. For example, to retrieve an *account* record given the account number, the database system would look up an index to find on which disk block the corresponding record resides, and then fetch the disk block, to get the *account* record.

Keeping a sorted list of account numbers would not work well on very large databases with millions of accounts, since the index would itself be very big; further,

even though keeping the index sorted reduces the search time, finding an account can still be rather time-consuming. Instead, more sophisticated indexing techniques may be used. We shall discuss several of these techniques in this chapter.

There are two basic kinds of indices:

- **Ordered indices**. Based on a sorted ordering of the values.
- **Hash indices**. Based on a uniform distribution of values across a range of buckets. The bucket to which a value is assigned is determined by a function, called a *hash function*.

We shall consider several techniques for both ordered indexing and hashing. No one technique is the best. Rather, each technique is best suited to particular database applications. Each technique must be evaluated on the basis of these factors:

- **Access types**: The types of access that are supported efficiently. Access types can include finding records with a specified attribute value and finding records whose attribute values fall in a specified range.
- **Access time**: The time it takes to find a particular data item, or set of items, using the technique in question.
- **Insertion time**: The time it takes to insert a new data item. This value includes the time it takes to find the correct place to insert the new data item, as well as the time it takes to update the index structure.
- **Deletion time**: The time it takes to delete a data item. This value includes the time it takes to find the item to be deleted, as well as the time it takes to update the index structure.
- **Space overhead**: The additional space occupied by an index structure. Provided that the amount of additional space is moderate, it is usually worthwhile to sacrifice the space to achieve improved performance.

We often want to have more than one index for a file. For example, libraries maintained several card catalogs: for author, for subject, and for title.

An attribute or set of attributes used to look up records in a file is called a **search key**. Note that this definition of *key* differs from that used in *primary key*, *candidate key*, and *superkey*. This duplicate meaning for *key* is (unfortunately) well established in practice. Using our notion of a search key, we see that if there are several indices on a file, there are several search keys.

12.2 Ordered Indices

To gain fast random access to records in a file, we can use an index structure. Each index structure is associated with a particular search key. Just like the index of a book or a library catalog, an ordered index stores the values of the search keys in sorted order, and associates with each search key the records that contain it.

The records in the indexed file may themselves be stored in some sorted order, just as books in a library are stored according to some attribute such as the Dewey deci-

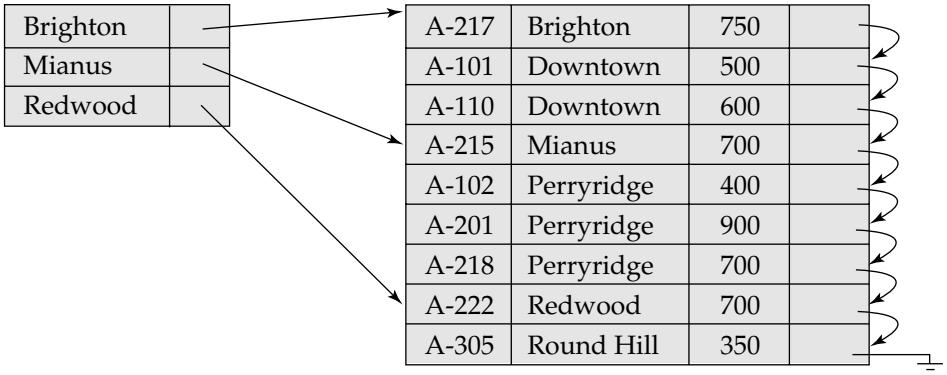


Figure 12.1 Sequential file for *account* records.

mal number. A file may have several indices, on different search keys. If the file containing the records is sequentially ordered, a **primary index** is an index whose search key also defines the sequential order of the file. (The term *primary index* is sometimes used to mean an index on a primary key. However, such usage is nonstandard and should be avoided.) Primary indices are also called **clustering indices**. The search key of a primary index is usually the primary key, although that is not necessarily so. Indices whose search key specifies an order different from the sequential order of the file are called **secondary indices**, or **nonclustering** indices.

12.2.1 Primary Index

In this section, we assume that all files are ordered sequentially on some search key. Such files, with a primary index on the search key, are called **index-sequential files**. They represent one of the oldest index schemes used in database systems. They are designed for applications that require both sequential processing of the entire file and random access to individual records.

Figure 12.1 shows a sequential file of *account* records taken from our banking example. In the example of Figure 12.1, the records are stored in search-key order, with *branch-name* used as the search key.

12.2.1.1 Dense and Sparse Indices

An **index record**, or **index entry**, consists of a search-key value, and pointers to one or more records with that value as their search-key value. The pointer to a record consists of the identifier of a disk block and an offset within the disk block to identify the record within the block.

There are two types of ordered indices that we can use:

- **Dense index**: An index record appears for every search-key value in the file. In a dense primary index, the index record contains the search-key value and a pointer to the first data record with that search-key value. The rest of the records with the same search key-value would be stored sequentially after the

first record, since, because the index is a primary one, records are sorted on the same search key.

Dense index implementations may store a list of pointers to all records with the same search-key value; doing so is not essential for primary indices.

- **Sparse index**: An index record appears for only some of the search-key values. As is true in dense indices, each index record contains a search-key value and a pointer to the first data record with that search-key value. To locate a record, we find the index entry with the largest search-key value that is less than or equal to the search-key value for which we are looking. We start at the record pointed to by that index entry, and follow the pointers in the file until we find the desired record.

Figures 12.2 and 12.3 show dense and sparse indices, respectively, for the *account* file. Suppose that we are looking up records for the Perryridge branch. Using the dense index of Figure 12.2, we follow the pointer directly to the first Perryridge record. We process this record, and follow the pointer in that record to locate the next record in search-key (*branch-name*) order. We continue processing records until we encounter a record for a branch other than Perryridge. If we are using the sparse index (Figure 12.3), we do not find an index entry for "Perryridge." Since the last entry (in alphabetic order) before "Perryridge" is "Mianus," we follow that pointer. We then read the *account* file in sequential order until we find the first Perryridge record, and begin processing at that point.

As we have seen, it is generally faster to locate a record if we have a dense index rather than a sparse index. However, sparse indices have advantages over dense indices in that they require less space and they impose less maintenance overhead for insertions and deletions.

There is a trade-off that the system designer must make between access time and space overhead. Although the decision regarding this trade-off depends on the specific application, a good compromise is to have a sparse index with one index entry per block. The reason this design is a good trade-off is that the dominant cost in pro-

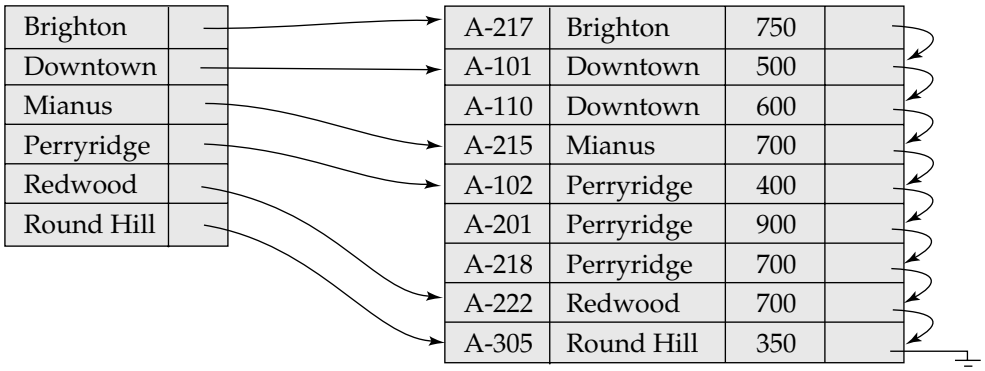


Figure 12.2 Dense index.

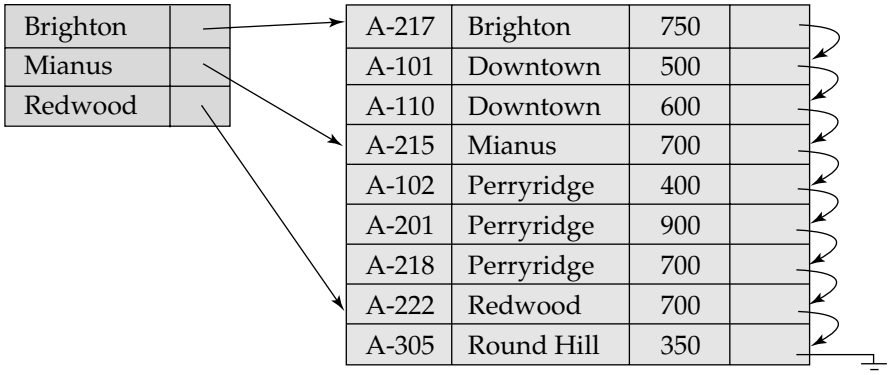


Figure 12.3 Sparse index.

cessing a database request is the time that it takes to bring a block from disk into main memory. Once we have brought in the block, the time to scan the entire block is negligible. Using this sparse index, we locate the block containing the record that we are seeking. Thus, unless the record is on an overflow block (see Section 11.7.1), we minimize block accesses while keeping the size of the index (and thus, our space overhead) as small as possible.

For the preceding technique to be fully general, we must consider the case where records for one search-key value occupy several blocks. It is easy to modify our scheme to handle this situation.

12.2.1.2 Multilevel Indices

Even if we use a sparse index, the index itself may become too large for efficient processing. It is not unreasonable, in practice, to have a file with 100,000 records, with 10 records stored in each block. If we have one index record per block, the index has 10,000 records. Index records are smaller than data records, so let us assume that 100 index records fit on a block. Thus, our index occupies 100 blocks. Such large indices are stored as sequential files on disk.

If an index is sufficiently small to be kept in main memory, the search time to find an entry is low. However, if the index is so large that it must be kept on disk, a search for an entry requires several disk block reads. Binary search can be used on the index file to locate an entry, but the search still has a large cost. If the index occupies b blocks, binary search requires as many as $\lceil \log_2(b) \rceil$ blocks to be read. ($\lceil x \rceil$ denotes the least integer that is greater than or equal to x; that is, we round upward.) For our 100-block index, binary search requires seven block reads. On a disk system where a block read takes 30 milliseconds, the search will take 210 milliseconds, which is long. Note that, if overflow blocks have been used, binary search will not be possible. In that case, a sequential search is typically used, and that requires b block reads, which will take even longer. Thus, the process of searching a large index may be costly.

To deal with this problem, we treat the index just as we would treat any other sequential file, and construct a sparse index on the primary index, as in Figure 12.4.

To locate a record, we first use binary search on the outer index to find the record for the largest search-key value less than or equal to the one that we desire. The pointer points to a block of the inner index. We scan this block until we find the record that has the largest search-key value less than or equal to the one that we desire. The pointer in this record points to the block of the file that contains the record for which we are looking.

Using the two levels of indexing, we have read only one index block, rather than the seven we read with binary search, if we assume that the outer index is already in main memory. If our file is extremely large, even the outer index may grow too large to fit in main memory. In such a case, we can create yet another level of index. Indeed, we can repeat this process as many times as necessary. Indices with two or more levels are called **multilevel** indices. Searching for records with a multilevel index requires significantly fewer I/O operations than does searching for records by binary search. Each level of index could correspond to a unit of physical storage. Thus, we may have indices at the track, cylinder, and disk levels.

A typical dictionary is an example of a multilevel index in the nondatabase world. The header of each page lists the first word alphabetically on that page. Such a book

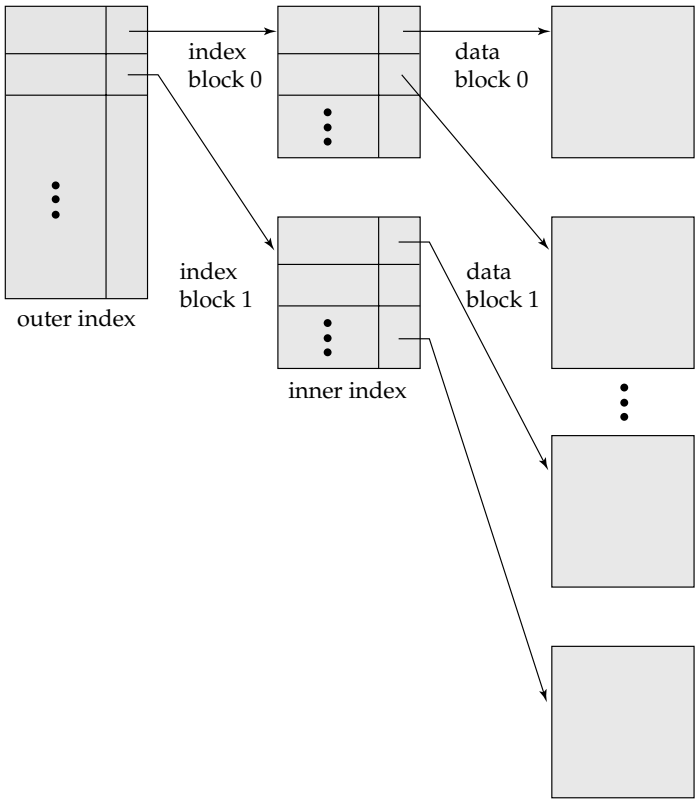


Figure 12.4 Two-level sparse index.

index is a multilevel index: The words at the top of each page of the book index form a sparse index on the contents of the dictionary pages.

Multilevel indices are closely related to tree structures, such as the binary trees used for in-memory indexing. We shall examine the relationship later, in Section 12.3.

12.2.1.3 Index Update

Regardless of what form of index is used, every index must be updated whenever a record is either inserted into or deleted from the file. We first describe algorithms for updating single-level indices.

- **Insertion**. First, the system performs a lookup using the search-key value that appears in the record to be inserted. Again, the actions the system takes next depend on whether the index is dense or sparse:
  - Dense indices:
    1. If the search-key value does not appear in the index, the system inserts an index record with the search-key value in the index at the appropriate position.
    2. Otherwise the following actions are taken:
       a. If the index record stores pointers to all records with the same search-key value, the system adds a pointer to the new record to the index record.
       b. Otherwise, the index record stores a pointer to only the first record with the search-key value. The system then places the record being inserted after the other records with the same search-key values.
  - Sparse indices: We assume that the index stores an entry for each block. If the system creates a new block, it inserts the first search-key value (in search-key order) appearing in the new block into the index. On the other hand, if the new record has the least search-key value in its block, the system updates the index entry pointing to the block; if not, the system makes no change to the index.
- **Deletion**. To delete a record, the system first looks up the record to be deleted. The actions the system takes next depend on whether the index is dense or sparse:
  - Dense indices:
    1. If the deleted record was the only record with its particular search-key value, then the system deletes the corresponding index record from the index.
    2. Otherwise the following actions are taken:
       a. If the index record stores pointers to all records with the same search-key value, the system deletes the pointer to the deleted record from the index record.

b. Otherwise, the index record stores a pointer to only the first record with the search-key value. In this case, if the deleted record was the first record with the search-key value, the system updates the index record to point to the next record.

- Sparse indices:
  1. If the index does not contain an index record with the search-key value of the deleted record, nothing needs to be done to the index.
  2. Otherwise the system takes the following actions:
     a. If the deleted record was the only record with its search key, the system replaces the corresponding index record with an index record for the next search-key value (in search-key order). If the next search-key value already has an index entry, the entry is deleted instead of being replaced.
     b. Otherwise, if the index record for the search-key value points to the record being deleted, the system updates the index record to point to the next record with the same search-key value.

Insertion and deletion algorithms for multilevel indices are a simple extension of the scheme just described. On deletion or insertion, the system updates the lowest-level index as described. As far as the second level is concerned, the lowest-level index is merely a file containing records—thus, if there is any change in the lowest-level index, the system updates the second-level index as described. The same technique applies to further levels of the index, if there are any.

12.2.2 Secondary Indices

Secondary indices must be dense, with an index entry for every search-key value, and a pointer to every record in the file. A primary index may be sparse, storing only some of the search-key values, since it is always possible to find records with intermediate search-key values by a sequential access to a part of the file, as described earlier. If a secondary index stores only some of the search-key values, records with intermediate search-key values may be anywhere in the file and, in general, we cannot find them without searching the entire file.

A secondary index on a candidate key looks just like a dense primary index, except that the records pointed to by successive values in the index are not stored sequentially. In general, however, secondary indices may have a different structure from primary indices. If the search key of a primary index is not a candidate key, it suffices if the index points to the first record with a particular value for the search key, since the other records can be fetched by a sequential scan of the file.

In contrast, if the search key of a secondary index is not a candidate key, it is not enough to point to just the first record with each search-key value. The remaining records with the same search-key value could be anywhere in the file, since the records are ordered by the search key of the primary index, rather than by the search key of the secondary index. Therefore, a secondary index must contain pointers to all the records.

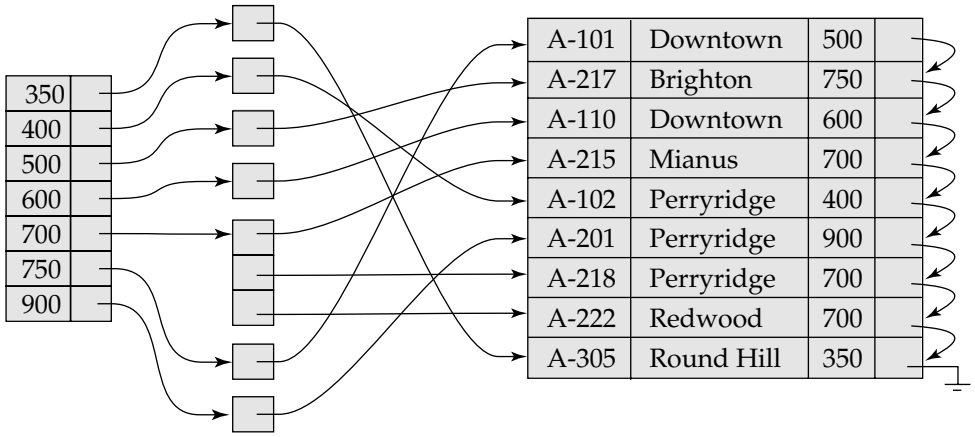


Figure 12.5 Secondary index on *account* file, on noncandidate key *balance*.

We can use an extra level of indirection to implement secondary indices on search keys that are not candidate keys. The pointers in such a secondary index do not point directly to the file. Instead, each points to a bucket that contains pointers to the file. Figure 12.5 shows the structure of a secondary index that uses an extra level of indirection on the *account* file, on the search key *balance*.

A sequential scan in primary index order is efficient because records in the file are stored physically in the same order as the index order. However, we cannot (except in rare special cases) store a file physically ordered both by the search key of the primary index, and the search key of a secondary index. Because secondary-key order and physical-key order differ, if we attempt to scan the file sequentially in secondary-key order, the reading of each record is likely to require the reading of a new block from disk, which is very slow.

The procedure described earlier for deletion and insertion can also be applied to secondary indices; the actions taken are those described for dense indices storing a pointer to every record in the file. If a file has multiple indices, whenever the file is modified, *every* index must be updated.

Secondary indices improve the performance of queries that use keys other than the search key of the primary index. However, they impose a significant overhead on modification of the database. The designer of a database decides which secondary indices are desirable on the basis of an estimate of the relative frequency of queries and modifications.

12.3 B+-Tree Index Files

The main disadvantage of the index-sequential file organization is that performance degrades as the file grows, both for index lookups and for sequential scans through the data. Although this degradation can be remedied by reorganization of the file, frequent reorganizations are undesirable.

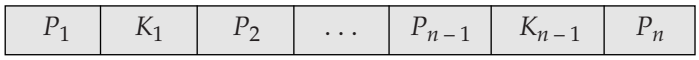


Figure 12.6 Typical node of a B$^+$-tree.

The **B$^+$-tree** index structure is the most widely used of several index structures that maintain their efficiency despite insertion and deletion of data. A B$^+$-tree index takes the form of a **balanced tree** in which every path from the root of the tree to a leaf of the tree is of the same length. Each nonleaf node in the tree has between $\lceil n/2 \rceil$ and n children, where n is fixed for a particular tree.

We shall see that the B$^+$-tree structure imposes performance overhead on insertion and deletion, and adds space overhead. The overhead is acceptable even for frequently modified files, since the cost of file reorganization is avoided. Furthermore, since nodes may be as much as half empty (if they have the minimum number of children), there is some wasted space. This space overhead, too, is acceptable given the performance benefits of the B$^+$-tree structure.

12.3.1 Structure of a B$^+$-Tree

A B$^+$-tree index is a multilevel index, but it has a structure that differs from that of the multilevel index-sequential file. Figure 12.6 shows a typical node of a B$^+$-tree. It contains up to $n - 1$ search-key values $K_1, K_2, \ldots, K_{n-1}$, and n pointers $P_1, P_2, \ldots, P_n$. The search-key values within a node are kept in sorted order; thus, if $i < j$, then $K_i < K_j$.

We consider first the structure of the leaf nodes. For $i = 1, 2, \ldots, n - 1$, pointer P_i points to either a file record with search-key value K_i or to a bucket of pointers, each of which points to a file record with search-key value K_i. The bucket structure is used only if the search key does not form a primary key, and if the file is not sorted in the search-key value order. Pointer P_n has a special purpose that we shall discuss shortly.

Figure 12.7 shows one leaf node of a B$^+$-tree for the *account* file, in which we have chosen n to be 3, and the search key is *branch-name*. Note that, since the account file is ordered by *branch-name*, the pointers in the leaf node point directly to the file.

Now that we have seen the structure of a leaf node, let us consider how search-key values are assigned to particular nodes. Each leaf can hold up to $n - 1$ values. We allow leaf nodes to contain as few as $\lceil (n-1)/2 \rceil$ values. The ranges of values in each leaf do not overlap. Thus, if L_i and L_j are leaf nodes and $i < j$, then every search-key value in L_i is less than every search-key value in L_j. If the B$^+$-tree index is to be a dense index, every search-key value must appear in some leaf node.

Now we can explain the use of the pointer P_n. Since there is a linear order on the leaves based on the search-key values that they contain, we use P_n to chain together the leaf nodes in search-key order. This ordering allows for efficient sequential processing of the file.

The nonleaf nodes of the B$^+$-tree form a multilevel (sparse) index on the leaf nodes. The structure of nonleaf nodes is the same as that for leaf nodes, except that all pointers are pointers to tree nodes. A nonleaf node may hold up to n pointers, and *must*

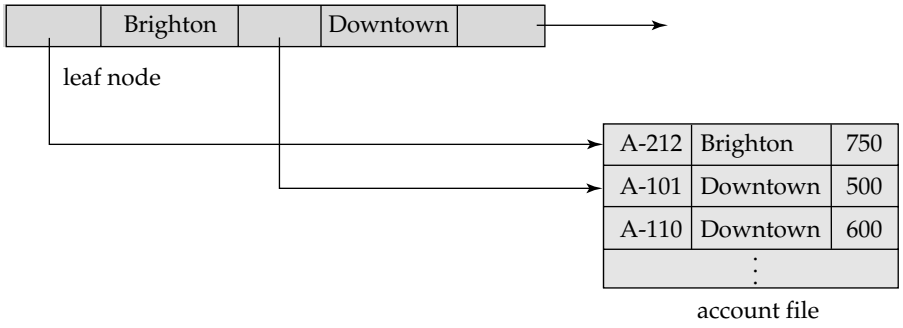


Figure 12.7 A leaf node for *account* B+-tree index ($n = 3$).

hold at least $\lceil n/2 \rceil$ pointers. The number of pointers in a node is called the *fanout* of the node.

Let us consider a node containing m pointers. For $i = 2, 3, \ldots, m - 1$, pointer P_i points to the subtree that contains search-key values less than K_i and greater than or equal to K_{i-1}. Pointer P_m points to the part of the subtree that contains those key values greater than or equal to K_{m-1}, and pointer P_1 points to the part of the subtree that contains those search-key values less than K_1.

Unlike other nonleaf nodes, the root node can hold fewer than $\lceil n/2 \rceil$ pointers; however, it must hold at least two pointers, unless the tree consists of only one node. It is always possible to construct a B+-tree, for any n, that satisfies the preceding requirements. Figure 12.8 shows a complete B+-tree for the *account* file ($n = 3$). For simplicity, we have omitted both the pointers to the file itself and the null pointers. As an example of a B+-tree for which the root must have less than $\lceil n/2 \rceil$ values, Figure 12.9 shows a B+-tree for the *account* file with $n = 5$.

These examples of B+-trees are all balanced. That is, the length of every path from the root to a leaf node is the same. This property is a requirement for a B+-tree. Indeed, the "B" in B+-tree stands for "balanced." It is the balance property of B+-trees that ensures good performance for lookup, insertion, and deletion.

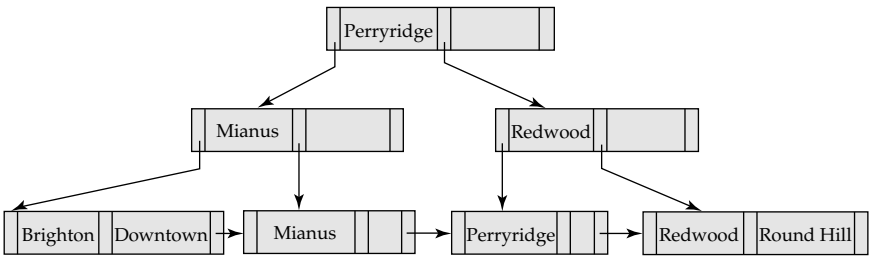


Figure 12.8 B+-tree for *account* file ($n = 3$).

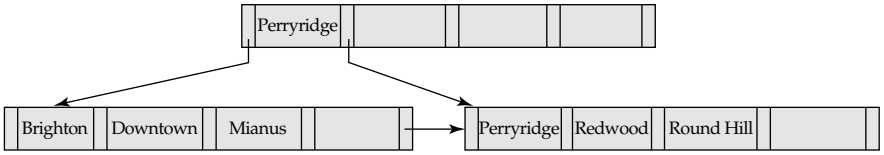


Figure 12.9 B$^+$-tree for *account* file with $n = 5$.

12.3.2 Queries on B$^+$-Trees

Let us consider how we process queries on a B$^+$-tree. Suppose that we wish to find all records with a search-key value of V. Figure 12.10 presents pseudocode for doing so. Intuitively, the procedure works as follows. First, we examine the root node, looking for the smallest search-key value greater than V. Suppose that we find that this search-key value is K_i. We then follow pointer P_i to another node. If we find no such value, then $k \geq K_{m-1}$, where m is the number of pointers in the node. In this case we follow P_m to another node. In the node we reached above, again we look for the smallest search-key value greater than V, and once again follow the corresponding pointer as above. Eventually, we reach a leaf node. At the leaf node, if we find search-key value K_i equals V, then pointer P_i directs us to the desired record or bucket. If the value V is not found in the leaf node, no record with key value V exists.

Thus, in processing a query, we traverse a path in the tree from the root to some leaf node. If there are K search-key values in the file, the path is no longer than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$.

In practice, only a few nodes need to be accessed, Typically, a node is made to be the same size as a disk block, which is typically 4 kilobytes. With a search-key size of 12 bytes, and a disk-pointer size of 8 bytes, n is around 200. Even with a more conservative estimate of 32 bytes for the search-key size, n is around 100. With $n = 100$, if we have 1 million search-key values in the file, a lookup requires only

```
procedure find(value V)
    set C = root node
    while C is not a leaf node begin
        Let K_i = smallest search-key value, if any, greater than V
        if there is no such value then begin
            Let m = the number of pointers in the node
            set C = node pointed to by P_m
        end
            else set C = the node pointed to by P_i
    end
    if there is a key value K_i in C such that K_i = V
        then pointer P_i directs us to the desired record or bucket
        else no record with key value k exists
end procedure
```

Figure 12.10 Querying a B$^+$-tree.

$\lceil \log_{50}(1{,}000{,}000) \rceil = 4$ nodes to be accessed. Thus, at most four blocks need to be read from disk for the lookup. The root node of the tree is usually heavily accessed and is likely to be in the buffer, so typically only three or fewer blocks need to be read from disk.

An important difference between B$^+$-tree structures and in-memory tree structures, such as binary trees, is the size of a node, and as a result, the height of the tree. In a binary tree, each node is small, and has at most two pointers. In a B$^+$-tree, each node is large—typically a disk block—and a node can have a large number of pointers. Thus, B$^+$-trees tend to be fat and short, unlike thin and tall binary trees. In a balanced binary tree, the path for a lookup can be of length $\lceil \log_2(K) \rceil$, where K is the number of search-key values. With $K = 1{,}000{,}000$ as in the previous example, a balanced binary tree requires around 20 node accesses. If each node were on a different disk block, 20 block reads would be required to process a lookup, in contrast to the four block reads for the B$^+$-tree.

12.3.3 Updates on B$^+$-Trees

Insertion and deletion are more complicated than lookup, since it may be necessary to **split** a node that becomes too large as the result of an insertion, or to **coalesce** nodes (that is, combine nodes) if a node becomes too small (fewer than $\lceil n/2 \rceil$ pointers). Furthermore, when a node is split or a pair of nodes is combined, we must ensure that balance is preserved. To introduce the idea behind insertion and deletion in a B$^+$-tree, we shall assume temporarily that nodes never become too large or too small. Under this assumption, insertion and deletion are performed as defined next.

- **Insertion**. Using the same technique as for lookup, we find the leaf node in which the search-key value would appear. If the search-key value already appears in the leaf node, we add the new record to the file and, if necessary, add to the bucket a pointer to the record. If the search-key value does not appear, we insert the value in the leaf node, and position it such that the search keys are still in order. We then insert the new record in the file and, if necessary, create a new bucket with the appropriate pointer.
- **Deletion**. Using the same technique as for lookup, we find the record to be deleted, and remove it from the file. We remove the search-key value from the leaf node if there is no bucket associated with that search-key value or if the bucket becomes empty as a result of the deletion.

We now consider an example in which a node must be split. Assume that we wish to insert a record with a *branch-name* value of "Clearview" into the B$^+$-tree of Figure 12.8. Using the algorithm for lookup, we find that "Clearview" should appear in the node containing "Brighton" and "Downtown." There is no room to insert the search-key value "Clearview." Therefore, the node is *split* into two nodes. Figure 12.11 shows the two leaf nodes that result from inserting "Clearview" and splitting the node containing "Brighton" and "Downtown." In general, we take the n search-key

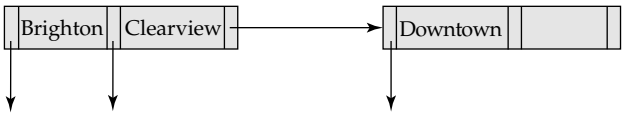


Figure 12.11 Split of leaf node on insertion of "Clearview."

values (the $n - 1$ values in the leaf node plus the value being inserted), and put the first $\lceil n/2 \rceil$ in the existing node and the remaining values in a new node.

Having split a leaf node, we must insert the new leaf node into the B$^+$-tree structure. In our example, the new node has "Downtown" as its smallest search-key value. We need to insert this search-key value into the parent of the leaf node that was split. The B$^+$-tree of Figure 12.12 shows the result of the insertion. The search-key value "Downtown" was inserted into the parent. It was possible to perform this insertion because there was room for an added search-key value. If there were no room, the parent would have had to be split. In the worst case, all nodes along the path to the root must be split. If the root itself is split, the entire tree becomes deeper.

The general technique for insertion into a B$^+$-tree is to determine the leaf node l into which insertion must occur. If a split results, insert the new node into the parent of node l. If this insertion causes a split, proceed recursively up the tree until either an insertion does not cause a split or a new root is created.

Figure 12.13 outlines the insertion algorithm in pseudocode. In the pseudocode, $L.K_i$ and $L.P_i$ denote the ith value and the ith pointer in node L, respectively. The pseudocode also makes use of the function $parent(L)$ to find the parent of a node L. We can compute a list of nodes in the path from the root to the leaf while initially finding the leaf node, and can use it later to find the parent of any node in the path efficiently. The pseudocode refers to inserting an entry (V, P) into a node. In the case of leaf nodes, the pointer to an entry actually precedes the key value, so the leaf node actually stores P before V. For internal nodes, P is stored just after V.

We now consider deletions that cause tree nodes to contain too few pointers. First, let us delete "Downtown" from the B$^+$-tree of Figure 12.12. We locate the entry for "Downtown" by using our lookup algorithm. When we delete the entry for "Downtown" from its leaf node, the leaf becomes empty. Since, in our example, $n = 3$ and $0 < \lceil (n-1)/2 \rceil$, this node must be eliminated from the B$^+$-tree. To delete a leaf node,

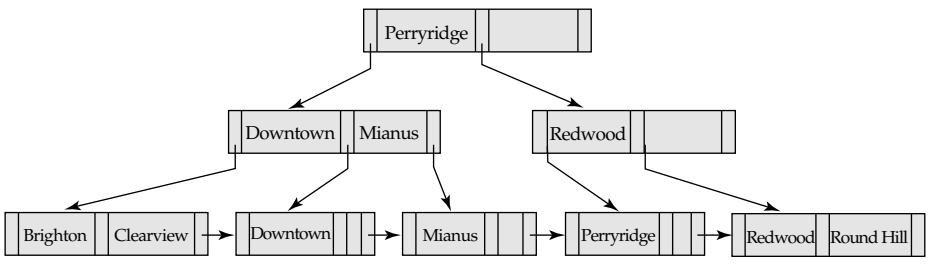


Figure 12.12 Insertion of "Clearview" into the B$^+$-tree of Figure 12.8.

```
procedure insert(value V, pointer P)
    find the leaf node L that should contain value V
    insert_entry(L, V, P)
end procedure

procedure insert_entry(node L, value V, pointer P)
    if (L has space for (V, P))
        then insert (V, P) in L
        else begin /* Split L */
            Create node L'
            Let V' be the value in L.K_1, ..., L.K_{n-1}, V such that exactly
                ⌈n/2⌉ of the values L.K_1, ..., L.K_{n-1}, V are less than V'
            Let m be the lowest value such that L.K_m ≥ V'
            /* Note: V' must be either L.K_m or V */
            if (L is a leaf) then begin
                move L.P_m, L.K_m, ..., L.P_{n-1}, L.K_{n-1} to L'
                if (V < V') then insert (P, V) in L
                else insert (P, V) in L'
            end
            else begin
                if (V = V') /* V is smallest value to go to L' */
                    then add P, L.K_m, ..., L.P_{n-1}, L.K_{n-1}, L.P_n to L'
                    else add L.P_m, ..., L.P_{n-1}, L.K_{n-1}, L.P_n to L'
                delete L.K_m, ..., L.P_{n-1}, L.K_{n-1}, L.P_n from L
                if (V < V') then insert (V, P) in L
                else if (V > V') then insert (V, P) in L'
                /* Case of V = V' handled already */
            end
            if (L is not the root of the tree)
                then insert_entry(parent(L), V', L');
                else begin
                    create a new node R with child nodes L and L' and
                        the single value V'
                    make R the root of the tree
                end
            if (L) is a leaf node then begin /* Fix next child pointers */
                set L'.P_n = L.P_n;
                set L.P_n = L'
            end
        end
end procedure
```

Figure 12.13 Insertion of entry in a B$^+$-tree.

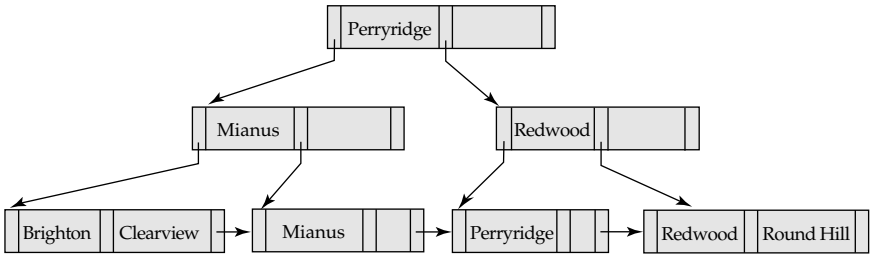


Figure 12.14 Deletion of "Downtown" from the B$^+$-tree of Figure 12.12.

we must delete the pointer to it from its parent. In our example, this deletion leaves the parent node, which formerly contained three pointers, with only two pointers. Since $2 \geq \lceil n/2 \rceil$, the node is still sufficiently large, and the deletion operation is complete. The resulting B$^+$-tree appears in Figure 12.14.

When we make a deletion from a parent of a leaf node, the parent node itself may become too small. That is exactly what happens if we delete "Perryridge" from the B$^+$-tree of Figure 12.14. Deletion of the Perryridge entry causes a leaf node to become empty. When we delete the pointer to this node in the latter's parent, the parent is left with only one pointer. Since $n = 3$, $\lceil n/2 \rceil = 2$, and thus only one pointer is too few. However, since the parent node contains useful information, we cannot simply delete it. Instead, we look at the sibling node (the nonleaf node containing the one search key, Mianus). This sibling node has room to accommodate the information contained in our now-too-small node, so we coalesce these nodes, such that the sibling node now contains the keys "Mianus" and "Redwood." The other node (the node containing only the search key "Redwood") now contains redundant information and can be deleted from its parent (which happens to be the root in our example). Figure 12.15 shows the result. Notice that the root has only one child pointer after the deletion, so it is deleted and its sole child becomes the root. So the depth of the B$^+$-tree has been decreased by 1.

It is not always possible to coalesce nodes. As an illustration, delete "Perryridge" from the B$^+$-tree of Figure 12.12. In this example, the "Downtown" entry is still part of the tree. Once again, the leaf node containing "Perryridge" becomes empty. The parent of the leaf node becomes too small (only one pointer). However, in this example, the sibling node already contains the maximum number of pointers: three. Thus, it cannot accommodate an additional pointer. The solution in this case is to **redistribute** the pointers such that each sibling has two pointers. The result appears in

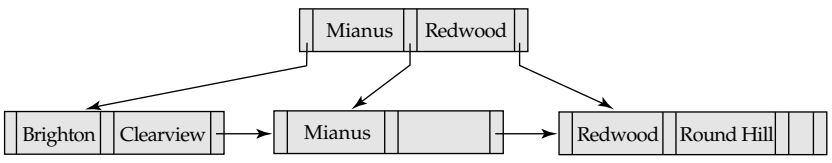


Figure 12.15 Deletion of "Perryridge" from the B$^+$-tree of Figure 12.14.

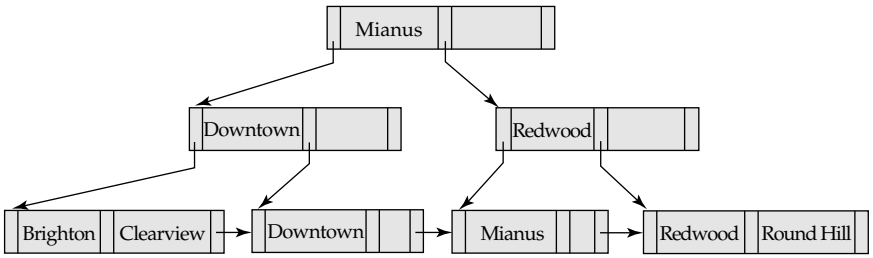


Figure 12.16 Deletion of "Perryridge" from the B+-tree of Figure 12.12.

Figure 12.16. Note that the redistribution of values necessitates a change of a search-key value in the parent of the two siblings.

In general, to delete a value in a B+-tree, we perform a lookup on the value and delete it. If the node is too small, we delete it from its parent. This deletion results in recursive application of the deletion algorithm until the root is reached, a parent remains adequately full after deletion, or redistribution is applied.

Figure 12.17 outlines the pseudocode for deletion from a B+-tree. The procedure swap_variables(L, L') merely swaps the values of the (pointer) variables L and L'; this swap has no effect on the tree itself. The pseudocode uses the condition "too few pointers/values." For nonleaf nodes, this criterion means less than $\lceil n/2 \rceil$ pointers; for leaf nodes, it means less than $\lceil (n-1)/2 \rceil$ values. The pseudocode redistributes entries by borrowing a single entry from an adjacent node. We can also redistribute entries by repartitioning entries equally between the two nodes. The pseudocode refers to deleting an entry (V, P) from a node. In the case of leaf nodes, the pointer to an entry actually precedes the key value, so the pointer P precedes the key value V. For internal nodes, P follows the key value V.

It is worth noting that, as a result of deletion, a key value that is present in an internal node of the B+-tree may not be present at any leaf of the tree.

Although insertion and deletion operations on B+-trees are complicated, they require relatively few I/O operations, which is an important benefit since I/O operations are expensive. It can be shown that the number of I/O operations needed for a worst-case insertion or deletion is proportional to $\log_{\lceil n/2 \rceil}(K)$, where n is the maximum number of pointers in a node, and K is the number of search-key values. In other words, the cost of insertion and deletion operations is proportional to the height of the B+-tree, and is therefore low. It is the speed of operation on B+-trees that makes them a frequently used index structure in database implementations.

12.3.4 B+-Tree File Organization

As mentioned in Section 12.3, the main drawback of index-sequential file organization is the degradation of performance as the file grows: With growth, an increasing percentage of index records and actual records become out of order, and are stored in overflow blocks. We solve the degradation of index lookups by using B+-tree indices on the file. We solve the degradation problem for storing the actual records by using the leaf level of the B+-tree to organize the blocks containing the actual records. We

```
procedure delete(value V, pointer P)
    find the leaf node L that contains (V, P)
    delete_entry(L, V, P)
end procedure
procedure delete_entry(node L, value V, pointer P)
    delete (V, P) from L
    if (L is the root and L has only one remaining child)
    then make the child of L the new root of the tree and delete L
    else if (L has too few values/pointers) then begin
        Let L' be the previous or next child of parent(L)
        Let V' be the value between pointers L and L' in parent(L)
        if (entries in L and L' can fit in a single node)
            then begin /* Coalesce nodes */
                if (L is a predecessor of L') then swap_variables(L, L')
                if (L is not a leaf)
                    then append V' and all pointers and values in L to L'
                    else  append all (K_i, P_i) pairs in L to L'; set L'.P_n = L.P_n
                delete_entry(parent(L), V', L); delete node L
            end
        else begin /* Redistribution: borrow an entry from L' */
            if (L' is a predecessor of L) then begin
                if (L is a non-leaf node) then begin
                    let m be such that L'.P_m is the last pointer in L'
                    remove (L'.K_{m-1}, L'.P_m) from L'
                    insert (L'.P_m, V') as the first pointer and value in L,
                        by shifting other pointers and values right
                    replace V' in parent(L) by L'.K_{m-1}
                end
                else begin
                    let m be such that (L'.P_m, L'.K_m) is the last pointer/value
                        pair in L'
                    remove (L'.P_m, L'.K_m) from L'
                    insert (L'.P_m, L'.K_m) as the first pointer and value in L,
                        by shifting other pointers and values right
                    replace V' in parent(L) by L'.K_m
                end
            end
            else  ... symmetric to the then case ...
        end
    end
end procedure
```

Figure 12.17 Deletion of entry from a B^+-tree.

use the B+-tree structure not only as an index, but also as an organizer for records in a file. In a **B+-tree file organization**, the leaf nodes of the tree store records, instead of storing pointers to records. Figure 12.18 shows an example of a B+-tree file organization. Since records are usually larger than pointers, the maximum number of records that can be stored in a leaf node is less than the number of pointers in a nonleaf node. However, the leaf nodes are still required to be at least half full.

Insertion and deletion of records from a B+-tree file organization are handled in the same way as insertion and deletion of entries in a B+-tree index. When a record with a given key value v is inserted, the system locates the block that should contain the record by searching the B+-tree for the largest key in the tree that is $\leq v$. If the block located has enough free space for the record, the system stores the record in the block. Otherwise, as in B+-tree insertion, the system splits the block in two, and redistributes the records in it (in the B+-tree–key order) to create space for the new record. The split propagates up the B+-tree in the normal fashion. When we delete a record, the system first removes it from the block containing it. If a block B becomes less than half full as a result, the records in B are redistributed with the records in an adjacent block B'. Assuming fixed-sized records, each block will hold at least one-half as many records as the maximum that it can hold. The system updates the nonleaf nodes of the B+-tree in the usual fashion.

When we use a B+-tree for file organization, space utilization is particularly important, since the space occupied by the records is likely to be much more than the space occupied by keys and pointers. We can improve the utilization of space in a B+-tree by involving more sibling nodes in redistribution during splits and merges. The technique is applicable to both leaf nodes and internal nodes, and works as follows.

During insertion, if a node is full the system attempts to redistribute some of its entries to one of the adjacent nodes, to make space for a new entry. If this attempt fails because the adjacent nodes are themselves full, the system splits the node, and splits the entries evenly among one of the adjacent nodes and the two nodes that it obtained by splitting the original node. Since the three nodes together contain one more record than can fit in two nodes, each node will be about two-thirds full. More precisely, each node will have at least $\lfloor 2n/3 \rfloor$ entries, where n is the maximum number of entries that the node can hold. ($\lfloor x \rfloor$ denotes the greatest integer that is less than or equal to x; that is, we drop the fractional part, if any.)

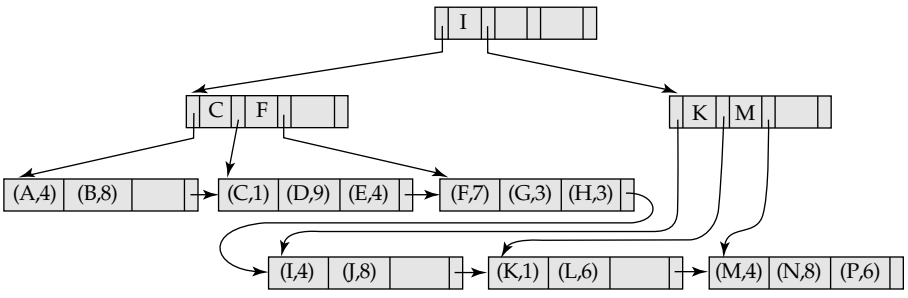


Figure 12.18 B+-tree file organization.

During deletion of a record, if the occupancy of a node falls below $\lfloor 2n/3 \rfloor$, the system attempts to borrow an entry from one of the sibling nodes. If both sibling nodes have $\lfloor 2n/3 \rfloor$ records, instead of borrowing an entry, the system redistributes the entries in the node and in the two siblings evenly between two of the nodes, and deletes the third node. We can use this approach because the total number of entries is $3\lfloor 2n/3 \rfloor - 1$, which is less than $2n$. With three adjacent nodes used for redistribution, each node can be guaranteed to have $\lfloor 3n/4 \rfloor$ entries. In general, if m nodes ($m - 1$ siblings) are involved in redistribution, each node can be guaranteed to contain at least $\lfloor (m - 1)n/m \rfloor$ entries. However, the cost of update becomes higher as more sibling nodes are involved in the redistribution.

12.4 B-Tree Index Files

B-tree indices are similar to B$^+$-tree indices. The primary distinction between the two approaches is that a B-tree eliminates the redundant storage of search-key values. In the B$^+$-tree of Figure 12.12, the search keys "Downtown," "Mianus," "Redwood," and "Perryridge" appear twice. Every search-key value appears in some leaf node; several are repeated in nonleaf nodes.

A B-tree allows search-key values to appear only once. Figure 12.19 shows a B-tree that represents the same search keys as the B$^+$-tree of Figure 12.12. Since search keys are not repeated in the B-tree, we may be able to store the index in fewer tree nodes than in the corresponding B$^+$-tree index. However, since search keys that appear in nonleaf nodes appear nowhere else in the B-tree, we are forced to include an additional pointer field for each search key in a nonleaf node. These additional pointers point to either file records or buckets for the associated search key.

A generalized B-tree leaf node appears in Figure 12.20a; a nonleaf node appears in Figure 12.20b. Leaf nodes are the same as in B$^+$-trees. In nonleaf nodes, the pointers P_i are the tree pointers that we used also for B$^+$-trees, while the pointers B_i are bucket or file-record pointers. In the generalized B-tree in the figure, there are $n - 1$ keys in the leaf node, but there are $m - 1$ keys in the nonleaf node. This discrepancy occurs because nonleaf nodes must include pointers B_i, thus reducing the number of

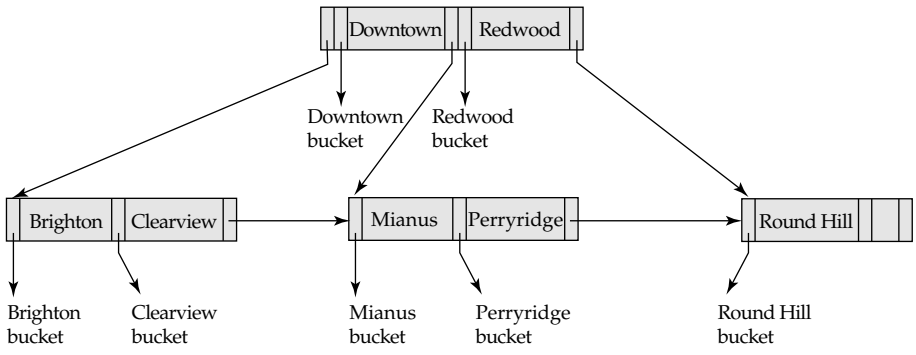


Figure 12.19 B-tree equivalent of B$^+$-tree in Figure 12.12.

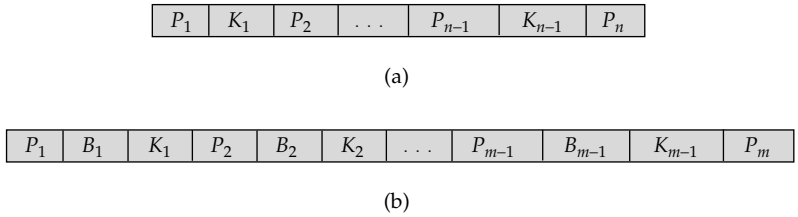


Figure 12.20 Typical nodes of a B-tree. (a) Leaf node. (b) Nonleaf node.

search keys that can be held in these nodes. Clearly, $m < n$, but the exact relationship between m and n depends on the relative size of search keys and pointers.

The number of nodes accessed in a lookup in a B-tree depends on where the search key is located. A lookup on a B$^+$-tree requires traversal of a path from the root of the tree to some leaf node. In contrast, it is sometimes possible to find the desired value in a B-tree before reaching a leaf node. However, roughly n times as many keys are stored in the leaf level of a B-tree as in the nonleaf levels, and, since n is typically large, the benefit of finding certain values early is relatively small. Moreover, the fact that fewer search keys appear in a nonleaf B-tree node, compared to B$^+$-trees, implies that a B-tree has a smaller fanout and therefore may have depth greater than that of the corresponding B$^+$-tree. Thus, lookup in a B-tree is faster for some search keys but slower for others, although, in general, lookup time is still proportional to the logarithm of the number of search keys.

Deletion in a B-tree is more complicated. In a B$^+$-tree, the deleted entry always appears in a leaf. In a B-tree, the deleted entry may appear in a nonleaf node. The proper value must be selected as a replacement from the subtree of the node containing the deleted entry. Specifically, if search key K_i is deleted, the smallest search key appearing in the subtree of pointer P_{i+1} must be moved to the field formerly occupied by K_i. Further actions need to be taken if the leaf node now has too few entries. In contrast, insertion in a B-tree is only slightly more complicated than is insertion in a B$^+$-tree.

The space advantages of B-trees are marginal for large indices, and usually do not outweigh the disadvantages that we have noted. Thus, many database system implementers prefer the structural simplicity of a B$^+$-tree. The exercises explore details of the insertion and deletion algorithms for B-trees.

12.5 Static Hashing

One disadvantage of sequential file organization is that we must access an index structure to locate data, or must use binary search, and that results in more I/O operations. File organizations based on the technique of **hashing** allow us to avoid accessing an index structure. Hashing also provides a way of constructing indices. We study file organizations and indices based on hashing in the following sections.

12.5.1 Hash File Organization

In a **hash file organization**, we obtain the address of the disk block containing a desired record directly by computing a function on the search-key value of the record. In our description of hashing, we shall use the term **bucket** to denote a unit of storage that can store one or more records. A bucket is typically a disk block, but could be chosen to be smaller or larger than a disk block.

Formally, let K denote the set of all search-key values, and let B denote the set of all bucket addresses. A **hash function** h is a function from K to B. Let h denote a hash function.

To insert a record with search key K_i, we compute $h(K_i)$, which gives the address of the bucket for that record. Assume for now that there is space in the bucket to store the record. Then, the record is stored in that bucket.

To perform a lookup on a search-key value K_i, we simply compute $h(K_i)$, then search the bucket with that address. Suppose that two search keys, K_5 and K_7, have the same hash value; that is, $h(K_5) = h(K_7)$. If we perform a lookup on K_5, the bucket $h(K_5)$ contains records with search-key values K_5 and records with search-key values K_7. Thus, we have to check the search-key value of every record in the bucket to verify that the record is one that we want.

Deletion is equally straightforward. If the search-key value of the record to be deleted is K_i, we compute $h(K_i)$, then search the corresponding bucket for that record, and delete the record from the bucket.

12.5.1.1 Hash Functions

The worst possible hash function maps all search-key values to the same bucket. Such a function is undesirable because all the records have to be kept in the same bucket. A lookup has to examine every such record to find the one desired. An ideal hash function distributes the stored keys uniformly across all the buckets, so that every bucket has the same number of records.

Since we do not know at design time precisely which search-key values will be stored in the file, we want to choose a hash function that assigns search-key values to buckets in such a way that the distribution has these qualities:

- The distribution is *uniform*. That is, the hash function assigns each bucket the same number of search-key values from the set of *all* possible search-key values.
- The distribution is *random*. That is, in the average case, each bucket will have nearly the same number of values assigned to it, regardless of the actual distribution of search-key values. More precisely, the hash value will not be correlated to any externally visible ordering on the search-key values, such as alphabetic ordering or ordering by the length of the search keys; the hash function will appear to be random.

As an illustration of these principles, let us choose a hash function for the *account* file using the search key *branch-name*. The hash function that we choose must have

the desirable properties not only on the example *account* file that we have been using, but also on an *account* file of realistic size for a large bank with many branches.

Assume that we decide to have 26 buckets, and we define a hash function that maps names beginning with the *i*th letter of the alphabet to the *i*th bucket. This hash function has the virtue of simplicity, but it fails to provide a uniform distribution, since we expect more branch names to begin with such letters as B and R than Q and X, for example.

Now suppose that we want a hash function on the search key *balance*. Suppose that the minimum balance is 1 and the maximum balance is 100,000, and we use a hash function that divides the values into 10 ranges, 1–10,000, 10,001–20,000 and so on. The distribution of search-key values is uniform (since each bucket has the same number of different *balance* values), but is not random. But records with balances between 1 and 10,000 are far more common than are records with balances between 90,001 and 100,000. As a result, the distribution of records is not uniform—some buckets receive more records than others do. If the function has a random distribution, even if there are such correlations in the search keys, the randomness of the distribution will make it very likely that all buckets will have roughly the same number of records, as long as each search key occurs in only a small fraction of the records. (If a single search key occurs in a large fraction of the records, the bucket containing it is likely to have more records than other buckets, regardless of the hash function used.)

Typical hash functions perform computation on the internal binary machine representation of characters in the search key. A simple hash function of this type first computes the sum of the binary representations of the characters of a key, then returns the sum modulo the number of buckets. Figure 12.21 shows the application of such a scheme, with 10 buckets, to the *account* file, under the assumption that the *i*th letter in the alphabet is represented by the integer *i*.

Hash functions require careful design. A bad hash function may result in lookup taking time proportional to the number of search keys in the file. A well-designed function gives an average-case lookup time that is a (small) constant, independent of the number of search keys in the file.

12.5.1.2 Handling of Bucket Overflows

So far, we have assumed that, when a record is inserted, the bucket to which it is mapped has space to store the record. If the bucket does not have enough space, a **bucket overflow** is said to occur. Bucket overflow can occur for several reasons:

- **Insufficient buckets**. The number of buckets, which we denote n_B, must be chosen such that $n_B > n_r/f_r$, where n_r denotes the total number of records that will be stored, and f_r denotes the number of records that will fit in a bucket. This designation, of course, assumes that the total number of records is known when the hash function is chosen.
- **Skew**. Some buckets are assigned more records than are others, so a bucket may overflow even when other buckets still have space. This situation is called bucket **skew**. Skew can occur for two reasons:

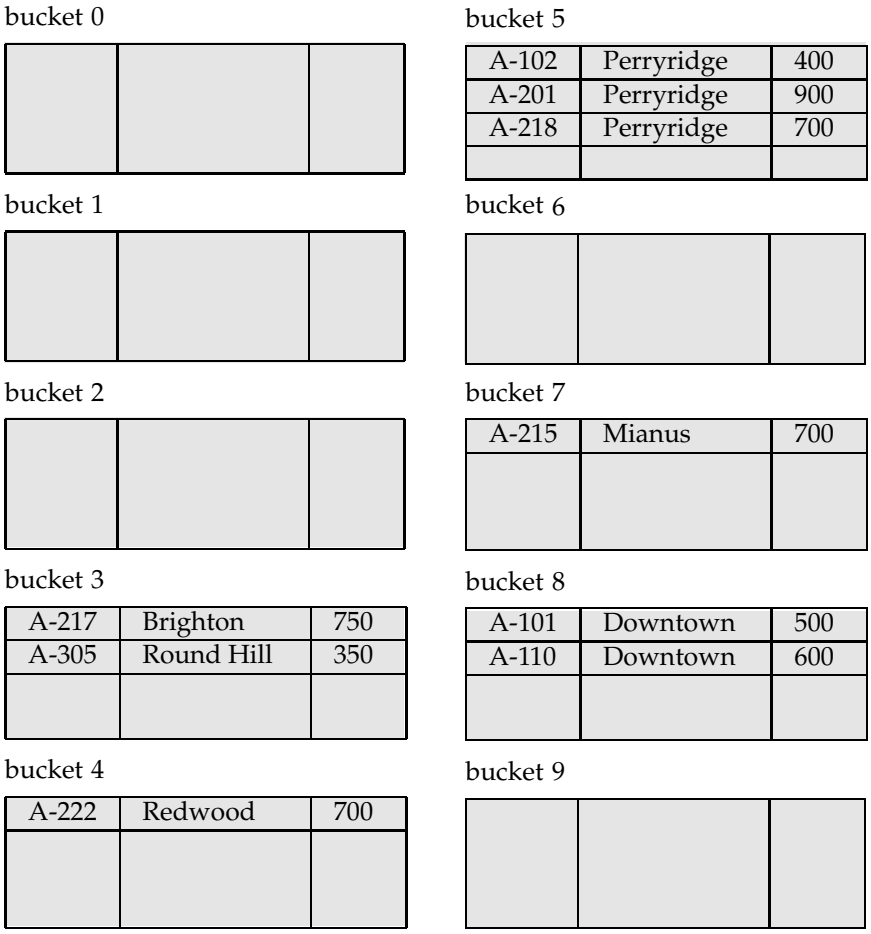


Figure 12.21 Hash organization of *account* file, with *branch-name* as the key.

1. Multiple records may have the same search key.
2. The chosen hash function may result in nonuniform distribution of search keys.

So that the probability of bucket overflow is reduced, the number of buckets is chosen to be $(n_r/f_r) * (1 + d)$, where d is a fudge factor, typically around 0.2. Some space is wasted: About 20 percent of the space in the buckets will be empty. But the benefit is that the probability of overflow is reduced.

Despite allocation of a few more buckets than required, bucket overflow can still occur. We handle bucket overflow by using **overflow buckets**. If a record must be inserted into a bucket b, and b is already full, the system provides an overflow bucket for b, and inserts the record into the overflow bucket. If the overflow bucket is also full, the system provides another overflow bucket, and so on. All the overflow buck-

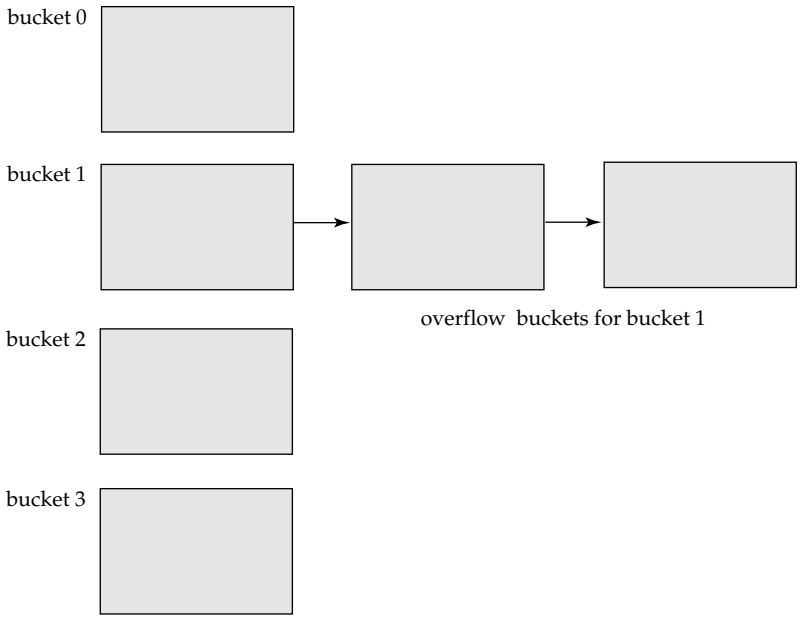


Figure 12.22 Overflow chaining in a hash structure.

ets of a given bucket are chained together in a linked list, as in Figure 12.22. Overflow handling using such a linked list is called **overflow chaining**.

We must change the lookup algorithm slightly to handle overflow chaining. As before, the system uses the hash function on the search key to identify a bucket b. The system must examine all the records in bucket b to see whether they match the search key, as before. In addition, if bucket b has overflow buckets, the system must examine the records in all the overflow buckets also.

The form of hash structure that we have just described is sometimes referred to as **closed hashing**. Under an alternative approach, called **open hashing**, the set of buckets is fixed, and there are no overflow chains. Instead, if a bucket is full, the system inserts records in some other bucket in the initial set of buckets B. One policy is to use the next bucket (in cyclic order) that has space; this policy is called *linear probing*. Other policies, such as computing further hash functions, are also used. Open hashing has been used to construct symbol tables for compilers and assemblers, but closed hashing is preferable for database systems. The reason is that deletion under open hashing is troublesome. Usually, compilers and assemblers perform only lookup and insertion operations on their symbol tables. However, in a database system, it is important to be able to handle deletion as well as insertion. Thus, open hashing is of only minor importance in database implementation.

An important drawback to the form of hashing that we have described is that we must choose the hash function when we implement the system, and it cannot be changed easily thereafter if the file being indexed grows or shrinks. Since the function h maps search-key values to a fixed set B of bucket addresses, we waste space if B is

made large to handle future growth of the file. If B is too small, the buckets contain records of many different search-key values, and bucket overflows can occur. As the file grows, performance suffers. We study later, in Section 12.6, how the number of buckets and the hash function can be changed dynamically.

12.5.2 Hash Indices

Hashing can be used not only for file organization, but also for index-structure creation. A **hash index** organizes the search keys, with their associated pointers, into a hash file structure. We construct a hash index as follows. We apply a hash function on a search key to identify a bucket, and store the key and its associated pointers in the bucket (or in overflow buckets). Figure 12.23 shows a secondary hash index on the *account* file, for the search key *account-number*. The hash function in the figure computes the sum of the digits of the account number modulo 7. The hash index has seven buckets, each of size 2 (realistic indices would, of course, have much larger

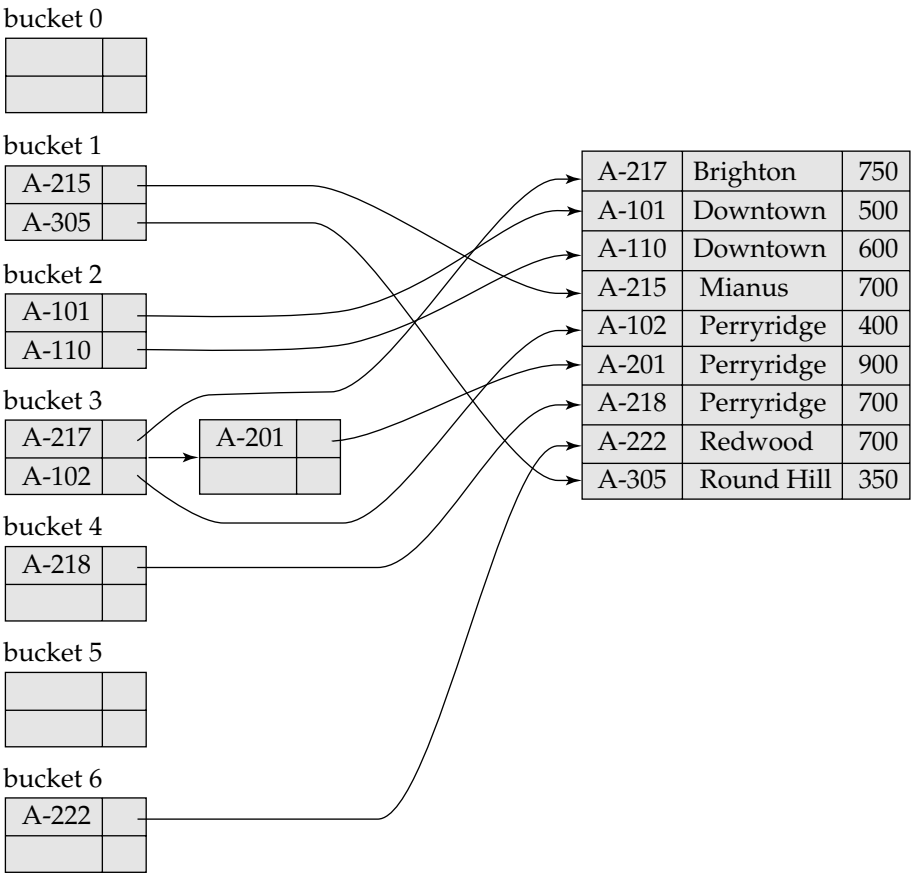


Figure 12.23 Hash index on search key *account-number* of *account* file.

bucket sizes). One of the buckets has three keys mapped to it, so it has an overflow bucket. In this example, *account-number* is a primary key for *account*, so each search-key has only one associated pointer. In general, multiple pointers can be associated with each key.

We use the term *hash index* to denote hash file structures as well as secondary hash indices. Strictly speaking, hash indices are only secondary index structures. A hash index is never needed as a primary index structure, since, if a file itself is organized by hashing, there is no need for a separate hash index structure on it. However, since hash file organization provides the same direct access to records that indexing provides, we pretend that a file organized by hashing also has a primary hash index on it.

12.6 Dynamic Hashing

As we have seen, the need to fix the set B of bucket addresses presents a serious problem with the static hashing technique of the previous section. Most databases grow larger over time. If we are to use static hashing for such a database, we have three classes of options:

1. Choose a hash function based on the current file size. This option will result in performance degradation as the database grows.
2. Choose a hash function based on the anticipated size of the file at some point in the future. Although performance degradation is avoided, a significant amount of space may be wasted initially.
3. Periodically reorganize the hash structure in response to file growth. Such a reorganization involves choosing a new hash function, recomputing the hash function on every record in the file, and generating new bucket assignments. This reorganization is a massive, time-consuming operation. Furthermore, it is necessary to forbid access to the file during reorganization.

Several **dynamic hashing** techniques allow the hash function to be modified dynamically to accommodate the growth or shrinkage of the database. In this section we describe one form of dynamic hashing, called **extendable hashing**. The bibliographical notes provide references to other forms of dynamic hashing.

12.6.1 Data Structure

Extendable hashing copes with changes in database size by splitting and coalescing buckets as the database grows and shrinks. As a result, space efficiency is retained. Moreover, since the reorganization is performed on only one bucket at a time, the resulting performance overhead is acceptably low.

With extendable hashing, we choose a hash function h with the desirable properties of uniformity and randomness. However, this hash function generates values over a relatively large range—namely, b-bit binary integers. A typical value for b is 32.

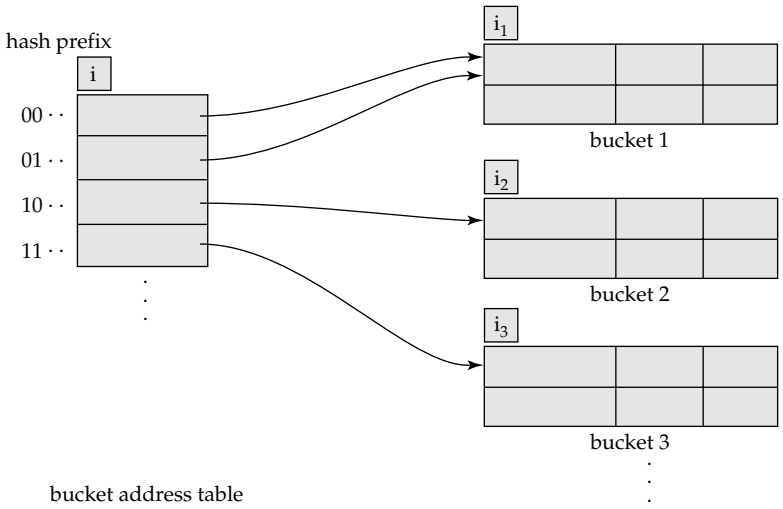


Figure 12.24 General extendable hash structure.

We do not create a bucket for each hash value. Indeed, 2^{32} is over 4 billion, and that many buckets is unreasonable for all but the largest databases. Instead, we create buckets on demand, as records are inserted into the file. We do not use the entire b bits of the hash value initially. At any point, we use i bits, where $0 \leq i \leq b$. These i bits are used as an offset into an additional table of bucket addresses. The value of i grows and shrinks with the size of the database.

Figure 12.24 shows a general extendable hash structure. The i appearing above the bucket address table in the figure indicates that i bits of the hash value $h(K)$ are required to determine the correct bucket for K. This number will, of course, change as the file grows. Although i bits are required to find the correct entry in the bucket address table, several consecutive table entries may point to the same bucket. All such entries will have a common hash prefix, but the length of this prefix may be less than i. Therefore, we associate with each bucket an integer giving the length of the common hash prefix. In Figure 12.24 the integer associated with bucket j is shown as i_j. The number of bucket-address-table entries that point to bucket j is

$$2^{(i - i_j)}$$

12.6.2 Queries and Updates

We now see how to perform lookup, insertion, and deletion on an extendable hash structure.

To locate the bucket containing search-key value K_l, the system takes the first i high-order bits of $h(K_l)$, looks at the corresponding table entry for this bit string, and follows the bucket pointer in the table entry.

To insert a record with search-key value K_l, the system follows the same procedure for lookup as before, ending up in some bucket—say, j. If there is room in the bucket,

the system inserts the record in the bucket. If, on the other hand, the bucket is full, it must split the bucket and redistribute the current records, plus the new one. To split the bucket, the system must first determine from the hash value whether it needs to increase the number of bits that it uses.

- If $i = i_j$, only one entry in the bucket address table points to bucket *j*. Therefore, the system needs to increase the size of the bucket address table so that it can include pointers to the two buckets that result from splitting bucket *j*. It does so by considering an additional bit of the hash value. It increments the value of *i* by 1, thus doubling the size of the bucket address table. It replaces each entry by two entries, both of which contain the same pointer as the original entry. Now two entries in the bucket address table point to bucket *j*. The system allocates a new bucket (bucket *z*), and sets the second entry to point to the new bucket. It sets i_j and i_z to *i*. Next, it rehashes each record in bucket *j* and, depending on the first *i* bits (remember the system has added 1 to *i*), either keeps it in bucket *j* or allocates it to the newly created bucket.

  The system now reattempts the insertion of the new record. Usually, the attempt will succeed. However, if all the records in bucket *j*, as well as the new record, have the same hash-value prefix, it will be necessary to split a bucket again, since all the records in bucket *j* and the new record are assigned to the same bucket. If the hash function has been chosen carefully, it is unlikely that a single insertion will require that a bucket be split more than once, unless there are a large number of records with the same search key. If all the records in bucket *j* have the same search-key value, no amount of splitting will help. In such cases, overflow buckets are used to store the records, as in static hashing.

- If $i > i_j$, then more than one entry in the bucket address table points to bucket *j*. Thus, the system can split bucket *j* without increasing the size of the bucket address table. Observe that all the entries that point to bucket *j* correspond to hash prefixes that have the same value on the leftmost i_j bits. The system allocates a new bucket (bucket *z*), and set i_j and i_z to the value that results from adding 1 to the original i_j value. Next, the system needs to adjust the entries in the bucket address table that previously pointed to bucket *j*. (Note that with the new value for i_j, not all the entries correspond to hash prefixes that have the same value on the leftmost i_j bits.) The system leaves the first half of the entries as they were (pointing to bucket *j*), and sets all the remaining entries to point to the newly created bucket (bucket *z*). Next, as in the previous case, the system rehashes each record in bucket *j*, and allocates it either to bucket *j* or to the newly created bucket *z*.

  The system then reattempts the insert. In the unlikely case that it again fails, it applies one of the two cases, $i = i_j$ or $i > i_j$, as appropriate.

Note that, in both cases, the system needs to recompute the hash function on only the records in bucket *j*.

To delete a record with search-key value K_l, the system follows the same procedure for lookup as before, ending up in some bucket—say, *j*. It removes both the

A-217	Brighton	750
A-101	Downtown	500
A-1 10	Downtown	600
A-215	Mianus	700
A-102	Perryridge	400
A-201	Perryridge	900
A-218	Perryridge	700
A-222	Redwood	700
A-305	Round Hill	350

Figure 12.25 Sample *account* file.

search key from the bucket and the record from the file. The bucket too is removed if it becomes empty. Note that, at this point, several buckets can be coalesced, and the size of the bucket address table can be cut in half. The procedure for deciding on which buckets can be coalesced and how to coalesce buckets is left to you to do as an exercise. The conditions under which the bucket address table can be reduced in size are also left to you as an exercise. Unlike coalescing of buckets, changing the size of the bucket address table is a rather expensive operation if the table is large. Therefore it may be worthwhile to reduce the bucket address table size only if the number of buckets reduces greatly.

Our example *account* file in Figure 12.25 illustrates the operation of insertion. The 32-bit hash values on *branch-name* appear in Figure 12.26. Assume that, initially, the file is empty, as in Figure 12.27. We insert the records one by one. To illustrate all the features of extendable hashing in a small structure, we shall make the unrealistic assumption that a bucket can hold only two records.

We insert the record (A-217, Brighton, 750). The bucket address table contains a pointer to the one bucket, and the system inserts the record. Next, we insert the record (A-101, Downtown, 500). The system also places this record in the one bucket of our structure.

When we attempt to insert the next record (Downtown, A-110, 600), we find that the bucket is full. Since $i = i_0$, we need to increase the number of bits that we use from the hash value. We now use 1 bit, allowing us $2^1 = 2$ buckets. This increase in

branch-name	h(*branch-name*)
Brighton	0010 1101 1111 1011 0010 1100 0011 0000
Downtown	1010 0011 1010 0000 1100 0110 1001 1111
Mianus	1100 0111 1110 1101 1011 1111 0011 1010
Perryridge	1111 0001 0010 0100 1001 0011 0110 1101
Redwood	0011 0101 1010 0110 1100 1001 1110 1011
Round Hill	1101 1000 0011 1111 1001 1100 0000 0001

Figure 12.26 Hash function for *branch-name*.

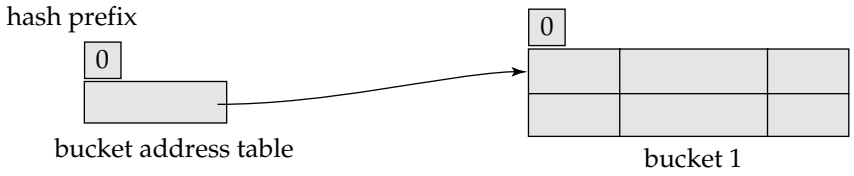


Figure 12.27 Initial extendable hash structure.

the number of bits necessitates doubling the size of the bucket address table to two entries. The system splits the bucket, placing in the new bucket those records whose search key has a hash value beginning with 1, and leaving in the original bucket the other records. Figure 12.28 shows the state of our structure after the split.

Next, we insert (A-215, Mianus, 700). Since the first bit of h(Mianus) is 1, we must insert this record into the bucket pointed to by the "1" entry in the bucket address table. Once again, we find the bucket full and $i = i_1$. We increase the number of bits that we use from the hash to 2. This increase in the number of bits necessitates doubling the size of the bucket address table to four entries, as in Figure 12.29. Since the bucket of Figure 12.28 for hash prefix 0 was not split, the two entries of the bucket address table of 00 and 01 both point to this bucket.

For each record in the bucket of Figure 12.28 for hash prefix 1 (the bucket being split), the system examines the first 2 bits of the hash value to determine which bucket of the new structure should hold it.

Next, we insert (A-102, Perryridge, 400), which goes in the same bucket as Mianus. The following insertion, of (A-201, Perryridge, 900), results in a bucket overflow, leading to an increase in the number of bits, and a doubling of the size of the bucket address table. The insertion of the third Perryridge record, (A-218, Perryridge, 700), leads to another overflow. However, this overflow cannot be handled by increasing the number of bits, since there are three records with exactly the same hash value. Hence the system uses an overflow bucket, as in Figure 12.30.

We continue in this manner until we have inserted all the *account* records of Figure 12.25. The resulting structure appears in Figure 12.31.

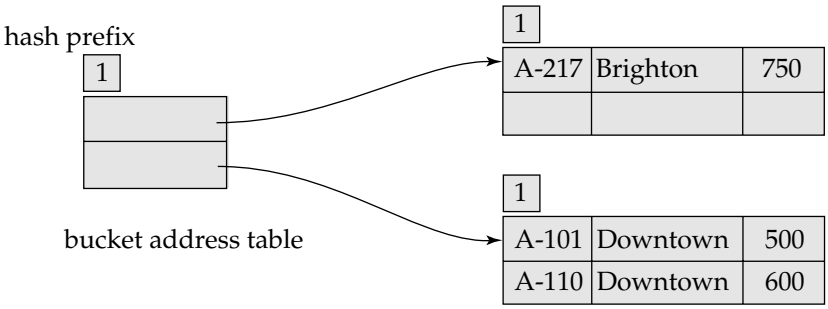


Figure 12.28 Hash structure after three insertions.

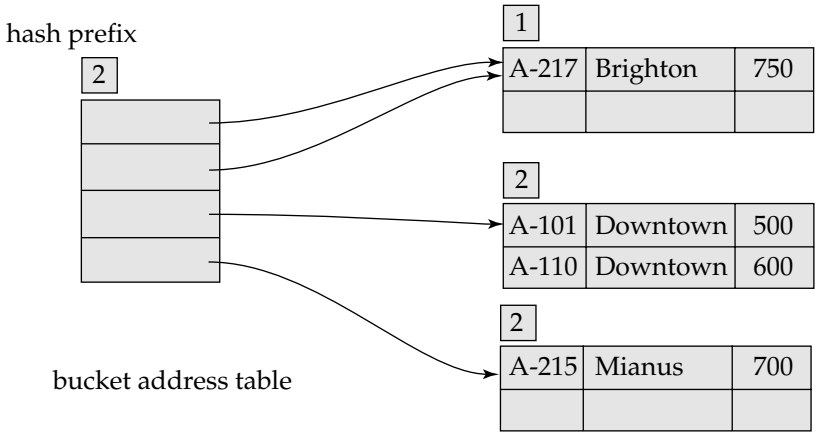


Figure 12.29 Hash structure after four insertions.

12.6.3 Comparison with Other Schemes

We now examine the advantages and disadvantages of extendable hashing, compared with the other schemes that we have discussed. The main advantage of extendable hashing is that performance does not degrade as the file grows. Furthermore, there is minimal space overhead. Although the bucket address table incurs additional overhead, it contains one pointer for each hash value for the current pre-

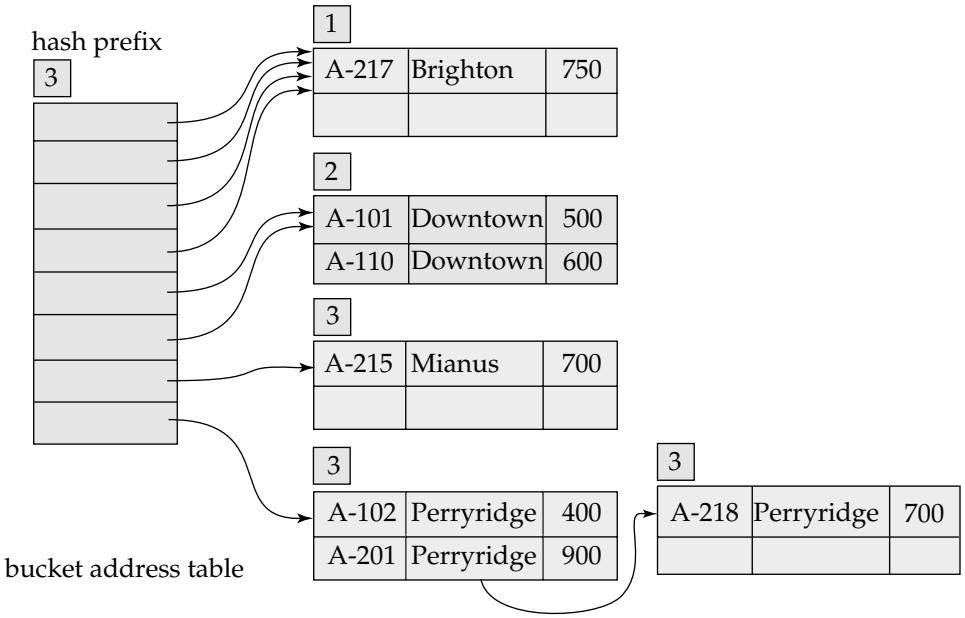


Figure 12.30 Hash structure after seven insertions.

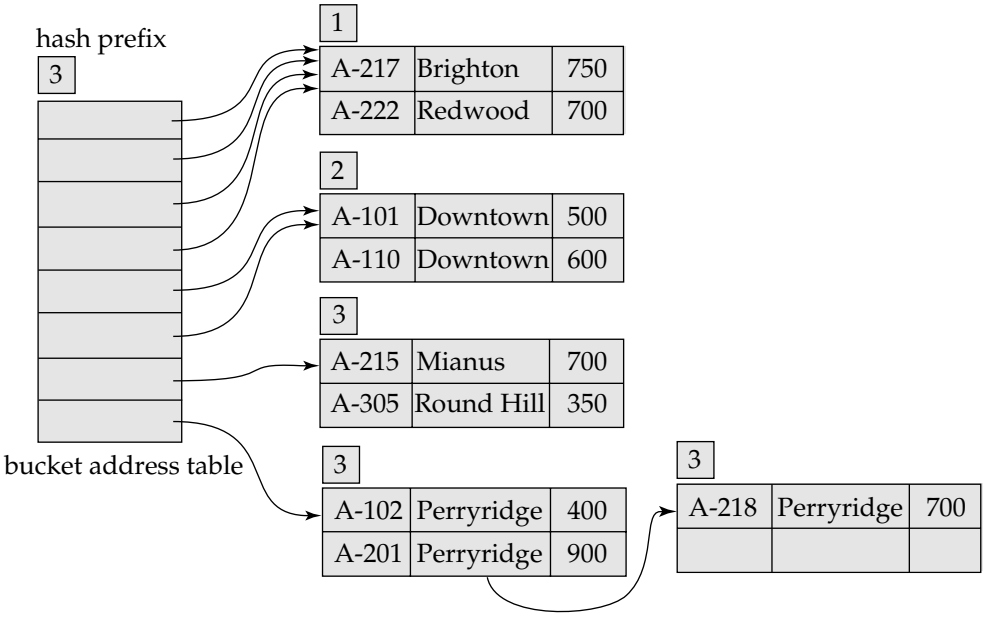


Figure 12.31 Extendable hash structure for the *account* file.

fix length. This table is thus small. The main space saving of extendable hashing over other forms of hashing is that no buckets need to be reserved for future growth; rather, buckets can be allocated dynamically.

A disadvantage of extendable hashing is that lookup involves an additional level of indirection, since the system must access the bucket address table before accessing the bucket itself. This extra reference has only a minor effect on performance. Although the hash structures that we discussed in Section 12.5 do not have this extra level of indirection, they lose their minor performance advantage as they become full.

Thus, extendable hashing appears to be a highly attractive technique, provided that we are willing to accept the added complexity involved in its implementation. The bibliographical notes reference more detailed descriptions of extendable hashing implementation. The bibliographical notes also provide references to another form of dynamic hashing called **linear hashing**, which avoids the extra level of indirection associated with extendable hashing, at the possible cost of more overflow buckets.

12.7 Comparison of Ordered Indexing and Hashing

We have seen several ordered-indexing schemes and several hashing schemes. We can organize files of records as ordered files, by using index-sequential organization or B^+-tree organizations. Alternatively, we can organize the files by using hashing. Finally, we can organize them as heap files, where the records are not ordered in any particular way.

Each scheme has advantages in certain situations. A database-system implementor could provide many schemes, leaving the final decision of which schemes to use to the database designer. However, such an approach requires the implementor to write more code, adding both to the cost of the system and to the space that the system occupies. Most database systems support B^+-trees and may additionally support some form of hash file organization or hash indices.

To make a wise choice of file organization and indexing techniques for a relation, the implementor or the database designer must consider the following issues:

- Is the cost of periodic reorganization of the index or hash organization acceptable?
- What is the relative frequency of insertion and deletion?
- Is it desirable to optimize average access time at the expense of increasing the worst-case access time?
- What types of queries are users likely to pose?

We have already examined the first three of these issues, first in our review of the relative merits of specific indexing techniques, and again in our discussion of hashing techniques. The fourth issue, the expected type of query, is critical to the choice of ordered indexing or hashing.

If most queries are of the form

select $A_1, A_2, \ldots, A_n$
from r
where $A_i = c$

then, to process this query, the system will perform a lookup on an ordered index or a hash structure for attribute A_i, for value c. For queries of this form, a hashing scheme is preferable. An ordered-index lookup requires time proportional to the log of the number of values in r for A_i. In a hash structure, however, the average lookup time is a constant independent of the size of the database. The only advantage to an index over a hash structure for this form of query is that the worst-case lookup time is proportional to the log of the number of values in r for A_i. By contrast, for hashing, the worst-case lookup time is proportional to the number of values in r for A_i. However, the worst-case lookup time is unlikely to occur with hashing, and hashing is preferable in this case.

Ordered-index techniques are preferable to hashing in cases where the query specifies a range of values. Such a query takes the following form:

select $A_1, A_2, ..., A_n$
from r
where $A_i \leq c_2$ **and** $A_i \geq c_1$

In other words, the preceding query finds all the records with A_i values between c_1 and c_2.

Let us consider how we process this query using an ordered index. First, we perform a lookup on value c_1. Once we have found the bucket for value c_1, we follow the pointer chain in the index to read the next bucket in order, and we continue in this manner until we reach c_2.

If, instead of an ordered index, we have a hash structure, we can perform a lookup on c_1 and can locate the corresponding bucket—but it is not easy, in general, to determine the next bucket that must be examined. The difficulty arises because a good hash function assigns values randomly to buckets. Thus, there is no simple notion of "next bucket in sorted order." The reason we cannot chain buckets together in sorted order on A_i is that each bucket is assigned many search-key values. Since values are scattered randomly by the hash function, the values in the specified range are likely to be scattered across many or all of the buckets. Therefore, we have to read all the buckets to find the required search keys.

Usually the designer will choose ordered indexing unless it is known in advance that range queries will be infrequent, in which case hashing would be chosen. Hash organizations are particularly useful for temporary files created during query processing, if lookups based on a key value are required, but no range queries will be performed.

12.8 Index Definition in SQL

The SQL standard does not provide any way for the database user or administrator to control what indices are created and maintained in the database system. Indices are not required for correctness, since they are redundant data structures. However, indices are important for efficient processing of transactions, including both update transactions and queries. Indices are also important for efficient enforcement of integrity constraints. For example, typical implementations enforce a key declaration (Chapter 6) by creating an index with the declared key as the search key of the index.

In principle, a database system can decide automatically what indices to create. However, because of the space cost of indices, as well as the effect of indices on update processing, it is not easy to automatically make the right choices about what indices to maintain. Therefore, most SQL implementations provide the programmer control over creation and removal of indices via data-definition-language commands.

We illustrate the syntax of these commands next. Although the syntax that we show is widely used and supported by many database systems, it is not part of the SQL:1999 standard. The SQL standards (up to SQL:1999, at least) do not support control of the physical database schema, and have restricted themselves to the logical database schema.

We create an index by the **create index** command, which takes the form

create index <index-name> **on** <relation-name> (<attribute-list>)

The *attribute-list* is the list of attributes of the relations that form the search key for the index.

To define an index name *b-index* on the *branch* relation with *branch-name* as the search key, we write

create index *b-index* **on** *branch* (*branch-name*)

If we wish to declare that the search key is a candidate key, we add the attribute **unique** to the index definition. Thus, the command

create unique index *b-index* **on** *branch* (*branch-name*)

declares *branch-name* to be a candidate key for *branch*. If, at the time we enter the **create unique index** command, *branch-name* is not a candidate key, the system will display an error message, and the attempt to create the index will fail. If the index-creation attempt succeeds, any subsequent attempt to insert a tuple that violates the key declaration will fail. Note that the **unique** feature is redundant if the database system supports the **unique** declaration of the SQL standard.

Many database systems also provide a way to specify the type of index to be used (such as B^+-tree or hashing). Some database systems also permit one of the indices on a relation to be declared to be clustered; the system then stores the relation sorted by the search-key of the clustered index.

The index name we specified for an index is required to drop an index. The **drop index** command takes the form:

drop index <index-name>

12.9 Multiple-Key Access

Until now, we have assumed implicitly that only one index (or hash table) is used to process a query on a relation. However, for certain types of queries, it is advantageous to use multiple indices if they exist.

12.9.1 Using Multiple Single-Key Indices

Assume that the *account* file has two indices: one for *branch-name* and one for *balance*. Consider the following query: "Find all account numbers at the Perryridge branch with balances equal to $1000." We write

select *loan-number*
from *account*
where *branch-name* = "Perryridge" **and** *balance* = 1000

There are three strategies possible for processing this query:

1. Use the index on *branch-name* to find all records pertaining to the Perryridge branch. Examine each such record to see whether *balance* = 1000.
2. Use the index on *balance* to find all records pertaining to accounts with balances of $1000. Examine each such record to see whether *branch-name* = "Perryridge."
3. Use the index on *branch-name* to find *pointers* to all records pertaining to the Perryridge branch. Also, use the index on *balance* to find pointers to all records

pertaining to accounts with a balance of $1000. Take the intersection of these two sets of pointers. Those pointers that are in the intersection point to records pertaining to both Perryridge and accounts with a balance of $1000.

The third strategy is the only one of the three that takes advantage of the existence of multiple indices. However, even this strategy may be a poor choice if all of the following hold:

- There are many records pertaining to the Perryridge branch.
- There are many records pertaining to accounts with a balance of $1000.
- There are only a few records pertaining to *both* the Perryridge branch and accounts with a balance of $1000.

If these conditions hold, we must scan a large number of pointers to produce a small result. An index structure called a "bitmap index" greatly speeds up the intersection operation used in the third strategy. Bitmap indices are outlined in Section 12.9.4.

12.9.2 Indices on Multiple Keys

An alternative strategy for this case is to create and use an index on a search key (*branch-name*, *balance*)—that is, the search key consisting of the branch name concatenated with the account balance. The structure of the index is the same as that of any other index, the only difference being that the search key is not a single attribute, but rather is a list of attributes. The search key can be represented as a tuple of values, of the form $(a_1, \ldots, a_n)$, where the indexed attributes are $A_1, \ldots, A_n$. The ordering of search-key values is the *lexicographic ordering*. For example, for the case of two attribute search keys, $(a_1, a_2) < (b_1, b_2)$ if either $a_1 < b_1$ or $a_1 = b_1$ and $a_2 < b_2$. Lexicographic ordering is basically the same as alphabetic ordering of words.

The use of an ordered-index structure on multiple attributes has a few shortcomings. As an illustration, consider the query

select *loan-number*
from *account*
where *branch-name* $<$ "Perryridge" **and** *balance* $= 1000$

We can answer this query by using an ordered index on the search key (*branch-name*, *balance*): For each value of *branch-name* that is less than "Perryridge" in alphabetic order, the system locates records with a *balance* value of 1000. However, each record is likely to be in a different disk block, because of the ordering of records in the file, leading to many I/O operations.

The difference between this query and the previous one is that the condition on *branch-name* is a comparison condition, rather than an equality condition.

To speed the processing of general multiple search-key queries (which can involve one or more comparison operations), we can use several special structures. We shall consider the *grid file* in Section 12.9.3. There is another structure, called the *R-tree*, that

can be used for this purpose. The R-tree is an extension of the B^+-tree to handle indexing on multiple dimensions. Since the R-tree is used primarily with geographical data types, we describe the structure in Chapter 23.

12.9.3 Grid Files

Figure 12.32 shows part of a **grid file** for the search keys *branch-name* and *balance* on the *account* file. The two-dimensional array in the figure is called the *grid array*, and the one-dimensional arrays are called *linear scales*. The grid file has a single grid array, and one linear scale for each search-key attribute.

Search keys are mapped to cells in this way. Each cell in the grid array has a pointer to a bucket that contains the search-key values and pointers to records. Only some of the buckets and pointers from the cells are shown in the figure. To conserve space, multiple elements of the array can point to the same bucket. The dotted boxes in the figure indicate which cells point to the same bucket.

Suppose that we want to insert in the grid-file index a record whose search-key value is ("Brighton", 500000). To find the cell to which the key is mapped, we independently locate the row and column to which the cell belongs.

We first use the linear scales on *branch-name* to locate the row of the cell to which the search key maps. To do so, we search the array to find the least element that is greater than "Brighton". In this case, it is the first element, so the search key maps to the row marked 0. If it were the ith element, the search key would map to row $i - 1$. If the search key is greater than or equal to all elements in the linear scale, it maps to

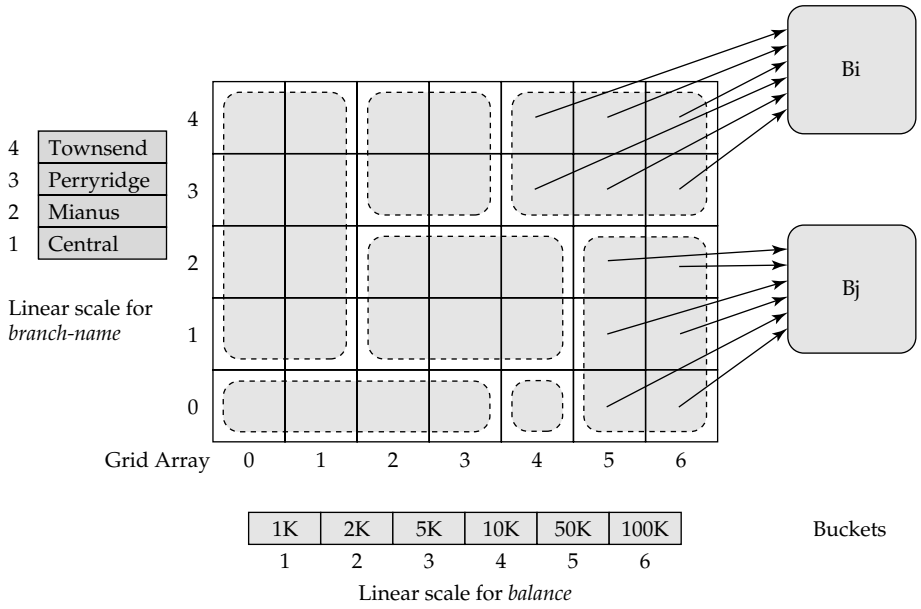


Figure 12.32 Grid file on keys *branch-name* and *balance* of the *account* file.

the final row. Next, we use the linear scale on *balance* to find out similarly to which column the search key maps. In this case, the balance 500000 maps to column 6.

Thus, the search-key value ("Brighton", 500000) maps to the cell in row 0, column 6. Similarly, ("Downtown", 60000) would map to the cell in row 1 column 5. Both cells point to the same bucket (as indicated by the dotted box), so, in both cases, the system stores the search-key values and the pointer to the record in the bucket labeled B_j in the figure.

To perform a lookup to answer our example query, with the search condition of

$$\textit{branch-name} < \text{"Perryridge"} \textbf{ and } \textit{balance} = 1000$$

we find all rows that can contain branch names less than "Perryridge", using the linear scale on *branch-name*. In this case, these rows are 0, 1, and 2. Rows 3 and beyond contain branch names greater than or equal to "Perryridge". Similarly, we find that only column 1 can contain a *balance* value of 1000. In this case, only column 1 satisfies this condition. Thus, only the cells in column 1, rows 0, 1, and 2, can contain entries that satisfy the search condition.

We therefore look up all entries in the buckets pointed to from these three cells. In this case, there are only two buckets, since two of the cells point to the same bucket, as indicated by the dotted boxes in the figure. The buckets may contain some search keys that do not satisfy the required condition, so each search key in the buckets must be tested again to see whether it satisfies the search condition. We have to examine only a small number of buckets, however, to answer this query.

We must choose the linear scales in such a way that the records are uniformly distributed across the cells. When a bucket—call it A—becomes full and an entry has to be inserted in it, the system allocates an extra bucket, B. If more than one cell points to A, the system changes the cell pointers so that some point to A and others to B. The entries in bucket A and the new entry are then redistributed between A and B according to the cells to which they map. If only one cell points to bucket A, B becomes an overflow bucket for A. To improve performance in such a situation, we must reorganize the grid file, with an expanded grid array and expanded linear scales. The process is much like the expansion of the bucket address table in extensible hashing, and is left for you to do as an exercise.

It is conceptually simple to extend the grid-file approach to any number of search keys. If we want our structure to be used for queries on n keys, we construct an n-dimensional grid array with n linear scales.

The grid structure is suitable also for queries involving one search key. Consider this query:

select *
from *account*
where *branch-name* = "Perryridge"

The linear scale on *branch-name* tells us that only cells in row 3 can satisfy this condition. Since there is no condition on *balance*, we examine all buckets pointed to by cells in row 3 to find entries pertaining to Perryridge. Thus, we can use a grid-file index on

two search keys to answer queries on either search key by itself, as well as to answer queries on both search keys. Thus, a single grid-file index can serve the role of three separate indices. If each index were maintained separately, the three together would occupy more space, and the cost of updating them would be high.

Grid files provide a significant decrease in processing time for multiple-key queries. However, they impose a space overhead (the grid directory can become large), as well as a performance overhead on record insertion and deletion. Further, it is hard to choose partitioning ranges for the keys such that the distribution of records is uniform. If insertions to the file are frequent, reorganization will have to be carried out periodically, and that can have a high cost.

12.9.4 Bitmap Indices

Bitmap indices are a specialized type of index designed for easy querying on multiple keys, although each bitmap index is built on a single key.

For bitmap indices to be used, records in a relation must be numbered sequentially, starting from, say, 0. Given a number n, it must be easy to retrieve the record numbered n. This is particularly easy to achieve if records are fixed in size, and allocated on consecutive blocks of a file. The record number can then be translated easily into a block number and a number that identifies the record within the block.

Consider a relation r, with an attribute A that can take on only one of a small number (for example, 2 to 20) values. For instance, a relation *customer-info* may have an attribute *gender*, which can take only values m (male) or f (female). Another example would be an attribute *income-level*, where income has been broken up into 5 levels: $L1$: $\$0 - 9999$, $L2$: $\$10,000 - 19,999$, $L3$: $20,000 - 39,999$, $L4$: $40,000 - 74,999$, and $L5$: $75,000 - \infty$. Here, the raw data can take on many values, but a data analyst has split the values into a small number of ranges to simplify analysis of the data.

12.9.4.1 Bitmap Index Structure

A **bitmap** is simply an array of bits. In its simplest form, a **bitmap index** on the attribute A of relation r consists of one bitmap for each value that A can take. Each bitmap has as many bits as the number of records in the relation. The ith bit of the bitmap for value v_j is set to 1 if the record numbered i has the value v_j for attribute A. All other bits of the bitmap are set to 0.

In our example, there is one bitmap for the value m and one for f. The ith bit of the bitmap for m is set to 1 if the *gender* value of the record numbered i is m. All other bits of the bitmap for m are set to 0. Similarly, the bitmap for f has the value 1 for bits corresponding to records with the value f for the *gender* attribute; all other bits have the value 0. Figure 12.33 shows an example of bitmap indices on a relation *customer-info*.

We now consider when bitmaps are useful. The simplest way of retrieving all records with value m (or value f) would be to simply read all records of the relation and select those records with value m (or f, respectively). The bitmap index doesn't really help to speed up such a selection.

record number	*name*	*gender*	*address*	*income -level*
0	John	m	Perryridge	L1
1	Diana	f	Brooklyn	L2
2	Mary	f	Jonestown	L1
3	Peter	m	Brooklyn	L4
4	Kathy	f	Perryridge	L3

Bitmaps for *gender*

m	10010
f	01101

Bitmaps for *income-level*

L1	10100
L2	01000
L3	00001
L4	00010
L5	00000

Figure 12.33 Bitmap indices on relation *customer-info*.

In fact, bitmap indices are useful for selections mainly when there are selections on multiple keys. Suppose we create a bitmap index on attribute *income-level*, which we described earlier, in addition to the bitmap index on *gender*.

Consider now a query that selects women with income in the range $10,000 - 19,999$. This query can be expressed as $\sigma_{gender=\mathsf{f} \wedge income\text{-}level=L2}(r)$. To evaluate this selection, we fetch the bitmaps for *gender* value f and the bitmap for *income-level* value $L2$, and perform an **intersection** (logical-and) of the two bitmaps. In other words, we compute a new bitmap where bit i has value 1 if the ith bit of the two bitmaps are both 1, and has a value 0 otherwise. In the example in Figure 12.33, the intersection of the bitmap for $gender = \mathsf{f}$ (01101) and the bitmap for $income\text{-}level = L1$ (10100) gives the bitmap 00100.

Since the first attribute can take 2 values, and the second can take 5 values, we would expect only about 1 in 10 records, on an average, to satisfy a combined condition on the two attributes. If there are further conditions, the fraction of records satisfying all the conditions is likely to be quite small. The system can then compute the query result by finding all bits with value 1 in the intersection bitmap, and retrieving the corresponding records. If the fraction is large, scanning the entire relation would remain the cheaper alternative.

Another important use of bitmaps is to count the number of tuples satisfying a given selection. Such queries are important for data analysis. For instance, if we wish to find out how many women have an income level $L2$, we compute the intersection of the two bitmaps, and then count the number of bits that are 1 in the intersection bitmap. We can thus get the desired result from the bitmap index, without even accessing the relation.

Bitmap indices are generally quite small compared to the actual relation size. Records are typically at least tens of bytes to hundreds of bytes long, whereas a single bit represents the record in a bitmap. Thus the space occupied by a single bitmap is usually less than 1 percent of the space occupied by the relation. For instance, if the record size for a given relation is 100 bytes, then the space occupied by a single bitmap would be $\frac{1}{8}$ of 1 percent of the space occupied by the relation. If an attribute A of the relation can take on only one of 8 values, a bitmap index on attribute A would consist of 8 bitmaps, which together occupy only 1 percent of the size of the relation.

Deletion of records creates gaps in the sequence of records, since shifting records (or record numbers) to fill gaps would be extremely expensive. To recognize deleted records, we can store an **existence bitmap**, in which bit i is 0 if record i does not exist and 1 otherwise. We will see the need for existence bitmaps in Section 12.9.4.2. Insertion of records should not affect the sequence numbering of other records. Therefore, we can do insertion either by appending records to the end of the file or by replacing deleted records.

12.9.4.2 Efficient Implementation of Bitmap Operations

We can compute the intersection of two bitmaps easily by using a **for** loop: the ith iteration of the loop computes the **and** of the ith bits of the two bitmaps. We can speed up computation of the intersection greatly by using bit-wise **and** instructions supported by most computer instruction sets. A *word* usually consists of 32 or 64 bits, depending on the architecture of the computer. A bit-wise **and** instruction takes two words as input and outputs a word where each bit is the logical **and** of the bits in corresponding positions of the input words. What is important to note is that a single bit-wise **and** instruction can compute the intersection of 32 or 64 bits *at once*.

If a relation had 1 million records, each bitmap would contain 1 million bits, or equivalently 128 Kbytes. Only 31,250 instructions are needed to compute the intersection of two bitmaps for our relation, assuming a 32-bit word length. Thus, computing bitmap intersections is an extremely fast operation.

Just like bitmap intersection is useful for computing the **and** of two conditions, bitmap union is useful for computing the **or** of two conditions. The procedure for bitmap union is exactly the same as for intersection, except we use bit-wise **or** instructions instead of bit-wise **and** instructions.

The complement operation can be used to compute a predicate involving the negation of a condition, such as **not** (*income-level* = $L1$). The complement of a bitmap is generated by complementing every bit of the bitmap (the complement of 1 is 0 and the complement of 0 is 1). It may appear that **not** (*income-level* = $L1$) can be implemented by just computing the complement of the bitmap for income level $L1$. If some records have been deleted, however, just computing the complement of a bitmap is not sufficient. Bits corresponding to such records would be 0 in the original bitmap, but would become 1 in the complement, although the records don't exist. A similar problem also arises when the value of an attribute is *null*. For instance, if the value of *income-level* is null, the bit would be 0 in the original bitmap for value $L1$, and 1 in the complement bitmap.

To make sure that the bits corresponding to deleted records are set to 0 in the result, the complement bitmap must be intersected with the existence bitmap to turn off the bits for deleted records. Similarly, to handle null values, the complement bitmap must also be intersected with the complement of the bitmap for the value *null*.[1]

Counting the number of bits that are 1 in a bitmap can be done fast by a clever technique. We can maintain an array with 256 entries, where the ith entry stores the

1. Handling predicates such as **is unknown** would cause further complications, which would in general require use of an extra bitmap to to track which operation results are unknown.

number of bits that are 1 in the binary representation of i. Set the total count initially to 0. We take each byte of the bitmap, use it to index into this array, and add the stored count to the total count. The number of addition operations would be $\frac{1}{8}$ of the number of tuples, and thus the counting process is very efficient. A large array (using $2^{16} = 65536$ entries), indexed by pairs of bytes, would give even higher speedup, but at a higher storage cost.

12.9.4.3 Bitmaps and B$^+$-Trees

Bitmaps can be combined with regular B$^+$-tree indices for relations where a few attribute values are extremely common, and other values also occur, but much less frequently. In a B$^+$-tree index leaf, for each value we would normally maintain a list of all records with that value for the indexed attribute. Each element of the list would be a record identifier, consisting of at least 32 bits, and usually more. For a value that occurs in many records, we store a bitmap instead of a list of records.

Suppose a particular value v_i occurs in $\frac{1}{16}$ of the records of a relation. Let N be the number of records in the relation, and assume that a record has a 64-bit number identifying it. The bitmap needs only 1 bit per record, or N bits in total. In contrast, the list representation requires 64 bits per record where the value occurs, or $64 * N/16 = 4N$ bits. Thus, a bitmap is preferable for representing the list of records for value v_i. In our example (with a 64-bit record identifier), if fewer than 1 in 64 records have a particular value, the list representation is preferable for identifying records with that value, since it uses fewer bits than the bitmap representation. If more than 1 in 64 records have that value, the bitmap representation is preferable.

Thus, bitmaps can be used as a compressed storage mechanism at the leaf nodes of B$^+$-trees, for those values that occur very frequently.

12.10 Summary

- Many queries reference only a small proportion of the records in a file. To reduce the overhead in searching for these records, we can construct *indices* for the files that store the database.
- Index-sequential files are one of the oldest index schemes used in database systems. To permit fast retrieval of records in search-key order, records are stored sequentially, and out-of-order records are chained together. To allow fast random access, we use an index structure.
- There are two types of indices that we can use: dense indices and sparse indices. Dense indices contain entries for every search-key value, whereas sparse indices contain entries only for some search-key values.
- If the sort order of a search key matches the sort order of a relation, an index on the search key is called a *primary index*. The other indices are called *secondary indices*. Secondary indices improve the performance of queries that use search keys other than the primary one. However, they impose an overhead on modification of the database.

- The primary disadvantage of the index-sequential file organization is that performance degrades as the file grows. To overcome this deficiency, we can use a B^+*-tree index*.
- A B^+-tree index takes the form of a *balanced* tree, in which every path from the root of the tree to a leaf of the tree is of the same length. The height of a B^+-tree is proportional to the logarithm to the base N of the number of records in the relation, where each nonleaf node stores N pointers; the value of N is often around 50 or 100. B^+-trees are much shorter than other balanced binary-tree structures such as AVL trees, and therefore require fewer disk accesses to locate records.
- Lookup on B^+-trees is straightforward and efficient. Insertion and deletion, however, are somewhat more complicated, but still efficient. The number of operations required for lookup, insertion, and deletion on B^+-trees is proportional to the logarithm to the base N of the number of records in the relation, where each nonleaf node stores N pointers.
- We can use B^+-trees for indexing a file containing records, as well as to organize records into a file.
- B-tree indices are similar to B^+-tree indices. The primary advantage of a B-tree is that the B-tree eliminates the redundant storage of search-key values. The major disadvantages are overall complexity and reduced fanout for a given node size. System designers almost universally prefer B^+-tree indices over B-tree indices in practice.
- Sequential file organizations require an index structure to locate data. File organizations based on hashing, by contrast, allow us to find the address of a data item directly by computing a function on the search-key value of the desired record. Since we do not know at design time precisely which search-key values will be stored in the file, a good hash function to choose is one that assigns search-key values to buckets such that the distribution is both uniform and random.
- *Static hashing* uses hash functions in which the set of bucket addresses is fixed. Such hash functions cannot easily accommodate databases that grow significantly larger over time. There are several *dynamic hashing techniques* that allow the hash function to be modified. One example is *extendable hashing*, which copes with changes in database size by splitting and coalescing buckets as the database grows and shrinks.
- We can also use hashing to create secondary indices; such indices are called *hash indices*. For notational convenience, we assume hash file organizations have an implicit hash index on the search key used for hashing.
- Ordered indices such as B^+-trees and hash indices can be used for selections based on equality conditions involving single attributes. When multiple

attributes are involved in a selection condition, we can intersect record identifiers retrieved from multiple indices.

- Grid files provide a general means of indexing on multiple attributes.
- Bitmap indices provide a very compact representation for indexing attributes with very few distinct values. Intersection operations are extremely fast on bitmaps, making them ideal for supporting queries on multiple attributes.

Review Terms

- Access types
- Access time
- Insertion time
- Deletion time
- Space overhead
- Ordered index
- Primary index
- Clustering index
- Secondary index
- Nonclustering index
- Index-sequential file
- Index record/entry
- Dense index
- Sparse index
- Multilevel index
- Sequential scan
- B^+-Tree index
- Balanced tree
- B^+-Tree file organization
- B-Tree index
- Static hashing
- Hash file organization
- Hash index
- Bucket
- Hash function
- Bucket overflow
- Skew
- Closed hashing
- Dynamic hashing
- Extendable hashing
- Multiple-key access
- Indices on multiple keys
- Grid files
- Bitmap index
- Bitmap operations
  - ☐ Intersection
  - ☐ Union
  - ☐ Complement
  - ☐ Existence bitmap

Exercises

12.1 When is it preferable to use a dense index rather than a sparse index? Explain your answer.

12.2 Since indices speed query processing, why might they not be kept on several search keys? List as many reasons as possible.

12.3 What is the difference between a primary index and a secondary index?

12.4 Is it possible in general to have two primary indices on the same relation for different search keys? Explain your answer.

12.5 Construct a B^+-tree for the following set of key values:

(2, 3, 5, 7, 11, 17, 19, 23, 29, 31)

Assume that the tree is initially empty and values are added in ascending order. Construct B^+-trees for the cases where the number of pointers that will fit in one node is as follows:

a. Four
b. Six
c. Eight

12.6 For each B^+-tree of Exercise 12.5, show the steps involved in the following queries:

a. Find records with a search-key value of 11.
b. Find records with a search-key value between 7 and 17, inclusive.

12.7 For each B^+-tree of Exercise 12.5, show the form of the tree after each of the following series of operations:

a. Insert 9.
b. Insert 10.
c. Insert 8.
d. Delete 23.
e. Delete 19.

12.8 Consider the modified redistribution scheme for B^+-trees described in page 463. What is the expected height of the tree as a function of n?

12.9 Repeat Exercise 12.5 for a B-tree.

12.10 Explain the distinction between closed and open hashing. Discuss the relative merits of each technique in database applications.

12.11 What are the causes of bucket overflow in a hash file organization? What can be done to reduce the occurrence of bucket overflows?

12.12 Suppose that we are using extendable hashing on a file that contains records with the following search-key values:

2, 3, 5, 7, 11, 17, 19, 23, 29, 31

Show the extendable hash structure for this file if the hash function is $h(x) = x$ mod 8 and buckets can hold three records.

12.13 Show how the extendable hash structure of Exercise 12.12 changes as the result of each of the following steps:

a. Delete 11.
b. Delete 31.
c. Insert 1.
d. Insert 15.

12.14 Give pseudocode for deletion of entries from an extendable hash structure, including details of when and how to coalesce buckets. Do not bother about reducing the size of the bucket address table.

12.15 Suggest an efficient way to test if the bucket address table in extendable hashing can be reduced in size, by storing an extra count with the bucket address table. Give details of how the count should be maintained when buckets are split, coalesced or deleted.

(Note: Reducing the size of the bucket address table is an expensive operation, and subsequent inserts may cause the table to grow again. Therefore, it is best not to reduce the size as soon as it is possible to do so, but instead do it only if the number of index entries becomes small compared to the bucket address table size.)

12.16 Why is a hash structure not the best choice for a search key on which range queries are likely?

12.17 Consider a grid file in which we wish to avoid overflow buckets for performance reasons. In cases where an overflow bucket would be needed, we instead reorganize the grid file. Present an algorithm for such a reorganization.

12.18 Consider the *account* relation shown in Figure 12.25.

a. Construct a bitmap index on the attributes *branch-name* and *balance*, dividing *balance* values into 4 ranges: below 250, 250 to below 500, 500 to below 750, and 750 and above.

b. Consider a query that requests all accounts in Downtown with a balance of 500 or more. Outline the steps in answering the query, and show the final and intermediate bitmaps constructed to answer the query.

12.19 Show how to compute existence bitmaps from other bitmaps. Make sure that your technique works even in the presence of null values, by using a bitmap for the value *null*.

12.20 How does data encryption affect index schemes? In particular, how might it affect schemes that attempt to store data in sorted order?

Bibliographical Notes

Discussions of the basic data structures in indexing and hashing can be found in Cormen et al. [1990]. B-tree indices were first introduced in Bayer [1972] and Bayer and McCreight [1972]. B^+-trees are discussed in Comer [1979], Bayer and Unterauer [1977] and Knuth [1973]. The bibliographic notes in Chapter 16 provides references to research on allowing concurrent accesses and updates on B^+-trees. Gray and Reuter [1993] provide a good description of issues in the implementation of B^+-trees.

Several alternative tree and treelike search structures have been proposed. **Tries** are trees whose structure is based on the "digits" of keys (for example, a dictionary thumb index, which has one entry for each letter). Such trees may not be balanced in the sense that B^+-trees are. Tries are discussed by Ramesh et al. [1989], Orenstein

[1982], Litwin [1981] and Fredkin [1960]. Related work includes the digital B-trees of Lomet [1981].

Knuth [1973] analyzes a large number of different hashing techniques. Several dynamic hashing schemes exist. Extendable hashing was introduced by Fagin et al. [1979]. Linear hashing was introduced by Litwin [1978] and Litwin [1980]; Larson [1982] presents a performance analysis of linear hashing. Ellis [1987] examined concurrency with linear hashing. Larson [1988] presents a variant of linear hashing. Another scheme, called dynamic hashing, was proposed by Larson [1978]. An alternative given by Ramakrishna and Larson [1989] allows retrieval in a single disk access at the price of a high overhead for a small fraction of database modifications. Partitioned hashing is an extension of hashing to multiple attributes, and is covered in Rivest [1976], Burkhard [1976] and Burkhard [1979].

The grid file structure appears in Nievergelt et al. [1984] and Hinrichs [1985]. Bitmap indices, and variants called **bit-sliced indices** and **projection indices** are described in O'Neil and Quass [1997]. They were first introduced in the IBM Model 204 file manager on the AS 400 platform. They provide very large speedups on certain types of queries, and are today implemented on most database systems. Recent research on bitmap indices includes Wu and Buchmann [1998], Chan and Ioannidis [1998], Chan and Ioannidis [1999], and Johnson [1999a].

© The McGraw–Hill Companies, 2001

CHAPTER 13

Query Processing

Query processing refers to the range of activities involved in extracting data from a database. The activities include translation of queries in high-level database languages into expressions that can be used at the physical level of the file system, a variety of query-optimizing transformations, and actual evaluation of queries.

13.1 Overview

The steps involved in processing a query appear in Figure 13.1. The basic steps are

1. Parsing and translation
2. Optimization
3. Evaluation

Before query processing can begin, the system must translate the query into a usable form. A language such as SQL is suitable for human use, but is ill-suited to be the system's internal representation of a query. A more useful internal representation is one based on the extended relational algebra.

Thus, the first action the system must take in query processing is to translate a given query into its internal form. This translation process is similar to the work performed by the parser of a compiler. In generating the internal form of the query, the parser checks the syntax of the user's query, verifies that the relation names appearing in the query are names of the relations in the database, and so on. The system constructs a parse-tree representation of the query, which it then translates into a relational-algebra expression. If the query was expressed in terms of a view, the translation phase also replaces all uses of the view by the relational-algebra expres-

© The McGraw–Hill Companies, 2001

CHAPTER 15

Transactions

Often, a collection of several operations on the database appears to be a single unit from the point of view of the database user. For example, a transfer of funds from a checking account to a savings account is a single operation from the customer's standpoint; within the database system, however, it consists of several operations. Clearly, it is essential that all these operations occur, or that, in case of a failure, none occur. It would be unacceptable if the checking account were debited, but the savings account were not credited.

Collections of operations that form a single logical unit of work are called **transactions**. A database system must ensure proper execution of transactions despite failures—either the entire transaction executes, or none of it does. Furthermore, it must manage concurrent execution of transactions in a way that avoids the introduction of inconsistency. In our funds-transfer example, a transaction computing the customer's total money might see the checking-account balance before it is debited by the funds-transfer transaction, but see the savings balance after it is credited. As a result, it would obtain an incorrect result.

This chapter introduces the basic concepts of transaction processing. Details on concurrent transaction processing and recovery from failures are in Chapters 16 and 17, respectively. Further topics in transaction processing are discussed in Chapter 24.

15.1 Transaction Concept

A **transaction** is a **unit** of program execution that accesses and possibly updates various data items. Usually, a transaction is initiated by a user program written in a high-level data-manipulation language or programming language (for example, SQL, COBOL, C, C++, or Java), where it is delimited by statements (or function calls) of the form **begin transaction** and **end transaction**. The transaction consists of all operations executed between the **begin transaction** and **end transaction**.

To ensure integrity of the data, we require that the database system maintain the following properties of the transactions:

- **Atomicity**. Either all operations of the transaction are reflected properly in the database, or none are.
- **Consistency**. Execution of a transaction in isolation (that is, with no other transaction executing concurrently) preserves the consistency of the database.
- **Isolation**. Even though multiple transactions may execute concurrently, the system guarantees that, for every pair of transactions T_i and T_j, it appears to T_i that either T_j finished execution before T_i started, or T_j started execution after T_i finished. Thus, each transaction is unaware of other transactions executing concurrently in the system.
- **Durability**. After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

These properties are often called the **ACID properties**; the acronym is derived from the first letter of each of the four properties.

To gain a better understanding of ACID properties and the need for them, consider a simplified banking system consisting of several accounts and a set of transactions that access and update those accounts. For the time being, we assume that the database permanently resides on disk, but that some portion of it is temporarily residing in main memory.

Transactions access data using two operations:

- read(X), which transfers the data item X from the database to a local buffer belonging to the transaction that executed the read operation.
- write(X), which transfers the data item X from the the local buffer of the transaction that executed the write back to the database.

In a real database system, the write operation does not necessarily result in the immediate update of the data on the disk; the write operation may be temporarily stored in memory and executed on the disk later. For now, however, we shall assume that the write operation updates the database immediately. We shall return to this subject in Chapter 17.

Let T_i be a transaction that transfers \$50 from account A to account B. This transaction can be defined as

T_i: read(A);
$A := A - 50$;
write(A);
read(B);
$B := B + 50$;
write(B).

Let us now consider each of the ACID requirements. (For ease of presentation, we consider them in an order different from the order A-C-I-D).

- **Consistency**: The consistency requirement here is that the sum of A and B be unchanged by the execution of the transaction. Without the consistency requirement, money could be created or destroyed by the transaction! It can

be verified easily that, if the database is consistent before an execution of the transaction, the database remains consistent after the execution of the transaction.

Ensuring consistency for an individual transaction is the responsibility of the application programmer who codes the transaction. This task may be facilitated by automatic testing of integrity constraints, as we discussed in Chapter 6.

- **Atomicity**: Suppose that, just before the execution of transaction T_i the values of accounts A and B are \$1000 and \$2000, respectively. Now suppose that, during the execution of transaction T_i, a failure occurs that prevents T_i from completing its execution successfully. Examples of such failures include power failures, hardware failures, and software errors. Further, suppose that the failure happened after the write(A) operation but before the write(B) operation. In this case, the values of accounts A and B reflected in the database are \$950 and \$2000. The system destroyed \$50 as a result of this failure. In particular, we note that the sum $A + B$ is no longer preserved.

  Thus, because of the failure, the state of the system no longer reflects a real state of the world that the database is supposed to capture. We term such a state an **inconsistent state**. We must ensure that such inconsistencies are not visible in a database system. Note, however, that the system must at some point be in an inconsistent state. Even if transaction T_i is executed to completion, there exists a point at which the value of account A is \$950 and the value of account B is \$2000, which is clearly an inconsistent state. This state, however, is eventually replaced by the consistent state where the value of account A is \$950, and the value of account B is \$2050. Thus, if the transaction never started or was guaranteed to complete, such an inconsistent state would not be visible except during the execution of the transaction. That is the reason for the atomicity requirement: If the atomicity property is present, all actions of the transaction are reflected in the database, or none are.

  The basic idea behind ensuring atomicity is this: The database system keeps track (on disk) of the old values of any data on which a transaction performs a write, and, if the transaction does not complete its execution, the database system restores the old values to make it appear as though the transaction never executed. We discuss these ideas further in Section 15.2. Ensuring atomicity is the responsibility of the database system itself; specifically, it is handled by a component called the **transaction-management component**, which we describe in detail in Chapter 17.

- **Durability:** Once the execution of the transaction completes successfully, and the user who initiated the transaction has been notified that the transfer of funds has taken place, it must be the case that no system failure will result in a loss of data corresponding to this transfer of funds.

  The durability property guarantees that, once a transaction completes successfully, all the updates that it carried out on the database persist, even if there is a system failure after the transaction completes execution.

We assume for now that a failure of the computer system may result in loss of data in main memory, but data written to disk are never lost. We can guarantee durability by ensuring that either

1. The updates carried out by the transaction have been written to disk before the transaction completes.
2. Information about the updates carried out by the transaction and written to disk is sufficient to enable the database to reconstruct the updates when the database system is restarted after the failure.

Ensuring durability is the responsibility of a component of the database system called the **recovery-management component**. The transaction-management component and the recovery-management component are closely related, and we describe them in Chapter 17.

- **Isolation:** Even if the consistency and atomicity properties are ensured for each transaction, if several transactions are executed concurrently, their operations may interleave in some undesirable way, resulting in an inconsistent state.

  For example, as we saw earlier, the database is temporarily inconsistent while the transaction to transfer funds from A to B is executing, with the deducted total written to A and the increased total yet to be written to B. If a second concurrently running transaction reads A and B at this intermediate point and computes $A + B$, it will observe an inconsistent value. Furthermore, if this second transaction then performs updates on A and B based on the inconsistent values that it read, the database may be left in an inconsistent state even after both transactions have completed.

  A way to avoid the problem of concurrently executing transactions is to execute transactions serially—that is, one after the other. However, concurrent execution of transactions provides significant performance benefits, as we shall see in Section 15.4. Other solutions have therefore been developed; they allow multiple transactions to execute concurrently.

  We discuss the problems caused by concurrently executing transactions in Section 15.4. The isolation property of a transaction ensures that the concurrent execution of transactions results in a system state that is equivalent to a state that could have been obtained had these transactions executed one at a time in some order. We shall discuss the principles of isolation further in Section 15.5. Ensuring the isolation property is the responsibility of a component of the database system called the **concurrency-control component**, which we discuss later, in Chapter 16.

15.2 Transaction State

In the absence of failures, all transactions complete successfully. However, as we noted earlier, a transaction may not always complete its execution successfully. Such a transaction is termed **aborted**. If we are to ensure the atomicity property, an aborted transaction must have no effect on the state of the database. Thus, any changes that

the aborted transaction made to the database must be undone. Once the changes caused by an aborted transaction have been undone, we say that the transaction has been **rolled back**. It is part of the responsibility of the recovery scheme to manage transaction aborts.

A transaction that completes its execution successfully is said to be **committed**. A committed transaction that has performed updates transforms the database into a new consistent state, which must persist even if there is a system failure.

Once a transaction has committed, we cannot undo its effects by aborting it. The only way to undo the effects of a committed transaction is to execute a **compensating transaction**. For instance, if a transaction added $20 to an account, the compensating transaction would subtract $20 from the account. However, it is not always possible to create such a compensating transaction. Therefore, the responsibility of writing and executing a compensating transaction is left to the user, and is not handled by the database system. Chapter 24 includes a discussion of compensating transactions.

We need to be more precise about what we mean by *successful completion* of a transaction. We therefore establish a simple abstract transaction model. A transaction must be in one of the following states:

- **Active**, the initial state; the transaction stays in this state while it is executing
- **Partially committed**, after the final statement has been executed
- **Failed**, after the discovery that normal execution can no longer proceed
- **Aborted**, after the transaction has been rolled back and the database has been restored to its state prior to the start of the transaction
- **Committed**, after successful completion

The state diagram corresponding to a transaction appears in Figure 15.1. We say that a transaction has committed only if it has entered the committed state. Similarly, we say that a transaction has aborted only if it has entered the aborted state. A transaction is said to have **terminated** if has either committed or aborted.

A transaction starts in the active state. When it finishes its final statement, it enters the partially committed state. At this point, the transaction has completed its execution, but it is still possible that it may have to be aborted, since the actual output may still be temporarily residing in main memory, and thus a hardware failure may preclude its successful completion.

The database system then writes out enough information to disk that, even in the event of a failure, the updates performed by the transaction can be re-created when the system restarts after the failure. When the last of this information is written out, the transaction enters the committed state.

As mentioned earlier, we assume for now that failures do not result in loss of data on disk. Chapter 17 discusses techniques to deal with loss of data on disk.

A transaction enters the failed state after the system determines that the transaction can no longer proceed with its normal execution (for example, because of hardware or logical errors). Such a transaction must be rolled back. Then, it enters the aborted state. At this point, the system has two options:

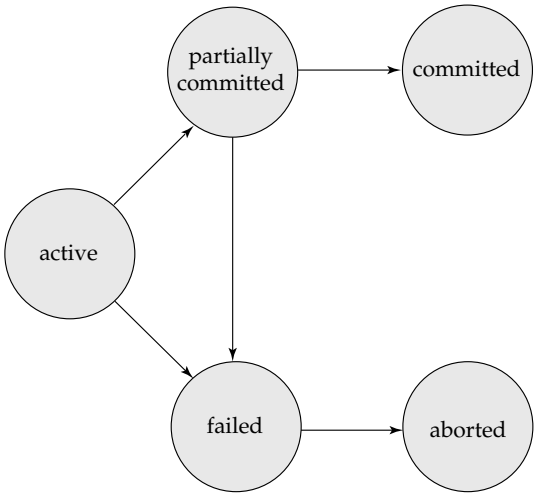


Figure 15.1 State diagram of a transaction.

- It can **restart** the transaction, but only if the transaction was aborted as a result of some hardware or software error that was not created through the internal logic of the transaction. A restarted transaction is considered to be a new transaction.
- It can **kill** the transaction. It usually does so because of some internal logical error that can be corrected only by rewriting the application program, or because the input was bad, or because the desired data were not found in the database.

We must be cautious when dealing with **observable external writes**, such as writes to a terminal or printer. Once such a write has occurred, it cannot be erased, since it may have been seen external to the database system. Most systems allow such writes to take place only after the transaction has entered the committed state. One way to implement such a scheme is for the database system to store any value associated with such external writes temporarily in nonvolatile storage, and to perform the actual writes only after the transaction enters the committed state. If the system should fail after the transaction has entered the committed state, but before it could complete the external writes, the database system will carry out the external writes (using the data in nonvolatile storage) when the system is restarted.

Handling external writes can be more complicated in some situations. For example suppose the external action is that of dispensing cash at an automated teller machine, and the system fails just before the cash is actually dispensed (we assume that cash can be dispensed atomically). It makes no sense to dispense cash when the system is restarted, since the user may have left the machine. In such a case a compensating transaction, such as depositing the cash back in the users account, needs to be executed when the system is restarted.

For certain applications, it may be desirable to allow active transactions to display data to users, particularly for long-duration transactions that run for minutes or hours. Unfortunately, we cannot allow such output of observable data unless we are willing to compromise transaction atomicity. Most current transaction systems ensure atomicity and, therefore, forbid this form of interaction with users. In Chapter 24, we discuss alternative transaction models that support long-duration, interactive transactions.

15.3 Implementation of Atomicity and Durability

The recovery-management component of a database system can support atomicity and durability by a variety of schemes. We first consider a simple, but extremely inefficient, scheme called the **shadow copy** scheme. This scheme, which is based on making copies of the database, called *shadow* copies, assumes that only one transaction is active at a time. The scheme also assumes that the database is simply a file on disk. A pointer called db-pointer is maintained on disk; it points to the current copy of the database.

In the shadow-copy scheme, a transaction that wants to update the database first creates a complete copy of the database. All updates are done on the new database copy, leaving the original copy, the **shadow copy**, untouched. If at any point the transaction has to be aborted, the system merely deletes the new copy. The old copy of the database has not been affected.

If the transaction completes, it is committed as follows. First, the operating system is asked to make sure that all pages of the new copy of the database have been written out to disk. (Unix systems use the flush command for this purpose.) After the operating system has written all the pages to disk, the database system updates the pointer db-pointer to point to the new copy of the database; the new copy then becomes the current copy of the database. The old copy of the database is then deleted. Figure 15.2 depicts the scheme, showing the database state before and after the update.

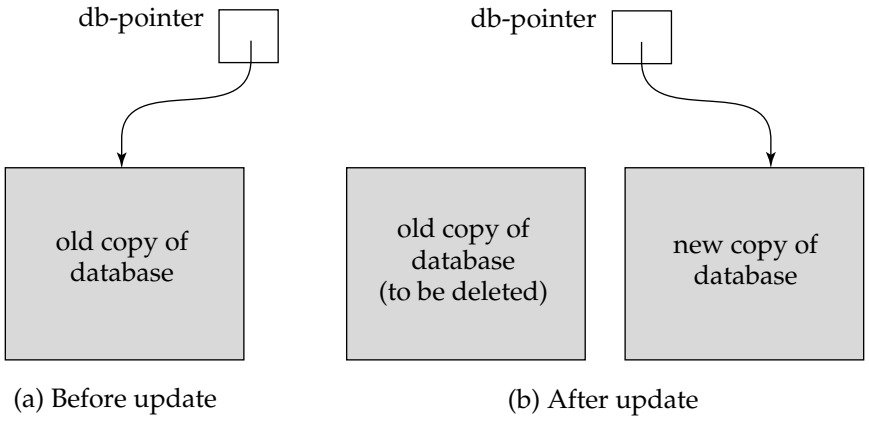


Figure 15.2 Shadow-copy technique for atomicity and durability.

The transaction is said to have been *committed* at the point where the updated db-pointer is written to disk.

We now consider how the technique handles transaction and system failures. First, consider transaction failure. If the transaction fails at any time before db-pointer is updated, the old contents of the database are not affected. We can abort the transaction by just deleting the new copy of the database. Once the transaction has been committed, all the updates that it performed are in the database pointed to by db-pointer. Thus, either all updates of the transaction are reflected, or none of the effects are reflected, regardless of transaction failure.

Now consider the issue of system failure. Suppose that the system fails at any time before the updated db-pointer is written to disk. Then, when the system restarts, it will read db-pointer and will thus see the original contents of the database, and none of the effects of the transaction will be visible on the database. Next, suppose that the system fails after db-pointer has been updated on disk. Before the pointer is updated, all updated pages of the new copy of the database were written to disk. Again, we assume that, once a file is written to disk, its contents will not be damaged even if there is a system failure. Therefore, when the system restarts, it will read db-pointer and will thus see the contents of the database *after* all the updates performed by the transaction.

The implementation actually depends on the write to db-pointer being atomic; that is, either all its bytes are written or none of its bytes are written. If some of the bytes of the pointer were updated by the write, but others were not, the pointer is meaningless, and neither old nor new versions of the database may be found when the system restarts. Luckily, disk systems provide atomic updates to entire blocks, or at least to a disk sector. In other words, the disk system guarantees that it will update db-pointer atomically, as long as we make sure that db-pointer lies entirely in a single sector, which we can ensure by storing db-pointer at the beginning of a block.

Thus, the atomicity and durability properties of transactions are ensured by the shadow-copy implementation of the recovery-management component.

As a simple example of a transaction outside the database domain, consider a text-editing session. An entire editing session can be modeled as a transaction. The actions executed by the transaction are reading and updating the file. Saving the file at the end of editing corresponds to a commit of the editing transaction; quitting the editing session without saving the file corresponds to an abort of the editing transaction.

Many text editors use essentially the implementation just described, to ensure that an editing session is transactional. A new file is used to store the updated file. At the end of the editing session, if the updated file is to be saved, the text editor uses a file *rename* command to rename the new file to have the actual file name. The rename, assumed to be implemented as an atomic operation by the underlying file system, deletes the old file as well.

Unfortunately, this implementation is extremely inefficient in the context of large databases, since executing a single transaction requires copying the *entire* database. Furthermore, the implementation does not allow transactions to execute concurrently with one another. There are practical ways of implementing atomicity and durability that are much less expensive and more powerful. We study these recovery techniques in Chapter 17.

15.4 Concurrent Executions

Transaction-processing systems usually allow multiple transactions to run concurrently. Allowing multiple transactions to update data concurrently causes several complications with consistency of the data, as we saw earlier. Ensuring consistency in spite of concurrent execution of transactions requires extra work; it is far easier to insist that transactions run **serially**—that is, one at a time, each starting only after the previous one has completed. However, there are two good reasons for allowing concurrency:

- **Improved throughput and resource utilization**. A transaction consists of many steps. Some involve I/O activity; others involve CPU activity. The CPU and the disks in a computer system can operate in parallel. Therefore, I/O activity can be done in parallel with processing at the CPU. The parallelism of the CPU and the I/O system can therefore be exploited to run multiple transactions in parallel. While a read or write on behalf of one transaction is in progress on one disk, another transaction can be running in the CPU, while another disk may be executing a read or write on behalf of a third transaction. All of this increases the **throughput** of the system—that is, the number of transactions executed in a given amount of time. Correspondingly, the processor and disk **utilization** also increase; in other words, the processor and disk spend less time idle, or not performing any useful work.
- **Reduced waiting time**. There may be a mix of transactions running on a system, some short and some long. If transactions run serially, a short transaction may have to wait for a preceding long transaction to complete, which can lead to unpredictable delays in running a transaction. If the transactions are operating on different parts of the database, it is better to let them run concurrently, sharing the CPU cycles and disk accesses among them. Concurrent execution reduces the unpredictable delays in running transactions. Moreover, it also reduces the **average response time**: the average time for a transaction to be completed after it has been submitted.

The motivation for using concurrent execution in a database is essentially the same as the motivation for using **multiprogramming** in an operating system.

When several transactions run concurrently, database consistency can be destroyed despite the correctness of each individual transaction. In this section, we present the concept of schedules to help identify those executions that are guaranteed to ensure consistency.

The database system must control the interaction among the concurrent transactions to prevent them from destroying the consistency of the database. It does so through a variety of mechanisms called **concurrency-control schemes**. We study concurrency-control schemes in Chapter 16; for now, we focus on the concept of correct concurrent execution.

Consider again the simplified banking system of Section 15.1, which has several accounts, and a set of transactions that access and update those accounts. Let T_1 and

T_2 be two transactions that transfer funds from one account to another. Transaction T_1 transfers \$50 from account A to account B. It is defined as

T_1: read(A);
$A := A - 50$;
write(A);
read(B);
$B := B + 50$;
write(B).

Transaction T_2 transfers 10 percent of the balance from account A to account B. It is defined as

T_2: read(A);
$temp := A * 0.1$;
$A := A - temp$;
write(A);
read(B);
$B := B + temp$;
write(B).

Suppose the current values of accounts A and B are \$1000 and \$2000, respectively. Suppose also that the two transactions are executed one at a time in the order T_1 followed by T_2. This execution sequence appears in Figure 15.3. In the figure, the sequence of instruction steps is in chronological order from top to bottom, with instructions of T_1 appearing in the left column and instructions of T_2 appearing in the right column. The final values of accounts A and B, after the execution in Figure 15.3 takes place, are \$855 and \$2145, respectively. Thus, the total amount of money in

T_1	T_2
read(A)	
$A := A - 50$	
write (A)	
read(B)	
$B := B + 50$	
write(B)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)

Figure 15.3 Schedule 1—a serial schedule in which T_1 is followed by T_2.

accounts A and B—that is, the sum $A + B$—is preserved after the execution of both transactions.

Similarly, if the transactions are executed one at a time in the order T_2 followed by T_1, then the corresponding execution sequence is that of Figure 15.4. Again, as expected, the sum $A + B$ is preserved, and the final values of accounts A and B are \$850 and \$2150, respectively.

The execution sequences just described are called **schedules**. They represent the chronological order in which instructions are executed in the system. Clearly, a schedule for a set of transactions must consist of all instructions of those transactions, and must preserve the order in which the instructions appear in each individual transaction. For example, in transaction T_1, the instruction write(A) must appear before the instruction read(B), in any valid schedule. In the following discussion, we shall refer to the first execution sequence (T_1 followed by T_2) as schedule 1, and to the second execution sequence (T_2 followed by T_1) as schedule 2.

These schedules are **serial**: Each serial schedule consists of a sequence of instructions from various transactions, where the instructions belonging to one single transaction appear together in that schedule. Thus, for a set of n transactions, there exist $n!$ different valid serial schedules.

When the database system executes several transactions concurrently, the corresponding schedule no longer needs to be serial. If two transactions are running concurrently, the operating system may execute one transaction for a little while, then perform a context switch, execute the second transaction for some time, and then switch back to the first transaction for some time, and so on. With multiple transactions, the CPU time is shared among all the transactions.

Several execution sequences are possible, since the various instructions from both transactions may now be interleaved. In general, it is not possible to predict exactly how many instructions of a transaction will be executed before the CPU switches to

T_1	T_2
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
	$B := B + temp$
	write(B)
read(A)	
$A := A - 50$	
write(A)	
read(B)	
$B := B + 50$	
write(B)	

Figure 15.4 Schedule 2—a serial schedule in which T_2 is followed by T_1.

T_1	T_2
read(A)	
$A := A - 50$	
write(A)	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
read(B)	
$B := B + 50$	
write(B)	
	read(B)
	$B := B + temp$
	write(B)

Figure 15.5 Schedule 3—a concurrent schedule equivalent to schedule 1.

another transaction. Thus, the number of possible schedules for a set of n transactions is much larger than $n!$.

Returning to our previous example, suppose that the two transactions are executed concurrently. One possible schedule appears in Figure 15.5. After this execution takes place, we arrive at the same state as the one in which the transactions are executed serially in the order T_1 followed by T_2. The sum $A + B$ is indeed preserved.

Not all concurrent executions result in a correct state. To illustrate, consider the schedule of Figure 15.6. After the execution of this schedule, we arrive at a state where the final values of accounts A and B are \$950 and \$2100, respectively. This final state is an *inconsistent state*, since we have gained \$50 in the process of the concurrent execution. Indeed, the sum $A + B$ is not preserved by the execution of the two transactions.

If control of concurrent execution is left entirely to the operating system, many possible schedules, including ones that leave the database in an inconsistent state, such as the one just described, are possible. It is the job of the database system to ensure that any schedule that gets executed will leave the database in a consistent state. The **concurrency-control component** of the database system carries out this task.

We can ensure consistency of the database under concurrent execution by making sure that any schedule that executed has the same effect as a schedule that could have occurred without any concurrent execution. That is, the schedule should, in some sense, be equivalent to a serial schedule. We examine this idea in Section 15.5.

15.5 Serializability

The database system must control concurrent execution of transactions, to ensure that the database state remains consistent. Before we examine how the database

T_1	T_2
read(A)	
$A := A - 50$	
	read(A)
	$temp := A * 0.1$
	$A := A - temp$
	write(A)
	read(B)
write(A)	
read(B)	
$B := B + 50$	
write(B)	
	$B := B + temp$
	write(B)

Figure 15.6 Schedule 4—a concurrent schedule.

system can carry out this task, we must first understand which schedules will ensure consistency, and which schedules will not.

Since transactions are programs, it is computationally difficult to determine exactly what operations a transaction performs and how operations of various transactions interact. For this reason, we shall not interpret the type of operations that a transaction can perform on a data item. Instead, we consider only two operations: read and write. We thus assume that, between a read(Q) instruction and a write(Q) instruction on a data item Q, a transaction may perform an arbitrary sequence of operations on the copy of Q that is residing in the local buffer of the transaction. Thus, the only significant operations of a transaction, from a scheduling point of view, are its read and write instructions. We shall therefore usually show only read and write instructions in schedules, as we do in schedule 3 in Figure 15.7.

In this section, we discuss different forms of schedule equivalence; they lead to the notions of **conflict serializability** and **view serializability**.

T_1	T_2
read(A)	
write(A)	
	read(A)
	write(A)
read(B)	
write(B)	
	read(B)
	write(B)

Figure 15.7 Schedule 3—showing only the read and write instructions.

15.5.1 Conflict Serializability

Let us consider a schedule S in which there are two consecutive instructions I_i and I_j, of transactions T_i and T_j, respectively ($i \neq j$). If I_i and I_j refer to different data items, then we can swap I_i and I_j without affecting the results of any instruction in the schedule. However, if I_i and I_j refer to the same data item Q, then the order of the two steps may matter. Since we are dealing with only read and write instructions, there are four cases that we need to consider:

1. I_i = read(Q), I_j = read(Q). The order of I_i and I_j does not matter, since the same value of Q is read by T_i and T_j, regardless of the order.
2. I_i = read(Q), I_j = write(Q). If I_i comes before I_j, then T_i does not read the value of Q that is written by T_j in instruction I_j. If I_j comes before I_i, then T_i reads the value of Q that is written by T_j. Thus, the order of I_i and I_j matters.
3. I_i = write(Q), I_j = read(Q). The order of I_i and I_j matters for reasons similar to those of the previous case.
4. I_i = write(Q), I_j = write(Q). Since both instructions are write operations, the order of these instructions does not affect either T_i or T_j. However, the value obtained by the next read(Q) instruction of S is affected, since the result of only the latter of the two write instructions is preserved in the database. If there is no other write(Q) instruction after I_i and I_j in S, then the order of I_i and I_j directly affects the final value of Q in the database state that results from schedule S.

Thus, only in the case where both I_i and I_j are read instructions does the relative order of their execution not matter.

We say that I_i and I_j **conflict** if they are operations by different transactions on the same data item, and at least one of these instructions is a write operation.

To illustrate the concept of conflicting instructions, we consider schedule 3, in Figure 15.7. The write(A) instruction of T_1 conflicts with the read(A) instruction of T_2. However, the write(A) instruction of T_2 does not conflict with the read(B) instruction of T_1, because the two instructions access different data items.

Let I_i and I_j be consecutive instructions of a schedule S. If I_i and I_j are instructions of different transactions and I_i and I_j do not conflict, then we can swap the order of I_i and I_j to produce a new schedule S'. We expect S to be equivalent to S', since all instructions appear in the same order in both schedules except for I_i and I_j, whose order does not matter.

Since the write(A) instruction of T_2 in schedule 3 of Figure 15.7 does not conflict with the read(B) instruction of T_1, we can swap these instructions to generate an equivalent schedule, schedule 5, in Figure 15.8. Regardless of the initial system state, schedules 3 and 5 both produce the same final system state.

We continue to swap nonconflicting instructions:

- Swap the read(B) instruction of T_1 with the read(A) instruction of T_2.
- Swap the write(B) instruction of T_1 with the write(A) instruction of T_2.
- Swap the write(B) instruction of T_1 with the read(A) instruction of T_2.

T_1	T_2
read(A)	
write(A)	
	read(A)
read(B)	
	write(A)
write(B)	
	read(B)
	write(B)

Figure 15.8 Schedule 5—schedule 3 after swapping of a pair of instructions.

The final result of these swaps, schedule 6 of Figure 15.9, is a serial schedule. Thus, we have shown that schedule 3 is equivalent to a serial schedule. This equivalence implies that, regardless of the initial system state, schedule 3 will produce the same final state as will some serial schedule.

If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**.

In our previous examples, schedule 1 is not conflict equivalent to schedule 2. However, schedule 1 is conflict equivalent to schedule 3, because the read(B) and write(B) instruction of T_1 can be swapped with the read(A) and write(A) instruction of T_2.

The concept of conflict equivalence leads to the concept of conflict serializability. We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule. Thus, schedule 3 is conflict serializable, since it is conflict equivalent to the serial schedule 1.

Finally, consider schedule 7 of Figure 15.10; it consists of only the significant operations (that is, the read and write) of transactions T_3 and T_4. This schedule is not conflict serializable, since it is not equivalent to either the serial schedule $<T_3,T_4>$ or the serial schedule $<T_4,T_3>$.

It is possible to have two schedules that produce the same outcome, but that are not conflict equivalent. For example, consider transaction T_5, which transfers \$10

T_1	T_2
read(A)	
write(A)	
read(B)	
write(B)	
	read(A)
	write(A)
	read(B)
	write(B)

Figure 15.9 Schedule 6—a serial schedule that is equivalent to schedule 3.

T_3	T_4
read(Q)	
	write(Q)
write(Q)	

Figure 15.10 Schedule 7.

from account B to account A. Let schedule 8 be as defined in Figure 15.11. We claim that schedule 8 is not conflict equivalent to the serial schedule $<T_1,T_5>$, since, in schedule 8, the write(B) instruction of T_5 conflicts with the read(B) instruction of T_1. Thus, we cannot move all the instructions of T_1 before those of T_5 by swapping consecutive nonconflicting instructions. However, the final values of accounts A and B after the execution of either schedule 8 or the serial schedule $<T_1,T_5>$ are the same —$960 and $2040, respectively.

We can see from this example that there are less stringent definitions of schedule equivalence than conflict equivalence. For the system to determine that schedule 8 produces the same outcome as the serial schedule $<T_1,T_5>$, it must analyze the computation performed by T_1 and T_5, rather than just the read and write operations. In general, such analysis is hard to implement and is computationally expensive. However, there are other definitions of schedule equivalence based purely on the read and write operations. We will consider one such definition in the next section.

15.5.2 View Serializability

In this section, we consider a form of equivalence that is less stringent than conflict equivalence, but that, like conflict equivalence, is based on only the read and write operations of transactions.

T_1	T_5
read(A)	
$A := A - 50$	
write(A)	
	read(B)
	$B := B - 10$
	write(B)
read(B)	
$B := B + 50$	
write(B)	
	read(A)
	$A := A + 10$
	write(A)

Figure 15.11 Schedule 8.

Consider two schedules S and S', where the same set of transactions participates in both schedules. The schedules S and S' are said to be **view equivalent** if three conditions are met:

1. For each data item Q, if transaction T_i reads the initial value of Q in schedule S, then transaction T_i must, in schedule S', also read the initial value of Q.
2. For each data item Q, if transaction T_i executes read(Q) in schedule S, and if that value was produced by a write(Q) operation executed by transaction T_j, then the read(Q) operation of transaction T_i must, in schedule S', also read the value of Q that was produced by the same write(Q) operation of transaction T_j.
3. For each data item Q, the transaction (if any) that performs the final write(Q) operation in schedule S must perform the final write(Q) operation in schedule S'.

Conditions 1 and 2 ensure that each transaction reads the same values in both schedules and, therefore, performs the same computation. Condition 3, coupled with conditions 1 and 2, ensures that both schedules result in the same final system state.

In our previous examples, schedule 1 is not view equivalent to schedule 2, since, in schedule 1, the value of account A read by transaction T_2 was produced by T_1, whereas this case does not hold in schedule 2. However, schedule 1 is view equivalent to schedule 3, because the values of account A and B read by transaction T_2 were produced by T_1 in both schedules.

The concept of view equivalence leads to the concept of view serializability. We say that a schedule S is **view serializable** if it is view equivalent to a serial schedule.

As an illustration, suppose that we augment schedule 7 with transaction T_6, and obtain schedule 9 in Figure 15.12. Schedule 9 is view serializable. Indeed, it is view equivalent to the serial schedule $<T_3, T_4, T_6>$, since the one read(Q) instruction reads the initial value of Q in both schedules, and T_6 performs the final write of Q in both schedules.

Every conflict-serializable schedule is also view serializable, but there are view-serializable schedules that are not conflict serializable. Indeed, schedule 9 is not conflict serializable, since every pair of consecutive instructions conflicts, and, thus, no swapping of instructions is possible.

Observe that, in schedule 9, transactions T_4 and T_6 perform write(Q) operations without having performed a read(Q) operation. Writes of this sort are called **blind writes**. Blind writes appear in any view-serializable schedule that is not conflict serializable.

T_3	T_4	T_6
read(Q)		
	write(Q)	
write(Q)		
		write(Q)

Figure 15.12 Schedule 9—a view-serializable schedule.

15.6 Recoverability

So far, we have studied what schedules are acceptable from the viewpoint of consistency of the database, assuming implicitly that there are no transaction failures. We now address the effect of transaction failures during concurrent execution.

If a transaction T_i fails, for whatever reason, we need to undo the effect of this transaction to ensure the atomicity property of the transaction. In a system that allows concurrent execution, it is necessary also to ensure that any transaction T_j that is dependent on T_i (that is, T_j has read data written by T_i) is also aborted. To achieve this surety, we need to place restrictions on the type of schedules permitted in the system.

In the following two subsections, we address the issue of what schedules are acceptable from the viewpoint of recovery from transaction failure. We describe in Chapter 16 how to ensure that only such acceptable schedules are generated.

15.6.1 Recoverable Schedules

Consider schedule 11 in Figure 15.13, in which T_9 is a transaction that performs only one instruction: read(A). Suppose that the system allows T_9 to commit immediately after executing the read(A) instruction. Thus, T_9 commits before T_8 does. Now suppose that T_8 fails before it commits. Since T_9 has read the value of data item A written by T_8, we must abort T_9 to ensure transaction atomicity. However, T_9 has already committed and cannot be aborted. Thus, we have a situation where it is impossible to recover correctly from the failure of T_8.

Schedule 11, with the commit happening immediately after the read(A) instruction, is an example of a *nonrecoverable* schedule, which should not be allowed. Most database system require that all schedules be *recoverable*. A **recoverable schedule** is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the commit operation of T_j.

15.6.2 Cascadeless Schedules

Even if a schedule is recoverable, to recover correctly from the failure of a transaction T_i, we may have to roll back several transactions. Such situations occur if transactions have read data written by T_i. As an illustration, consider the partial schedule

T_8	T_9
read(A)	
write(A)	
	read(A)
read(B)	

Figure 15.13 Schedule 11.

T_{10}	T_{11}	T_{12}
read(A)		
read(B)		
write(A)		
	read(A)	
	write(A)	
		read(A)

Figure 15.14 Schedule 12.

of Figure 15.14. Transaction T_{10} writes a value of A that is read by transaction T_{11}. Transaction T_{11} writes a value of A that is read by transaction T_{12}. Suppose that, at this point, T_{10} fails. T_{10} must be rolled back. Since T_{11} is dependent on T_{10}, T_{11} must be rolled back. Since T_{12} is dependent on T_{11}, T_{12} must be rolled back. This phenomenon, in which a single transaction failure leads to a series of transaction rollbacks, is called **cascading rollback**.

Cascading rollback is undesirable, since it leads to the undoing of a significant amount of work. It is desirable to restrict the schedules to those where cascading rollbacks cannot occur. Such schedules are called *cascadeless* schedules. Formally, a **cascadeless schedule** is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i, the commit operation of T_i appears before the read operation of T_j. It is easy to verify that every cascadeless schedule is also recoverable.

15.7 Implementation of Isolation

So far, we have seen what properties a schedule must have if it is to leave the database in a consistent state and allow transaction failures to be handled in a safe manner. Specifically, schedules that are conflict or view serializable and cascadeless satisfy these requirements.

There are various **concurrency-control scheme**s that we can use to ensure that, even when multiple transactions are executed concurrently, only acceptable schedules are generated, regardless of how the operating-system time-shares resources (such as CPU time) among the transactions.

As a trivial example of a concurrency-control scheme, consider this scheme: A transaction acquires a **lock** on the entire database before it starts and releases the lock after it has committed. While a transaction holds a lock, no other transaction is allowed to acquire the lock, and all must therefore wait for the lock to be released. As a result of the locking policy, only one transaction can execute at a time. Therefore, only serial schedules are generated. These are trivially serializable, and it is easy to verify that they are cascadeless as well.

A concurrency-control scheme such as this one leads to poor performance, since it forces transactions to wait for preceding transactions to finish before they can start. In other words, it provides a poor degree of concurrency. As explained in Section 15.4, concurrent execution has several performance benefits.

The goal of concurrency-control schemes is to provide a high degree of concurrency, while ensuring that all schedules that can be generated are conflict or view serializable, and are cascadeless.

We study a number of concurrency-control schemes in Chapter 16. The schemes have different trade-offs in terms of the amount of concurrency they allow and the amount of overhead that they incur. Some of them allow only conflict serializable schedules to be generated; others allow certain view-serializable schedules that are not conflict-serializable to be generated.

15.8 Transaction Definition in SQL

A data-manipulation language must include a construct for specifying the set of actions that constitute a transaction.

The SQL standard specifies that a transaction begins implicitly. Transactions are ended by one of these SQL statements:

- **Commit work** commits the current transaction and begins a new one.
- **Rollback work** causes the current transaction to abort.

The keyword **work** is optional in both the statements. If a program terminates without either of these commands, the updates are either committed or rolled back—which of the two happens is not specified by the standard and depends on the implementation.

The standard also specifies that the system must ensure both serializability and freedom from cascading rollback. The definition of serializability used by the standard is that a schedule must have the *same effect* as would some serial schedule. Thus, conflict and view serializability are both acceptable.

The SQL-92 standard also allows a transaction to specify that it may be executed in a manner that causes it to become nonserializable with respect to other transactions. We study such weaker levels of consistency in Section 16.8.

15.9 Testing for Serializability

When designing concurrency control schemes, we must show that schedules generated by the scheme are serializable. To do that, we must first understand how to determine, given a particular schedule S, whether the schedule is serializable.

We now present a simple and efficient method for determining conflict serializability of a schedule. Consider a schedule S. We construct a directed graph, called a **precedence graph**, from S. This graph consists of a pair $G = (V, E)$, where V is a set of vertices and E is a set of edges. The set of vertices consists of all the transactions participating in the schedule. The set of edges consists of all edges $T_i \rightarrow T_j$ for which one of three conditions holds:

1. T_i executes write(Q) before T_j executes read(Q).
2. T_i executes read(Q) before T_j executes write(Q).
3. T_i executes write(Q) before T_j executes write(Q).



Figure 15.15 Precedence graph for (a) schedule 1 and (b) schedule 2.

If an edge $T_i \rightarrow T_j$ exists in the precedence graph, then, in any serial schedule S' equivalent to S, T_i must appear before T_j.

For example, the precedence graph for schedule 1 in Figure 15.15a contains the single edge $T_1 \rightarrow T_2$, since all the instructions of T_1 are executed before the first instruction of T_2 is executed. Similarly, Figure 15.15b shows the precedence graph for schedule 2 with the single edge $T_2 \rightarrow T_1$, since all the instructions of T_2 are executed before the first instruction of T_1 is executed.

The precedence graph for schedule 4 appears in Figure 15.16. It contains the edge $T_1 \rightarrow T_2$, because T_1 executes read(A) before T_2 executes write(A). It also contains the edge $T_2 \rightarrow T_1$, because T_2 executes read(B) before T_1 executes write(B).

If the precedence graph for S has a cycle, then schedule S is not conflict serializable. If the graph contains no cycles, then the schedule S is conflict serializable.

A **serializability order** of the transactions can be obtained through **topological sorting**, which determines a linear order consistent with the partial order of the precedence graph. There are, in general, several possible linear orders that can be obtained through a topological sorting. For example, the graph of Figure 15.17a has the two acceptable linear orderings shown in Figures 15.17b and 15.17c.

Thus, to test for conflict serializability, we need to construct the precedence graph and to invoke a cycle-detection algorithm. Cycle-detection algorithms can be found in standard textbooks on algorithms. Cycle-detection algorithms, such as those based on depth-first search, require on the order of n^2 operations, where n is the number of vertices in the graph (that is, the number of transactions). Thus, we have a practical scheme for determining conflict serializability.

Returning to our previous examples, note that the precedence graphs for schedules 1 and 2 (Figure 15.15) indeed do not contain cycles. The precedence graph for schedule 4 (Figure 15.16), on the other hand, contains a cycle, indicating that this schedule is not conflict serializable.

Testing for view serializability is rather complicated. In fact, it has been shown that the problem of testing for view serializability is itself *NP*-complete. Thus, almost certainly there exists no efficient algorithm to test for view serializability. See

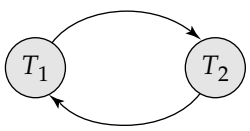


Figure 15.16 Precedence graph for schedule 4.

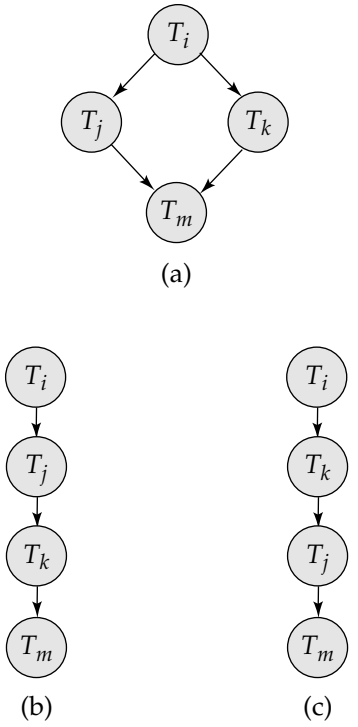


Figure 15.17 Illustration of topological sorting.

the bibliographical notes for references on testing for view serializability. However, concurrency-control schemes can still use *sufficient conditions* for view serializability. That is, if the sufficient conditions are satisfied, the schedule is view serializable, but there may be view-serializable schedules that do not satisfy the sufficient conditions.

15.10 Summary

- A *transaction* is a *unit* of program execution that accesses and possibly updates various data items. Understanding the concept of a transaction is critical for understanding and implementing updates of data in a database, in such a way that concurrent executions and failures of various forms do not result in the database becoming inconsistent.
- Transactions are required to have the ACID properties: atomicity, consistency, isolation, and durability.
  - Atomicity ensures that either all the effects of a transaction are reflected in the database, or none are; a failure cannot leave the database in a state where a transaction is partially executed.
  - Consistency ensures that, if the database is initially consistent, the execution of the transaction (by itself) leaves the database in a consistent state.

- Isolation ensures that concurrently executing transactions are isolated from one another, so that each has the impression that no other transaction is executing concurrently with it.
- Durability ensures that, once a transaction has been committed, that transaction's updates do not get lost, even if there is a system failure.

- Concurrent execution of transactions improves throughput of transactions and system utilization, and also reduces waiting time of transactions.
- When several transactions execute concurrently in the database, the consistency of data may no longer be preserved. It is therefore necessary for the system to control the interaction among the concurrent transactions.
  - Since a transaction is a unit that preserves consistency, a serial execution of transactions guarantees that consistency is preserved.
  - A *schedule* captures the key actions of transactions that affect concurrent execution, such as read and write operations, while abstracting away internal details of the execution of the transaction.
  - We require that any schedule produced by concurrent processing of a set of transactions will have an effect equivalent to a schedule produced when these transactions are run serially in some order.
  - A system that guarantees this property is said to ensure *serializability*.
  - There are several different notions of equivalence leading to the concepts of *conflict serializability* and *view serializability*.
- Serializability of schedules generated by concurrently executing transactions can be ensured through one of a variety of mechanisms called *concurrency-control* schemes.
- Schedules must be recoverable, to make sure that if transaction a sees the effects of transaction b, and b then aborts, then a also gets aborted.
- Schedules should preferably be cascadeless, so that the abort of a transaction does not result in cascading aborts of other transactions. Cascadelessness is ensured by allowing transactions to only read committed data.
- The concurrency-control–management component of the database is responsible for handling the concurrency-control schemes. Chapter 16 describes concurrency-control schemes.
- The recovery-management component of a database is responsible for ensuring the atomicity and durability properties of transactions.

  The shadow copy scheme is used for ensuring atomicity and durability in text editors; however, it has extremely high overheads when used for database systems, and, moreover, it does not support concurrent execution. Chapter 17 covers better schemes.
- We can test a given schedule for conflict serializability by constructing a *precedence graph* for the schedule, and by searching for absence of cycles in the

graph. However, there are more efficient concurrency control schemes for ensuring serializability.

Review Terms

- Transaction
- ACID properties
  - Atomicity
  - Consistency
  - Isolation
  - Durability
- Inconsistent state
- Transaction state
  - Active
  - Partially committed
  - Failed
  - Aborted
  - Committed
  - Terminated
- Transaction
  - Restart
  - Kill
- Observable external writes
- Shadow copy scheme
- Concurrent executions
- Serial execution
- Schedules
- Conflict of operations
- Conflict equivalence
- Conflict serializability
- View equivalence
- View serializability
- Blind writes
- Recoverability
- Recoverable schedules
- Cascading rollback
- Cascadeless schedules
- Concurrency-control scheme
- Lock
- Serializability testing
- Precedence graph
- Serializability order

Exercises

15.1 List the ACID properties. Explain the usefulness of each.

15.2 Suppose that there is a database system that never fails. Is a recovery manager required for this system?

15.3 Consider a file system such as the one on your favorite operating system.

a. What are the steps involved in creation and deletion of files, and in writing data to a file?

b. Explain how the issues of atomicity and durability are relevant to the creation and deletion of files, and to writing data to files.

15.4 Database-system implementers have paid much more attention to the ACID properties than have file-system implementers. Why might this be the case?

15.5 During its execution, a transaction passes through several states, until it finally commits or aborts. List all possible sequences of states through which a transaction may pass. Explain why each state transition may occur.

15.6 Justify the following statement: Concurrent execution of transactions is more important when data must be fetched from (slow) disk or when transactions are long, and is less important when data is in memory and transactions are very short.

15.7 Explain the distinction between the terms *serial schedule* and *serializable schedule*.

15.8 Consider the following two transactions:

T_1: read(A);
read(B);
if $A = 0$ **then** $B := B + 1$;
write(B).

T_2: read(B);
read(A);
if $B = 0$ **then** $A := A + 1$;
write(A).

Let the consistency requirement be $A = 0 \vee B = 0$, with $A = B = 0$ the initial values.

a. Show that every serial execution involving these two transactions preserves the consistency of the database.

b. Show a concurrent execution of T_1 and T_2 that produces a nonserializable schedule.

c. Is there a concurrent execution of T_1 and T_2 that produces a serializable schedule?

15.9 Since every conflict-serializable schedule is view serializable, why do we emphasize conflict serializability rather than view serializability?

15.10 Consider the precedence graph of Figure 15.18. Is the corresponding schedule conflict serializable? Explain your answer.

15.11 What is a recoverable schedule? Why is recoverability of schedules desirable? Are there any circumstances under which it would be desirable to allow nonrecoverable schedules? Explain your answer.

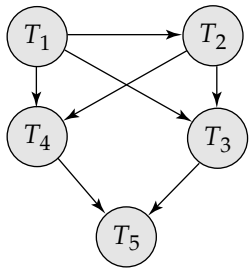


Figure 15.18 Precedence graph.

15.12 What is a cascadeless schedule? Why is cascadelessness of schedules desirable? Are there any circumstances under which it would be desirable to allow noncascadeless schedules? Explain your answer.

Bibliographical Notes

Gray and Reuter [1993] provides detailed textbook coverage of transaction-processing concepts, techniques and implementation details, including concurrency control and recovery issues. Bernstein and Newcomer [1997] provides textbook coverage of various aspects of transaction processing.

Early textbook discussions of concurrency control and recovery included Papadimitriou [1986] and Bernstein et al. [1987]. An early survey paper on implementation issues in concurrency control and recovery is presented by Gray [1978].

The concept of serializability was formalized by Eswaran et al. [1976] in connection to work on concurrency control for System R. The results concerning serializability testing and NP-completeness of testing for view serializability are from Papadimitriou et al. [1977] and Papadimitriou [1979]. Cycle-detection algorithms as well as an introduction to NP-completeness can be found in standard algorithm textbooks such as Cormen et al. [1990].

References covering specific aspects of transaction processing, such as concurrency control and recovery, are cited in Chapters 16, 17, and 24.

© The McGraw–Hill Companies, 2001

CHAPTER 16

Concurrency Control

We saw in Chapter 15 that one of the fundamental properties of a transaction is isolation. When several transactions execute concurrently in the database, however, the isolation property may no longer be preserved. To ensure that it is, the system must control the interaction among the concurrent transactions; this control is achieved through one of a variety of mechanisms called *concurrency-control* schemes.

The concurrency-control schemes that we discuss in this chapter are all based on the serializability property. That is, all the schemes presented here ensure that the schedules are serializable. In Chapter 24, we discuss concurrency control schemes that admit nonserializable schedules. In this chapter, we consider the management of concurrently executing transactions, and we ignore failures. In Chapter 17, we shall see how the system can recover from failures.

16.1 Lock-Based Protocols

One way to ensure serializability is to require that data items be accessed in a mutually exclusive manner; that is, while one transaction is accessing a data item, no other transaction can modify that data item. The most common method used to implement this requirement is to allow a transaction to access a data item only if it is currently holding a lock on that item.

16.1.1 Locks

There are various modes in which a data item may be locked. In this section, we restrict our attention to two modes:

1. **Shared**. If a transaction T_i has obtained a **shared-mode lock** (denoted by S) on item Q, then T_i can read, but cannot write, Q.
2. **Exclusive**. If a transaction T_i has obtained an **exclusive-mode lock** (denoted by X) on item Q, then T_i can both read and write Q.

	S	X
S	true	false
X	false	false

Figure 16.1 Lock-compatibility matrix comp.

We require that every transaction **request** a lock in an appropriate mode on data item Q, depending on the types of operations that it will perform on Q. The transaction makes the request to the concurrency-control manager. The transaction can proceed with the operation only after the concurrency-control manager **grants** the lock to the transaction.

Given a set of lock modes, we can define a **compatibility function** on them as follows. Let A and B represent arbitrary lock modes. Suppose that a transaction T_i requests a lock of mode A on item Q on which transaction T_j ($T_i \neq T_j$) currently holds a lock of mode B. If transaction T_i can be granted a lock on Q immediately, in spite of the presence of the mode B lock, then we say mode A is **compatible** with mode B. Such a function can be represented conveniently by a matrix. The compatibility relation between the two modes of locking discussed in this section appears in the matrix comp of Figure 16.1. An element comp(A, B) of the matrix has the value *true* if and only if mode A is compatible with mode B.

Note that shared mode is compatible with shared mode, but not with exclusive mode. At any time, several shared-mode locks can be held simultaneously (by different transactions) on a particular data item. A subsequent exclusive-mode lock request has to wait until the currently held shared-mode locks are released.

A transaction requests a shared lock on data item Q by executing the lock-S(Q) instruction. Similarly, a transaction requests an exclusive lock through the lock-X(Q) instruction. A transaction can unlock a data item Q by the unlock(Q) instruction.

To access a data item, transaction T_i must first lock that item. If the data item is already locked by another transaction in an incompatible mode, the concurrency-control manager will not grant the lock until all incompatible locks held by other transactions have been released. Thus, T_i is made to **wait** until all incompatible locks held by other transactions have been released.

```
T1: lock-X(B);
    read(B);
    B := B − 50;
    write(B);
    unlock(B);
    lock-X(A);
    read(A);
    A := A + 50;
    write(A);
    unlock(A).
```

Figure 16.2 Transaction T_1.

T_2: lock-S(A);
read(A);
unlock(A);
lock-S(B);
read(B);
unlock(B);
display($A + B$).

Figure 16.3 Transaction T_2.

Transaction T_i may unlock a data item that it had locked at some earlier point. Note that a transaction must hold a lock on a data item as long as it accesses that item. Moreover, for a transaction to unlock a data item immediately after its final access of that data item is not always desirable, since serializability may not be ensured.

As an illustration, consider again the simplified banking system that we introduced in Chapter 15. Let A and B be two accounts that are accessed by transactions T_1 and T_2. Transaction T_1 transfers \$50 from account B to account A (Figure 16.2). Transaction T_2 displays the total amount of money in accounts A and B—that is, the sum $A + B$ (Figure 16.3).

T_1	T_2	concurrency-control manager
lock-X(B)		
		grant-X(B, T_1)
read(B)		
$B := B - 50$		
write(B)		
unlock(B)		
	lock-S(A)	
		grant-S(A, T_2)
	read(A)	
	unlock(A)	
	lock-S(B)	
		grant-S(B, T_2)
	read(B)	
	unlock(B)	
	display($A + B$)	
lock-X(A)		
		grant-X(A, T_2)
read(A)		
$A := A + 50$		
write(A)		
unlock(A)		

Figure 16.4 Schedule 1.

```
T3: lock-X(B);
    read(B);
    B := B − 50;
    write(B);
    lock-X(A);
    read(A);
    A := A + 50;
    write(A);
    unlock(B);
    unlock(A).
```

Figure 16.5 Transaction T_3.

Suppose that the values of accounts A and B are \$100 and \$200, respectively. If these two transactions are executed serially, either in the order T_1, T_2 or the order T_2, T_1, then transaction T_2 will display the value \$300. If, however, these transactions are executed concurrently, then schedule 1, in Figure 16.4 is possible. In this case, transaction T_2 displays \$250, which is incorrect. The reason for this mistake is that the transaction T_1 unlocked data item B too early, as a result of which T_2 saw an inconsistent state.

The schedule shows the actions executed by the transactions, as well as the points at which the concurrency-control manager grants the locks. The transaction making a lock request cannot execute its next action until the concurrency-control manager grants the lock. Hence, the lock must be granted in the interval of time between the lock-request operation and the following action of the transaction. Exactly when within this interval the lock is granted is not important; we can safely assume that the lock is granted just before the following action of the transaction. We shall therefore drop the column depicting the actions of the concurrency-control manager from all schedules depicted in the rest of the chapter. We let you infer when locks are granted.

Suppose now that unlocking is delayed to the end of the transaction. Transaction T_3 corresponds to T_1 with unlocking delayed (Figure 16.5). Transaction T_4 corresponds to T_2 with unlocking delayed (Figure 16.6).

You should verify that the sequence of reads and writes in schedule 1, which lead to an incorrect total of \$250 being displayed, is no longer possible with T_3 and T_4.

```
T4: lock-S(A);
    read(A);
    lock-S(B);
    read(B);
    display(A + B);
    unlock(A);
    unlock(B).
```

Figure 16.6 Transaction T_4.

T_3	T_4
lock-X(B)	
read(B)	
$B := B - 50$	
write(B)	
	lock-S(A)
	read(A)
	lock-S(B)
lock-X(A)	

Figure 16.7 Schedule 2.

Other schedules are possible. T_4 will not print out an inconsistent result in any of them; we shall see why later.

Unfortunately, locking can lead to an undesirable situation. Consider the partial schedule of Figure 16.7 for T_3 and T_4. Since T_3 is holding an exclusive-mode lock on B and T_4 is requesting a shared-mode lock on B, T_4 is waiting for T_3 to unlock B. Similarly, since T_4 is holding a shared-mode lock on A and T_3 is requesting an exclusive-mode lock on A, T_3 is waiting for T_4 to unlock A. Thus, we have arrived at a state where neither of these transactions can ever proceed with its normal execution. This situation is called **deadlock**. When deadlock occurs, the system must roll back one of the two transactions. Once a transaction has been rolled back, the data items that were locked by that transaction are unlocked. These data items are then available to the other transaction, which can continue with its execution. We shall return to the issue of deadlock handling in Section 16.6.

If we do not use locking, or if we unlock data items as soon as possible after reading or writing them, we may get inconsistent states. On the other hand, if we do not unlock a data item before requesting a lock on another data item, deadlocks may occur. There are ways to avoid deadlock in some situations, as we shall see in Section 16.1.5. However, in general, deadlocks are a necessary evil associated with locking, if we want to avoid inconsistent states. Deadlocks are definitely preferable to inconsistent states, since they can be handled by rolling back of transactions, whereas inconsistent states may lead to real-world problems that cannot be handled by the database system.

We shall require that each transaction in the system follow a set of rules, called a **locking protocol**, indicating when a transaction may lock and unlock each of the data items. Locking protocols restrict the number of possible schedules. The set of all such schedules is a proper subset of all possible serializable schedules. We shall present several locking protocols that allow only conflict-serializable schedules. Before doing so, we need a few definitions.

Let $\{T_0, T_1, \ldots, T_n\}$ be a set of transactions participating in a schedule S. We say that T_i **precedes** T_j in S, written $T_i \rightarrow T_j$, if there exists a data item Q such that T_i has held lock mode A on Q, and T_j has held lock mode B on Q later, and comp(A,B) = false. If $T_i \rightarrow T_j$, then that precedence implies that in any equivalent serial schedule, T_i must appear before T_j. Observe that this graph is similar to the precedence

graph that we used in Section 15.9 to test for conflict serializability. Conflicts between instructions correspond to noncompatibility of lock modes.

We say that a schedule S is **legal** under a given locking protocol if S is a possible schedule for a set of transactions that follow the rules of the locking protocol. We say that a locking protocol **ensures** conflict serializability if and only if all legal schedules are conflict serializable; in other words, for all legal schedules the associated $\rightarrow$ relation is acyclic.

16.1.2 Granting of Locks

When a transaction requests a lock on a data item in a particular mode, and no other transaction has a lock on the same data item in a conflicting mode, the lock can be granted. However, care must be taken to avoid the following scenario. Suppose a transaction T_2 has a shared-mode lock on a data item, and another transaction T_1 requests an exclusive-mode lock on the data item. Clearly, T_1 has to wait for T_2 to release the shared-mode lock. Meanwhile, a transaction T_3 may request a shared-mode lock on the same data item. The lock request is compatible with the lock granted to T_2, so T_3 may be granted the shared-mode lock. At this point T_2 may release the lock, but still T_1 has to wait for T_3 to finish. But again, there may be a new transaction T_4 that requests a shared-mode lock on the same data item, and is granted the lock before T_3 releases it. In fact, it is possible that there is a sequence of transactions that each requests a shared-mode lock on the data item, and each transaction releases the lock a short while after it is granted, but T_1 never gets the exclusive-mode lock on the data item. The transaction T_1 may never make progress, and is said to be **starved**.

We can avoid starvation of transactions by granting locks in the following manner: When a transaction T_i requests a lock on a data item Q in a particular mode M, the concurrency-control manager grants the lock provided that

1. There is no other other transaction holding a lock on Q in a mode that conflicts with M.
2. There is no other transaction that is waiting for a lock on Q, and that made its lock request before T_i.

Thus, a lock request will never get blocked by a lock request that is made later.

16.1.3 The Two-Phase Locking Protocol

One protocol that ensures serializability is the **two-phase locking protocol**. This protocol requires that each transaction issue lock and unlock requests in two phases:

1. **Growing phase**. A transaction may obtain locks, but may not release any lock.
2. **Shrinking phase**. A transaction may release locks, but may not obtain any new locks.

Initially, a transaction is in the growing phase. The transaction acquires locks as needed. Once the transaction releases a lock, it enters the shrinking phase, and it can issue no more lock requests.

For example, transactions T_3 and T_4 are two phase. On the other hand, transactions T_1 and T_2 are not two phase. Note that the unlock instructions do not need to appear at the end of the transaction. For example, in the case of transaction T_3, we could move the unlock(B) instruction to just after the lock-X(A) instruction, and still retain the two-phase locking property.

We can show that the two-phase locking protocol ensures conflict serializability. Consider any transaction. The point in the schedule where the transaction has obtained its final lock (the end of its growing phase) is called the **lock point** of the transaction. Now, transactions can be ordered according to their lock points—this ordering is, in fact, a serializability ordering for the transactions. We leave the proof as an exercise for you to do (see Exercise 16.1).

Two-phase locking does *not* ensure freedom from deadlock. Observe that transactions T_3 and T_4 are two phase, but, in schedule 2 (Figure 16.7), they are deadlocked.

Recall from Section 15.6.2 that, in addition to being serializable, schedules should be cascadeless. Cascading rollback may occur under two-phase locking. As an illustration, consider the partial schedule of Figure 16.8. Each transaction observes the two-phase locking protocol, but the failure of T_5 after the read(A) step of T_7 leads to cascading rollback of T_6 and T_7.

Cascading rollbacks can be avoided by a modification of two-phase locking called the **strict two-phase locking protocol**. This protocol requires not only that locking be two phase, but also that all exclusive-mode locks taken by a transaction be held until that transaction commits. This requirement ensures that any data written by an uncommitted transaction are locked in exclusive mode until the transaction commits, preventing any other transaction from reading the data.

Another variant of two-phase locking is the **rigorous two-phase locking protocol**, which requires that all locks be held until the transaction commits. We can easily

T_5	T_6	T_7
lock-X(A)		
read(A)		
lock-S(B)		
read(B)		
write(A)		
unlock(A)		
	lock-X(A)	
	read(A)	
	write(A)	
	unlock(A)	
		lock-S(A)
		read(A)

Figure 16.8 Partial schedule under two-phase locking.

verify that, with rigorous two-phase locking, transactions can be serialized in the order in which they commit. Most database systems implement either strict or rigorous two-phase locking.

Consider the following two transactions, for which we have shown only some of the significant read and write operations:

T_8: read(a_1);
read(a_2);
...
read(a_n);
write(a_1).

T_9: read(a_1);
read(a_2);
display($a_1 + a_2$).

If we employ the two-phase locking protocol, then T_8 must lock a_1 in exclusive mode. Therefore, any concurrent execution of both transactions amounts to a serial execution. Notice, however, that T_8 needs an exclusive lock on a_1 only at the end of its execution, when it writes a_1. Thus, if T_8 could initially lock a_1 in shared mode, and then could later change the lock to exclusive mode, we could get more concurrency, since T_8 and T_9 could access a_1 and a_2 simultaneously.

This observation leads us to a refinement of the basic two-phase locking protocol, in which **lock conversions** are allowed. We shall provide a mechanism for upgrading a shared lock to an exclusive lock, and downgrading an exclusive lock to a shared lock. We denote conversion from shared to exclusive modes by **upgrade**, and from exclusive to shared by **downgrade**. Lock conversion cannot be allowed arbitrarily. Rather, upgrading can take place in only the growing phase, whereas downgrading can take place in only the shrinking phase.

Returning to our example, transactions T_8 and T_9 can run concurrently under the refined two-phase locking protocol, as shown in the incomplete schedule of Figure 16.9, where only some of the locking instructions are shown.

T_8	T_9
lock-S (a_1)	
	lock-S (a_1)
lock-S (a_2)	
	lock-S (a_2)
lock-S (a_3)	
lock-S (a_4)	
	unlock(a_1)
	unlock(a_2)
lock-S (a_n)	
upgrade (a_1)	

Figure 16.9 Incomplete schedule with a lock conversion.

Note that a transaction attempting to upgrade a lock on an item Q may be forced to wait. This enforced wait occurs if Q is currently locked by *another* transaction in shared mode.

Just like the basic two-phase locking protocol, two-phase locking with lock conversion generates only conflict-serializable schedules, and transactions can be serialized by their lock points. Further, if exclusive locks are held until the end of the transaction, the schedules are cascadeless.

For a set of transactions, there may be conflict-serializable schedules that cannot be obtained through the two-phase locking protocol. However, to obtain conflict-serializable schedules through non-two-phase locking protocols, we need either to have additional information about the transactions or to impose some structure or ordering on the set of data items in the database. In the absence of such information, two-phase locking is necessary for conflict serializability—if T_i is a non-two-phase transaction, it is always possible to find another transaction T_j that is two-phase so that there is a schedule possible for T_i and T_j that is not conflict serializable.

Strict two-phase locking and rigorous two-phase locking (with lock conversions) are used extensively in commercial database systems.

A simple but widely used scheme automatically generates the appropriate lock and unlock instructions for a transaction, on the basis of read and write requests from the transaction:

- When a transaction T_i issues a read(Q) operation, the system issues a lock-S(Q) instruction followed by the read(Q) instruction.
- When T_i issues a write(Q) operation, the system checks to see whether T_i already holds a shared lock on Q. If it does, then the system issues an upgrade(Q) instruction, followed by the write(Q) instruction. Otherwise, the system issues a lock-X(Q) instruction, followed by the write(Q) instruction.
- All locks obtained by a transaction are unlocked after that transaction commits or aborts.

16.1.4 Implementation of Locking**

A **lock manager** can be implemented as a process that receives messages from transactions and sends messages in reply. The lock-manager process replies to lock-request messages with lock-grant messages, or with messages requesting rollback of the transaction (in case of deadlocks). Unlock messages require only an acknowledgment in response, but may result in a grant message to another waiting transaction.

The lock manager uses this data structure: For each data item that is currently locked, it maintains a linked list of records, one for each request, in the order in which the requests arrived. It uses a hash table, indexed on the name of a data item, to find the linked list (if any) for a data item; this table is called the **lock table**. Each record of the linked list for a data item notes which transaction made the request, and what lock mode it requested. The record also notes if the request has currently been granted.

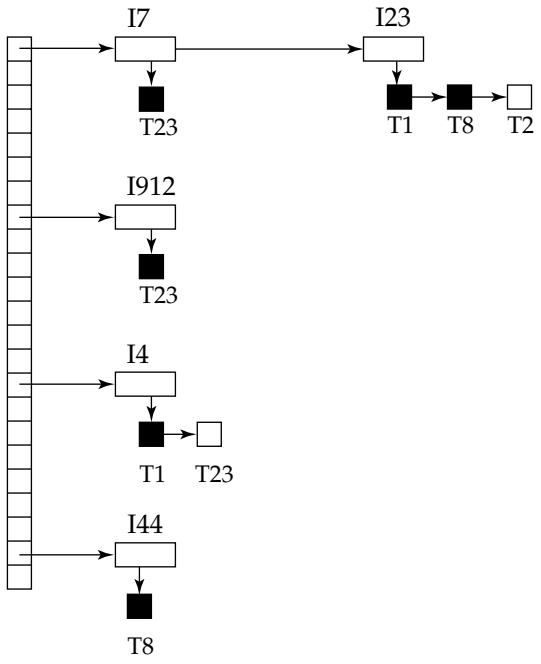


Figure 16.10 Lock table.

Figure 16.10 shows an example of a lock table. The table contains locks for five different data items, I4, I7, I23, I44, and I912. The lock table uses overflow chaining, so there is a linked list of data items for each entry in the lock table. There is also a list of transactions that have been granted locks, or are waiting for locks, for each of the data items. Granted locks are the filled-in (black) rectangles, while waiting requests are the empty rectangles. We have omitted the lock mode to keep the figure simple. It can be seen, for example, that T23 has been granted locks on I912 and I7, and is waiting for a lock on I4.

Although the figure does not show it, the lock table should also maintain an index on transaction identifiers, so that it is possible to determine efficiently the set of locks held by a given transaction.

The lock manager processes requests this way:

- When a lock request message arrives, it adds a record to the end of the linked list for the data item, if the linked list is present. Otherwise it creates a new linked list, containing only the record for the request.

  It always grants the first lock request on a data item. But if the transaction requests a lock on an item on which a lock has already been granted, the lock manager grants the request only if it is compatible with all earlier requests, and all earlier requests have been granted already. Otherwise the request has to wait.

- When the lock manager receives an unlock message from a transaction, it deletes the record for that data item in the linked list corresponding to that transaction. It tests the record that follows, if any, as described in the previous paragraph, to see if that request can now be granted. If it can, the lock manager grants that request, and processes the record following it, if any, similarly, and so on.
- If a transaction aborts, the lock manager deletes any waiting request made by the transaction. Once the database system has taken appropriate actions to undo the transaction (see Section 17.3), it releases all locks held by the aborted transaction.

This algorithm guarantees freedom from starvation for lock requests, since a request can never be granted while a request received earlier is waiting to be granted. We study how to detect and handle deadlocks later, in Section 16.6.3. Section 18.2.1 describes an alternative implementation—one that uses shared memory instead of message passing for lock request/grant.

16.1.5 Graph-Based Protocols

As noted in Section 16.1.3, the two-phase locking protocol is both necessary and sufficient for ensuring serializability in the absence of information concerning the manner in which data items are accessed. But, if we wish to develop protocols that are not two phase, we need additional information on how each transaction will access the database. There are various models that can give us the additional information, each differing in the amount of information provided. The simplest model requires that we have prior knowledge about the order in which the database items will be accessed. Given such information, it is possible to construct locking protocols that are not two phase, but that, nevertheless, ensure conflict serializability.

To acquire such prior knowledge, we impose a partial ordering $\rightarrow$ on the set $\mathbf{D} = \{d_1, d_2, \ldots, d_h\}$ of all data items. If $d_i \rightarrow d_j$, then any transaction accessing both d_i and d_j must access d_i before accessing d_j. This partial ordering may be the result of either the logical or the physical organization of the data, or it may be imposed solely for the purpose of concurrency control.

The partial ordering implies that the set **D** may now be viewed as a directed acyclic graph, called a **database graph**. In this section, for the sake of simplicity, we will restrict our attention to only those graphs that are rooted trees. We will present a simple protocol, called the *tree protocol*, which is restricted to employ only *exclusive* locks. References to other, more complex, graph-based locking protocols are in the bibliographical notes.

In the **tree protocol**, the only lock instruction allowed is lock-X. Each transaction T_i can lock a data item at most once, and must observe the following rules:

1. The first lock by T_i may be on any data item.
2. Subsequently, a data item Q can be locked by T_i only if the parent of Q is currently locked by T_i.

3. Data items may be unlocked at any time.
4. A data item that has been locked and unlocked by T_i cannot subsequently be relocked by T_i.

All schedules that are legal under the tree protocol are conflict serializable.

To illustrate this protocol, consider the database graph of Figure 16.11. The following four transactions follow the tree protocol on this graph. We show only the lock and unlock instructions:

T_{10}: lock-X(B); lock-X(E); lock-X(D); unlock(B); unlock(E); lock-X(G); unlock(D); unlock(G).
T_{11}: lock-X(D); lock-X(H); unlock(D); unlock(H).
T_{12}: lock-X(B); lock-X(E); unlock(E); unlock(B).
T_{13}: lock-X(D); lock-X(H); unlock(D); unlock(H).

One possible schedule in which these four transactions participated appears in Figure 16.12. Note that, during its execution, transaction T_{10} holds locks on two *disjoint* subtrees.

Observe that the schedule of Figure 16.12 is conflict serializable. It can be shown not only that the tree protocol ensures conflict serializability, but also that this protocol ensures freedom from deadlock.

The tree protocol in Figure 16.12 does not ensure recoverability and cascadelessness. To ensure recoverability and cascadelessness, the protocol can be modified to not permit release of exclusive locks until the end of the transaction. Holding exclusive locks until the end of the transaction reduces concurrency. Here is an alternative that improves concurrency, but ensures only recoverability: For each data item with an uncommitted write we record which transaction performed the last write to the data item. Whenever a transaction T_i performs a read of an uncommitted data item, we record a **commit dependency** of T_i on the transaction that performed the

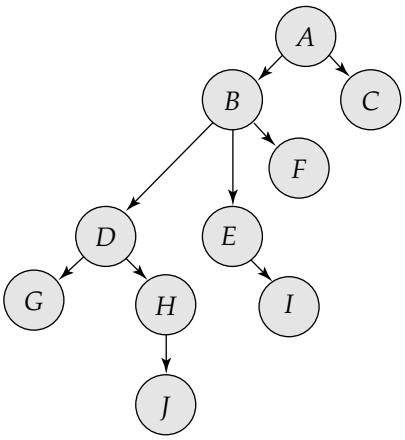


Figure 16.11 Tree-structured database graph.

T_{10}	T_{11}	T_{12}	T_{13}
lock-X(B)			
	lock-X(D)		
	lock-X(H)		
	unlock(D)		
lock-X(E)			
lock-X(D)			
unlock(B)			
unlock(E)			
		lock-X(B)	
		lock-X(E)	
	unlock(H)		
lock-X(G)			
unlock(D)			
			lock-X(D)
			lock-X(H)
			unlock(D)
			unlock(H)
		unlock(E)	
		unlock(B)	
unlock (G)			

Figure 16.12 Serializable schedule under the tree protocol.

last write to the data item. Transaction T_i is then not permitted to commit until the commit of all transactions on which it has a commit dependency. If any of these transactions aborts, T_i must also be aborted.

The tree-locking protocol has an advantage over the two-phase locking protocol in that, unlike two-phase locking, it is deadlock-free, so no rollbacks are required. The tree-locking protocol has another advantage over the two-phase locking protocol in that unlocking may occur earlier. Earlier unlocking may lead to shorter waiting times, and to an increase in concurrency.

However, the protocol has the disadvantage that, in some cases, a transaction may have to lock data items that it does not access. For example, a transaction that needs to access data items A and J in the database graph of Figure 16.11 must lock not only A and J, but also data items B, D, and H. This additional locking results in increased locking overhead, the possibility of additional waiting time, and a potential decrease in concurrency. Further, without prior knowledge of what data items will need to be locked, transactions will have to lock the root of the tree, and that can reduce concurrency greatly.

For a set of transactions, there may be conflict-serializable schedules that cannot be obtained through the tree protocol. Indeed, there are schedules possible under the two-phase locking protocol that are not possible under the tree protocol, and vice versa. Examples of such schedules are explored in the exercises.

16.2 Timestamp-Based Protocols

The locking protocols that we have described thus far determine the order between every pair of conflicting transactions at execution time by the first lock that both members of the pair request that involves incompatible modes. Another method for determining the serializability order is to select an ordering among transactions in advance. The most common method for doing so is to use a *timestamp-ordering* scheme.

16.2.1 Timestamps

With each transaction T_i in the system, we associate a unique fixed timestamp, denoted by TS(T_i). This timestamp is assigned by the database system before the transaction T_i starts execution. If a transaction T_i has been assigned timestamp TS(T_i), and a new transaction T_j enters the system, then TS(T_i) $<$ TS(T_j). There are two simple methods for implementing this scheme:

1. Use the value of the *system clock* as the timestamp; that is, a transaction's timestamp is equal to the value of the clock when the transaction enters the system.
2. Use a **logical counter** that is incremented after a new timestamp has been assigned; that is, a transaction's timestamp is equal to the value of the counter when the transaction enters the system.

The timestamps of the transactions determine the serializability order. Thus, if TS(T_i) $<$ TS(T_j), then the system must ensure that the produced schedule is equivalent to a serial schedule in which transaction T_i appears before transaction T_j.

To implement this scheme, we associate with each data item Q two timestamp values:

- **W-timestamp**(Q) denotes the largest timestamp of any transaction that executed write(Q) successfully.
- **R-timestamp**(Q) denotes the largest timestamp of any transaction that executed read(Q) successfully.

These timestamps are updated whenever a new read(Q) or write(Q) instruction is executed.

16.2.2 The Timestamp-Ordering Protocol

The **timestamp-ordering protocol** ensures that any conflicting read and write operations are executed in timestamp order. This protocol operates as follows:

1. Suppose that transaction T_i issues read(Q).
   a. If TS(T_i) $<$ W-timestamp(Q), then T_i needs to read a value of Q that was already overwritten. Hence, the read operation is rejected, and T_i is rolled back.

b. If TS(T_i) $\geq$ W-timestamp(Q), then the read operation is executed, and R-timestamp(Q) is set to the maximum of R-timestamp(Q) and TS(T_i).

2. Suppose that transaction T_i issues write(Q).

a. If TS(T_i) $<$ R-timestamp(Q), then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced. Hence, the system rejects the write operation and rolls T_i back.

b. If TS(T_i) $<$ W-timestamp(Q), then T_i is attempting to write an obsolete value of Q. Hence, the system rejects this write operation and rolls T_i back.

c. Otherwise, the system executes the write operation and sets W-timestamp(Q) to TS(T_i).

If a transaction T_i is rolled back by the concurrency-control scheme as result of issuance of either a read or write operation, the system assigns it a new timestamp and restarts it.

To illustrate this protocol, we consider transactions T_{14} and T_{15}. Transaction T_{14} displays the contents of accounts A and B:

T_{14}: read(B);
read(A);
display($A + B$).

Transaction T_{15} transfers \$50 from account A to account B, and then displays the contents of both:

T_{15}: read(B);
$B := B - 50$;
write(B);
read(A);
$A := A + 50$;
write(A);
display($A + B$).

In presenting schedules under the timestamp protocol, we shall assume that a transaction is assigned a timestamp immediately before its first instruction. Thus, in schedule 3 of Figure 16.13, TS(T_{14}) $<$ TS(T_{15}), and the schedule is possible under the timestamp protocol.

We note that the preceding execution can also be produced by the two-phase locking protocol. There are, however, schedules that are possible under the two-phase locking protocol, but are not possible under the timestamp protocol, and vice versa (see Exercise 16.20).

The timestamp-ordering protocol ensures conflict serializability. This is because conflicting operations are processed in timestamp order.

The protocol ensures freedom from deadlock, since no transaction ever waits. However, there is a possibility of starvation of long transactions if a sequence of conflicting short transactions causes repeated restarting of the long transaction. If

T_{14}	T_{15}
read (B)	
	read (B)
	$B := B - 50$
	write (B)
read (A)	
	read (A)
display ($A + B$)	
	$A := A + 50$
	write (A)
	display ($A + B$)

Figure 16.13 Schedule 3.

a transaction is found to be getting restarted repeatedly, conflicting transactions need to be temporarily blocked to enable the transaction to finish.

The protocol can generate schedules that are not recoverable. However, it can be extended to make the schedules recoverable, in one of several ways:

- Recoverability and cascadelessness can be ensured by performing all writes together at the end of the transaction. The writes must be atomic in the following sense: While the writes are in progress, no transaction is permitted to access any of the data items that have been written.
- Recoverability and cascadelessness can also be guaranteed by using a limited form of locking, whereby reads of uncommitted items are postponed until the transaction that updated the item commits (see Exercise 16.22).
- Recoverability alone can be ensured by tracking uncommitted writes, and allowing a transaction T_i to commit only after the commit of any transaction that wrote a value that T_i read. Commit dependencies, outlined in Section 16.1.5, can be used for this purpose.

16.2.3 Thomas' Write Rule

We now present a modification to the timestamp-ordering protocol that allows greater potential concurrency than does the protocol of Section 16.2.2. Let us consider schedule 4 of Figure 16.14, and apply the timestamp-ordering protocol. Since T_{16} starts before T_{17}, we shall assume that TS(T_{16}) < TS(T_{17}). The read(Q) operation of T_{16} succeeds, as does the write(Q) operation of T_{17}. When T_{16} attempts its write(Q) operation, we find that TS(T_{16}) < W-timestamp(Q), since W-timestamp(Q) = TS(T_{17}). Thus, the write(Q) by T_{16} is rejected and transaction T_{16} must be rolled back.

Although the rollback of T_{16} is required by the timestamp-ordering protocol, it is unnecessary. Since T_{17} has already written Q, the value that T_{16} is attempting to write is one that will never need to be read. Any transaction T_i with TS(T_i) < TS(T_{17}) that attempts a read(Q) will be rolled back, since TS(T_i) < W-timestamp(Q). Any

T_{16}	T_{17}
read(Q)	
	write(Q)
write(Q)	

Figure 16.14 Schedule 4.

transaction T_j with TS(T_j) > TS(T_{17}) must read the value of Q written by T_{17}, rather than the value written by T_{16}.

This observation leads to a modified version of the timestamp-ordering protocol in which obsolete write operations can be ignored under certain circumstances. The protocol rules for read operations remain unchanged. The protocol rules for write operations, however, are slightly different from the timestamp-ordering protocol of Section 16.2.2.

The modification to the timestamp-ordering protocol, called **Thomas' write rule**, is this: Suppose that transaction T_i issues write(Q).

1. If TS(T_i) < R-timestamp(Q), then the value of Q that T_i is producing was previously needed, and it had been assumed that the value would never be produced. Hence, the system rejects the write operation and rolls T_i back.
2. If TS(T_i) < W-timestamp(Q), then T_i is attempting to write an obsolete value of Q. Hence, this write operation can be ignored.
3. Otherwise, the system executes the write operation and sets W-timestamp(Q) to TS(T_i).

The difference between these rules and those of Section 16.2.2 lies in the second rule. The timestamp-ordering protocol requires that T_i be rolled back if T_i issues write(Q) and TS(T_i) < W-timestamp(Q). However, here, in those cases where TS(T_i) ≥ R-timestamp(Q), we ignore the obsolete write.

Thomas' write rule makes use of view serializability by, in effect, deleting obsolete write operations from the transactions that issue them. This modification of transactions makes it possible to generate serializable schedules that would not be possible under the other protocols presented in this chapter. For example, schedule 4 of Figure 16.14 is not conflict serializable and, thus, is not possible under any of two-phase locking, the tree protocol, or the timestamp-ordering protocol. Under Thomas' write rule, the write(Q) operation of T_{16} would be ignored. The result is a schedule that is view equivalent to the serial schedule $<T_{16}, T_{17}>$.

16.3 Validation-Based Protocols

In cases where a majority of transactions are read-only transactions, the rate of conflicts among transactions may be low. Thus, many of these transactions, if executed without the supervision of a concurrency-control scheme, would nevertheless leave the system in a consistent state. A concurrency-control scheme imposes overhead of code execution and possible delay of transactions. It may be better to use an alterna-

tive scheme that imposes less overhead. A difficulty in reducing the overhead is that we do not know in advance which transactions will be involved in a conflict. To gain that knowledge, we need a scheme for **monitoring** the system.

We assume that each transaction T_i executes in two or three different phases in its lifetime, depending on whether it is a read-only or an update transaction. The phases are, in order,

1. **Read phase**. During this phase, the system executes transaction T_i. It reads the values of the various data items and stores them in variables local to T_i. It performs all write operations on temporary local variables, without updates of the actual database.
2. **Validation phase**. Transaction T_i performs a validation test to determine whether it can copy to the database the temporary local variables that hold the results of write operations without causing a violation of serializability.
3. **Write phase**. If transaction T_i succeeds in validation (step 2), then the system applies the actual updates to the database. Otherwise, the system rolls back T_i.

Each transaction must go through the three phases in the order shown. However, all three phases of concurrently executing transactions can be interleaved.

To perform the validation test, we need to know when the various phases of transactions T_i took place. We shall, therefore, associate three different timestamps with transaction T_i:

1. **Start**(T_i), the time when T_i started its execution.
2. **Validation**(T_i), the time when T_i finished its read phase and started its validation phase.
3. **Finish**(T_i), the time when T_i finished its write phase.

We determine the serializability order by the timestamp-ordering technique, using the value of the timestamp Validation(T_i). Thus, the value TS(T_i) = Validation(T_i) and, if TS(T_j) $<$ TS(T_k), then any produced schedule must be equivalent to a serial schedule in which transaction T_j appears before transaction T_k. The reason we have chosen Validation(T_i), rather than Start(T_i), as the timestamp of transaction T_i is that we can expect faster response time provided that conflict rates among transactions are indeed low.

The **validation test** for transaction T_j requires that, for all transactions T_i with TS(T_i) $<$ TS(T_j), one of the following two conditions must hold:

1. Finish(T_i) $<$ Start(T_j). Since T_i completes its execution before T_j started, the serializability order is indeed maintained.
2. The set of data items written by T_i does not intersect with the set of data items read by T_j, and T_i completes its write phase before T_j starts its validation phase (Start(T_j) $<$ Finish(T_i) $<$ Validation(T_j)). This condition ensures that

T_{14}	T_{15}
read(B)	
	read(B)
	$B := B - 50$
	read(A)
	$A := A + 50$
read(A)	
⟨*validate*⟩	
display($A + B$)	
	⟨*validate*⟩
	write(B)
	write(A)

Figure 16.15 Schedule 5, a schedule produced by using validation.

the writes of T_i and T_j do not overlap. Since the writes of T_i do not affect the read of T_j, and since T_j cannot affect the read of T_i, the serializability order is indeed maintained.

As an illustration, consider again transactions T_{14} and T_{15}. Suppose that TS(T_{14}) $<$ TS(T_{15}). Then, the validation phase succeeds in the schedule 5 in Figure 16.15. Note that the writes to the actual variables are performed only after the validation phase of T_{15}. Thus, T_{14} reads the old values of B and A, and this schedule is serializable.

The validation scheme automatically guards against cascading rollbacks, since the actual writes take place only after the transaction issuing the write has committed. However, there is a possibility of starvation of long transactions, due to a sequence of conflicting short transactions that cause repeated restarts of the long transaction. To avoid starvation, conflicting transactions must be temporarily blocked, to enable the long transaction to finish.

This validation scheme is called the **optimistic concurrency control** scheme since transactions execute optimistically, assuming they will be able to finish execution and validate at the end. In contrast, locking and timestamp ordering are pessimistic in that they force a wait or a rollback whenever a conflict is detected, even though there is a chance that the schedule may be conflict serializable.

16.4 Multiple Granularity

In the concurrency-control schemes described thus far, we have used each individual data item as the unit on which synchronization is performed.

There are circumstances, however, where it would be advantageous to group several data items, and to treat them as one individual synchronization unit. For example, if a transaction T_i needs to access the entire database, and a locking protocol is used, then T_i must lock each item in the database. Clearly, executing these locks is time consuming. It would be better if T_i could issue a *single* lock request to lock the

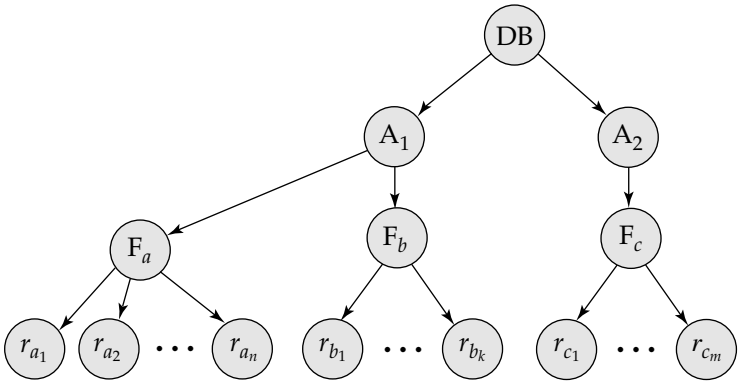


Figure 16.16 Granularity hierarchy.

entire database. On the other hand, if transaction T_j needs to access only a few data items, it should not be required to lock the entire database, since otherwise concurrency is lost.

What is needed is a mechanism to allow the system to define multiple levels of **granularity**. We can make one by allowing data items to be of various sizes and defining a hierarchy of data granularities, where the small granularities are nested within larger ones. Such a hierarchy can be represented graphically as a tree. Note that the tree that we describe here is significantly different from that used by the tree protocol (Section 16.1.5). A nonleaf node of the multiple-granularity tree represents the data associated with its descendants. In the tree protocol, each node is an independent data item.

As an illustration, consider the tree of Figure 16.16, which consists of four levels of nodes. The highest level represents the entire database. Below it are nodes of type *area*; the database consists of exactly these areas. Each area in turn has nodes of type *file* as its children. Each area contains exactly those files that are its child nodes. No file is in more than one area. Finally, each file has nodes of type *record*. As before, the file consists of exactly those records that are its child nodes, and no record can be present in more than one file.

Each node in the tree can be locked individually. As we did in the two-phase locking protocol, we shall use **shared** and **exclusive** lock modes. When a transaction locks a node, in either shared or exclusive mode, the transaction also has implicitly locked all the descendants of that node in the same lock mode. For example, if transaction T_i gets an **explicit lock** on file F_c of Figure 16.16, in exclusive mode, then it has an **implicit lock** in exclusive mode all the records belonging to that file. It does not need to lock the individual records of F_c explicitly.

Suppose that transaction T_j wishes to lock record r_{b_6} of file F_b. Since T_i has locked F_b explicitly, it follows that r_{b_6} is also locked (implicitly). But, when T_j issues a lock request for r_{b_6}, r_{b_6} is not explicitly locked! How does the system determine whether T_j can lock r_{b_6}? T_j must traverse the tree from the root to record r_{b_6}. If any node in that path is locked in an incompatible mode, then T_j must be delayed.

	IS	IX	S	SIX	X
IS	true	true	true	true	false
IX	true	true	false	false	false
S	true	false	true	false	false
SIX	true	false	false	false	false
X	false	false	false	false	false

Figure 16.17 Compatibility matrix.

Suppose now that transaction T_k wishes to lock the entire database. To do so, it simply must lock the root of the hierarchy. Note, however, that T_k should not succeed in locking the root node, since T_i is currently holding a lock on part of the tree (specifically, on file F_b). But how does the system determine if the root node can be locked? One possibility is for it to search the entire tree. This solution, however, defeats the whole purpose of the multiple-granularity locking scheme. A more efficient way to gain this knowledge is to introduce a new class of lock modes, called **intention lock modes**. If a node is locked in an intention mode, explicit locking is being done at a lower level of the tree (that is, at a finer granularity). Intention locks are put on all the ancestors of a node before that node is locked explicitly. Thus, a transaction does not need to search the entire tree to determine whether it can lock a node successfully. A transaction wishing to lock a node—say, Q—must traverse a path in the tree from the root to Q. While traversing the tree, the transaction locks the various nodes in an intention mode.

There is an intention mode associated with shared mode, and there is one with exclusive mode. If a node is locked in **intention-shared (IS) mode**, explicit locking is being done at a lower level of the tree, but with only shared-mode locks. Similarly, if a node is locked in **intention-exclusive (IX) mode**, then explicit locking is being done at a lower level, with exclusive-mode or shared-mode locks. Finally, if a node is locked in **shared and intention-exclusive (SIX) mode**, the subtree rooted by that node is locked explicitly in shared mode, and that explicit locking is being done at a lower level with exclusive-mode locks. The compatibility function for these lock modes is in Figure 16.17.

The **multiple-granularity locking protocol**, which ensures serializability, is this: Each transaction T_i can lock a node Q by following these rules:

1. It must observe the lock-compatibility function of Figure 16.17.
2. It must lock the root of the tree first, and can lock it in any mode.
3. It can lock a node Q in S or IS mode only if it currently has the parent of Q locked in either IX or IS mode.
4. It can lock a node Q in X, SIX, or IX mode only if it currently has the parent of Q locked in either IX or SIX mode.
5. It can lock a node only if it has not previously unlocked any node (that is, T_i is two phase).
6. It can unlock a node Q only if it currently has none of the children of Q locked.